



Birch Stream System Hang and MCE

Issue Debug Handbook

Rev. 1.0

June 2024

Intel Confidential



Notice: This document contains information on products in the design phase of development. The information here is subject to change without notice. Do not finalize a design with this information.

Intel technologies may require enabled hardware, software or service activation.

You may not use or facilitate the use of this document in connection with any infringement or other legal analysis concerning Intel products described herein. You agree to grant Intel a non-exclusive, royalty-free license to any patent claim thereafter drafted which includes subject matter disclosed herein.

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

Performance results are based on testing as of dates shown in configurations and may not reflect all publicly available updates. See backup for configuration details. No product or component can be absolutely secure.

Performance varies by use, configuration, and other factors. Learn more on the [Performance Index site](#).

Your costs and results may vary.

"Conflict-free" refers to products, suppliers, supply chains, smelters, and refiners that, based on our due diligence, do not contain or source tantalum, tin, tungsten or gold (referred to as "conflict minerals" by the U.S. Securities and Exchange Commission) that directly or indirectly finance or benefit armed groups in the Democratic Republic of the Congo or adjoining countries.

All product plans and roadmaps are subject to change without notice.

Code names are used by Intel to identify products, technologies, or services that are in development and not publicly available. These are not "commercial" names and not intended to function as trademarks.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

Altering clock frequency or voltage may void any product warranties and reduce stability, security, performance, and life of the processor and other components. Check with system and component manufacturers for details.

Results have been estimated or simulated.

Intel contributes to the development of benchmarks by participating in, sponsoring, and/or contributing technical support to various benchmarking groups, including the BenchmarkXPRT Development Community administered by Principled Technologies.

Copies of documents which have an order number and are referenced in this document may be obtained by calling 1-800-548-4725 or visiting the [Intel Resource and Document Center](#).

Intel, the Intel logo, Xeon, Simics, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

Copyright © 2024, Intel Corporation. All rights reserved.

Revision History

Revision Number	Description	Date
1.0	- Updated Section 2.4 Birch Stream System Topology and CPU Block Diagram - Updated Section 2.5 Birch Stream Machine Check Bank Assignment and Hierarchy - Updated Section 2.16.6 HW.MCE.UBOX - Added Section 2.16.7 HW.MCE.MSE - Added Section 3.2.6 How to Get CPU Crash Log - Updated Section 3.17 How to Verify Processor Getting into Shutdown State - Added Section 3.31 How to Check Merged MCA Bank Instance ID - Added Section 3.32 How to Check First MCERR/IERR Source ID - Updated 4.1.1.2 Shutdown State. MCE when CR4.MCE=0 - Removed 4.2.2.2 HW.MCE.DCU.0x20.0x401 - Added Section 4.3 Error Pin Injection and Verification - Added Appendix A.9 What is AWR and AGR - Error injection recipes add into RAS Automation Toolkit v2.5.0 release	June 2024
0.7	- Checked all recipe and error injection method on Intel® Xeon® 6900/6700-Series Processors with E-Cores - Updated Section 2.5 - Added MCA bank mapping and hierarchy diagram for Intel® Xeon® 6900/6700-Series Processors with E-Cores - Added Section 2.6 to summary Intel® Xeon® 6900/6700-Series Processors with E-Cores core MCA bank difference with Granite Rapids - Updated Section 2.7 - Added BUS, L2, MEC, FEC tags for Intel® Xeon® 6900/6700-Series Processors with E-Cores - Updated Section 2.12 and 2.15 - Added ROB timeout for Intel® Xeon® 6900/6700-Series Processors with E-Cores. - Added Section 2.16.6 HW.MCE.Ubox - Updated Section 2.18 - Added poison error changes on Intel® Xeon® 6900/6700-Series Processors with E-Cores - Updated Section 3.8, 3.9 for ROB timeout on Intel® Xeon® 6900/6700-Series Processors with E-Cores - Added Section 3.11 - Added Section 3.25 - Added Section 3.28 - Added Section 3.29 - Added Section 3.30 - Updated Section 4.1.5 error injection recipe for Intel® Xeon® 6900/6700-Series Processors with E-Cores - Updated Chapter 4.2.1, 4.2.2 FEC and MEC error injection recipe for Intel® Xeon® 6900/6700-Series Processors with E-Cores - Added Section 4.2.6 Ubox Misaligned error injection - Updated Appendix A.4 for Intel® Xeon® 6900/6700-Series Processors with E-Cores	December 2023

continued...

Revision Number	Description	Date
	Added Appendix A.9 BMC Crashdump online parsing tool: BMC Assisted FRU Isolation (BAFI)	
0.5	- Updated Section 2.6 system hang issue classification. - Updated Section 2.8 system hang issue triage flow, take shutdown state into consideration. - Added Section 2.10 Shutdown state error triage - Updated Section 2.13 PCU error triage, add 2.13.2 S3M error triage - Updated Section 2.14 TOR timeout error triage flow, add 2.14.4 OOBMSM TOR timeout error triage - Added Section 2.18 S3M issue debug - Added Section 3.1.4 how to verify if it is not a known issue recipe and flow - Added 3.2.5 How to get crashdump and 3.2.6 how to get selftest result - Added 3.15 how to find which socket report first IERR - Updated Section 3.20 how to use BMC dump to decode errors - Updated Section 3.21 how to collect and analyze IEH errors - Added 3.22 how to verify S3M error happens - Added 3.23 how to check S3M errors - Added 3.24 how to enable S3M UART trace log - Added 3.26 how to check TOR_Timeout.OOBMSM type - Updated Section 4, error injection cases verified on Granite Rapids-AP and Granite Rapids-SP CPU	September 2023
0.3	- Initial release of the document - Focus on limited debug recipes on Birch Stream Platform, platform power on experience, and real cases also has been added into this release	April 2023

Contents

Revision History.....	3
1.0 Introduction.....	10
1.1 Contents Introduction.....	10
1.2 How to Use This Debug Handbook.....	11
1.3 References.....	11
1.4 Terminology.....	12
2.0 System Hang and MCE Issue Triage.....	13
2.1 Scientific Debugging Process.....	13
2.2 Pre-Work before Debug.....	15
2.3 Things to Do Before Contacting Intel.....	15
2.4 Birch Stream System Topology and CPU Block Diagram.....	16
2.5 Birch Stream Machine Check Bank Assignment and Hierarchy.....	17
2.6 Core MCA Bank Difference between Granite Rapids and Intel® Xeon® 6900/6700- Series Processors with E-Cores.....	19
2.7 System Hang Issue Triage.....	20
2.7.1 Hard Hang.IERR.....	20
2.7.2 Hard Hang.non-IERR.....	23
2.7.3 Soft Hang.....	23
2.8 Birch Stream Issue Triage Tag Introduction.....	24
2.9 System Hang Issue Triage Flow.....	26
2.9.1 Flow Option 1.....	27
2.9.2 Flow Option 2.....	31
2.10 Soft Hang.Non_MCE.....	32
2.11 Hang.Shutdown_State.....	33
2.12 Hang.3strike_Only.....	34
2.13 Hang.Nested_MCE.....	35
2.14 Hang.PCU.....	36
2.14.1 HW.MCE.PCU.SVID	38
2.14.2 HW.MCE.PCU.S3M.....	38
2.14.3 HW.MCE.PCU.Others	39
2.15 Hang.3Strike.TOR.....	39
2.15.1 Hang.TOR.LOCK.....	42
2.15.2 Hang.TOR.PCIe.....	43
2.15.3 Hang.TOR.PCIe_Int.....	44
2.15.4 Hang.TOR.OOBMSM.....	44
2.15.5 Hang.TOR.MEM.....	45
2.16 Single MCERR Issue Debug.....	45
2.16.1 HW.MCE.IFU.....	46
2.16.2 HW.MCE.DCU.....	46
2.16.3 HW.MCE.DTLB.....	46
2.16.4 HW.MCE.MLC.....	46
2.16.5 HW.MCE.CHA.....	47
2.16.6 HW.MCE.Ubox.....	47
2.16.7 HW.MCE.MSE.....	48
2.17 Other IERR Issue Debug.....	49
2.18 UCR Issue Debug.....	49

2.19 S3M Issue Debug.....	49
3.0 Useful Recipes for Triaging System Hang and MCE Issues.....	51
3.1 Recipes on Prewrite.....	51
3.1.1 How to Verify the System Hang Issue is Not a Single System Issue.....	51
3.1.2 How to Speed Up Debugging Your System Hang Issue.....	52
3.1.3 How to Get Reference BIOS and Drivers Files.....	53
3.1.4 How to Verify if It is Not a Known Issue.....	53
3.1.5 How to Write a System Hang Issue Title.....	55
3.1.6 How to Write a Failure Signature.....	55
3.2 General Recipes.....	56
3.2.1 How to Enable/Disable Poison.....	56
3.2.2 How to Set System Event Breakpoint with Intel® In-Target Probe (Intel® ITP)...	57
3.2.3 How to Check the Break is Setting Well.....	58
3.2.4 How to Get Uncore Dump.....	59
3.2.5 How to Get Crashdump.....	59
3.2.6 How to Get CPU Crash Log.....	60
3.2.7 How to Get SelfTest Result.....	60
3.3 How to Verify if System Hang Issue is Hard Hang or Soft Hang.....	61
3.4 How to Verify if Soft Hang Issue with MCE Happens or Not.....	62
3.5 How to Verify if IERR Happens.....	63
3.6 How to Verify if MCERR Happens.....	65
3.7 How to Verify Whether IERR is Caused by Real PCU Errors.....	66
3.8 How to Verify if 3Strike-Timeout/ROB-Timeout Only is True.....	68
3.9 How to Enabled/Disable 3StrikeTimer.....	69
3.10 How to Verify if TOR Timeout is Enabled or Disabled.....	69
3.11 How to Check TOR Entry if TOR Timeout is Disabled.....	71
3.12 How to Verify IERR is Caused by Nested MCE.....	72
3.13 How to Check Multiple MCA Banks Issue MCE in System.....	72
3.14 How to Find Which Socket Report First MCERR.....	73
3.15 How to Find Which Socket Report First IERR.....	74
3.16 How to Decode MCA Bank.....	75
3.17 How to Verify Processor Getting into Shutdown State.....	78
3.18 How to Decode CHA Bank Address Register when TOR Timeout Happens.....	80
3.19 How to Draw a Transaction Flow.....	83
3.20 How to Use BMC Dump to Decode Errors.....	87
3.21 How to Collect and Analyze Integrated Error Handler (IEH) Errors.....	87
3.22 How to Verify S3M Fatal Error Happens.....	88
3.23 How to Check S3M Errors.....	88
3.24 How to Enable S3M UART Trace Log.....	90
3.25 How to Dump S3M TREGs Via <code>irdrscan()</code>	90
3.26 How to Check PCIe* Errors.....	97
3.27 How to Check TOR_Timeout.OOBMSM Type.....	97
3.28 How to Verify First MCE is from CHA Bank and Error Code is TOR Timeout.....	98
3.29 How to Get CPU SKU Type.....	98
3.30 How to Find which Event Triggers Triple Fault.....	99
3.31 How to Check Merged MCA Bank Instance ID.....	99
3.32 How to Check First MCERR/IERR Source ID.....	100
4.0 Error Injection and Verification.....	103
4.1 Hard Hang Error Injection.....	103

4.1.1 Hard Hang.Shutdown.....	103
4.1.2 Hang.Nested_MCE.....	110
4.1.3 Hang.PCU.....	119
4.1.4 Hang.3strike.TOR.....	124
4.1.5 Hang.3strike_Only.....	140
4.2 MCE Error Injection.....	142
4.2.1 HW.MCE.IFU.....	142
4.2.2 HW.MCE.DCU.....	146
4.2.3 HW.MCE.CHA	155
4.2.4 TOR Timeout without 3Strike-Timeout.....	157
4.2.5 Uncorrected Recoverable Error Injection under Linux*	162
4.2.6 Ubox Misaligned Error Injection	164
4.3 Error Pin Injection and Verification.....	166
4.3.1 RMCA Error Pin.....	166
4.3.2 CATERR_N Pin.....	171
4.3.3 Error_N[0,1,2] Pin.....	174
4.3.4 GLB_RST_Warn_N Pin.....	179
5.0 Power-On Preparation.....	182
5.1 Reference Documents.....	182
5.2 Preparation Check.....	182
5.2.1 BIOS Preparation Check.....	183
5.2.2 BMC Preparation Check.....	184
6.0 Real Cases.....	185
6.1 [System hang] System Hang at BIOS Checkpoint 0xB7 When Memory VDD Voltage over 1.145v.....	185
6.2 [Display Issue] No Display on Monitor When LegacyVgaStack Setting Do Not Match Platform Design.....	187
Appendix A Overview.....	190
A.1 What is MCERR and IERR?.....	190
A.2 Errors Classification.....	191
A.3 What is Nested MCE?.....	192
A.4 What is 3Strike-Timeout?.....	192
A.5 What is TOR Timeout?.....	193
A.6 What Is PCIe* Completion Timeout?.....	193
A.7 What Is Double Fault (#DF)?.....	194
A.8 What Is Triple Fault?.....	194
A.9 What Is AWR and AGR?.....	194
A.9.1 Asynchronous Warm Reset (AWR).....	194
A.9.2 Asynchronous Global Reset (AGR).....	195
A.10 BMC Crashdump Online Parsing Tool: BMC Assisted FRU Isolation (BAFI)	196

Figures

1	Scientific Debugging Process.....	14
2	System Topology Example.....	16
3	Granite Rapids-AP X1 (1 Compute Die) CPU Block Concept.....	17
4	Machine Check Bank Hierarchy.....	19
5	System Hang Issue Classification.....	20
6	Shutdown Suppression.....	22
7	System Hang and MCE Issue Tag Category.....	24
8	System Hang Issue Triage Flow Option 1.....	27
9	System Hang Issue Triage Flow Option 2.....	31
10	System Hang with or without MCE Issue Classification.....	32
11	Hard Hang.Shutdown_State Debug Flow.....	34
12	Nested MCE Issue Debug Flow.....	36
13	PCU Error Debug Flow.....	37
14	TOR Timeout Issue Debug Flow.....	41
15	External PCIe* Device or Unstable PCIe* Link Causes System Hang Issue Debug Flow Chart.....	43
16	Platform Codename Categorization.....	53
17	Know Issue Verifying.....	54
18	SelfTest Result.....	61
19	Processor Configuration.....	69
20	IA32_MCi_STATUS MSRs.....	76
21	S0C0 Local Read LLC Missing (2S Platform).....	84
22	S0C0 Lock Transaction Flow.....	86
23	S3M TREGs.....	92
24	S3M Error.....	124
25	Ubox MMIO Space Range.....	164
26	BIOS Memory Voltage.....	186
27	MCERR and IERR.....	191
28	Error Classification.....	192
29	Server Crash.....	196
30	BAFI Output.....	197



Tables

1	Core MCA Bank for Granite Rapids.....	17
2	Core MCA Bank for Intel® Xeon® 6900/6700-Series Processors with E-Cores.....	18
3	Uncore MCA Bank for Granite Rapids and Intel® Xeon® 6900/6700-Series Processors with E-Cores.....	18
4	System Hang and MCE Issue Tag Index.....	24
5	HW.MCE.PCU.S3M.....	38
6	Merged MCA Bank Instance IDs.....	99
7	Summary.....	170
8	Preparation Check.....	182

1.0 Introduction

This debug handbook will focus on system hang and Machine Check Exception (MCE) issues debugging on Birch Stream Server platform, although some concepts may apply to previous generations, including Intel® Xeon® processor Scalable family, Intel® Xeon® E5 v4/v5 processor.

System hang issues may behave in different ways but can generally be divided into **two** categories: **Soft Hang** or **Hard Hang**.

Soft Hang means the system does not respond but **all** cores in processors may still be "alive" and able to execute instructions even though a Machine Check Architecture (MCA) bank status may be set to Processor Context Corrupted (PCC). No response in Soft Hang category may be caused by a software dead loop or processors running to an empty code, regardless of whether an MCE has occurred.

Hard Hang means the system encounters a significant error, and one or more cores in processor may not be able to continue execution.

Start from this debug handbook, platform power on experience and real cases were also added.

1.1 Contents Introduction

This Debug Handbook includes:

- **Chapter 1** - Introduction. This chapter is a general introduction of the system hang and MCE issue debug handbook, which includes system hang issue definition and classification and the scope of this book. It also lists recommended methods on how to use this handbook.
- **Chapter 2** - System Hang and MCE Issue Triage. This chapter introduces the issue triage method and issue triage **tag** used in this book. Debugging flow and recommendations for next steps will be introduced based on unified issue triage **tag**.
- **Chapter 3** - Useful Recipes for Triaging System Hang and MCE Issues. This chapter lists most popular recipes on how to make decisions about next steps in the debugging process. These recipes come from hyperlinks in the debugging flowchart or sections in System Hang and MCE Issue Triage.
- **Chapter 4** - Error Injection and Verification. This chapter introduces how to inject different types of errors based on issue triage tag to trigger IERR or Recoverable MCA (RMCA) event. Through these methods, you can practice your debugging tools, recipes, and become familiar with debugging process and different types of errors. Some verification methods will demonstrate how to use special designed experiments to verify some guesses. Feedback to Intel if any recipe does not work.
- **Chapter 5** - Sharing power on related experience.

- **Chapter 6** - Real cases sharing to show debug process and skills. See the Useful Recipes for Triaging System Hang and MCE Issues.
- **Chapter 7 Appendix** - This chapter introduces and clarifies some technical terms.

1.2 How to Use This Debug Handbook

If you have a system hang issue on hand, you can:

Complete [Pre-Work](#) part before debugging and try to triage the issue as a specific issue tag according to triage guide of this book. Tentative methods:

Method 1: Follow triage and debugging flow in System Hang and MCE Issue Triage to triage your system hang issue. If you already know how to do it, skip this. Finally, you will find a unique issue triage tag.

Method 2: If you are dealing with MCE errors one by one, you can check individual MCA Bank Issue Debugging in System Hang and MCE Issue Triage to see if this error code is covered by this book. An injection method may be provided, and you can use it to familiarize yourself with the error.

1.3 References

See the next documents for additional information.

Document	Document ID
<i>Birch Stream Platform DDR5 and MRC Issue Debug Handbook</i>	776519
<i>Birch Stream Platform PCI Express*(PCIe*) and IIO Issue Debug Handbook</i>	776833
<i>Birch Stream Intel® Ultra Path Interconnect (Intel® UPI) Debug Handbook</i>	788995
<i>Management Diagnostic Guide of MCTP in Birch Stream Intel® Out-of-Band Management Services Module</i>	781870
<i>Birch Stream Platform Power On and Reset Issue Debug Handbook</i>	726890
<i>Granite Rapids Processor Architecture Specification</i>	639866
<i>Granite Rapids Processor Register Specification</i>	637776
<i>Birch Stream-SP Platform Electrical Specification</i>	645781
<i>Sierra Forest Processor Architecture Specification</i>	736624
<i>Sierra Forest Processor Registers Specification</i>	737845
<i>Birch Stream AP Platform Design Guide</i>	635354
<i>Birch Stream-SP Platform Design Guide</i>	633502
<i>Birch Stream Platform: Granite Rapids/Sierra Forest Error Architecture and RAS</i>	728644
<i>Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4</i>	671200

1.4 Terminology

Term	Description
SDM	Software Developer Manual, which is public documents (3 volumes) and can be downloaded from www.intel.com
MSCOD	Model Specific Code. Check Volume 3 Chapter 15 Machine Check Architecture of Software Developer Manual (SDM)
MCACOD	Machine Check Architecture Code. Check Volume 3 Chapter 15 Machine Check Architecture of SDM
MCE	Machine Check Exception or Machine Check Error
MCERR	Machine Check ERROR
IERR	Internal ERROR
RMCA	Recoverable MCA event
UCE	UnCorrectable Error
PCC	Processor Context Corrupted. Check Volume 3 Chapter 15 Machine Check Architecture of SDM
BKC	Best Know Configuration
SRAO	Software Recoverable Action Optional (PCC = 0, EN = x, S = 1, AR = 0)
SRAR	Software Recoverable Action Request (PCC = 0, EN = 1, S = 1, AR = 1)
UCNA	UnCorrected No Action Required (PCC = 0, EN = x, S = 0, AR = 0)
INT18	Interrupt 18 which is exception with interrupt vector number 18. Check volume 3 Chapter 6 and 15 of SDM
CHA	Combined caching and Home Agent (HA). Within a socket, the Intel® UPI caching and HAs are implemented as a set of CHA instances.
Tag	Issue Triage tag. Based on issue classification, a tag consists of classification hierarchy information represents a batch of similar issues in this direction. The deeper hierarchy tag means more detail debug process, but more tags we need. More similar issues in a direction, then the tag in this direction may need deeper.
ROB	Reader Buffer

NOTE

There is no Software Recoverable Action Optional (SRAO) error type support on Birch Stream platform.

2.0 System Hang and MCE Issue Triage

The goal of this chapter is to demonstrate a scientific and systematic methodology to triage the system hangs and MCE issues, for who develop product and debug issues on Birch Stream Platform.

This chapter will introduce basic concepts about system hang and MCE, system hang and MCE classification, triage flow for each issue class and a scientific and systematic debugging process, furthermore, the system hang architecture update on Birch Stream Platform is also introduced.

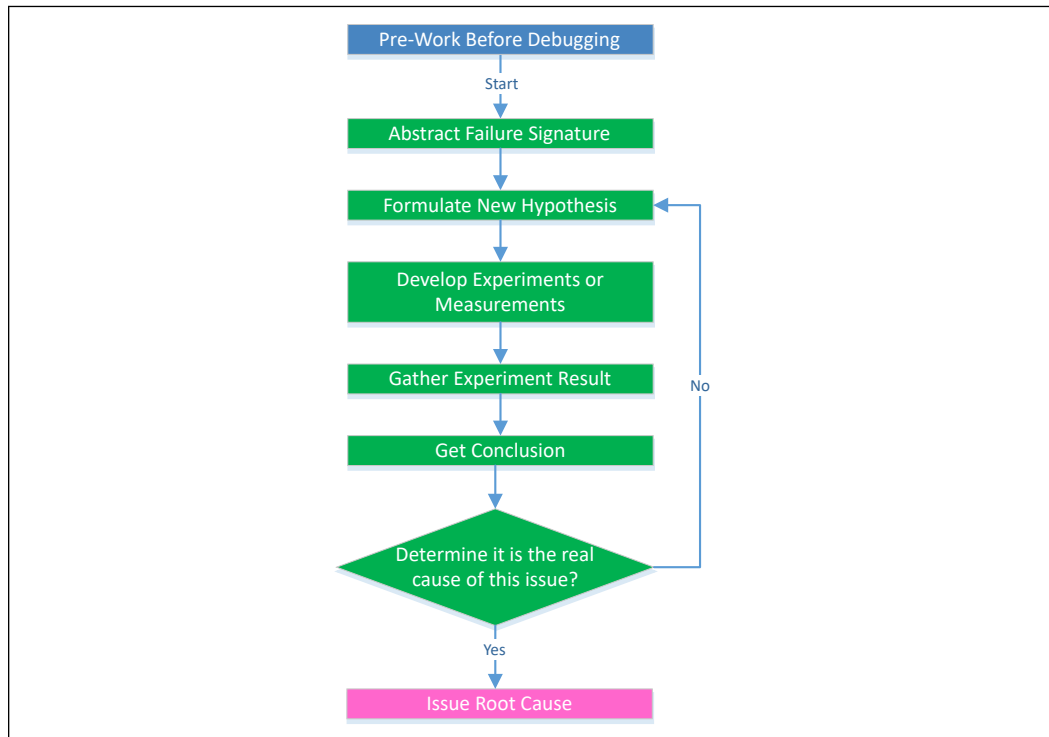
This chapter is intent on providing general guidance on the most common issues, however, noted that the methods and triage flows in this chapter could not cover all the possible issue scenarios.

During system hang and MCE issue debugging and triaging, the recipes used to determine the direction of subsequent debugging are detailed in [Section 3 Useful Recipes for Triaging System Hang and MCE Issues](#).

2.1 Scientific Debugging Process

Referring to scientific method, scientific debug process is strongly recommended during issue debug. It is a set of debug activities to narrow down the scope of the issue and find the root cause. See the next figure.

Figure 1. Scientific Debugging Process



It is recommended to complete the debug **pre-work** before proceeding to the real issue.

A high-quality issue title and failure signature will speed up your issue debug.

Traditionally, the debug process can be divided into two stages: first stage is the issue triage, second stage is to debug in a specific direction. See [Section 2.8 Birch Stream Issue Tag Introduction](#) to obtain the first stage completion flag, which means, the goal of first stage is to find the issue tag.

The scientific debug process is a continuous cycling task, which starts with observed systematic and repeatable data. Based on survey results, error logs and dump data, formulate a theory or hypothesis that is most likely to be a clue to the root cause.

One or more experiments must be developed and followed to determine whether observations (and the results) are consistent or conflicting with the predictions derived from hypotheses.

To conclude, you can formulate the next level of assumptions and continue the debug process cycling.

If you are stuck on the next hypothetical loop level, you can:

- Review your existing data again
- Dump deep level information or other data
- Look up similar real cases sharing
- Consult your peer or looking for database

- Read Debug handbook related sections

The root cause will not be found until the conclusion on the current assumptions are satisfied.

NOTE

Hypothesis, experiments, and theories could be revised based on actual test data.

2.2 Pre-Work before Debug

This is a very important section. Read this section before you start investigating the issue.

When you start working on Birch Stream Platform system power on preparation, see [Section 5 Power On Preparation](#).

When you received a system hang issue, you need to:

- Check that this is not a single system issue
- Check that you have already had a stable recipe to reproduce the issue
- Check that your debug target is with the latest of FW image (BKC or Intel's Platform Updates [IPU])
- Check that you have written a high-quality failure signature
- Check that the debug environment are working well
- Check that you are familiar with scientific debug process
- Check that this system is not hitting a known issue
- Check that the issue can be reproduced on the reference board
- Check that the smallest configuration can reproduce the issue
- See [Section 3.1 Recipes on Pework](#)

File an Intel® Premier Support if you want to call help from Intel or debug flows point to **contact Intel**. You need to collect system configuration, error log or crash dump data for Intel analysis, see [Section 2.3 Things to Do Before Contacting Intel](#) for more details.

2.3 Things to Do Before Contacting Intel

Before contacting Intel via Intel® Premier Support, make sure you have completed next process, it is very helpful for the issue debug and makes the process more effective.

- Pass [pre-work](#) before debugging section, eliminate single system, and known issues
- File an Intel® Premier Support, provide system configuration, detail reproduce recipes and high-quality issue title and failure signature
- Crashdump data. See [Section 3.2.5 How to Get Crashdump](#)
- BIOS serial log
- May require Uncore register dump or BIOS self-test log

See [Section 3.2.4 How to get Uncore dump](#) for uncore register dump.

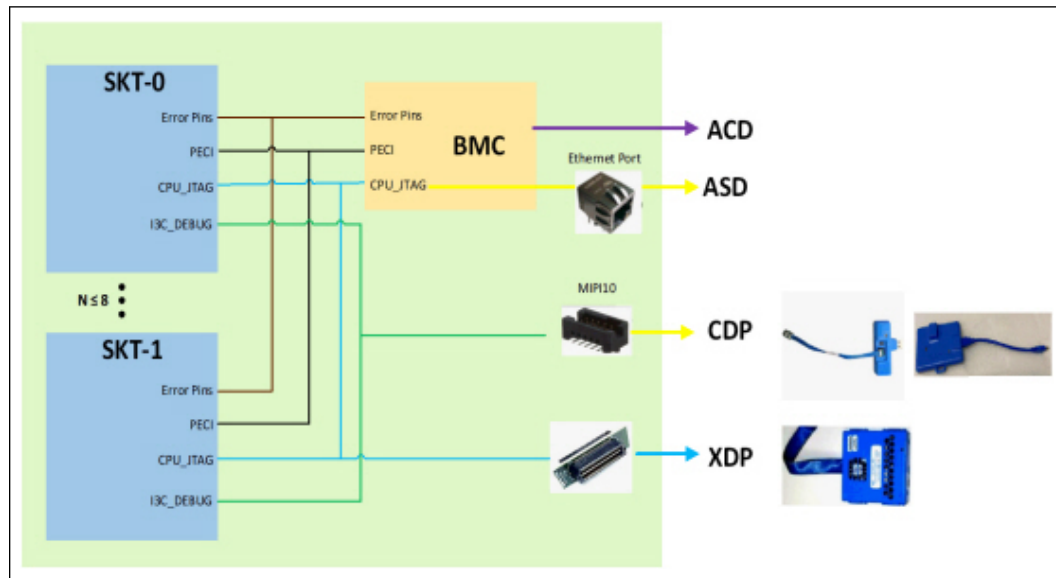
See [Section 3.2.6 How to get SelfTest result](#) for self-test log.

- It is recommended if you can provide an issue triage tag

For more details, see *Intel® Premier Support Case Guidance*, document number 598743.

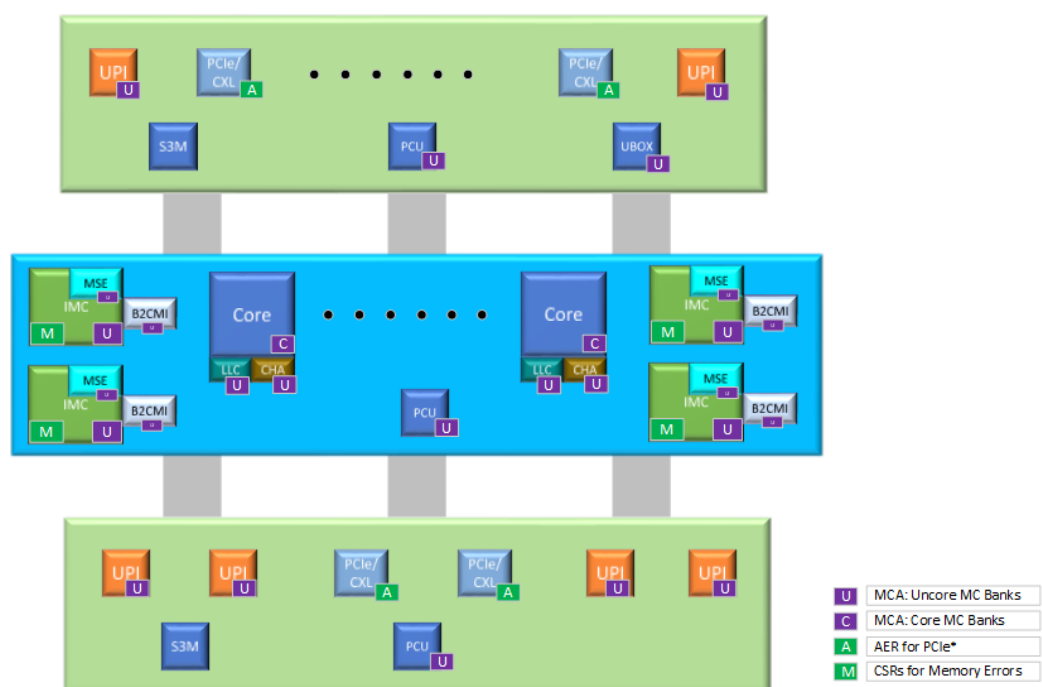
2.4 Birch Stream System Topology and CPU Block Diagram

Figure 2. System Topology Example



NOTE

The previous system topology diagram is an Avenue City RP example.

Figure 3. Granite Rapids-AP X1 (1 Compute Die) CPU Block Concept**NOTE**

This diagram shows a concept. It may vary between different CPU SKUs.

2.5 Birch Stream Machine Check Bank Assignment and Hierarchy

Birch Stream Machine Check Bank ID Mapping Tables

Table 1. Core MCA Bank for Granite Rapids

MCA Bank ID	IP	IA32_MCI_CTL	IA32_MCI_STAT_US	IA32_MCI_AD_DR	IA32_MCI_MI_SC	IA32_MCI_CTL2	Merged?
0	IFU	400h	401h	402h	403h	280h	No
1	DCU	404h	405h	406h	407h	281h	No
2	DTLB	408h	409h	40Ah	40Bh	282h	No
3	MLC	40Ch	40Dh	40Eh	40Fh	283h	No

Table 2. Core MCA Bank for Intel® Xeon® 6900/6700-Series Processors with E-Cores

MCA Bank ID	IP	IA32_MCi_CTL	IA32_MCi_STAT_US	IA32_MCi_AD_DR	IA32_MCi_MI_SC	IA32_MCi_CTL2	Merged?
0	BUS	400h	401h	402h	403h	280h	No
1	L2	404h	405h	406h	407h	281h	No
2	FEC	408h	409h	40Ah	40Bh	282h	No
3	MEC	40Ch	40Dh	40Eh	40Fh	283h	No

Table 3. Uncore MCA Bank for Granite Rapids and Intel® Xeon® 6900/6700-Series Processors with E-Cores

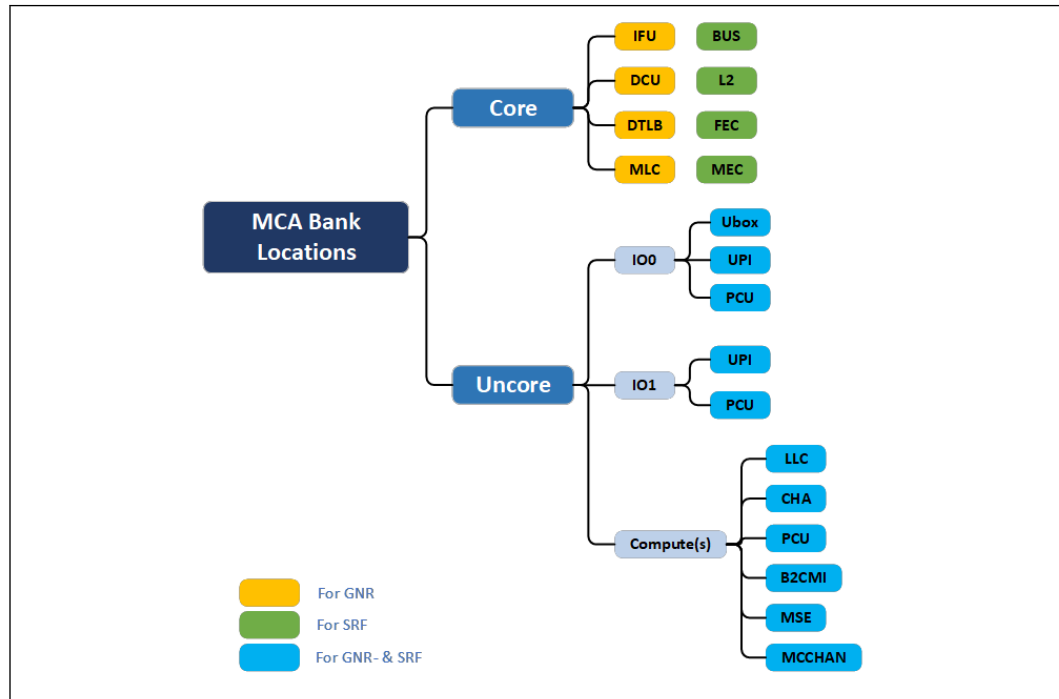
MCA Bank ID	IP	IA32_MCi_CTL	IA32_MCi_STAT_US	IA32_MCi_AD_DR	IA32_MCi_MI_SC	IA32_MCi_CTL2	Merged?
4	Ubox	410h	411h	412h	413h	284h	No
5	UPI	414h	415h	416h	417h	285h	Yes
6	PCU	418h	419h	41Ah	41Bh	286h	Yes
7	CHA	41Ch	41Dh	41Eh	41Fh	287h	Yes
9	LLC	424h	425h	426h	427h	289h	Yes
11	MSE	42Ch	42Dh	42Eh	42Fh	288h	Yes
12	B2CMI	430h	431h	432h	433h	28Ch	Yes
13	MCCHA N0	434h	435h	436h	437h	28Dh	No
14	MCCHA N1	438h	439h	43Ah	43Bh	28Eh	No
15	MCCHA N2	43Ch	43Dh	43Eh	43Fh	28Fh	No
16	MCCHA N3	440h	441h	442h	443h	290h	No
17	MCCHA N4	444h	445h	446h	447h	291h	No
18	MCCHA N5	448h	449h	44Ah	44Bh	292h	No
19	MCCHA N6	44Ch	44Dh	44Eh	44Fh	293h	No
20	MCCHA N7	450h	451h	452h	453h	294h	No
21	MCCHA N8	454h	455h	456h	457h	295h	No
22	MCCHA N9	458h	459h	45Ah	45Bh	296h	No
23	MCCHA N10	45Ch	45Dh	45Eh	45Fh	297h	No
24	MCCHA N11	460h	461h	462h	463h	298h	No

NOTE

Banks 21 to 24 are applicable only to Granite Rapids-AP X3 processors.

See *Birch Stream Platform: Granite Rapids/Sierra Forest Error Architecture and RAS*, document number 728644, chapter “MCA Bank ID Mapping” for more information.

Figure 4. Machine Check Bank Hierarchy



2.6 Core MCA Bank Difference between Granite Rapids and Intel® Xeon® 6900/6700-Series Processors with E-Cores

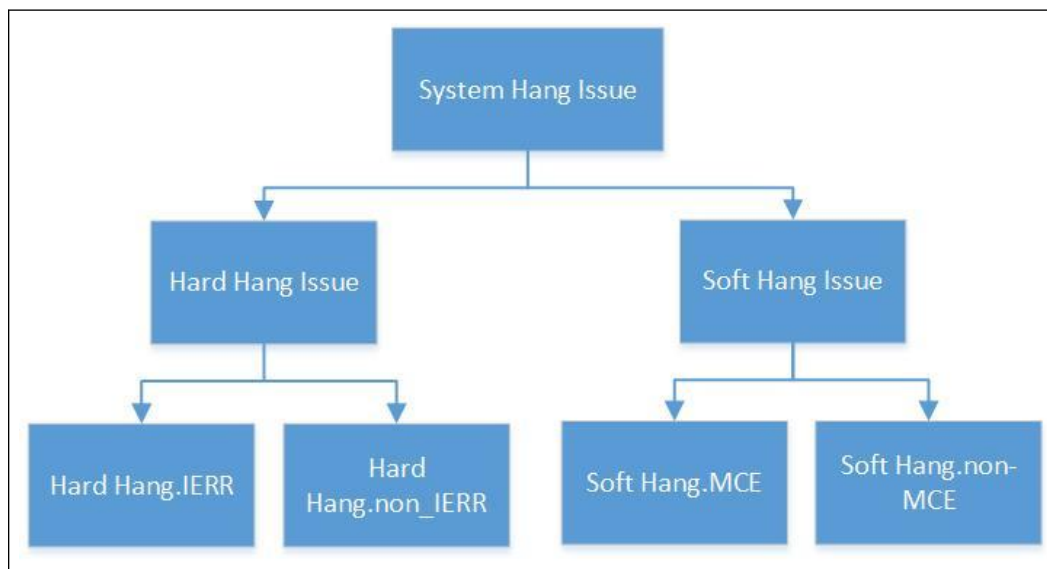
Due to core architecture difference, core IPs and MCA bank on Intel® Xeon® 6900/6700-Series Processors with E-Cores have significant difference with Granite Rapids and previous generation CPU.

- Bank0: BUS bank, logging MCA errors are primarily related to events that occur on the intra-die interconnect but also include events such as the "internal timer" timeout event.
- Bank1: L2 Bank, logging level 2 cache errors.
- Bank2: Front End Cluster (FEC) Bank, logging FEC errors, including Level 1 Instruction cache. Instruction fetch poison error will be logged into FEC bank with MCACOD 0x150.
- Bank3: Memory Execution Cluster (MEC) Bank, logging MEC errors, including Level 1 Data cache. Data load poison error will be logged into MEC bank with MCACOD 0x134.

2.7 System Hang Issue Triage

Before you start to debug system hang issues, review the steps required to categorize system hang issues. See the next figure.

Figure 5. System Hang Issue Classification



Intel categorizes system hang as either hard hang or soft hang. See Introduction to get definition of hard hang and soft hang and see [Section 3.3](#) to know how to verify hang issue is hard Hang or soft Hang.

For hard hang issues, most of them are caused by an IERR, which Intel calls it as Hard Hang.IERR in this book, for the other 'hard hang' cases without capturing any IERR. Intel calls it as Hard Hang. non_IERR .

NOTE

During normal triage flow, you may not need to figure out if the system hang is hard hang or soft hang but follow the first MCERR error source if there is MCE happens in the system.

If you have already seen IERR flag, then you can directly know your system hang is hard hang with IERR. For example, if you see CATERR# signal is asserted or related registers to show IERR happens.

Do not try to halt the processors if you have already known it is hard hang.

2.7.1 Hard Hang.IERR

What is IERR?

IERR is asserted by a processor when the processor is experiencing a serious internal error. IERR will be asserted until the processor is reset. Some source of IERR may have log in related MCA bank with both Uncorrectable Error (UCE) and Processor Context Corrupted (PCC) are set.

IERR sources

Some common sources of IERR are:

- Real PCU error
- 3Strike-Timeout error
- Nested MCERR
- MCE when CR4.MCE=0
- Factory - configuration download failures
- Shutdown State (With MSR (60h) SUPPRESS_SHUTDOWN bit0 enabled)

For real PCU error, it is recommended to decode its error code and follow the error code to further debug.

For 3Strike-Timeout and Nested MCERR, it is recommended to follow first MCERR hint to triage the issue.

For entering Shutdown State, with MSR (60h) SUPPRESS_SHUTDOWN bit0 set, system will trigger IERR.

2.7.1.1 Shutdown State

What is Shutdown State?

If the system encounters specific HW or software error conditions, the processor may enter in the Shutdown State, at this state, the processor stops executing instructions until a Non-Maskable Interrupt (NMI) interrupt, System Management Interrupt (SMI), hardware reset, or INIT# is received, the processor generates a special bus cycle to indicate that it has entered shutdown state. For example, hardware may turn on an indicator light on the front panel, generate an NMI interrupt to record diagnostic information, invoke reset initialization, generate an INIT initialization, or generate an SMI.

MSR (60h) SUPPRESS_SHUTDOWN is used to configure CPU Shutdown behavior.

See the *Granite Rapids Processor Register Specification*, document number 637776 for details:

Figure 6. Shutdown Suppression

(60h) SHUTDOWN_SUPPRESSION			
Shutdown suppression feature			
Type	Size	Scope	Default
MSR	8 bit	Package	00h
Register Level Access:			
Read Access		Write Access	
R		BIOS_DONE	
Bit Range	Default & Access	Field Name (ID): Description	
7:2	0h RO	Reserved	
1	0h RW	SHUTDOWN_LOGMCA_IERR: Trigger IERR and log errors for shutdown cycle	
0	0h RW	SUPPRESS_SHUTDOWN_AND_LOGMCA_IERR: Suppress shutdown special cycle and send IERR with logging	

When Bit0(SUPPRESS_SHUTDOWN_AND_LOGMCA_IERR) is set to 1, enables suppression of all shutdown special cycle while also logging the shutdown cycle cause and escalation to an IERR if the IERR was not already present.

For Birch Stream platform, MSR (60h) SUPPRESS_SHUTDOWN bit 0 is set to 1 by default. It is recommended to enable shutdown suppress function to enhance the log capturing for shutdown cycle.

Shutdown source(Condition):

Shutdown Condition	Description	Enhancement	
		Additional MC4 log (MC4_STATUS)	Additional MC4 log (MC4_STATUS)
		MCACOD[15:0]	MSCOD[31:16]
MCE when CR4.MCE is clear	1. Suppress shutdown (Optional) 2. Additional MC4 logs for shutdown reasons (Optional)	0x040c	0x0001
MCE when MCIP set	1. Suppress shutdown (Optional) 2. Additional MC4 logs for shutdown reasons (Optional)	0x040c	0x0002
MCE under WFS	1. Suppress shutdown (Optional) 2. Additional MC4 logs for shutdown reasons (Optional)	0x040c	0x0003
continued...			

Shutdown Condition	Enhancement		
	Description	Additional MC4 log (MC4_STATUS)	Additional MC4 log (MC4_STATUS)
		MCACOD[15:0]	MSCOD[31:16]
Unrecoverable error during security flow execution (Example: microcode patch authentication)	1. Suppress shutdown (Optional) 2. Additional MC4 logs for shutdown reasons (Optional)	0x040c	0x0004
SW triple fault shutdown	1. Suppress shutdown (Optional) 2. Additional MC4 logs for shutdown reasons (Optional) 3. Assert IERR signal (Optional)	0x040c	0x0005
RSM consistency check failures	1. Suppress shutdown (Optional) 2. Additional MC4 logs for shutdown reasons (Optional) 3. Assert IERR signal (Optional)	0x040c	0x0007
Invalid conditions on protected mode System Management Mode (SMM) entry	1. Suppress shutdown (Optional) 2. Additional MC4 logs for shutdown reasons (Optional) 3. Assert IERR signal (Optional)	0x040c	0x0008
Unrecoverable error during security flow execution (Example: Intel® Software Guard Extensions [Intel® SGX] flows)	1. Suppress shutdown (Optional) 2. Additional MC4 logs for shutdown reasons (Optional) 3. Assert IERR signal (Optional)	0x040c	0x0009
Virtual Machine Extension (VMX) exit consistency check failures	1. Suppress shutdown (Optional) 2. Additional MC4 logs for shutdown reasons (Optional) 3. Assert IERR signal (Optional)	0x040c	0x0006

2.7.2 Hard Hang.non-IERR

If you encounter other hard hang issue without IERR, contact Intel.

2.7.3 Soft Hang

For soft hang issues, Intel uses the presence of an MCE as a flag to determine whether the soft hang issue is more likely to be a pure software issue or not.

2.8 Birch Stream Issue Triage Tag Introduction

In this book, Intel uses **tag** to mark the issue based on issue triage and machine check error code characteristics. A detailed triage flow and debug suggestions are given based on **tag**. Figure 6 is the hierarchical tag category. Table 2 lists all **tags** Intel used in this book. Note that these tags may not cover the whole MCA error code combination, but Intel covers the most popular issues.

Figure 7. System Hang and MCE Issue Tag Category

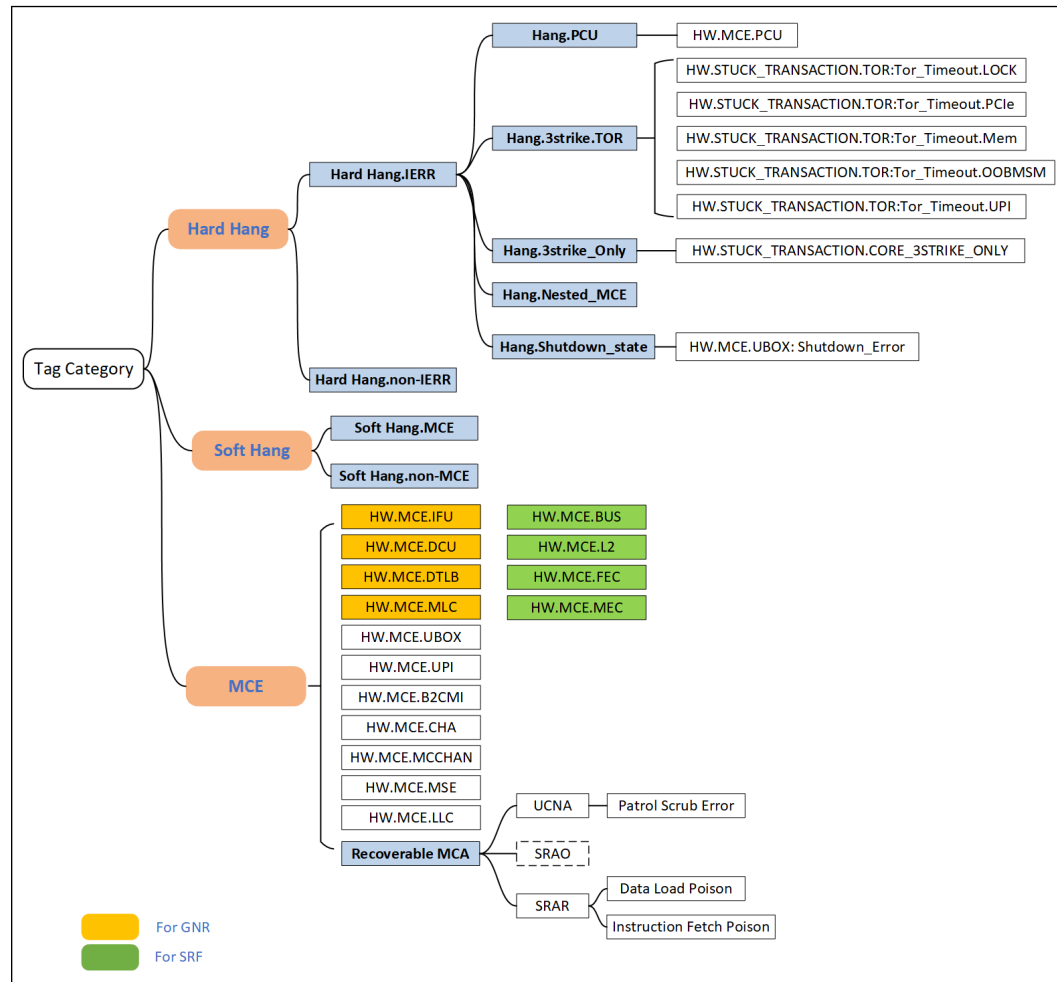


Table 4. System Hang and MCE Issue Tag Index

Issue Category	Issue Tag	Triage Flow and Debug Suggestion	Error Injection
Hang		System Hang Issue Triage Flow on page 26	
Soft Hang.Non_MCE		Soft Hang.Non_MCE	N/A
continued...			

Issue Category	Issue Tag	Triage Flow and Debug Suggestion	Error Injection
Hard Hang.Non_IERR		Debug case by case	N/A
Hang.3strike_Only	HW.STUCK_TRANSACTION.CORE_3STRIKE_ONLY	Hang.3strike_Only on page 34	Hang.3strike_Only on page 34
Hang.Nested_MCE	HW.MCE.OTHER:Shutdown_Error:_Mce_When_Mcip_Bit_Is_Set	Hang.Nested_MCE on page 35	Hang.Nested_MCE on page 35
Hang.PCU	HW.MCE.PUNIT	Hang.PCU on page 36	Hang.PCU on page 36
Hang.Shutdown_State	HW.MCE.UBOX:Shutdown_Error	Hang.Shutdown_State on page 33	
Hang.3strike.TOR		Hang.3Strike.TOR on page 39	Hang.3Strike.TOR on page 39
	HW.STUCK_TRANSACTION.TOR:TOR_Timeout.LOCK	Hang.TOR.LOCK on page 42	N/A
	HW.STUCK_TRANSACTION.TOR:TOR_Timeout.PCIe	Hang.TOR.PCIe on page 43	HW.STUCK_TRANSACTION.TOR: TOR_Timeout.PCIe on page 124
	HW.STUCK_TRANSACTION.TOR:TOR_Timeout.PCIe_Int	Hang.TOR.PCIe_Int on page 44	HW.STUCK_TRANSACTION.TOR:TOR_Timeout.PCIe on page 132
	HW.STUCK_TRANSACTION.TOR:TOR_Timeout.MEM	Hang.TOR.MEM	HW.STUCK_TRANSACTION.TOR:TOR_Timeout.MEM on page 135
MCE		Single MCERR Issue Debug on page 45	MCE Error Injection on page 142
	HW.MCE.IFU	HW.MCE.IFU on page 46	HW.MCE.IFU on page 142
	HW.MCE.DCU	HW.MCE.DCU on page 46	HW.MCE.DCU on page 46
	HW.MCE.DTLB	HW.MCE.DTLB on page 46	N/A
	HW.MCE.MLC	HW.MCE.MLC on page 46	N/A
	HW.MCE.BUS		
	HW.MCE.L2		
	HW.MCE.FEC		
	HW.MCE.MEC		
	HW.MCE.UBOX	For IIO error, see <i>Intel® Xeon® Scalable Processor (Granite Rapids) PCIe* and IIO Issue Debug Handbook</i>	
	HW.MCE.UPI	see <i>Intel® Xeon® Processor Scalable Family (Granite Rapids) Intel® Ultra Path Interconnect (Intel® UPI) Issue Debug Handbook</i>	
	HW.MCE.CHA	HW.MCE.CHA on page 47	HW.MCE.CHA on page 47
	HW.MCE.LLC		
	HW.MCE.MSE		
	HW.MCE.B2CMI		
continued...			

Issue Category	Issue Tag	Triage Flow and Debug Suggestion	Error Injection
	HW.MCE.MCCHAN	See <i>Birch Stream Platform DDR5 and MRC Issue- Debug Handbook</i> , document number 776519	
RMCA		Uncorrected Recoverable Issue Debug	Uncorrectable Recoverable error injection under Linux*
UCNA			UCR - UCNA on page 162
SRAR			UCR - SRAR on page 163

2.9 System Hang Issue Triage Flow

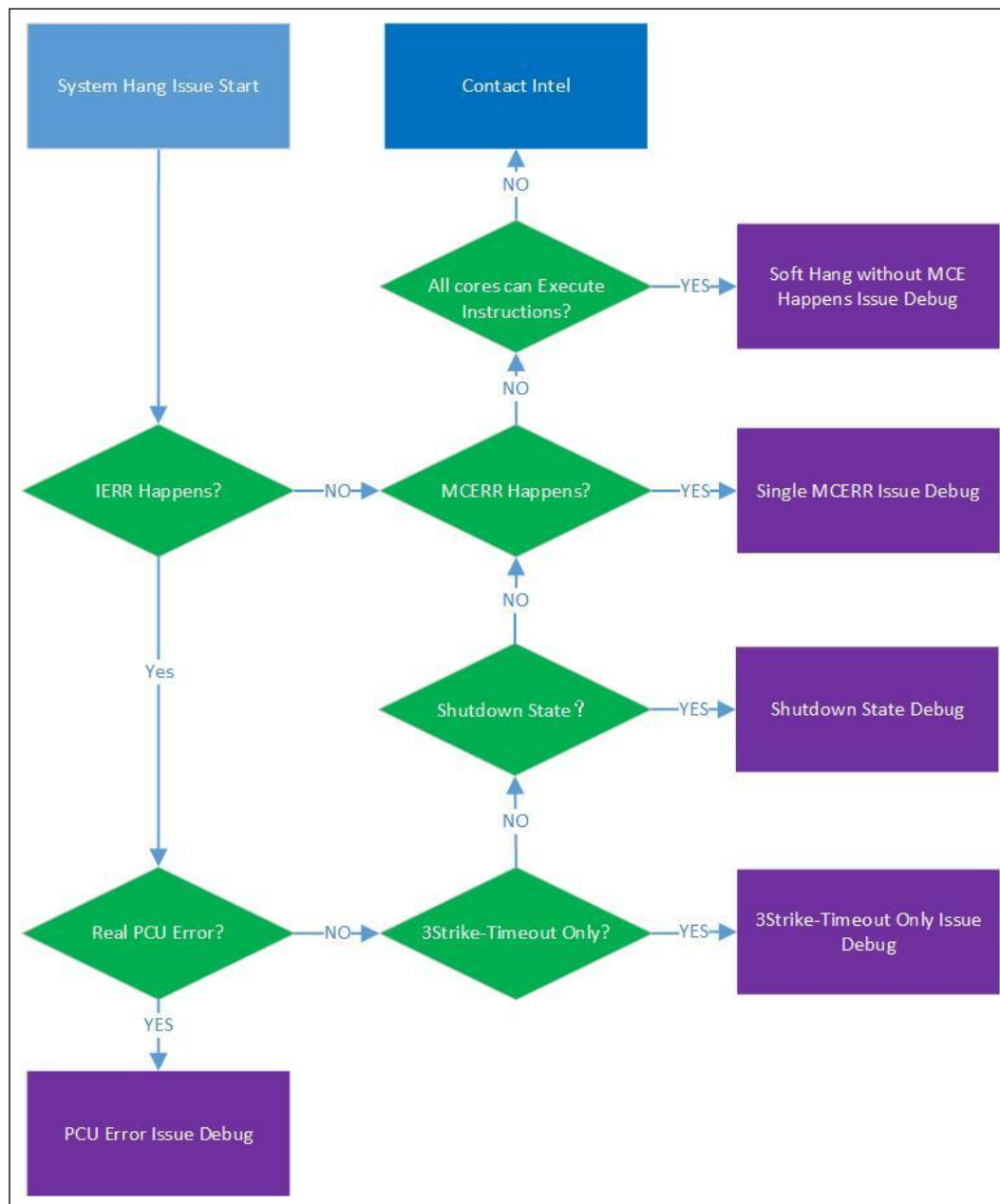
See the next triage flow:

Color code in flow chart:

- [Light Blue](#): debug starting point for this flow
- [Green](#): verification recipes (that is, analyze data)
- [Purple](#): follow a new sub-flow (later in this section)
- [Orange](#): action Request
- [Blue](#): contact Intel
- [Pink](#): done

2.9.1 Flow Option 1

Figure 8. System Hang Issue Triage Flow Option 1



Again, make sure to complete [Section 2.2 Pre-Work Before Debug](#)

1. IERR Happens?

It is recommended to figure out if the processors report IERR or not. The easy way is to check CATERR LED or **CATERR#** signals if LED is not implemented.

You can also get the same information from processors' register through BMC dump, Crashdump or CScripts command.

See [Section 3.5 How to Verify IERR Happens?](#)

- Next step will be "Real PCU Error?" if IERR happens.
- Next step will be "Shutdown State ?" if IERR happens and it is neither real PCU error nor 3Strike-Timeout only error.

2. Real PCU Error?

When IERR flag is set in whole system, then it is recommended to deal with real PCU error first.

See [Section 3.7 How to Verify IERR is Caused by Real PCU Errors?](#)

- Next step will be real PCU error debug if the first IERR source in whole system is from a PCU bank in a socket.

See [Section 2.14 Hang.PCU](#)

- Next step will be "3Strike-Timeout Only?" if the IERR is not caused by real PCU error.

3. 3Strike-Timeout Only?

3Strike-Timeout is one of the most common sources of IERR. Typically, a core 3Strike-Timeout will be accompanied with a Coherency and Home Agent (CHA) Table of Requests (TOR) timeout (HW.MCE.CHA.0xC), but there are cases where a core 3Strike-Timeout could occur without a TOR timeout which Intel called **Hang.3Strike_Only** in this book.

Hang.3Strike_Only will be true if one or several MLC MCA banks report 3Strike-Timeout but no associated CHA MCA banks in the same socket report TOR Timeout.

See [Section 3.8 How to Verify 3Strike-Timeout Only is True?](#)

NOTE

Use Crashdump function in CScripts to check if there is valid stuck TOR entry associated with the core which happened 3Strike-Timeout if TOR timeout is disabled.

See [Section 3.11 How to Check TOR Entry if TOR Timeout is Disabled](#)

The next step will be 3Strike-Timeout ONLY issue debug if 3Strike-Timeout ONLY is true and **no other** MCA banks report MCERR.

See [Section 2.12 Hang.3strike_Only](#)

For the Hang.3Strike_Only combine with other MCERR errors, suggest investigating the MCERR error first based on the first MCERR information.

- Next step will be "Shutdown State?" if IERR happens and it is neither real PCU error nor 3Strike-Timeout only error.

4. Shutdown State?

Check if CPU have entered Shutdown State?

See [Section 3.17 How to Verify Processor Getting into Shutdown State?](#)

Next step will be Shutdown State issue debug if Shutdown State happened.

See [Section 2.11 Hard Hang.Shutdown_State](#)

- Next step will be "MCERR Happens?" when there is no IERR happens.

5. MCERR Happens?

The concept is that except real PCU error, Hang.3Strike_Only and Shutdown State cases, Intel is going to follow first MCERR source lead in whole system to debug the issue.

Next step will be MCE bank error debug if any MCE bank reported error.

See [Section 2.16 Single MCERR Issue Debug](#)

- Next step will be "All Cores Can Execute Instructions?" if there is no IERR/MCERR happens in whole system.

See [Section 3.3 How to Verify System Hang Issue is Hard Hang or Soft Hang](#) Method 2 to check if all cores are still able to execute instructions.

6. All Cores Can Execute Instructions?

Next step will be soft hang without MCE issue debug if all cores are alive.

See [Section 2.10 Soft Hang.Non_MCE](#).

For other cases, contact Intel by needed.

7. Key Register Information

ierrloggingreg.firstierrsrcid: First IERR Source ID, socket level (that is, each processor has one field). There are two types of source IDs, one is PCU, and the other is cores. When PCU Error happens first in a processor, PCU ID will be recorded. When 3Strike-Timeout or other core errors which may be promoted to IERR happens first in a processor, Core ID will be recorded.

mcerrloggingreg.firstmcerrsrcid: First MCERR Source ID, socket level. See *Granite Rapids Processor Register Specification*, document number 637776, to get decode information.

rmcerrloggingreg.firstrmcerrsrcid: First RMCA Source ID, socket level. See *Granite Rapids Processor Register Specification*, document number 637776, to get decode information.

mca_err_src_log.IERR_Internal/MSMI_IERR_Internal: Die level, it shows that current processor may hit IERR/MSMI_IERR internally.

mca_err_src_log.IERR/MSMI_IERR: Die level, it shows that current processor observes external IERR/MSMI_IERR.

mca_err_src_log.MCERR_Internal/MSMI_MCERR_Internal: Die level, it shows that current processor may hit MCERR/MSMI_MCERR internally.

mca_err_src_log.MCERR/MSMI_MCERR: Die level, it shows that current processor observes external MCERR/MSMI_MCERR.

mca_err_src_log.RMCA_Internal/RMSMI_Internal: Die level, it shows that current processor asserted recoverable MCA/MSMI.

mca_err_src_log.RMCA/RMSMI: Die level, it shows that the current processor observed recoverable MCA/MSMI.

pcu_first_ierr_tsc: Die level, it shows Time Stamp Counter (TSC) record when current processor IERR happens. that is, when current processor hits an IERR.

pcu_first_mcerr_tsc: Die level, it shows TSC record when current processor MCERR happens.

pcu_first_rmca_tsc: Die level, it shows TSC record when current processor RMCA happens.

NOTE

Just primary Punit (io0 die) report mca_err_src _log.

8. Analysis Concept

Case 1: There is valid first MCERR in the whole system.

For multiple sockets report MCERR case, try to figure out the first MCERR, then debug from the first MCERR.

See [Section 3.14 How to Find Which Socket Report First MCERR?](#)

Case 2: There is IERR only without MCERR in the whole system.

For the IERR only without MCERR case, it means the source is from real PCU error or 3Strike-Timeout only or some Shutdown State cases. Try to figure out which socket happens IERR first, then debug from the first IERR.

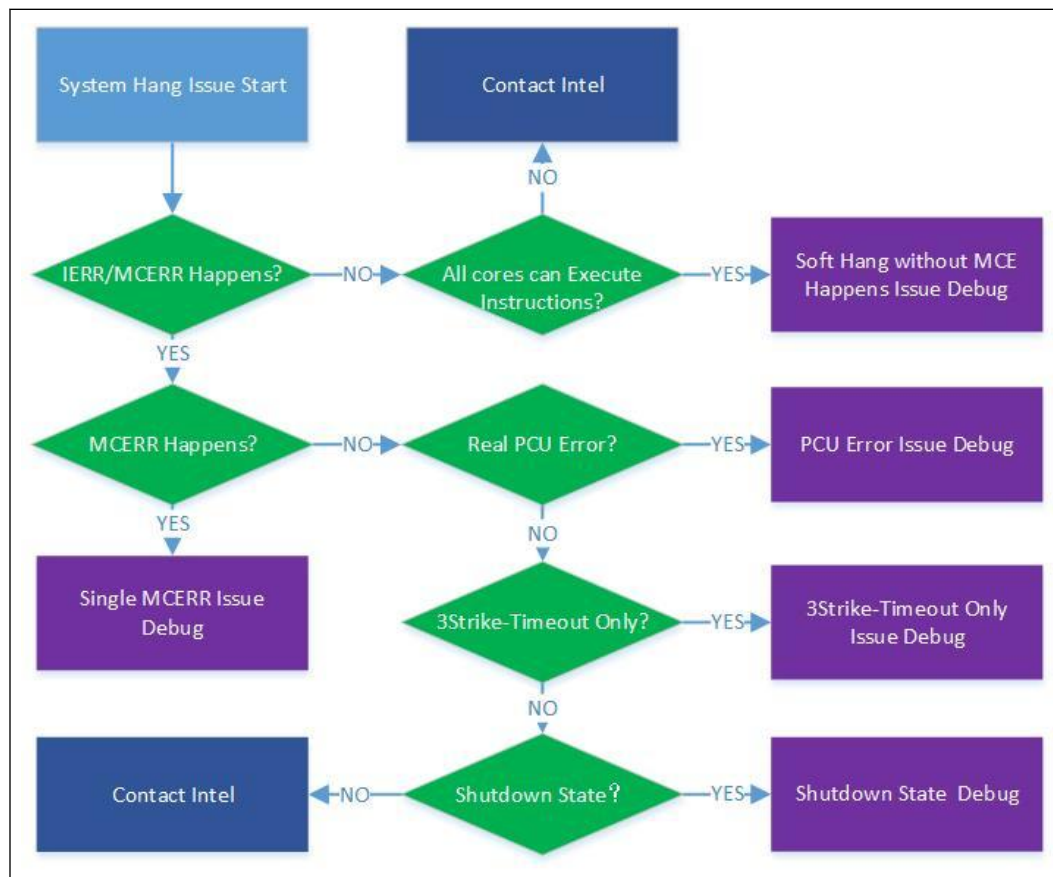
See [Section 3.15 How to Find Which Socket Report First IERR](#)

NOTE

Disable VIRAL mode and Enhanced Machine Check Architecture (eMCA) 2.0 to simplify failure if need.

2.9.2 Flow Option 2

Figure 9. System Hang Issue Triage Flow Option 2



IERR/MCERR Happens?

Figure out if IERR or MCERR happens,

See [Section 3.5 How to Verify if IERR Happens?](#) and [Section 3.6 How to Verify if MCERR Happens?](#)

Next Step: If MCERR happens, find out the first MCERR, then jump to [Section 2.16 Single MCERR Issue Debug](#).

Next Step: If no MCERR happens, but only IERR, then check if it is Real PCU Error? If yes, jump to [Section 2.14 Hang.PCU](#).

Next Step: If no MCERR happens, but only IERR, then check if it is 3Strike-Timeout only? If yes, jump to [Section 2.12 Hang.3strike_Only](#).

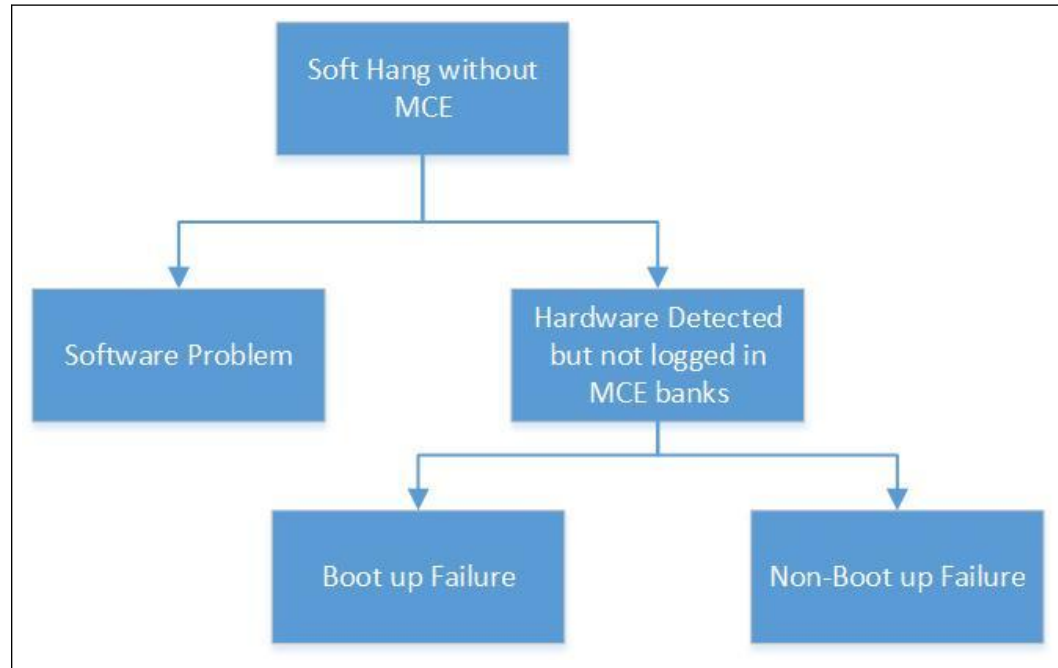
Next Step: If no MCERR happens, but only IERR, then check if CPU have entered Shutdown State? If yes, jump to [Section 2.11 Hard Hang.Shutdown_State](#).

Next Step: If no MCERR or IERR happens, and all cores can execute instructions, then this is more like pure Soft Hang issue Debug, which is not cover in this book currently.

2.10 Soft Hang.Non_MCE

This type of error might be a "pure" software error, or some types of error detected by hardware but not logged in MCA bank registers.

Figure 10. System Hang with or without MCE Issue Classification



Pure software issue will not be covered in this debug handbook. Contact your software vendor, OS vendor or BIOS vendor to debug such issues.

For the case where the hardware detects an error but is not logged in the MCA bank registers, Intel has two sources.

One type of case comes from boot up phase, some boot up failures are related to MCERR (not covered in this section), other failures do not have MCERR. Note that boot up failures related to Intel® Ultra Path Interconnect (Intel® UPI) Memory Reference Code (MRC) are **not** covered in this section. Intel® UPI or Memory debug handbooks will cover them.

Another type of case comes from non-boot up phase, that is, the error occurs while application is running in OS. Errors are detected by hardware/silicon and logged in non-MCA bank registers, which can also cause system hang. For example, some error status registers are in Control and Status Register (CSR) or Model-Specific Register (MSR). Note that Intel deals with these on a case-by-case basis.

Here are just some examples Intel has seen:

- An incorrect configuration of the external PCI Express* (PCIe*) switch built-in driver (the switch detects this error) will cause some versions of Windows* to become unresponsive when the PCIe* devices under the PCIe* switch are disabled.
- System hangs during OS installation due to UEFI code compilation issues.

- System boot failure caused by power sequence or CPLD coding, and so on.
- Windows* blue screen due to patch issues.

2.11 Hang.Shutdown_State

For Shutdown State definition, see to [Section 2.7.1.1](#).

In this case, you should have checked the CPU has entered Shutdown State.

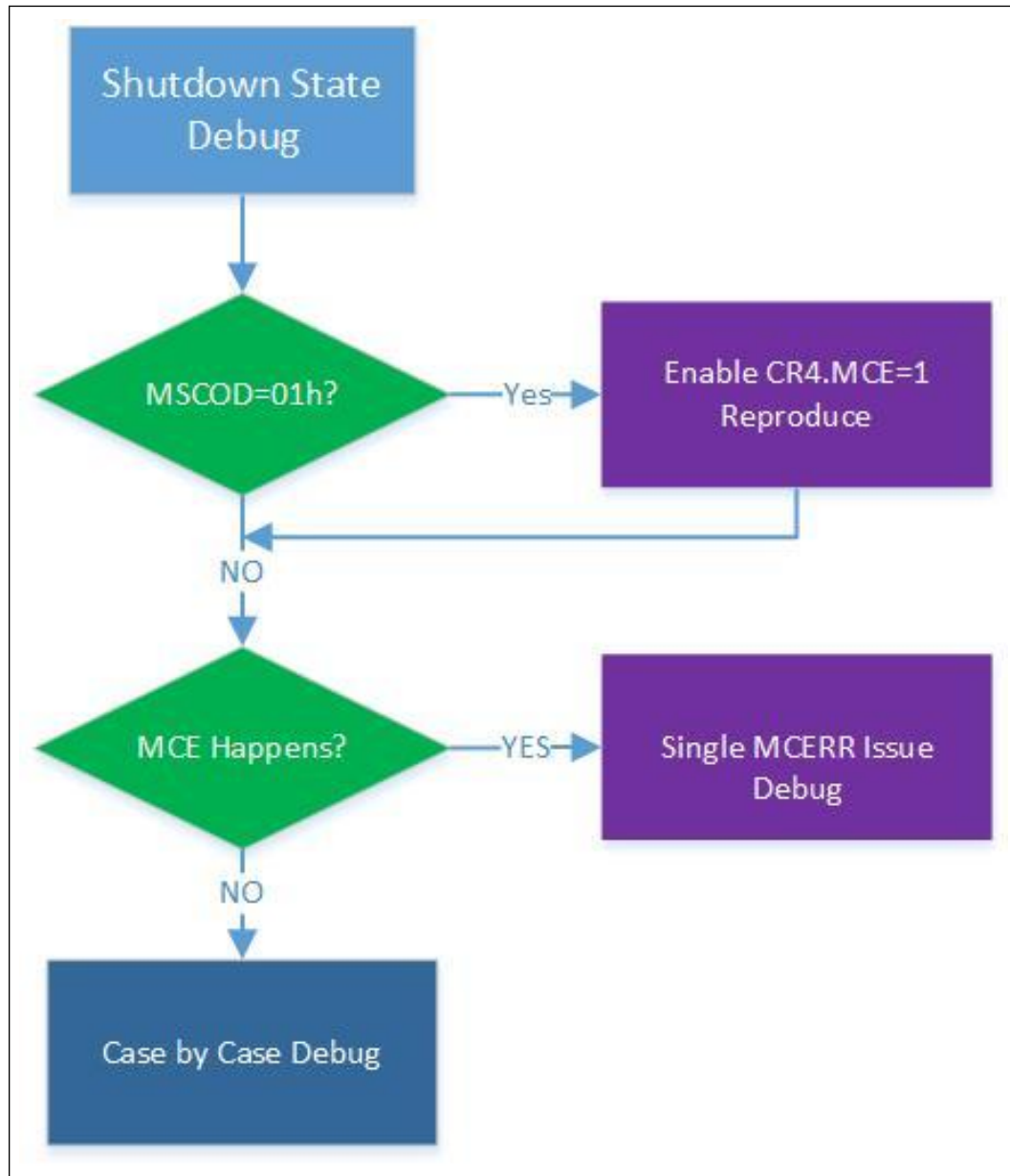
See [Section 3.17 How to Verify Processor Getting into Shutdown State?](#)

For Shutdown State cases, some cases accompanied by MCERR, for such cases, follow "Single MCERR Issue Debug".

For another Shutdown State cases without MCERR happen, do not covered by this book Intel suggests handle such cases case by case.

For Shutdown State triage, see the next figure.

Figure 11. Hard Hang.Shutdown_State Debug Flow



2.12 Hang.3strike_Only

See [How to Verify if 3Strike-Timeout Only is True?](#) to confirm you do see a true Hang.3Strike_Only case.

The possible reason that causes a core 3Strike-Timeout without a corresponding Coherency Home Agent (CHA) Table of Requests (TOR) timeout maybe either the request from the core was never able to make it outside of the core, the request/ completion got "lost" along the core/uncore interface, or instructions that take longer to complete, such as WBINVD or instructions that generate buslocks.

Suggestions: check whether software is heavily utilizing WBINVD or buslock instructions.

Check items:

Based on "Hang.3Strike_Only" definition, Intel assumes you already checked:

- CHA TOR Timeout is enabled
- CHA MCA bank control register TOR Timeout is enabled
- Make sure there is no valid stuck entries in the TOR entry

See [How to Verify if TOR Timeout is Enabled or Disabled?](#)

Suggestions:

If you have already checked all the previous items, consider contacting Intel to get further help.

NOTE

If 3Strike-Timeout happen in very early boot phase, then TOR Timeout may not be enabled by BIOS, like in the very early Intel® UPI link establish stage.

For Granite Rapids: Memory Latency Checker (MLC) bank address register may provide a physical address which is not the same as CHA's. MLC valid address usually points to IP, not real timeout target address which is shown in CHA bank address register.

For Intel® Xeon® 6900/6700-Series Processors with E-Cores: 3Strike-Timeout is called as ROB-Timeout, it will log in BUS Bank. BUS Bank address register not provide a valid physical address.

2.13 Hang.Nested_MCE

Nested MCE means while one MCE handler is running, another MCE occurs. When MCE occurs, Silicon will check Machine Check in Progress (MCIP) bit which is in Machine Check Global Status register (MSR 0x17A). Once the first instruction of the MCE handler is executed, hardware will automatically set MCIP bit.

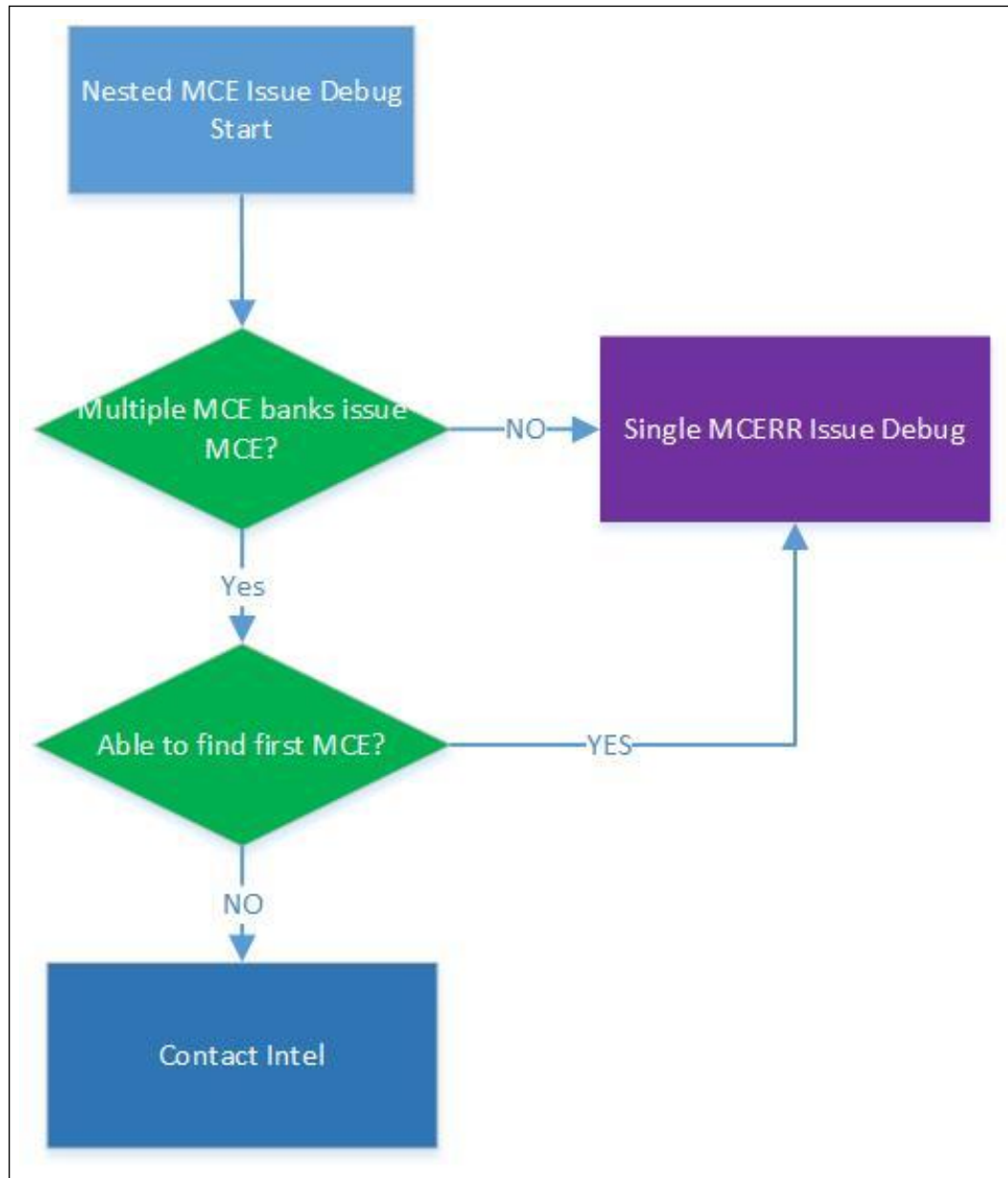
Because nested MCE is combined with several MCEs, the debug concept is to deal with MCEs one by one. Find out FIRST MCE and try to fix it, then reproduce issue again and process the next 'FIRST' MCE.

NOTE

There is no need to figure out if the IERR is caused by Nested MCE. If there are not real PCU errors or PCU errors are not the first IERR source in whole system, just follow first MCERR source hint for debugging.

The debug flow as shown:

Figure 12. Nested MCE Issue Debug Flow



You can also consider:

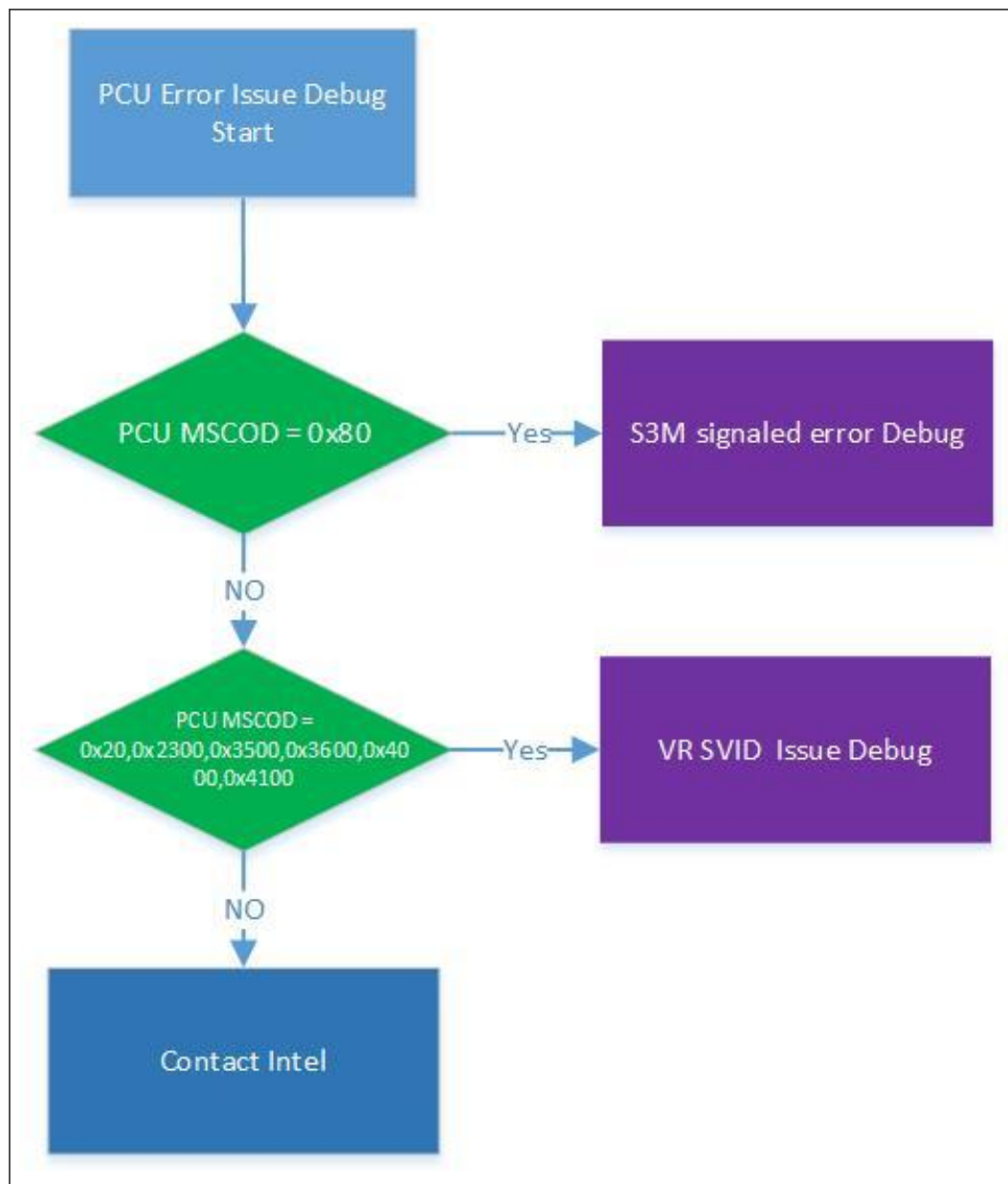
- Disable Viral Mode.
- Setting `itp.cv.machinecheckbreak = 1` and `itp.cv.smmentrybreak = 1` and try to break the system when first MCERR happens.

2.14 Hang.PCU

Customer design or 3rd party issues may cause this TAG, especially related to SVID or Secure Startup Services Module (S3M) signal error.

The general PCU errors debug flow chart is as follow:

Figure 13. PCU Error Debug Flow



The general PCU errors debug method is as follow:

- Check if MCACOD = 0x402
- Check MSCOD error code

PCU error is also one of main sources of IERR. The key notes here are:

- All PCU MCE errors will be promoted to **IERR**.

- The high-level approach to debugging PCU errors varies based on the MCA Encoding for PCU. See *Granite Rapids Processor Architecture Specification*, document number 639866, volume 2 MCA encoding for PCU table to check detail.

2.14.1 HW.MCE.PCU.SVID

For this type of issue, which is reported by PCU, it is needed to get the PCU Pcode error code first. The next table list the issue Intel has seen so far.

Tag	MSCOD	Debug Suggestion
Hang.PCU.HW	0x0020 "Power Management Agent signaled error"	1. Check VR solution selection or VR FW setting 2. Check VR SVID address setting 3. Capture SVID waveform and use Intel SVID Decoder Protocol Checker to check SVID behavior 4. Capture SVID trace with SVID protocol analyzer. 5. Contact VR vendor 6. Contact Intel
Tag	MSCOD[31:24]	
Hang.PCU.FW	0x2300 "MCA_PCU_SVID_ERROR: SVID error"	
	0x3500 "MCA_SVID_LOADLINE_INVALID: Invalid SVID VCCIN Loadline resistance value"	
	0x3600 "MCA_SVID_ICCMAX_INVALID: Invalid SVID ICCMAX value"	
	0x4000 "MCA_SVID_VIDMAX_INVALID: Invalid SVID VID_MAX value"	
	0x4100 "MCA_SVID_VDDRAMP_INVALID: Invalid ramp voltage for VDD_RAMP"	

NOTE

For other error code decoded as SVID related issue, also follow the suggestion in this table and Crashdump summarizer indicates MSCOD as well. See *Granite Rapids Processor Registers Specification*, document number 637776.

2.14.2 HW.MCE.PCU.S3M

Notice that not all S3M errors logged in PCU bank. In early boot phase before CPU come out of reset, S3M errors will not log into PCU bank but assert GLB_RST_WARN#.

Table 5. HW.MCE.PCU.S3M

Tag	MSCOD	MCACOD[15:0]	Debug Suggestion
Hang.PCU.HW	0x80 "S3M signaled error"	0x402	1. Check platform hardware pin strap and soft straps. 2. Check power sequence.

Tag	MSCOD	MCACOD[15:0]	Debug Suggestion
			3. Contact Intel and collect S3M UART logs for Intel analyzing.

2.14.3 HW.MCE.PCU.Others

For the PCU errors: always remember to check the unified patch version firstly, try disabling Core/Pkg P-states or C-states to triage the issue.

Contact Intel if the PCU error types are not SVID or S3M. In this document, the focus is on the PCU Pcode errors, which mainly occur in all the PCU errors.

2.15 Hang.3Strike.TOR

To deal with Hang.TOR issue, Intel needs to determine which transaction cannot be completed, and what the transaction flow is.

A TOR Timeout is an error logged by the CHA IP when a transaction is sent from CHA, but the TOR does not receive a response within a certain time (usually a few seconds).

The CHA MCA bank will report TOR Timeout when TOR Timeout event occurs, the error code (mscod bit [31:16]) is 0xC. Regardless of whether the transaction is posted or non-posted.

As usual, Intel will focus on the situation where the completion never returns, for the situation where the TOR timer is not fit for transactions roundtrip delay, Intel recommends increasing TOR timer or disable timer and reproduce issue again.

You may drop to this direction when:

1. Follow Single MCA bank Issue Debug to find out the first MCERR source in whole system is from a CHA MCA bank and its error code is 0xC.
2. Cannot figure out which MCA bank is first MCERR source in whole system and need analyze all valid MCA banks in whole system and CHA TOR Timeout is one of them.

NOTE

For the TOR Timeout issue along with bunch of correctable errors such as from Intel® UPI or memory, and so on. suggest fixing those correctable errors first.

TOR Timeout Symptom

1. Typically, TOR Timeout will accompany with 3Strike-Timeout which are reported by MLC MCA bank (for Granite Rapids) or ROB-Timeout which are reported by BUS MCA bank (for Intel® Xeon® 6900/6700-Series Processors with E-Cores). In this case, you will see IERR flag also.
2. In some cases, there is TOR Timeout but without 3Strike-Timeout.

For example:

If the transaction type is PCIe * Request (0x21E) which may be issued by external PCIe* device. Such as a DMA read request. In this case, IERR will not happen , and 3Strike-Timeout does not happen.

TOR Timeout Debug Concept

The key debug concept is to figure out the flow of the transaction who caused TOR Timeout on current specific configuration.

Then, check each point on the transaction path to figure out if the request is reached to the target component and if the completion is back to the CHA normally.

How to get the flow of the transaction who caused TOR Timeout?

First is **Transaction Type**, different transaction type will have different transaction flow. Second is **Target Component**. Even the transaction type are the same, different target component **may** cause different transaction flow.

Transaction type can be obtained from **origreq** field of CHA MCA bank **misc** register while the target component can be obtained from CHA MCA bank address registers physical address decoding. **TOR dump** and **Crash dump** will provide the same information when CHA MCA bank is not available to access.

Draw transaction flow base on **Transaction Type and Target Component**

See [How to Draw a Transaction Flow?](#)

TOR Timeout Possible Sources Example

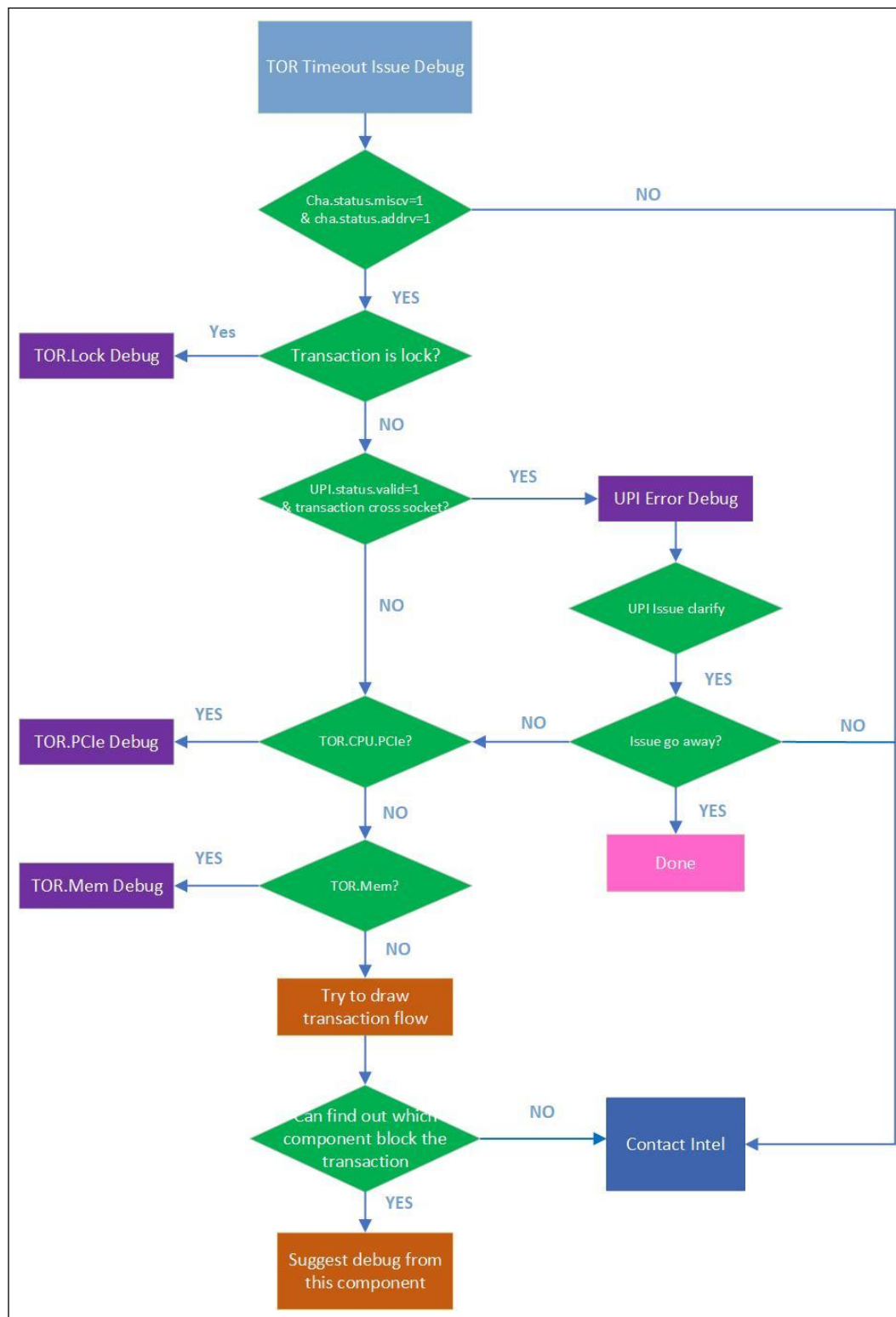
There are many sources that may cause a completion never to return. The most popular sources include but not limited:

- B2CMI.0x09 (Timeout error)
- Intel® UPI Interfaces errors, including correctable errors and Uncorrectable Errors (UCE)
 - For example: CE.UPI.0x20 (COR Phy Initialization Abort)
- External Devices problems, including PCIe*, BMC, SPI ROM, and so on.
- DDR Interfaces or DIMMs problems
 - For example: memory CE/UCE

TOR Timeout Debug Flow

See the next figure:

Figure 14. TOR Timeout Issue Debug Flow



1. To get TOR timeout transaction type and target component, CHA MCE bank status register miscv and addrv bits should be 1. If one of them is not "1", you can double check TOR dump to get the same information to TOR pipe. Contact Intel for further help if need.
2. Decode CHA MCA bank misc and addrv registers. If the transaction type is Lock/SplitLock/Unlock, see [Hang.TOR.LOCK](#). To get the transaction type and target component, see [How to Decode CHA Bank Address Register when TOR Timeout Happens?](#)
3. If Intel® UPI error occurred, and TOR timeout issue transaction is cross socket, suggest clarify the Intel® UPI error first. The key debug concept is clarify if this TOR timeout issue related to Intel® UPI. You can consider minimizing the system topology to 1S and reproduce the issue, this is a good way to eliminate the impact from Intel® UPI errors.

For Intel® UPI error debug, see the Intel® UPI debug handbook.

After debugging Intel® UPI issue, if root cause and workaround can be found, reproduce TOR timeout issue again after Intel® UPI errors fixed. If Intel® UPI issue is hard to root cause or unable to find a workable workaround, contact Intel for further help.

If TOR Timeout issue is gone when Intel® UPI errors are fixed, the TOR timeout issue may be secondary effect of Intel® UPI errors and we are done.

If TOR timeout error still happen, you must figure out the exact transaction flow and check every possible location may cause the transaction timeout. Suggest checking target component first. such as replace to another component if possible and try.

4. When target address points to an external PCIe* device on CPU side, see [Hang.TOR.PCIe](#).
5. When target address points to main memory, see [Hang.TOR.MEM](#).
6. Now you need to draw a transaction flow, even if it is hard to draw, but you have to try it. Contact Intel if you need further help. See [How to Draw a Transaction Flow?](#)
7. If you can find out which component whatever it is internal or external to block the transaction and causes TOR timeout, then you are done and congratulations!

2.15.1 Hang.TOR.LOCK

How to identify if it is lock type

When CHA MCA bank misc register origreq (bit[49:39]) field shows Lock (0x1df) / splitLock (0x1de) /Unlock (0x1d3): that means a lock transaction flow cannot finish in time for some reason.

The most probable cause is the external PCIe* device does not send one or more Ack and Nak to the root port. Those Transaction Layer Packet (TLPs) replay always replayed and the IIO cannot be quiet.

Lock Transaction TOR Timeout Possible Sources

1. TOR.Lock.IIO – IIO cannot be quiet (As usual, Ubox will report Lock master timeout first)
2. TOR.Lock.Core – Core cannot be quiet
3. TOR.Lock.UPI – Bad Intel® UPI Link may cause Lock transaction timeout

Debug Suggestion:

1. Try disable those root ports or remove all external PCIe* devices and reproduce the issue again.
2. Try to see if it can be reproduced with single socket.

Contact Intel if you need more help.

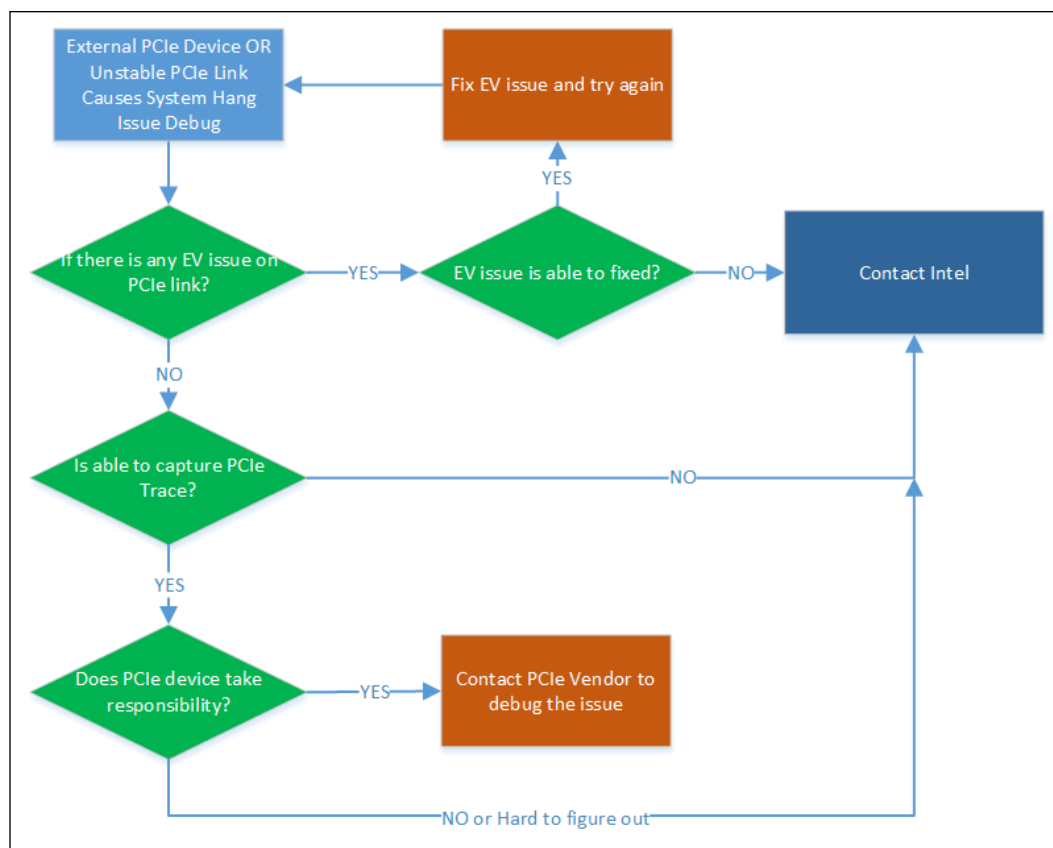
2.15.2 Hang.TOR.PCIe

How to identify it is HW.STUCK_TRANSACTION.TOR:TOR_Timeout.PCIe type?

An external PCIe* device on CPU side or its PCIe* link causes a TOR Timeout; the address of CHA MCA bank belongs to the PCIe* hierarchy under a root port.

Debug Flow:

Figure 15. External PCIe* Device or Unstable PCIe* Link Causes System Hang Issue Debug Flow Chart

**Debug Suggestion:**

1. Double-check it is the external PCIe* device causes the TOR Timeout error.
Try: removing this PCIe* device and reproduce the issue again.
2. Check PCIe* link quality

See the *Birch Stream PCI Express* (PCIe*) and IIO Issue Debug Handbook*, document number 776833.

Some tips:

Set link as generation 1 and reproduce again (may need to extend the reproduce time window longer, maybe three times than before)

Downgrade to 1 PCIe* lane. Like keep lane0 and reproduce again (tape lane1 – lane (max) signals)

Disable Active State Power Management (ASPM)

Checking if any power or power sequence issue on the PCIe* card

Checking if any clock issue (such as disable Scalable Communications Core (SSC), putting Clock buffer into Bypass mode if it is in Phase Lock Loop (PLL) mode)

3. Check with PCIe* device vendor for any known issue
4. Capture PCIe* trace to analyze the logic issue if it is not PCIe* link EV issue.
 - a. Check if any non-posted transaction cannot get completion back – Generally Completion Timeout error will be logged in register ERRUNCSTS in root port.
 - b. To know the first target address in CHA, you may disable Completion Timeout in this root port through the next setting: Completion Timeout Disable = 1 (bit 4 of register DEVCTL2)
 - c. Check if any abnormal Flow Control Information in DLLP Packets.
 - d. Check PCIe* trace to see if any PCIe* specification violation.

2.15.3 Hang.TOR.PCIe_Int

How to identify it is HW.STUCK_TRANSACTION.TOR:TOR_Timeout.PCIe_Int type

A CPU internal device or mesh causes a TOR timeout, the CHA bank address points to non-external devices.

Debug Suggestion:

1. Check the issues on the sighting report.
2. Contact Intel for the further debug help.

2.15.4 Hang.TOR.OOBMSM

How to identify if it is HW.STUCK_TRANSACTION.TOR:TOR_Timeout.OOBMSM type?

OOBMSM causes TOR timeout, the CHA bank address points to OO BMSM.

See [to Section 3.27 How to check TOR_Timeout.OOBMSM type](#)

Debug Suggestion:

1. Check OOBMSM debug handbook for the further debug.

See *Sapphire Rapids Server Processor Debug Handbook - OOBMSM Diagnostics*, document number 728327.

2.15.5 Hang.TOR.MEM

Key Concept:

Check if IMC bank has errors (Correctable Errors [CE] or Unexpected Completion [UC]), if yes, try to fix those errors.

Double check FW version especially unified patch and RC versions again.

Check DIMM date code.

Setting lower speed of DIMM and reproduce the issue again.

Change different DIMM population and reproduce the issue again.

Change different brand of DIMMs and reproduce the issue again.

See *Birch Stream Platform DDR5 and MRC Issue- Debug Handbook*, document number 776519.

NOTE

Not all Hang.TOR.MEM are point to memory issue. Contact Intel if you need additional support.

2.16 Single MCERR Issue Debug

As usual, there might be several MCA banks report MCERR when system hang happens. This section not only covers SINGLE MCA bank reporting MCERR in whole system, but also covers multiple MCA banks report MCERRs. The recommended way is to figure out which MCA bank is the **first MCERR source** in the whole system.

See [Section 3.14 How to Find Which Socket Report First MCERR?](#)

See *Granite Rapids Processor Register Specification*, document number 637776, the supported machine check bank number range from 0 around 24.

Bank 6: PCU MCE errors will be treated as IERR, so for PCU MCA bank errors, it will be covered in [Section 2.14 Hang.PCU](#).

Before you jump to specific bank to debug, you must know how to decode the MCA bank registers and get useful debug information to do next step.

See [Section 3.16 How to Decode MCA Bank?](#)

Different types of errors will be logged and signaled by one specific MCA bank.

Tips: You can check MCA bank control register to get error types.

NOTE

If your real issue Model Specified Code (MSCOD) codes are being covered in this book, contact Intel and feedback your crash dump log to us, Intel will consider adding them into this book's coverage.

NOTE

As Intel does not have other sections to cover correctable error type in each MCA bank, for correctable errors, Intel will introduce it in this section. Note that these correctable errors will not trigger MCERR handler. It does not belong to Hang.MCE. You can consider using [CE.MCA_Bank. Type] as issue tag. For example, [CE.IFU.0xN]

2.16.1 HW.MCE.IFU

See MSCOD and MCACOD to classify your Instruction Fetch Unit (IFU) errors, the next list are common codes and suggestions:

NOTE

For the IFU type error, always check the latest unified patch version first.

Tag	MSCOD [31:16]	MCACOD [10:0]	Debug Suggestion
HW.MCE.IFU.0xe.0x5	0xE	0x5	1. Try switching Intel® Advanced Vector Extensions 512 (Intel® AVX-512) feature 2. Put the CPU on Intel's reference board to see if can reproduce 3. Contact Intel
HW.MCE.IFU.0x7.0x5	0x7	0x5	
HW.MCE.IFU.0xc.0x150	0xC	0x150	Check MCCHAN, B2CMI and IIO MCA bank or IIO root ports Advanced Error Reporting (AER) uncerrsts.poisoned_tlp_status register

2.16.2 HW.MCE.DCU

See MSCOD and MCACOD to classify your Data Cache Unit (DCU) errors, the next list are common codes and suggestions:

Tag	MSCOD [37:16]	MCACOD [15:0]	Debug Suggestion
HW.MCE.DCU.0x0.0x124	0x0	0x124	1. Put the CPU on Intel's Reference Board to see if can reproduce 2. Contact Intel
HW.MCE.DCU.0x0.0x184	0x0	0x184	
HW.MCE.DCU.0x10.0x134	0x10	0x134	Check MCCHAN, B2CMI and IIO MCA bank or IIO root ports AER uncerrsts.poisoned_tlp_status register
HW.MCE.DCU.0x20.0x401	0x20	0x401	Check Advanced Programmable Interrupt Controller (APIC) memory type setting

2.16.3 HW.MCE.DTLB

No cases to show customer design or third-party issues will cause this tag so far.

If an MCE happens in this bank, contact Intel.

2.16.4 HW.MCE.MLC

See MSCOD and MCACOD to classify your MLC errors, the next list are common codes and suggestions: (3Strike-Timeout code is not listed in here).

Tag	MSCOD [31:16]	MCACOD [15:0]	Debug Suggestion
HW.MCE.MLC.0x30.0x135	0x30	0x135	Check MCCHAN, MB2CMI and IIO MCA bank or IIO root ports AER uncrrsts.poisoned_tlp_status register

NOTE

Your processor must be standard RAS SKU. For Standard RAS SKU, the poison data will not forward to Data Cache Unit (DCU) and it will be consumed in MLC, and no valid address will be provided.

NOTE

MCCHAN MCA Bank maybe set PCC = 1 if patrol scrub engine found this cacheline is uncorrectable and set poison flag with it.

2.16.5 HW.MCE.CHA

See MSCOD and MCACOD to classify your CHA errors, the next list are common codes we may hit in real issue debug:

Tag	MSCOD [31:16]	Debug Suggestion
HW.MCE.CHA.0x3	0x3	Happens when a CHA tries to writeback a cacheline out of Last-Level Cache (LLC) or MLC, however, this cacheline is not to a cacheable region. 1. If the CHA MCA bank has valid address, check if this address makes sense and find out which application will use this address in system. Then eliminate any application, driver that may use this address.
HW.MCE.CHA.0x5	0x5	1. Check valid CHA MCA bank address if belongs to MMIO region or VGA region. 2. Check MTRR setting to check if the address region was set as WB type
HW.MCE.CHA.0x6	0x6	Try to set PAM region to UC type and try again.

2.16.6 HW.MCE.Ubox

UBOX bank is used to log UBOX internal errors, Shutdown Suppression errors, and SCF Bridges errors: B2HOT, B2CXL, B2UPI.

MCACOD	Description
0x407	Ubox HW Errors
0x40C	Shutdown Suppression errors
0x412	B2HOT Errors
0x412	B2CXL Errors
0x412	B2UPI Errors

NOTE

Bridge IPs share the same MCACOD 0x412 when reporting machine check errors. IA32_MC4_MISC corresponding bits indicate senderID of the IP that generated the MCE.

The supported Ubox misaligned error types and descriptions are shown in the next table, MSCOD for Ubox HW Errors (MCACOD 0x407).

The processor has an MMIO Ubox decoder since the Ubox is a target for MMIO. The MMIO Ubox decoder will always target the Ubox and its limit addresses are 4M granular but can access more than 4G and hence its address bits have to cover bits [51:22]. The MMIO Ubox decoder has precedence over MMIO-H and MMIO-L.

If Ubox receives a 8B misaligned MMIO Read/Write and the Security Attributes of the Initiator (SAI) on the access does not indicate the access was from SMM BIOS, then Ubox must signal an error with the next characteristics:

Error Type	MSCOD[31:16]	MCACOD[15:0]
Misaligned, CFG Rd, SMM	0x8002	0x407 (Ubox HW Errors)
Misaligned, CFG Wr, SMM	0x8003	0x407 (Ubox HW Errors)
Misaligned, CFG Rd, Non-SMM	0x8004	0x407 (Ubox HW Errors)
Misaligned, CFG Wr, Non-SMM	0x8005	0x407 (Ubox HW Errors)
Misaligned, MMIO Rd, SMM	0x8006	0x407 (Ubox HW Errors)
Misaligned, MMIO Wr, SMM	0x8007	0x407 (Ubox HW Errors)
Misaligned, MMIO Rd, Non-SMM	0x8008	0x407 (Ubox HW Errors)
Misaligned, MMIO Wr, Non-SMM	0x8009	0x407 (Ubox HW Errors)

2.16.7 HW.MCE.MSE

Memory Security Engine (MSE) is the memory subsystem IP which used to support Intel TME, Intel MKTME, Intel SGX and Intel TDX for system security enhancements.

MSE supports a standard MCA bank. On Birch Stream, bank 11 is used to log MSE errors.

MCAC OD	MSCOD	Decode	Error Details
0x444	0x5	Security Error - sgx ns	MSE check the NS bit stored in memory with the incoming transaction's secure bit. A mismatch is flagged as security error.
0x444	0x6	Security Error - SGX ns 2lm	MSE check the NS bit stored in memory with the incoming transaction's secure bit. A mismatch is flagged as security error.
0x444	0x7	Security Error - Integrity	Memory integrity uses metadata bits in DRAM with each cache line to store the MAC bits and notifies trusted software of an integrity mismatch.
0x444	0x8	Security Error - td mismatch	MSE check if read request is come from TD and compare with the TD bit read from memory. MSE will signal TD mismatch if mismatch detected.

2.17 Other IERR Issue Debug

Contact Intel if you have system hang issues drop in this area. It may include:

- System Management Interrupt (SMI) and MCERR racing (happens simultaneously) causes IERR. This scenario will be seen only when you do error injection with eMCA enabled. (Disable eMCA).
- MCERR happens while viral mode is enabled. You can try to disable viral mode and reproduce again. (Disable viral mode).
- Intel internal fuse error. (Contact Intel)
- Others and contact Intel.

2.18 UCR Issue Debug

Software layer assisted recovery from uncorrectable data errors. Enables software layers (OS, Virtual Machine Manager [VMM], Database Management Systems [DBMS], Application) to assist in system recovery from errors that are not correctable at the hardware level.

Linux* message/dmesg will log uncorrected [Hardware Error], but do not trigger hang or reboot/shutdown event.

RMCA_N pin is designed to signaling Recoverable MCA event. RMCA_N :16 Bclks pulse signaling an UCR Error.

NOTE

Due to CORE MCA difference, data load and instruction fetch poison error will be logged to MEC (bank3) and FEC (bank2) bank respectively for Intel® Xeon® 6900/6700-Series Processors with E-Cores while will be logged into DCU and IFU for Granite Rapids.

2.19 S3M Issue Debug

S3M is a CPU IP subsystem that provides hardware and firmware infrastructure to deliver a diverse set of features.

S3M is divided into two domains: Non-self-boot domain and self-boot domain. Non-self-boot domain is the S3M micro controller hardware and firmware. Self-boot domain is the subsystem of S3M including eSPI, SPI, SMBus, UART, ACPI PM and P2SB.

Non-self-boot domain errors will be logged into S3M error reporting logic registers list:

- S3M_ERR_STS
- S3M_HW_ERR_FATAL_STATUS
- S3M_FW_ERR_FATAL_STATUS
- S3M_NVRAM_CONTENT_AUTH_ERR_STS

Before system boot phase 4, S3M will assert GLOBAL_RST_WARN# to signal S3M errors. After boot phase4, Fatal HW and FW errors will be signaled with CATERR# and be logged into Punit machine check bank (bank6) with MCACOD 0x402 and MSCOD0x80. See [Section 2.14.2 HW.MCE.PCU.S3M](#), [Section 3.22 How to verify S3M Error Happens](#) and [Section 3.23 Howto Check S3M Errors](#).

Self-boot domain is the new S3M functionality on Birch Stream platform that moved from PCH. The subsystem IPs will send uncorrectable AER message to sIEH. And UBOX machine check bank (bank4) will log IIO error with MCACOD 0xe0b finally.

3.0 Useful Recipes for Triaging System Hang and MCE Issues

In this chapter, a few useful recipes which can be used in real issue debugging are provided, these recipes will use one or more tools like CScripts through Intel® In-Target Probe (Intel® ITP), Intel's Compact Debug Port, Intel® At-Scale Debug (Intel® ASD), BMC through PECI, Autonomous Crash Dump (ACD) or Intel® Crash Log Technology. You could select specific tools based on your need, however, you shall understand the application scope of each tool.

3.1 Recipes on Prewrite

3.1.1 How to Verify the System Hang Issue is Not a Single System Issue

Step 1: Test multiple systems

Follow the same recipe to reproduce the issue on multiple systems:

1. If the issue happens on more than one system, then we consider that it is not a single system/configuration issue (End)
2. If the issue happens on only one system, go to step 2 to address which component causes the issue.

NOTE

These multiple systems must have the same HW and SW configurations including Printed Circuit Board Assembly (PCBA) (PCB Assembly) version, board configuration (DIMM, PCIe* add in cards and other peripheral devices), silicon stepping, Unified patch, RC version, BMC version, BIOS knob setting, CPLD versions, and so on.

Step 2: Swap parts to do cross check

Pick up two or more "good" systems and "bad" system, swap the suspected parts between 'good' systems and 'bad' system to see if the issue still happens.

1. If the issue could be reproduced on 'good' systems with the suspected part in all tests and can also be reproduced with the same failure signature on Intel's reference board, probably this issue is caused by this specific part.
2. If putting the suspected part onto a known-good system, there is still have issue but not the exact same failure symptom. The issue may be caused by the bad board and suspected part. So it is not a single device issue probably.

NOTE

Be careful to address it as a single part issue when you have very limited samples to do the cross check.

3.1.2 How to Speed Up Debugging Your System Hang Issue

Not all system hang issues are 100% reproducible. Most of cases are hard to reproduce due to low failure rate. You may spend over one week or one month to reproduce again. This recipe will show you how to accelerate the replication work of your system hang issues, noted that this concept is fit for all kinds of issues.

Whether the failure signature changed or not during replication is a key indicator to verify the success of your failure acceleration. Therefore, writing a high-quality failure signature is very important. After you dump related registers when issue occurs, you must make sure the key error register(s) match the MCA banks' error code if failure has MCA banks valid.

Here are some suggestions when you want to accelerate a failure replication:

1. Arrange More Systems to Reproduce the Issue

2. Adjusting Frequency, Voltage or Temperature

Adjusting Frequency - You can try to adjust core frequency or uncore frequency or bus speed, such as Intel® UPI, PCIe*, DDR5 interfaces.

Adjusting Voltage - You can try to adjust voltage of different power rails. For example, VNN, VCCFA, VCCIN of CPU's

Adjusting Temperature - You can try to adjust CPU and/or DIMM Temperature through Fan speed control, or other suspicious cooling condition of devices according to the failure signature

3. Change Different Power Management Settings

You can consider Packet C6, P-State, C-State, ASPM, Intel® Ultra Path Interconnect (Intel® UPI) LOP to be disabled or enabled. Note that the issue may be gone if you do some changes.

4. Try Other Setting

You can consider switching these features: Intel® Virtualization Technology (Intel® VT) for Directed I/O (Intel® VT-d), Intel® Advanced Vector Extensions (Intel® AVX). Note that the issue may be gone if you do some changes.

5. Run Stress Test

If you are not familiar with the issue you debug, you can consider increasing overall stress. For example, installing VMWare* and enable more guest OS and running third party benchmark tools. Full population configuration with maximum memory, DDR speed, processors with highest frequency, and hard disks with RAID.

If you already knew which part, which interface or which types of transactions may be related to failure, you can write specific stress application to accelerate the failure.

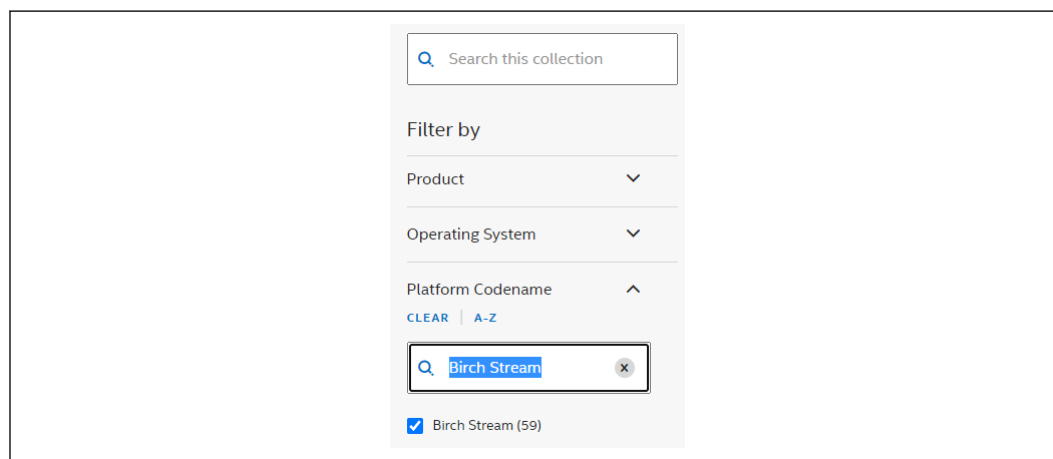
3.1.3 How to Get Reference BIOS and Drivers Files

BKC and IPU are a FW and drivers' package that has been validated by Intel validation team, when you meet issues, firstly you must use these images to double check or compare on reference board.

You can find it on RDC website: <https://www.intel.com/content/www/us/en/resources-documentation/developer.html>

You can check "Birch Stream" in "Platform Codename" category as shown:

Figure 16. Platform Codename Categorization



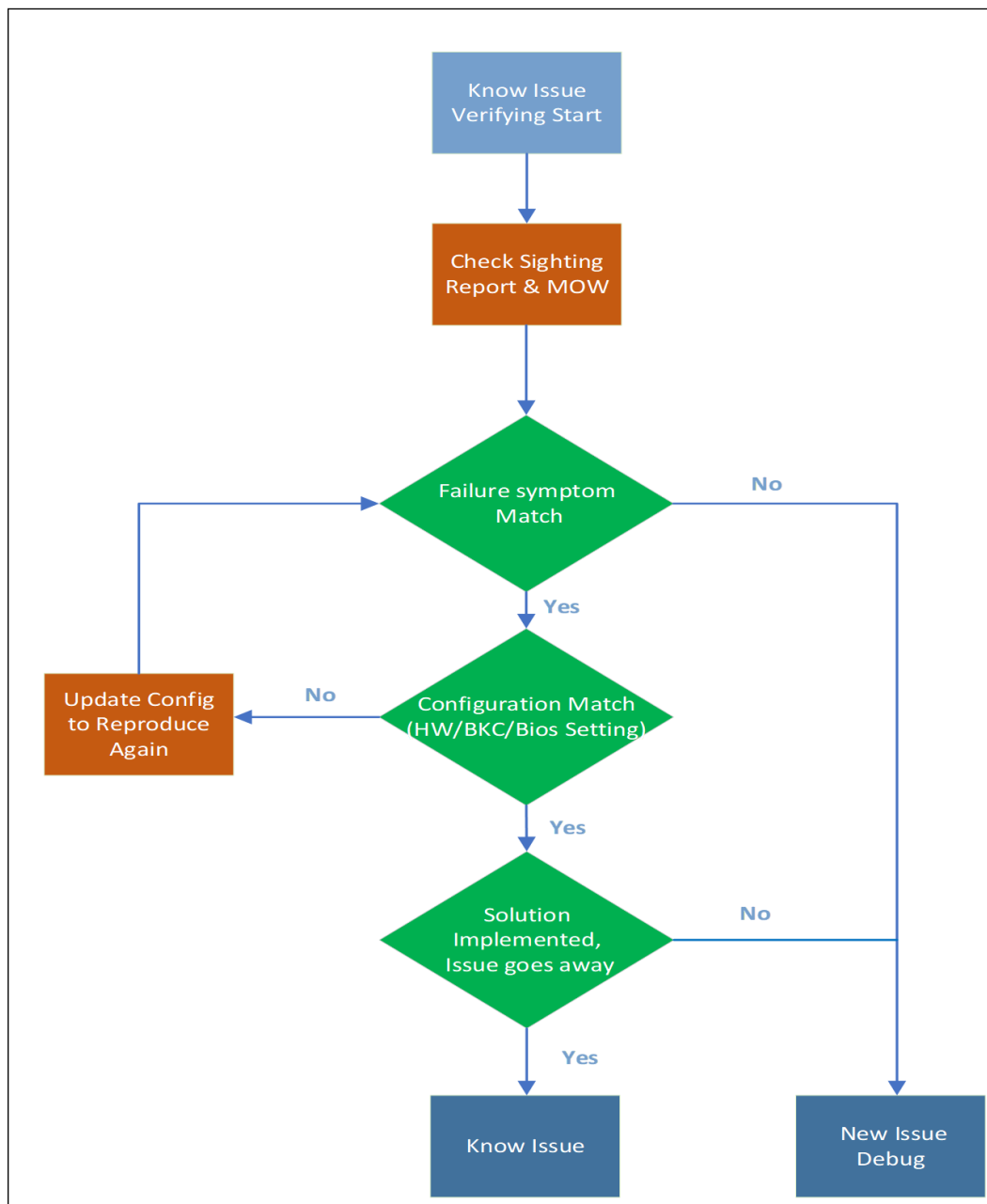
You will see all Birch Stream platform firmware image download link at right side.

The firmware image includes the IFWI, Unified Patches, BMC, CPLD, Authenticated Code Module (ACM), Linux* and Windows* drivers, and so on.

3.1.4 How to Verify if It is Not a Known Issue

Always check latest *Granite Rapids Server Processor Sightings Report*, document number 748867; *Birch Stream Platform Message of the Week [MoW]*, document number 645322 or contact Intel to check if your issue drops to a known issue list.

Figure 17. Know Issue Verifying



Step 1: Use key words, like 3Strike-Timeout or TOR Timeout, to search in the sighting report or MoW, and see if there is a similar case.

Step 2: Compare the failure symptom if it looks as similar as you got.

Step 3: Compare the HW configuration/RC version/Unified patches version/BMC version/BIOS knob setting, and so on.

If they are not the same configuration/version, update them, then reproduce the issue again, return to step 2.

If they are the same version, go to step 4.

Step 4: Implement the workaround or solution, check if the issue is resolved or not.

Step 5: If you can fix your issue following previous steps, it must be a known issue.

Step 6: If you cannot find a similar case by searching, or after updating the configuration/version, the failure symptom is changed and the new failure symptom still cannot match any case in the sighting report, it may not be a known issue.

When you debug an unknown issue, if you fix part of the issue and the issue symptom is changed, you can restart from step 1 to check if it is a known issue and find a solution quickly.

3.1.5 How to Write a System Hang Issue Title

A high-quality issue title and failure signature will accelerate your issue debugging progress, and be easier for other support teams quick involving and better for known issue correlating.

As usual, we must have several key information inside title, here are our suggestion format and key ingredients:

[Issue Triage tag] when and how the system hang happens with which key component on what specific configuration.

For example:

[Hang.TOR.CPU_PCIe] Intel 82599 PCIe* LAN Card Causes System 3Strike-Timeout when ASPM is Enabled, and OS is in Idle State.

[HW.MCE.CHA.0x5] System logs CHA MCERR in OS when doing cold boot stress test.

3.1.6 How to Write a Failure Signature

A Failure Signature is a description of the system at the time of failure.

The Failure Signature must be **Unique**, **Detailed**, and **Specific**.

Unique means the Failure Signature you write for this issue must be easily distinguished away from other similar issues unless they share the same root cause.

Detailed and **Specific** mean the failure signature you write for this issue must list all related pieces of information you observed and must be accurate and detail enough. For example, if an issue can be only reproduced on Windows Server* 2022, mention this finding in your failure signature because it is key information, and it will be better to write down the OS build version you were using too.

It is reasonable that you may update failure signature when you need correct some information or update more key information during your debug process.

An example of failure signature:

Granite Rapids with vendor B PCIe* device will be system hang during the installation of Windows Server* 2022. 100% failure rate and issue will be gone if you remove vendor B PCIe* device.

When you have other observations, but you are not sure if they are key information you can list in finding. Update them to failure signature when you find some pieces of information is key information.

When you find more information, you may update failure signature:

Granite Rapids with vendor B PCIe* device will be system hang during the installation of Windows Server* 2022. 100% failure rate and issue will be gone if you remove vendor B PCIe* device. CATERR# pin keeps low when system hang happens.

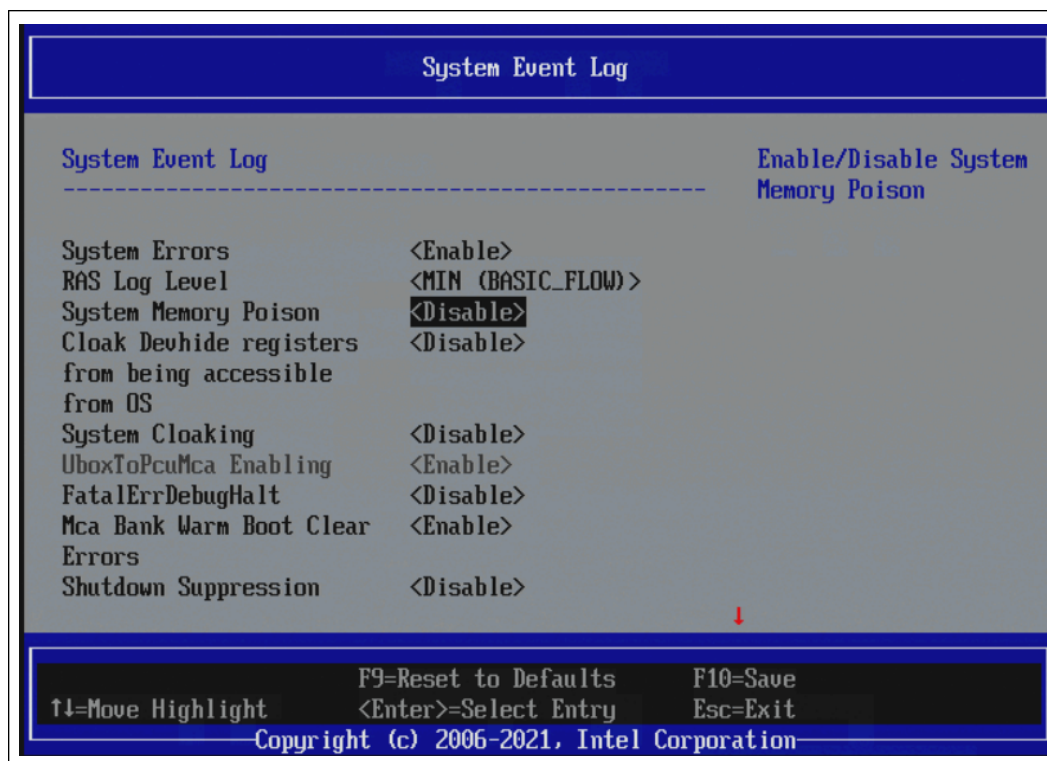
3.2 General Recipes

3.2.1 How to Enable/Disable Poison

Method 1: Enable/Disable it through BIOS setup menu

EDKII Menu → Platform Configuration → System Event Log → System Memory Poison

Example: Disable System Memory Poison in BIOS setup menu



Method 2: Enable/Disable it through MSR

Example: Enable Poison

```
>>> itp.msr(0x178,1)
```

NOTE

Be careful to use `msr(0x178,1)`, this command modifies socket 0 core 0's threads only. If you need to modify all cores, uses command `itp.msr(0x178,1)`.

Verification: Check MSR 0x178h MCG_CONTAIN bit 0 value

0: legacy behavior

1: poison enable

Example:

```
>>>itp.threads[0].msr(0x178)
[64b]0x0000000000000001
...
```

3.2.2 How to Set System Event Breakpoint with Intel® In-Target Probe (Intel® ITP)

The Intel® ITP provides a set of system event breakpoints which could be used in debugging. You can type "**itp.cv.**" + **Tab** in CScripts to see the full list of the breakpoints.

- - initbreak
- - machinecheckbreak
- - resetbreak
- - shutdownbreak
- - smmentrybreak
- - smmexitbreak
- - smmintobreak
- - vmentrybreak
- - vmexitbreak

Example: Set machinecheckbreak

```
>>> halt
>>> itp.cv.machinecheckbreak=1
>>> go
```

Setting other breaks share the same method.

Intel® ITP uses "brnew" to create breakpoints with a memory or I/O address.

Example: use a memory address to set breakpoint

```
>>> itp.threads[0].brnew("0x20000000", "acc global")
Breakpoint Name: #0001 - Id: 1
Address: 0x0000000020000000L - Data Size: 1 - DBReg: 0 - Enabled: True
>>> itp.brnew('0x80p', 'io')
```

You can check the value of `itp.cv.xxxbreak` to see if the break is setting well.

Example: check if machinecheckbreak has been setting well.

```
>>> itp.cv.machinecheckbreak
NodeGroup([
('GNR0_C0_T0.breaks.machinecheck', 1),
('GNR0_C0_T1.breaks.machinecheck', 1),
('GNR0_C1_T0.breaks.machinecheck', 1),
('GNR0_C1_T1.breaks.machinecheck', 1),
...
('GNR1_C95_T1.breaks.machinecheck', 1)])
```

3.2.3 How to Check the Break is Setting Well

The checkpoint code which indicates the BIOS POST stage could be used to set BIOS debug breakpoint by filling the major code and minor code of checkpoint in `biosscratchpad6_cfg` register. The `biosscratchpad6_cfg` details:

`biosscratchpad6_cfg[31:24]`: major code

`biosscratchpad6_cfg[23:16]`: minor code

The checkpoint code details in specific platform could be collected and found in BIOS serial log when set the message level to normal:

EDKII Menu → Platform Configuration → Miscellaneous Configuration → Serial Debug Message Level to Normal

An example to checkpoint code in serial log as follows:

Checkpoint Code: Socket 0, 0xB7, 0x12, 0x0000

An example to set `biosscratchpad6_cfg` after getting target checkpoint :

```
>>> sv.socket0.io0.uncore.ubox.ncdecs.biosscratchpad6_cfg=0xb7120000
```

When message level is set to Normal and breakpoint triggered, the serial log will show below message:

Breakpoint match found. S0 waiting...

NOTE

Use biosnonstickyscratchpad7_cfg to check current checkpoint. Set biosscratchpad6_cfg to 0x0 for clearing target breakpoint to continue POST.

```
>>> sv.socket0.io0.uncore.ubox.ncdecs.biosnonstickyscratchpad7_cfg
0xb7120000
>>> sv.socket0.io0.uncore.ubox.ncdecs.biosscratchpad6_cfg=0x0
```

3.2.4 How to Get Uncore Dump

We need to dump Uncore register to check for some cases. It may take some time to complete.

Here is an example to get Socket0 IO 0/ IO 1 /Compute DieUncore dump

```
>>> sv.socket0.io0.uncore.logregisters(r's0_io0_uncore_dump.txt')
>>> sv.socket0.io1.uncore.logregisters(r's0_io1_uncore_dump.txt')
>>> sv.socket0.compute.uncore.logregisters(r's0_compute_uncore_dump.txt')
```

3.2.5 How to Get Crashdump

Crashdump is a very useful tool for dumping critical CPU built-in registers when system is in a crash state.

Next is an example of running cscript built-in C rashdump command. In this example, we save the output to "c:\work\log\ C rashdump_2023_5.json". And it will show the time it takes, in this example, total time taken was about 490 seconds.

```
>>> error.crashdump(filename='crashdump_2023_5.json',outdir=r'c:\work\log',
slow=True)
Please wait...
Collecting socket0 io0 Die
Collecting socket0 io1 Die registers
Collecting socket0 compute0 Diesters
Collecting socket1 io0 Die registers
Collecting socket1 io1 Die registers
Collecting socket1 compute0 Diesters
----- Elapsed Time:490.28314447402954seconds -----
crashdump() log file created at: c:\work\log\crashdump_2023_5.json

>>> error.crashdump()
Please
wait...
Collecting socket0 io0 Die
...
Collecting socket1 io1 Die registers
Collecting socket1 compute0 Diesters
-----Elapsed Time:334.34407782554626 seconds-----
crashdump() log file created at: C:\Users\Administrator\PythonSv
\crashdump_20230507_153332.json
```

If no parameters are provided as error.crashdump(), the default "crashdump.json" will automatically generated in the default folder as "\\Documents\pythonsv".

Then we can use `crashdump_summary` to decode this C rashdump file. Following is an example.

```
>>> error.crashdump_summary(r"c:\work\log\crashdump_2023_5.json")
Output file location and Operating on Directory tree here: c:\work\log
* Found GNR project file crashdump_2023_5.json
+ Processing crashdump file: c:\work\log\crashdump_2023_5.json
=> Output summary file: summary_crashdump_2023_5.txt
>>>
```

Finally, open `summary_crashdump_2023_5.txt` to check error information.

NOTE

Crashdump could also be decoded with Intel® Baseboard Management Controller Autonomous Crashdump (Intel® BMC Autonomous Crashdump) online parsing tool: BMC Assisted FRU isolation (BAFI). See Appendix - System Crashdump and Decode for more details.

3.2.6 How to Get CPU Crash Log

There are two ways to trigger CPU crash logs:

- Manual Trigger over JTAG/Redfish* PECI
- IERR or MCERR

How to extract the CPU crash log:

- Manual extract over JTAG/Redfish* PECI
- Post -Reset Discovery and Extraction Using AWR
- Pre-Reset: CATERR trigger ACD then AGR

NOTE

What's meaning of AWR and AGR that you can reference the chapter of "A.9 What Is AWR and AGR?".

For more details, see *Granite Rapids CPU Crash Log Quick Start Guide*, document number 783720, and *Sierra Forest CPU Crash Log Quick Start Guide*, document number 788337

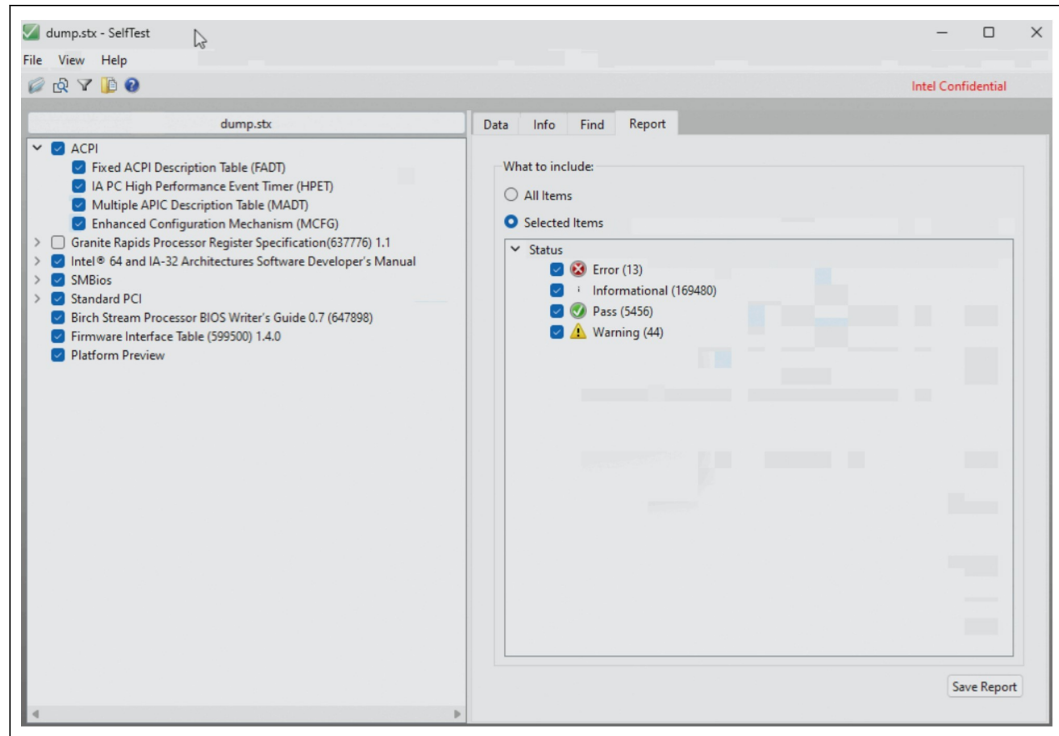
3.2.7 How to Get SelfTest Result

SelfTest performs a snapshot of BIOS configuration registers and compares the actual values to suggested values listed in the EDS, BWG, BIOS Specification, and related collateral.

1. Firstly, see *SelfTest 7 - Windows* x64*, document number 524152 to get the latest SelfTest tool and install `SelfTest7.msi`.
2. See chapter 3 data collector of *SelfTest 7 Help.pdf* under installation folder: `C:\Program Files\Intel\SelfTest 7\doc`, and follow the detailed steps to collect SelfTest dump.stx

- See chapter 4 *Analyzer of SelfTest 7 Help.pdf* under installation folder: C:\Program Files\Intel\SelfTest 7\doc, Launch the SelfTest 7 UI to click analyzer.exe for analyzing dump.stx files.

Figure 18. SelfTest Result



3.3 How to Verify if System Hang Issue is Hard Hang or Soft Hang

Hang is the situation in which the system or the test is stuck at some point. How to verify is it a hard hang or a soft hang? There are some indicators to help you to identify it.

Method 1: Check CATERR# LED or CATERR# signal status

Soft hang: CATERR# LED must not keep lighting.

Hard hang with IERR: You will see CATERR# LED keep lighting.

Hard hang without IERR: CATERR# LED must not keep lighting. Check if the CPU enter Shutdown State.

Method 2: Use Intel® ITP to see if all cores can be halted.

Soft hang:

Target machine is still able to be halted by Intel® ITP.

Example:

```
>>> halt
>>>
[GPC core group] Halt on 48 devices -- 22:06:42.417835 2023-03-14
>>> go
>>>
[GPC core group] Resuming on 48 devices -- 22:06:46.339661 2023-03-14
>>>
```

Hard Hang.IERR:

Target machine is not able to be halted by Intel® ITP.

NOTE

If you see only two threads are not able to be halted but others are ok to be halted, this is still considered to be "not able to be halted".

If you set shutdown break, you may still be able to halt.

Hard Hang.non-IERR:

Target machine is still able to be halted by Intel® ITP. Check if the CPU enter Shutdown State.

Method 3: Press Num Lock/Caps Lock/Scroll lock key to see if keyboard LED is toggles.

Soft hang: you may see LED toggling.

Hard hang with IERR: LED will not be toggling.

3.4 How to Verify if Soft Hang Issue with MCE Happens or Not

Method 1 : Checking punit.ptpcfsms.ptpcfsms.mca_err_src_log register

```
>>>sv.sockets.ios.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log
```

Example 1:

```
>>> sv.socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21);...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
```

```
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMT_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMT#. This is 0...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMT_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMT_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMT
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMT
0x00000000 : first_core_ierr (04:00) -- Reserved
```

Check `msmi_internal`, `msmi_mcerr_internal`, `ierr_internal` and `msmi_ierr_internal` bits to determine if MCE happened.

3.5 How to Verify if IERR Happens

Check Items:

1. CATERR# LED: keep lighting
2. CATERR# pin status: keep low
3. Register: `punit.ptpcfsms.ptpcfsms.mca_err_src`: see next example 1.
4. Register `punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_hi/lo_cfg` value is not '0': see next example 2.
5. Register: `ubox.ncevents.ierrloggingreg`: see next example 3

Example 1: Check register `mca_err_src_log.msmi_ierr_internal` and `mca_err_src_log.ierr_internal`.

```
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000001 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000001 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMT# (for any
reason). It is or(bit 22, bit21);...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMT_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMT_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMT#. This is 0...
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMT_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMT_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMT
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMT
0x00000000 : first_core_ierr (04:00) -- Reserved
```

```
0x00000001 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit...
0x00000001 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21);...
0x00000001 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is 0...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved
```

Example 2: check punit.pcu_first_ierr_tsc_lo/hi_cfg value is not '0'.

```
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg - 0x2b3b
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg - 0x2b3b
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0x77535938
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0x7753593a
```

Example 3 : check ubox.ncevents.ierrloggingreg is not all '0'.

```
>>> sv.sockets.io0.uncore.ubox.ncevents.ierrloggingreg.show()
0x00000000 : secondierrsrcfromcore (49:49) -- Set to '1' if the SecondIerrSrcID is
from a C2U
0x00000001 : secondierrsrcvalid (48:48) -- set to 1 if the SecondIerrSrcID is
valid
0x000006f1 : secondierrsrcid (47:32) -- Physical Port ID of the end point with
the second IERR
0x00000001 : firstierrsrcfromcore (17:17) -- Set to '1' if the FirstIerrSrcID is
from a C2U
0x00000001 : firstierrsrcvalid (16:16) -- Set to '1' if the FirstIerrSrcID is valid
0x00004800 : firstierrsrcid (15:00) -- Physical Port ID of the end point with the
first IERR

0x00000001 : secondierrsrcfromcore (49:49) -- Set to '1' if the SecondIerrSrcID is
from a C2U
0x00000001 : secondierrsrcvalid (48:48) -- set to 1 if the SecondIerrSrcID is
valid
0x00004811 : secondierrsrcid (47:32) -- Physical Port ID of the end point with
the second IERR
0x00000001 : firstierrsrcfromcore (17:17) -- Set to '1' if the FirstIerrSrcID is
from a C2U
```

```
0x00000001 : firstierrsrcvalid (16:16) -- Set to '1' if the FirstIerrSrcID is valid
0x0000480a : firstierrsrcid (15:00) -- Physical Port ID of the end point with the
first IERR
```

NOTE

It is not necessary to go through each item (1) around (5) to identify IERR happens. But if you only capture item (1) and (2), but no records of (3), (4), (5) and no MCA errors, then you may check your CATERR#/RMCA_N hardware design to make sure it is not a fake IERR alarm.

3.6 How to Verify if MCERR Happens

Check Items:

1. Register: 'sv.socket0.ios.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()': see next example 1
2. Register: 'sv.socket0.io0.uncore.ubox.ncevents.mcerrloggingreg.show()': see next example 2
3. Register: 'sv.socket0.ios.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo/hi_cfg': see next example 3

Example 1: Check msmi_mcerr_internal bit [18] and mcerr_internal bit [26].

```
>>> sv.socket0.ios.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions as a Valid bit for the other two
pack...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); functions as a Valid bit for the other two
package ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (bit 19,18).
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions as a Valid bit for the other
two pack...
```

```
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMT# (for any
reason). It is or(bit 22, bit21); functions as a Valid bit for the other two
package ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMT_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMT_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMT#. This is OR (bit 19,18).
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMT_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMT_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMT
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMT
0x00000000 : first_core_ierr (04:00) -- Reserved
```

Example 2: 'sv .socket0.io0.uncore.ubox.ncevents.mcerrloggingreg .show()' is not all '0'.

```
>>> sv.socket0.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1' if the SecondMCerrSrcID
is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCERR
0x00000001 : firstmcerrsrcfromcore (17:17) -- Set to '1' if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00004800 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR
```

Example 3: 'sv.socket 0.ios.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo/hi_cfg' value is not all '0'.

```
>>> sv.socket0.ios.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_hi_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_hi_cfg - 0x00000009
socket0.io1.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_hi_cfg - 0x00000000
>>> sv.socket0.ios.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo_cfg - 0x68b299dc
socket0.io1.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo_cfg - 0x00000000
```

3.7 How to Verify Whether IERR is Caused by Real PCU Errors

Check items:

1. Punit MCA bank 6 status register valid bit [63]: set as '1'.
2. Punit MCA bank 6 status register MCACOD [15:0]: is 0x402 .

Example:

```
##### Decode each Valid Machine Check #####
*****
*****
=====
=====
|skt|die|cor|thr|bank|Status      |Address      |Misc      |
Ctl      |
-----
|1  |io0|0  |0  |6  |0xfa0000000200402|0x0000000000000000|0x0000022800340400|
0x000000000000000f|
=====
=====

Decoding Status Register:

Range Field
Description
Description
Value Value Decode
-----
63:63 VALID
register is valid 0x1 Indicates that the information within this
Valid
62:62 OVERFLOW
occurred while 0x1 Overflow
Occurred
the results of a previous error were still in the
error-
reporting register bank (the VALID bit was
already
set)
61:61 UC
condition 0x1 UC A '1' indicates that an uncorrectable the error
Detected was
detected.
60:60 EN
error 0x1 Error reporting enabled. A '1' indicates that
Enabled reporting is enabled by the associated flag
bit(s) in
the
IA32_MCi_CTL
register.
59:59 MISCV
content is 0x1 IA32_MCi_MISC If this flag is '1', the IA32_MCi_MISC register
Valid valid and can be
read.
58:58 ADDR
content is 0x0 IA32_MCi_ADDR Not
Valid valid and can be
read.
57:57 PCC
reliable 0x1 Processor Context Processor Context
Corrupted restarting of the processor may not be possible
and a
reset
```

```

recommended.
56:56 S error. 0x0 Signaling an uncorrectable recoverable
---
55:55 AR error. 0x0 Recovery action required for UCR
---
54:53 THRS_ERR_ST No hardware status tracking is provided...
of corrected 0x0 errors has exceeded the safe threshold to the
point where an uncorrected error has become more likely to
happen.

52:38 CORRECTED_ERROR_COUNT Count of filtered correctable MCA errors
including sticky 0x0
---
52) overflow indicator (bit

37:37 FW_UPD Indicates that the BIOS has updated the contents
of the MC 0x0
---

Bank

36:32 OTHER_INFO Available when Enhanced MCA is in
use 0x0 ---
31:16 MSCOD Machine Specific Architecture Error
Code 0x20 Power Management Agent Signaled
Error

15:0 MCACOD Machine Check Architecture Error
Code 0x402 PUnit errors

```

3.8 How to Verify if 3Strike-Timeout/ROB-Timeout Only is True

For Granite Rapids: 3Strike-Timeout Flag in MLC MCA bank status register value is ended with MCACOD 0x400.

Valid	Over	uc	en	miscv	addrv	pcc	mcacod
1	X	1	1	1	1	1	0x400

For Intel® Xeon® 6900/6700-Series Processors with E-Cores: 3Strike-Timeout is reported as ROB timeout in BUS bank status register value is ended with MCACOD 0x0400.

Valid	Over	uc	en	miscv	addrv	pcc	mcacod
1	X	1	1	0	0	1	0x400

Example:

1. Checking MLC (for Granite Rapids)/BUS (for Intel® Xeon® 6900/6700-Series Processors with E-Cores) banks status

```
>>> error.check_machine_check_errors()
```

2. Checking TOR Timeout status

Register:

'sv.sockets.computes.uncore.cha.chas.sad.qpi_timeout_ctrl_cfg.enabletortimeout '

```
>>> sv.sockets.computes.uncore.cha.chas.sad.qpi_timeout_ctrl_cfg.enabletortimeout
socket0.compute0.uncore.cha.cha0.sad.qpi_timeout_ctrl_cfg.enabletortimeout -
0x00000001
socket0.compute0.uncore.cha.cha2.sad.qpi_timeout_ctrl_cfg.enabletortimeout -
0x00000001
...
```

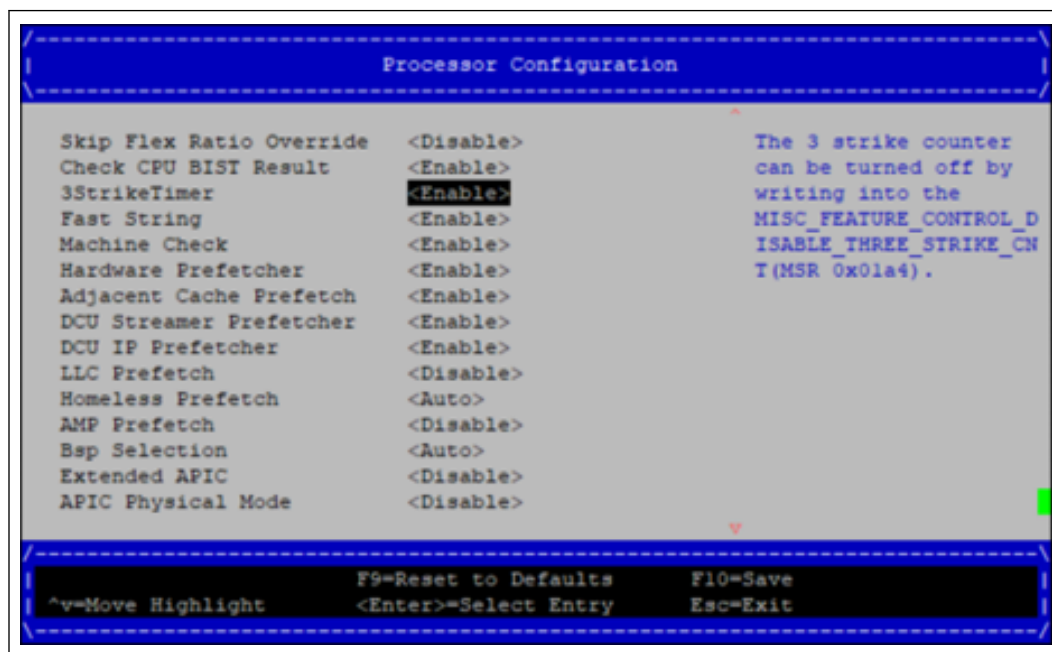
Double check TOR entry to make sure there is no valid stuck entry.

3.9 How to Enabled/Disable 3StrikeTimer

BIOS knob "3StrikeTimer" is used to enable/disable 3Strike-Timeout function.

EDKII Menu → Socket Configuration → Processor Configuration → 3StrikeTimer

Figure 19. Processor Configuration



3.10 How to Verify if TOR Timeout is Enabled or Disabled

There are two ways to verify that a TOR Timeout is enabled.

Method 1 : Using pci.config() to read the status of QPI_Timeout_Ctrl bit0

Socket 0 CHA:

```
>>> halt
>>>
[GPC core group] Halt on 48 devices -- 20:15:26.112195 2023-03-15
>>> pci.scan
Scan Segment# 0 ...
Seg 0, Bus 0xEA, Dev 0x1f, Fnc 0
PCI scan complete: 1 segments, 255 Buses, 104 Devices, 270 Functions
# Check CHA BDF.
>>> pci.devices()
...
|00|7e|0d|0|344f|8086|00|80|0000|8086|...|...|CHA - ALLO
...
# Check register offset 0x2d0. Bit 0 is the TOR Timeout enable bit.
>>> pci.config(0x7e,0xd,0,0x2d0)
0x16b5ad6d
```

Socket 1 CHA:

```
>>> pci.config(0xfe,0xd,0,0x2d0)
0x16b5ad6d
>>> sv.socket1.compute0.uncore.cha.cha6.sad.qpi_timeout_ctrl_cfg.read
0x16b5ad6d
```

Method 2 : Using Cscript register read

```
>>> sv.socket0.compute0.uncore.cha.cha6.sad.qpi_timeout_ctrl_cfg.show()
0x00000016 : level4_offset (28:24) -- Encoded Intel UPI Level 4 Timeout offset.
Offsets must be > 1
0x00000016 : level3_offset (23:19) -- Encoded Intel UPI Level 3 Timeout offset.
Offsets must be > 1
0x00000016 : level2_offset (18:14) -- Encoded Intel UPI Level 2 Timeout offset.
Offsets must be > 1
0x00000016 : level1_offset (13:09) -- Encoded Intel UPI Level 1 Timeout offset.
Offsets must be > 1
0x00000016 : level0_offset (08:04) -- Encoded Intel UPI Level 0 Timeout offset.
Offsets must be > 1.
0x00000006 : entry_ctr_inc_ctl (03:01) -- Indication of which bit of upper base
counter increments entry counter.
0x00000001 : enabletortimeout (00:00) -- Enable Intel UPI Timeout for certain
classes of Intel UPI transactions..
```

NOTE

Also need to check bit10 of the CHA MCA bank ctrl register. A value of '0' in Bit10 means that the CHA MCA bank will only log an error when the TOR Timeout occurs but will not issue the MCERR signal.

If using method 2 to check if CHA TOR Timeout is enabled, the previous example only shows socket0 CHA6. You can use the next command to show all of the CHAs' register:

```
>>> sv.sockets.computes.uncore.cha.chas.sad.qpi_timeout_ctrl_cfg
```

3.11 How to Check TOR Entry if TOR Timeout is Disabled

When 3Strike-Timeout occurs with `qpi_timeout_ctrl_cfg.enabletimeout=0` (disable), TOR timer will not timeout, so only 3Strike-Timeout is logged. You may capture 3Strike-Timeout, but no any other MCE bank records. For Intel® Xeon® 6900/6700-Series Processors with E-Cores, it will log to BUS bank as ROB-Timeout.

```
>>> sv.sockets.compute0.uncore.cha.cha1.sad.qpi_timeout_ctrl_cfg.enabletimeout
socket0.compute0.uncore.cha.cha1.sad.qpi_timeout_ctrl_cfg.enabletimeout -
0x00000000
socket0.compute0.uncore.cha.cha3.sad.qpi_timeout_ctrl_cfg.enabletimeout -
0x00000000
```

Find the first socket reporting IERR or MCERR.

Example:

```
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg - 0x00000000
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg - 0x0000001b
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0x00000000
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0x90e785a0
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_hi_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_hi_cfg - 0x00000000
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_hi_cfg - 0x0000001b
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo_cfg - 0x00000000
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo_cfg - 0x90e7859c
```

In this case, find Socket 1 is the first one in the system that reports an IERR and MCERR.

Find out what core of socket reports a 3Strike-Timeout.

Example: checking Crashdump "Machine Check Summary"

```
##### Status[63]=1 Machine Check summary #####
=====
|skt|die      |cor|thr|bank|Status      |Address      |
Misc      |CTL      |
Error      |
-----
|1 |compute0|9 |1 |3 |0xbe000000e18c0400|0x0000000000000000|
0x0000000000000000|0x0000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xe18c_0x0400|
=====
```

From this output, see that Core9 of Socket 1 has a 3strike timeout. "Machine Check summary" does not log any TOR timeout error. So need to check if there is a valid stuck entry in the TOR entry on Socket1 Core9.

```
TOR (Table of Requests) Stuck Valid Entries
=====
=====
|SKT|Die |CHA|TOR|InUse |Valid|InPipe|OpCode|Fsm      |System Address      |
Thread|Core |Die|IP|Target|TargetNode|RTId|SadResult|Snpsent|MemReadSent|
```

```
MemWriteSent|SnpPend|MemReadPend|MemWritePend|B/D/F|Offset CL|MCA MSCOD|
=====
|1 |soc |0 |30 |IN USE|1 |0 |DRd |Wdata_Wcmp_Split|0x0000031680000000|
0 |Core9|- |- |MEM0 |1 |- |HOM |0 |1 |
0 |0 |0 |0 |0 |0 |0 |0 |0 |0 |
=====
=====
```

Example for Intel® Xeon® 6900/6700-Series Processors with E-Cores, you will see ROB-Timeout instead.

```
##### Status[63]=1 Machine Check summary #####
=====
|skt|die |module |core|bank|Status |Address |
Misc |CTL |Error |
|1 |compute0|module1 |1 |0 |0xf200000000000400|0x0000000000000000|
0x0000000000000000|0x0000000000000000|CoreBUS:UNCORR_ROB_Timeout|
=====
```

Check if there is a valid stuck entry in the TOR entry on socket1 core1.

```
TOR (Table of Requests) Stuck Valid Entries
=====
=====
|SKT|Die |CHA|TOR|InUse |Valid|InPipe|OpCode |Fsm |System Address |
Thread|Core |Die|IP |Target|TargetNode|RTId |SadResult|SnpSent|MemReadSent|
MemWriteSent|SnpPend|MemReadPend|MemWritePend|B/D/F|Offset CL|MCA MSCOD|
=====
=====
|1 |soc |10 |30 |IN USE|1 |0 |DRd |Wdata_Wcmp_Split|0x0000000073553d00|
2 |Core1 |- |- |MEM0 |0 |- |HOM |7 |0 |
1 |0 |0 |0 |0 |0 |0 |0 |0 |0 |
=====
```

From this output, see a valid stuck entry on socket 1 core1. So further investigation on this entry is recommended.

3.12 How to Verify IERR is Caused by Nested MCE

There is no need to figure out if IERR is caused by nested MCE. Follow the First MCERR source hint to debug your system hang issue.

Here are some hints for IERR caused by nested MCERR:

1. You may not capture core 3Strike Timeout error or PCU error
2. You may capture multiple MCEs
3. CPU may enter Shutdown State

3.13 How to Check Multiple MCA Banks Issue MCE in System

Step 1 : Checking system error with Crashdump

If you find more than one MCA bank valid bits are set to 1 and UC bits are set to 1, then you can consider that multiple banks MCE happen even all banks are same type of MCA bank. For example, two B2CMI banks signal MCE will be considered as multiple banks MCE.

Step 2: set `itp.cv.machinecheckbreak = 1` and try to break processor at first MCERR happens. This setting may let processor halt when first MCERR happens.

3.14 How to Find Which Socket Report First MCERR

Method 1: Check Register `mcerrloggingreg`. `firstmcerrs rcid`

Example:

```
>>> sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCERR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00000088 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR

0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCERR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000000 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00000000 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR
```

In this two sockets Granite Rapids example, MCERRLOGGINGREG register shows only socket0 report MCERR. And the first MCERR source ID is 0x88.

If there are multiple sockets report first MCERR source ID, then go to method 2

Method 2: Check `punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo/hi_cfg` register on each die

Example:

```
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo_cfg - 0x00e89d0c
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_lo_cfg - 0x304ac740
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_hi_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_hi_cfg - 0x0000000e
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_mcerr_tsc_hi_cfg - 0x00000018
```

In this example, both socket0 and socket1 has TSC value. In this situation, the smallest TSC value indicates the first MCERR happens. For this case, the first MCERR happens in socket0.

Method 3: Set `itp.cv.machinecheckbreak = 1` and reproduce issue again

This setting may let processor halt when first MCERR happens, then use method 1 around 3 to double check.

Example:

```
>>>
[GNR0_C0_T0] Machine check Break at [0x38:0000000071cb77db] --
22:53:50.744505 2023-03-14
[GNR0_C0_T1] Machine check Break at [0x38:0000000071c944f1] --
22:53:50.747498 2023-03-14
[GNR0_C1_T0] Machine check Break at [0x38:0000000071c944f1] --
22:53:50.747715 2023-03-14
[GNR0_C1_T1] Machine check Break at [0x38:0000000071c944f1] --
22:53:50.748713 2023-03-14
[GNR0_C2_T0] Machine check Break at [0x38:0000000071c944f1] --
22:53:50.748713 2023-03-14
[GNR0_C2_T1] Machine check Break at [0x38:0000000071c944f1] --
22:53:50.749711 2023-03-14
[GNR0_C3_T0] Machine check Break at [0x38:0000000071c944f1] --
22:53:50.750709 2023-03-14
[GNR0_C3_T1] Machine check Break at [0x38:0000000071c944f1] --
22:53:50.750709 2023-03-14
[GNR0_C4_T0] Machine check Break at [0x38:0000000071c944f1] --
22:53:50.751707 2023-03-14
```

NOTE

Try cross check method 1 around 3 if you met exception on some methods

3.15 How to Find Which Socket Report First IERR

Method 1: Check Register `ierrloggingreg.firstmcerrsrcid`

Example:

```
>>> sv.sockets.io0.uncore.ubox.ncevents.ierrloggingreg.show()
0x00000000 : secondierrsrcfromcore (49:49) -- Set to '1' if the SecondIerrSrcID is
from a C2U
0x00000000 : secondierrsrcvalid (48:48) -- set to 1 if the SecondIerrSrcID is
valid
0x00000000 : secondierrsrcid (47:32) -- Physical Port ID of the end point with
the second IERR
0x00000001 : firstierrsrcfromcore (17:17) -- Set to '1' if the FirstIerrSrcID is
from a C2U
0x00000001 : firstierrsrcvalid (16:16) -- Set to '1' if the FirstIerrSrcID is valid
0x00004800 : firstierrsrcid (15:00) -- Physical Port ID of the end point with the
first IERR

0x00000000 : secondierrsrcfromcore (49:49) -- Set to '1' if the SecondIerrSrcID is
from a C2U
0x00000000 : secondierrsrcvalid (48:48) -- set to 1 if the SecondIerrSrcID is
valid
0x00000000 : secondierrsrcid (47:32) -- Physical Port ID of the end point with
the second IERR
0x00000000 : firstierrsrcfromcore (17:17) -- Set to '1' if the FirstIerrSrcID is
from a C2U
0x00000000 : firstierrsrcvalid (16:16) -- Set to '1' if the FirstIerrSrcID is valid
0x00000000 : firstierrsrcid (15:00) -- Physical Port ID of the end point with the
first IERR
```

In this two sockets Granite Rapids example, I ERRLOGGINGREG register shows only socket0 report I ERR. And the first I ERR source ID is 0x4800 .

If there are multiple sockets report first I ERR source ID, then go to method 2 .

Method 2: Check punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo/hi_cfg register on each die

Example:

```
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0xddad86d0
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0x00ade1d0
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg - 0x0000000f
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg - 0x0000007d
```

In this example, both socket0 and socket1 has TSC value. In this situation, the smallest TSC value indicates the first I ERR happens. For this case, the first I ERR happens in socket0.

Method 3: Set itp.cv.machinecheckbreak = 1 and reproduce issue again

This setting may let processor halt when first IERR happens, then use method 1 around 3 to double check.

3.16 How to Decode MCA Bank

MCA Banks are part of Mache Check Architecture. See *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*, document number 671200. The document provides detail decode information of most of files of MCA bank registers except MSCOD field. This field will be defined in *Granite Rapids Processor Registers Specification*, document number 637776.

List some general introduction:

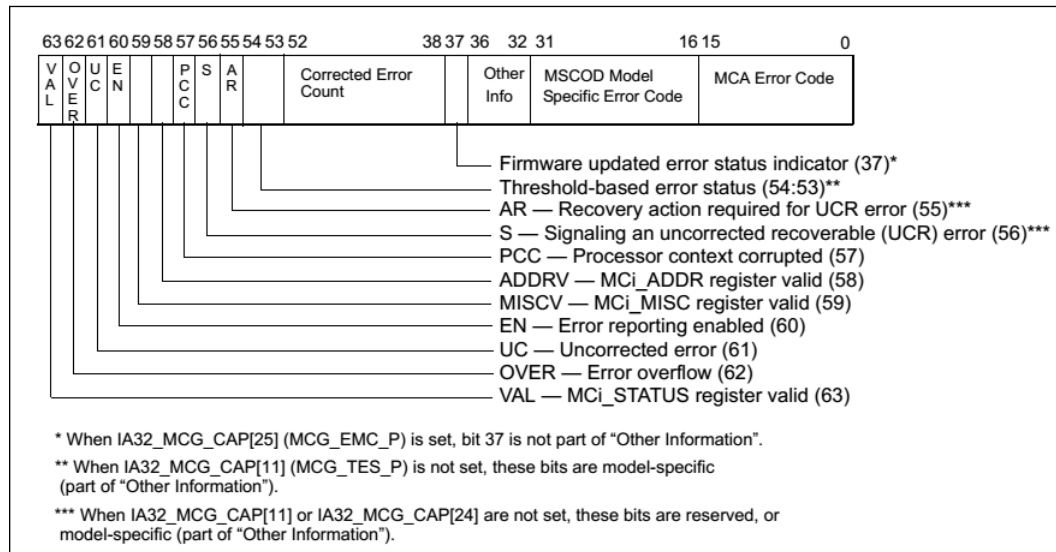
1. IA32_MCG_CAP MSR:

Bits [7:0] indicates the number of hardware unit error-reporting banks.

For example, 0x1C means total 28 MCA banks in a processor through a thread view.

2. IA32_MCI_STATUS MSRs:

Figure 20. IA32_MCi_STATUS MSRs



Bit [63], VAL: If set, you can check other bits, or ignore

Bit [62], OVER: Over bit will be set if previous MCA bank has valid log, even previous error is correctable error and current error is UC. However, when MCG_CMCI_P is set, correctable error is allowed not to set the over flag. See *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*, document number 671200 for details.

Bit [61], UC: If set, indicates hardware cannot fix this error. However, MCERR may not be signaled.

Bit [59], MISC: If set, you can trust MCA bank misc register content unless additional information.

Bit [58], ADDR: If set, you can trust MCA bank addr register content unless additional information.

Bit [57], PCC: If set, you cannot recovery this error by software. You cannot fully trust the CPU, even it can still run code. Note that some UC errors will not set PCC bit.

Bit [56], S: When UC is set to 1 but PCC is not set, you can check S and AR bits for UC sub-types.

Bit [55], AR: When UC is set to 1 but PCC is not set, you can check S and AR bits for UC sub-types.

NOTE

Bits [56:55] implemented as "S" and "AR", it is related to IA32_MCG_CAP.

Bits [54:53], when IA32_MCG_CAP [11]=1 and UC=0. There is an indicator when the number of corrected errors has exceeded the safe threshold to the point where an uncorrected error has become more likely to happen.

Bits [54:53]	Meaning
00	No tracking
01	Green: the number is at/below a pre-defined threshold
10	Yellow: the number exceeds the threshold
11	Reserved

For example, if an MCA bank status register bit [54:53] shows 0b10 (yellow) indicates this module inside the processor will report UCE or correctable error storm soon. It may be couple of weeks or months.

Bits [31:16], MSCOD: see the *EDS Volume 2* to get decode information. These 16 bits are key flags to distinguish error types in this module.

Bits [15:0], MCA Error Code: See *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*, document number 671200, volume 3, chapter 15.

NOTE

Some MCA bank status registers are not the same as MSCOD and MCACOD, such as Intel® UPI MCA bank, B2CMI MCA bank.

3. IA32_MCi_ADDR MSRs:

It contains the address of the code or data memory location that produced the machine-check error if the ADDR_V flag in the IA32_MCI_STATUS register is set.

For example, MLC MCA bank address register will record a linear address of an instruction when this instruction cannot be finished (3Strike-Timeout).

CHA MCA bank address register will record a physical address of a transaction when this transaction records in TOR but cannot be completed in time (TOR_Timeout).

4. IA32_MCi_MISC MSRs:

It contains additional information describing the machine-check error if the MISC_V flag in the IA32_MCI_STATUS register is set. Many useful information will be provided if MISC register is valid. The decode information is in *Granite Rapids Process Register*, document number 637776.

Before you decode the fields of MCA bank registers, you must get their values. There are many methods, including through Intel® ITP, Intel's CDP, Intel® ASD, BMC, OS dump log, ACD, and so on.

Method 1: Directly get fields of MCA bank registers through Cscript

You can use Cscript to get MCA bank information through Intel® ITP, Intel's CDP, Intel® ASD.

Example:

```
>>> sv.socket0.compute0.uncore.memss.mc0.ch0.mcchan.imc0_mc_status_shadow.show()
0x00000000 : valid (63:63) -- Error logged in this bank is valid
0x00000000 : over (62:62) -- (Overflow) Indicates (when set) that a machine check
error occurred while the...
```

```

0x00000000 : uc (61:61) -- Error logged in this banks is an uncorrected error.
0x00000000 : en (60:60) -- Indicates (when set) that signaling of the machine
check exception for this err...
0x00000000 : miscv (59:59) -- Indicates (when set) that the MCiMISC register
contains additional informati...
0x00000000 : addrv (58:58) -- indicates (when set) that the MCiADDR register
contains the address where th...
0x00000000 : pcc (57:57) -- Indicates (when set) that the state of the processor
might have been corrupted...
0x00000000 : s (56:56) -- S - Signaling. When set it indicates that the error was
signaled with an MCERR. ...
0x00000000 : ar (55:55) -- AR - Action REquired. When set it indicates that SW
needs to take immediate rec...
0x00000000 : cor_err_stat (54:53) -- These bits are used to indicate when the
number of corrected errors h...
0x00000000 : cor_err_cnt (52:38) -- A 15 bit counter that is incremented each
time a corrected (or uncorre...
0x00000000 : fw_upd (37:37) -- Indicates that the BIOS has updated the contents
of the MC Bank. HW should ...
0x00000000 : other_info (36:32) -- The functions of the bits in this field are
implementation specific and...
0x00000000 : mscod (31:16) -- Specifies a model specific error code that uniquely
identifies the machine c...
0x00000000 : mcacod (15:00) -- Specifies the machine check architecture defined
error code for the machine...

```

NOTE

Note that not all MCA bank registers are able to directly access and decode through register hierarchy.

If processor is still able to be halted after failure happens, you can consider halting processors and read MCA bank register through MSRs.

3.17 How to Verify Processor Getting into Shutdown State

For Shutdown State definition, see [Shutdown State](#) on page 21.

Intel provides two methods to indicate that processor is getting into shutdown state.

Method 1 - Set `itp.cv.shutdownbreak = 1` in `cscript`

If the following message is output in CScript prompt window after we enabled shutdown break, it means that processor is entering shutdown state.

```

>>> itp.cv.shutdownbreak=1
>>> go()
>>>
[GNR0_C0_T0] Shutdown Break at [0x38:0000000071c944f1] -- 10:43:10.779251
2023-03-15
[GPC core group] Break on 127 devices -- 10:43:10.797015 2023-03-1

```

Method 2 - Checking UBox Bank (Bank4) for shutdown

Granite Rapids provides shutdown reason in UBOX bank. To enable this, make sure that MSR (60h) SUPPRESS_SHUTDOWN bit 0 is enabled.

By checking UBOX MC4_STATUS(MSR,0x411), bit[15:0], if error code is 0x40c, it shows processor is in shutdown mode. And Ubox will record "IERR" status.

Here is an example: In MCACOD, bit[15:0], if error code is 0x40c, it shows processor is in shutdown mode, and bit[19:16] shows which shutdown condition is met. And Ubox will record 'IERR' status.

```
>>>error.crashdump()
...
Status[63]=1 Machine Check summary #####
=====
|skt|die|cor|thr|bank|Status      |Address      |Misc      |
CTL      |Error
-----
|0  |io0|-  |-  |4  |0xfa0000000005040c|0x0000000000000000|0x0000000000000000|
0x0000000000000001f|Ubox:Shutdown_Error:_SW_triple_fault_shutdown|
=====
Decoding Status Register:
  Range  Field
Description
Description
-----
63:63  VALID
register is valid  0x1    Valid    Indicates that the information within this
62:62  OVERFLOW
occurred while   0x1    Overflow Occurred
the error-
reporting register bank (the VALID bit was
already set)
61:61  UC
condition        0x1    UC Detected  A '1' indicates that an uncorrectable the error
detected.
was
60:60  EN
error            0x1    Enabled    Error reporting enabled. A '1' indicates that
reporting is enabled by the associated flag
bit(s) in the
IA32_MCi_CTL
register.
59:59  MISCV
content is       0x1    IA32_MCi_MISC Valid
read.
If this flag is '1', the IA32_MCi_MISC register
valid and can be
58:58  ADDR
content is       0x0    IA32_MCi_ADDR Not Valid
read.
If this flag is '1', the IA32_MCi_ADDR register
valid and can be
57:57  PCC
reliable        0x1    Processor Context Corrupted
and a reset
restarting of the processor may not be possible
recommended.
is
56:56  S
error.          0x0    ---
55:55  AR
error.          0x0    ---
54:53  THRESHOLD_BASED_STATUS
status          0x0    Threshold-based error
---
52:38  CORRECTED_ERROR_COUNT
IIO            0x0    ---
Corrected error count. Reserved for
```

37:37	FW	Indicates that the BIOS has updated the contents of the MC
0x0	---	
Bank		
36:32	OTHER_INFO	Not part of Machine-Check Architecture
---	---	0x0
31:16	MSCOD	Model Specific Error
Code	0x5	Shutdown Error: SW triple fault
shutdown		
15:0	MCACOD	Machine Check Architecture Error
Code	0x40c	Shutdown Error

3.18 How to Decode CHA Bank Address Register when TOR Timeout Happens

Step 1 - Checking CHA Bank address and misc registers are valid

CHA Bank status register bit59 (miscv) and bit58 (addrv)

addrv set to 1 means CHA bank address register is valid

miscv set to 1 means CHA bank misc register is valid

Step 2 - Checking if physical address is in IO space

Checking misc register bit [49:39]

If misc register bit [49:39] == 0x1D5 or 0x1DC, physical address is in IO space. Or else physical address is in memory space

For example: msr 0x41F= 0x183c817800100886

Bit [49:39] = (0x183c817800100886 >> 39) and 0x3FF = 0x102

0x102 is not equal to 0x1D5 or 0x1DC, so the physical address is in memory space.

Step 3 - Decode physical address if it is in IO space

For example:

```
>>> pci.scan
Scan Segment# 0 ...
Seg 0, Bus 0xEA, Dev 0x1f, Fnc 0
PCI scan complete: 1 segments, 255 Buses, 104 Devices, 270 Functions
>>> pci.resources()
+-----+-----+-----+-----+
|S|B|D|F|          Memory Range          |          I/O Range          |
|e|u|e|n|          (*=disabled, #=prefetchable) |          (*=disabled)      |
|g|s|v|c|          +-----+-----+-----+-----+
+-----+-----+-----+-----+
|00|00|01|0|0|#*00001e0fff980000-00001e0fff99ffff|*.....-.....|
|00|00|01|0|0|#*00001e0fff900000-00001e0fff93ffff|*.....-.....|
|00|00|02|0|0|#*00001e0fff9c0000-00001e0fff9cffff|*.....-.....|
|00|00|02|0|0|#*00001e0fff940000-00001e0fff97ffff|*.....-.....|
...
|00|16|00|0|0|*.....-.....|*00005000-0000501f|
|00|16|00|0|0|*0000000093280000-0000000093283fff|*.....-.....|
|00|17|00|0|0|0000000092000000-00000000930fffff|00004000-00004fff|
|00|17|00|0|0|#*ffffffffffff000000-0000000000ffff|.....-.....|
|00|18|00|0|0|0000000092000000-0000000092fffff|.....-.....|
```

```
|00|18|00|0| 0000000093000000-000000009303ffff| .....-.....|
|00|18|00|0| .....-.....| 00004000-0000407f|
...
```

In this example, 0x4000 to 0x4FFF IO space is assigned to device 0x17:0:0

If the physical I/O address belongs to CPU side, then find out the external device and verify if this device causes system hang or the path between the CHA and the device. For the IO address exceed legacy 64k, it is not legacy IO address, contact Intel for further debug.

Step 4 - Decode Transaction Type when the address is in memory space

Step 5 - Decode the address in memory space

Suggest to use `pci.scan()` and `pci.resource()` in CScripts which can both list I/O and MMIO of CPU and external PCI/PCIe* resources. See *Granite Rapids Processor Architecture Specification*, document number 639866, volume 1 - Source Address Decoder chapter to get more detail.

For example:

```
>>> pci.scan
Scan Segment# 0 ...
Seg 0, Bus 0xEA, Dev 0x1f, Fnc 0
PCI scan complete: 1 segments, 255 Buses, 104 Devices, 270 Functions
>>> pci.resources()
+---+---+---+---+---+---+---+---+---+---+
|S|B|D|F|                                     |I/O Range|
|e|u|e|n|         Memory Range                |(*=disabled)|
|g|s|v|c|      (*=disabled, #=prefetchable)    |(*=disabled)|
+---+---+---+---+---+---+---+---+---+---+
|00|00|01|0|#*00001e0fff980000-00001e0fff99ffff|*.....-.....|
|00|00|01|0|#*00001e0fff900000-00001e0fff93ffff|*.....-.....|
|00|00|02|0|#*00001e0fff9c0000-00001e0fff9cffff|*.....-.....|
|00|00|02|0|#*00001e0fff940000-00001e0fff97ffff|*.....-.....|
|00|00|03|0| *0000000090200000-00000000902fffff|*.....-.....|
|00|00|03|1|  0000000090300000-0000000090307fff|*.....-.....|
|00|00|03|1|  0000000090000000-00000000901fffff|*.....-.....|
|00|00|04|0| *00001e0fff800000-00001e0fff8fffff|*.....-.....|
...
```

If you cannot find your physical address in any memory range list in table. You need to consider the address may point to DRAM.

Suggest to use `mc.addTran()` in Cscript to decode address on CPU side:

```
>>> mc.addTran()

*****
*****          graniterapids Address Translator          *****
*****

1) Translate Address
2) Show naming convention
3) Show core address map
4) Print SAD arrays
5) Print RT arrays
6) Print TAD arrays
7) Print RIR arrays
8) Print NM SAD arrays
9) Tests
10) Decode memicals error log
```

```
11) Turn ON debug logging
q) Quit
Selection:
```

Press 1) Translate Address a 1) System address a 1) Physical DDR address. Then input the address you get from CHA Bank address register.

The translator tool will let you know if the address belongs to main memory.

For example: (if the address is in main memory, it will show detail information of dram address)

```
core address (52-bit) (0x0-0x3da7ffffff): 0x30000000
*****
Core Addr_____ = 0x3000_0000 (768.00 M)
Cluster_____ = 0x0
Set Index_____ = 0x0
Tag_____ = 0x1800
Cha_____ = 0
Perspective_____ = FM
*****
FM ddr5 1LM:
PSA Addr_____ = 0x3000_0000 (768.00 M)
Ch Addr_____ = 0x3000_0000 (768.00 M)
Sub_Ch Addr_____ = 0x1800_0000 (384.00 M)
pseudo_Ch Addr_____ = 0xc00_0000 (192.00 M)
Rank Addr_____ = 0x600_0000 (96.00 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 0x6
Ch_____ = 0x0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 3
Bank in Bank Grp = 0
Bank Addr_____ = 12
Row_____ = 0x600
Column_____ = 0x0
Mirror_____ = 0
Primary_____ = -1
*****
Hit Enter key to continue
```

NOTE

If the address points to PCH side, remember to disable DMI completion timeout and reproduce issue again. This tip will let you get the real first timeout transaction.

For example: (not in main memory)

```
core address (52-bit) (0x0-0x3da7ffffff): 0xa0000000
Core Address 0xa000_0000 (2.50 GB) maps to MMIO_L not a dram
Forward Translation of 0xa000_0000 (2.50 G) failed
*****
Core Addr_____ = 0xa000_0000 (2.50 G)
Cluster_____ = -0x1
Set Index_____ = -0x1
Tag_____ = -0x1
Cha_____ = -1
```

```
Perspective___ = FM
*****
FM :
Hit Enter key to continue
```

3.19 How to Draw a Transaction Flow

Concept:

- Get the transaction type from origreq field of CHA MCA bank misc register and decode it.
- Based on Intel® UPI specification and system topology to draw the transaction flow¹².

NOTE

Some register/feature setting may change transaction flow, such as D2C, D2K, \$HitMe, directory mode.

NOTE

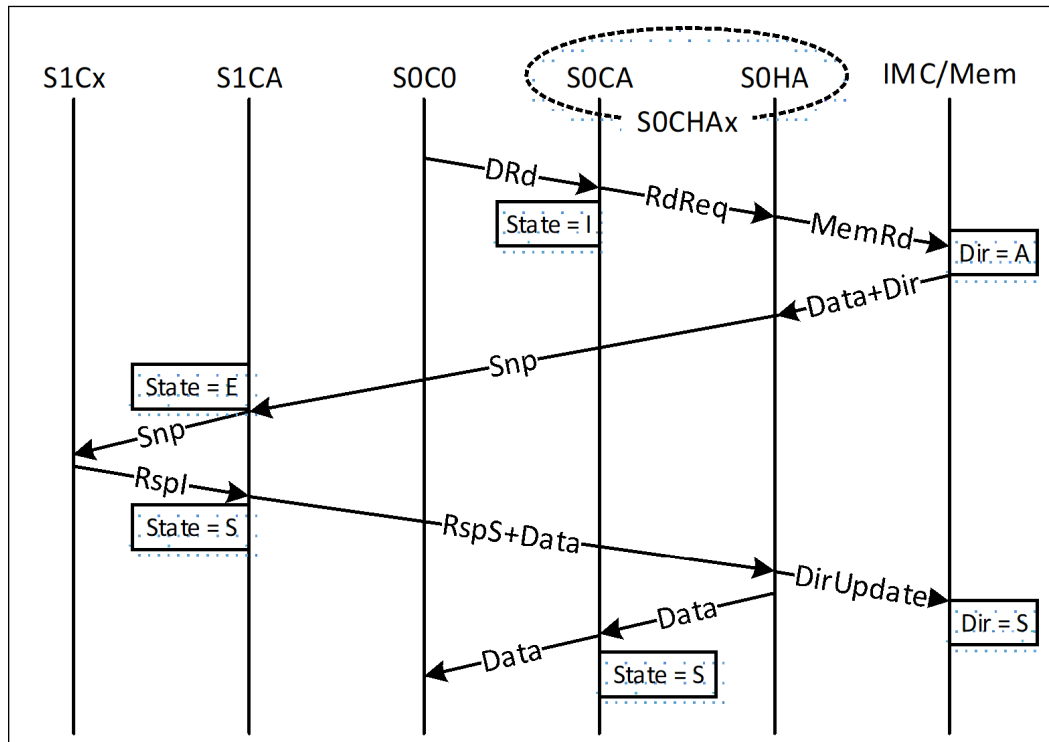
For cacheable address, different cache line state will impact transaction flow.

Tips:see Intel® UPI specification or 64-ia-32-architectures-optimization-manual to get more transaction flow information

Transaction Flow

Example 1

Figure 21. S0C0 Local Read LLC Missing (2S Platform)



CA – Cache Agent, there is one CA in a socket. A CA is composed of all CBOs (Part of CHA)

HA – Home Agent, for any cacheline in main memory, there is a HA to represent it. As we study a cacheline in transaction flow, there will be a HA in flow whatever how many sockets you are.

CHA – Since Intel® Xeon® Processor Scalable Family generation, CA and HA are distributed to piece of CHA. In logic, you can still use CA and HA instead in flow.

There are six components in transaction flow, S1Cx, S1CA, S0C0, S0CA, S0HA and IMC/Mem. S0CHAx represents CA and HA of a cacheline in S0.

1. DRd from S0C0 to S0CA

Socket0 Core0 sends a DRd package to Socket0 CA with a physical address. This address points to a cacheline which is owned by S0CHAx.

2. RdReq from S0CA to S0HA

This is an internal request inside the CHA component. TOR timer is here.

3. MemRd from S0HA to IMC/Mem

This is a memory request, and the destination is the main memory under S0. For simplicity, Intel removed M2M between S0HA and IMC.

4. Data+Dir from IMC/Mem to S0HA

Besides data, directory of this cacheline is gotten with data and it shows 'A'. It means other CAs requested this cacheline before, a Snp (Snoop) package need to be sent by HA.

5. Snp from S0HA to S1CA

This Snp package will cross socket0 to socket1 through Intel® UPI link between them.

As usual, S1CA is part of S1CHAx, x is the same number of socket0's. Because the same cacheline will be owned by the same logical CHA based on address hash algorithm.

S1CA cache hit and show the cacheline state is "E" (Exclusive) which means S1CA owns this cacheline.

6. Snp from S1CA to S1Cx

Based on core valid bits, this cacheline may be captured by one or some cores. One or more Snp requests may need to send to these cores.

7. RspI from S1Cx to S1CA

If all snooped cores did not modify this cacheline, they can easily discard the cacheline and send RspI to S1CA.

8. RspS+Data from S1CA to S0HA

S1CA can keep the cacheline but cannot own it, so the state will change to "S" which means this cacheline cannot be modified but can read. S1CA can send RspS and Data back to S0HA.

9. DirUpdate from S0HA to IMC/Mem

When S0HA receives RspS, it knows that this cacheline is "S" state in CAs, then it will update directory of the cacheline to "S".

Next time, any CA requests the same cacheline to S0HA, S0HA does not need to snoop but send data to the requester immediately.

10. Data from S0HA to S0CA

S0CA will store this cacheline and mark its state to "S".

11. Data from S0CA to S0C0

In the end, the Data will send to Socket0 Core0.

When DRd is sent out of S0C0, 3Strike timer starts. If TOR timer does not start, then 3Strike-Timeout ONLY is true.

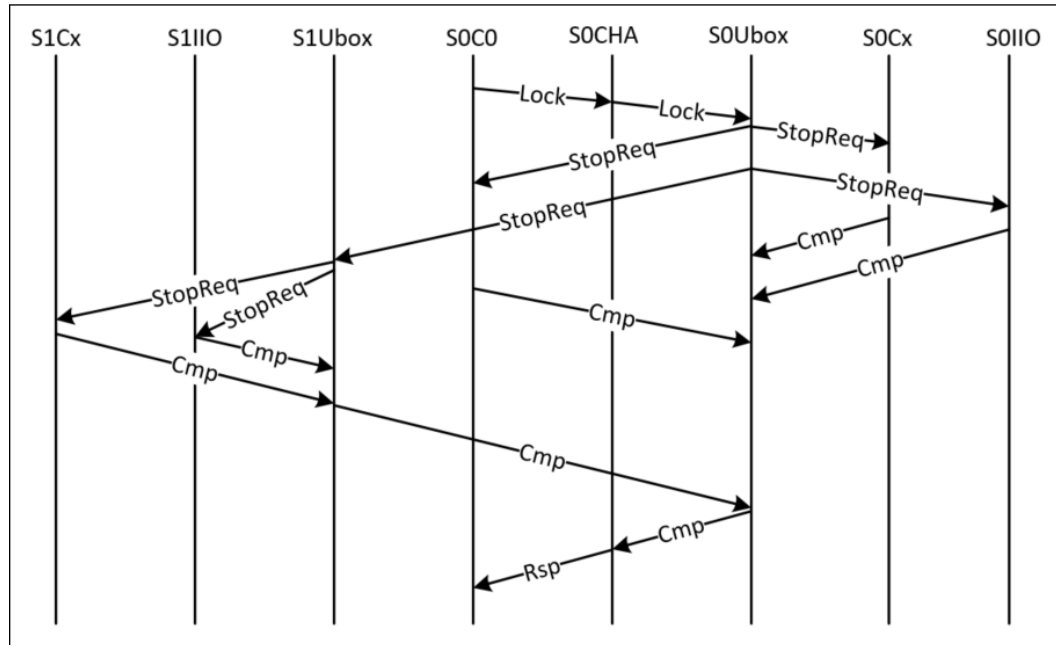
When TOR Timer starts, and the completion does not return in time, TOR timer will timeout.

Based on the previous transaction flow, any step blocked between (1) and (11) will cause TOR Timeout.

That is why an Intel® UPI error can cause a TOR timeout when S0C0 read a cacheline which is under S0's main memory even there is no memory under S1.

Example 2

Figure 22. SOC0 Lock Transaction Flow



Core can issue Lock transaction in order to do atomic operation. When the target cacheline is cacheable, cache lock will be used. When the cacheline is non-cacheable, a “bus lock” will be used. See *Intel® 64 and IA-32 Architectures Software Developer’s Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*, document number 671200, volume 2, chapter 8.

Bus lock must stop all potential requestors activities and then grant the bus and the cacheline ownership to the first one. When the core gets response from Lock Primary (Ubox), it can do read and write to the cacheline.

1. SOC0 sends Lock request to SOCHA
2. SOCHA forward Lock request to Lock Primary S0Ubox
3. S0Ubox boardcasts StopReqs to local cores, S0IIO and S1Ubox
4. S1Ubox boardcasts StopReqs in S1, including S1IIO and S1Cx
5. All potential requestors stop processing their transactions and send Cmp back to Uboxes.
6. Lock Primary (S0Ubox) receives all Cmp and send Cmp to SOCHA
7. SOCHA de-allocates the Lock transaction in its TOR entry
8. SOCHA forward grant information to SOC0

After Lock transaction finished, a normal read and write transactions will be issued. After read and write transactions finish, an Unlock transaction will be issued to release the “bus”.

Check IIO Memory direction when transaction type is “Lock” as pure Lock transaction flow will not touch main memory.

3.20 How to Use BMC Dump to Decode Errors

Intel has ACD and Intel® At-Scale Debug (Intel® ASD) debug solutions in BMC. For Intel® ASD, see *Intel® At-Scale Debug (Intel® ASD) BMC-JTAG Version 1.5.0*, document number 594948 for more information. For ACD, contact with your IFV.

See BMC Assisted FRU Isolation (BAFI) Solution to decode the data if you are using ACD.

3.21 How to Collect and Analyze Integrated Error Handler (IEH) Errors

If a PCIe* error was reported through IEHs during run-time, BIOS RAScode will process the errors by SMI and output SMI Serial login Firmware first mode. Here is an example to show you how to identify a device that generated an error connected to a Satellite IEH.

```
#
# Show SATIEH - BUS Mapping info
#
>>> ras.io.check_config_ieh()
=====
|          |          |          |          |
| IEHTYPEVER |          | IEHTYPEVER |          |
| Instance   |          | sat_bitmap |          |
| ieh_ver    |          | ieh_busnum |          | |
|---|---|---|---|---|
| socket0-io0-satieh0 |          | 0x2      |          | 0x1      |
| 0x3             | 0x0      |          |          |          |
| socket0-io0-satieh1 |          | 0x3      |          | 0x1      |
| 0x3             | 0x0      |          |          |          |
| socket0-io0-satieh2 |          | 0x4      |          | 0x1      |
| 0x3             | 0x32     |          |          |          |
| socket0-io0-satieh3 |          | 0x5     |          | 0x1     |
| 0x3             | 0xa     |          |          |          |
| socket0-io0-satieh4 |          | 0x6      |          | 0x1      |
| 0x3             | 0x1f     |          |          |          |
| socket0-io0-satieh5 |          | 0x7      |          | 0x1      |
| 0x3             | 0x45     |          |          |          |
| ...             |          |          |          |          |
|-----|-----|-----|-----|
#
# inject a PCIe UCE using CScript
#
>>> ei.io.injectPcieError(socket=0, port='0f',errType='uce',showErrorRegs=True)
#
#BIOS serial log
#
[IEH] error found at IEH(S:0x0 B:0x7E D:0x2 F:0x0) Sev: IEH NON FATAL ERROR
[IEH] ErrorStatus 0x20, MaxBitIdx 0x11
[IEH] BitIdx 0x5, ShareIdx 0x0
[IEH] error device is (S:0x0 B:0xA D:0x0 F:0x4) BitIdx 0x5, ShareIdx 0x0 [IEH]
error found at IEH(S:0x0 B:0xA D:0x0 F:0x4) Sev: IEH NON FATAL ERROR
[IEH] ErrorStatus 0x100, MaxBitIdx 0x11
[IEH] BitIdx 0x8, ShareIdx 0x0
[IEH] error device is (S:0x0 B:0xA D:0x7 F:0x0) BitIdx 0x8, ShareIdx 0x0
[Device Error] error on skt:0x0 Bus:0xA Device:0x7 func:0x0
PcieRootPortErrorHandler MailBox->PcieInitPar.SerrEmuTestEn = 0x0
```

```
ERROR: C00000002:V03071008 IO C698D0D1-904A-42E9-8406-152A6FF6549B 7AFE9410
IEH NON FATAL ERROR
IEH NON FATAL ERROR
```

3.22 How to Verify S3M Fatal Error Happens

On Birch Stream platform, S3M fatal errors that occur before the end of boot phase4 will be signaled as GLB_RST_WARN# assertion. After CPU has come out of reset, S3M fatal errors will be signaled as CPU CATERR# and logging into PCU machine check bank (bank6)with MCACOD 0x402 and MSCOD 0x80.

Example:

```
##### Status[63]=1 Machine Check summary #####
=====
|skt|die|cor|thr|bank|Status      |Address      |Misc      |
CTL      |Error
=====
|0  |io0|-  |-  |6  |0xb200000000800402|0x0000000000000000|0x0000000000000000|
0x000000000000000f|Punit:Non-Power_Management_Unit_Generic_Signaled_Error|
```

3.23 How to Check S3M Errors

Useful CScripts commands for checking S3M errors

```
>>> s3m.boot_info.boot_error_status()
S3M Boot Error for socket0
##### S3M BOOT ERROR #####
=====
|          Field          |          Description          | Value |
=====
| s3m_hw_err_fatal_sts   | Fatal Hardware Error         | 0x0   |
| s3m_hw_err_nonfatal_sts | NoFatal Hardware Error       | 0x0   |
| s3m_hw_err_cor_sts     | Correctable Hardware Error    | 0x0   |
| s3m_fw_err_fatal_sts   | Fatal Firmware Error         | 0x0   |
| s3m_fw_err_nonfatal_sts | NoFatal Firmware Error       | 0x0   |
| s3m_fw_err_cor_sts     | Correctable Firmware Error    | 0x0   |
=====

#####
##### NO S3M BOOT ERROR DETECTED #####
#####
##### S3M NVRAM Error Status #####
Could not read nvram_ssku_decryption_error. Skipping field.
=====
|          Field          |          Value          |
Description              |
=====
| nvram_vabd_content_auth_error | Failure of VAB/VAD Region | 0x0 |
| nvram_vad_content_auth_error  | Failure of VAD pin Content | 0x0 |
| nvram_pucode_content_decryption_error | Pucode Content Decryption Failure | 0x0 |
| nvram_ssku_decryption_error    | SSKU Decryption Failure | Error |
| nvram_fuse_decryption_error    | Fuse Decryption Failure | 0x0 |
| nvram_fuse_content_auth_error  | Fuse Content Authentication |
```

```

Failure | 0x0 |
| nvram_ssku_content_auth_error | SSKU Content Authentication
Failure | 0x0 |
| nvram_pucode_auth_error | Fuse offload Imagen ICV
Failure | 0x0 |
| nvram_s3m_fw_auth_error | RSA Authentication Failure of S3M FW
Image | 0x0 |
=====
=====

S3M Boot Error for socket1
##### S3M BOOT ERROR #####
=====
| Field | Description | Value |
-----
| s3m_hw_err_fatal_sts | Fatal Hardware Error | 0x0 |
| s3m_hw_err_nonfatal_sts | NoFatal Hardware Error | 0x0 |
| s3m_hw_err_cor_sts | Correctable Hardware Error | 0x0 |
| s3m_fw_err_fatal_sts | Fatal Firmware Error | 0x0 |
| s3m_fw_err_nonfatal_sts | NoFatal Firmware Error | 0x0 |
| s3m_fw_err_cor_sts | Correctable Firmware Error | 0x0 |
=====

#####
##### NO S3M BOOT ERROR DETECTED #####
#####
##### S3M NVRAM Error Status #####
Could not read nvram_ssku_decryption_error. Skipping field.
=====
| Field | Value |
Description |
-----
| nvram_vabd_content_auth_error | Failure of VAB/VAD
Region | 0x0 |
| nvram_vad_content_auth_error | Failure of VAD pin Content
Region | 0x0 |
| nvram_pucode_content_decryption_error | Pucode Content Decryption
Failure | 0x0 |
| nvram_ssku_decryption_error | SSKU Decryption
Failure | Error |
| nvram_fuse_decryption_error | Fuse Decryption
Failure | 0x0 |
| nvram_fuse_content_auth_error | Fuse Content Authentication
Failure | 0x0 |
| nvram_ssku_content_auth_error | SSKU Content Authentication
Failure | 0x0 |
| nvram_pucode_auth_error | Fuse offload Imagen ICV
Failure | 0x0 |
| nvram_s3m_fw_auth_error | RSA Authentication Failure of S3M FW
Image | 0x0 |
=====
=====

# Double check with MMIO register.
>>> sv.sockets.ios.uncore.s3m.main_treg.ibl_treg.mmio.showsearch('err', 'f') ##
for GNR CPU
>>> sv.sockets.ios.uncore.s3m.main_treg.ibl_treg.showsearch('err', 'f') ## for
SFR CPU
s3m_err_sts.s3m_hw_err_fatal_sts = 0x0
s3m_err_sts.s3m_hw_err_nonfatal_sts = 0x0
s3m_err_sts.s3m_hw_err_cor_sts = 0x0
s3m_err_sts.s3m_fw_err_fatal_sts = 0x0
s3m_err_sts.s3m_fw_err_nonfatal_sts = 0x0
s3m_err_sts.s3m_fw_err_cor_sts = 0x0
...

```

See *Birch Stream Platform: Granite Rapids/Sierra Forest Error Architecture and RAS*, document number 728644, chapter "S3M RAS" for more S3M information.

3.24 How to Enable S3M UART Trace Log

S3M will output its trace log with S3M UART. Analyzing the S3Mtrace logs is a useful debug method especially for S3M errors happening in early boot phase.

To enable S3M UART trace log output, you should enable UART_DEBUG in FIT4 DEBUG_POLICY (Set FIT type 4 offset 0x10 bit 2). Then you can collect S3MFW logs with S3M UART (USB 2.0 to TTL Serial Adapter is needed for Intel's RP). S3M FW UART logs are encoded and need decode script and XML for decoding. Contact Intel for decoding.

NOTE

S3M UARTFW Trace is an unsupported non-POR feature for debug purposes and needs to be used under Intel PAE's assistance. Intel does not guarantee its functionality and will not provide any support on it.

3.25 How to Dump S3M TREGs Via irdrscan()

S3M Hardware provides a dedicated space called TREG that is accessible to a list of consumers. Within TREG, certain registers are designated as pre-defined scratch-pads that can be read from and written to by the S3M firmware and other agents.

The S3M TREG domain encompasses a range of registers that indicate S3M and certain system status and configuration. Customers can leverage these registers using the "irdrscan()" command to retrieve critical information for triage in scenarios such as system hang, power-on hang, and other critical issues.

There are two methods to access these registers: S3M TREG commands on Cscripts and irdrscan() commands on OpenIPC*.

However, note that Cscripts cannot be used in the early stages of system boot. Therefore, in such situations, only irdrscan() over OpenIPC* is available. Otherwise, customers can utilize both methods to extract information from these TREGs.

Method1: Cscripts command

TREG Cscripts commands to retrieve data:

```
>>> sv.socket0.io0.uncore.s3m.main_treg.ibl_treg.cr.s3m_cfg_setup
0x0
```

irdrscan () command to retrieve data:

```
>>> itp.irdrscan(device="IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0",instruction=0x35,
bitCount=5, data=0, writeData=None)
[5b] 0x00
>>> itp.irdrscan(device="IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0",instruction=0x3a,
bitCount=8, data=0, writeData=None)
[8b] 0x00
>>> itp.irdrscan(device="IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0",instruction=0x3a,
bitCount=96, data=175640452039869704, writeData=None)
[96b] 0x00000000000000000000000000000000
```

```
>>> itp.irldrscan(device="IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0",instruction=0x31,
bitCount=96, data=None, writeData=None)
[96b] 0x000000000000000008006850B
```

To obtain a detailed description of each bit on a register:

```
>>>
sv.socket0.io0.uncore.s3m.main_treg.ibl_treg.cr.s3m_enabled_features.show()

0x000000000 : reserved (63:27) -- Reserved for future use cases
0x000000000 : ssku_telemetry_storage (26:26) -- SSKU Telemetry Storage
Enabled
0x000000000 : kat_fail_required (25:25) -- Known answer testing
required
0x000000000 : oem_key_wipe (24:24) -- OEM Key wipe required
0x000000000 : puf_for_gkek (23:23) -- Uses for PUF for GKEK
0x000000000 : core_gkek (22:22) -- Supports Hosting of Core GKEK
0x000000001 : soc_ip_patching (21:21) -- Supports SOC IP patching flows
0x000000001 : seamless_update (20:20) -- Supports seamless update feature for
itself
0x000000001 : isclk_service (19:19) -- Supports ISCLK service
0x000000001 : btg_w_prot (18:18) -- Supports BtG using PROT
0x000000001 : dfx_policy_auth (17:17) -- Security Policy Authentication Enabled
0x000000000 : prot_mutual_auth (16:16) -- Support Mutual Auth flow with a PROT
0x000000000 : bmc_mutual_auth (15:15) -- Support Mutual Auth flow with a BMC
0x000000000 : os_cpu_attestation (14:14) -- Support OS based CPU attestation
feature
0x000000000 : bmc_cpu_attestation (13:13) -- Supports BMC based CPU attestation
0x000000000 : ssku_attestation (12:12) -- SSKU Attestation Enabled
0x000000000 : prot_cpu_attestation (11:11) -- PROT Attestation Enabled
0x000000000 : bios_cpu_attestation (10:10) -- CPU Attestation Feature Enabled
0x000000000 : dpp (09:09) -- DPP support enabled
0x000000001 : hash_reporting_enabled (08:08) -- VAB or Hash Verification is
enabled
0x000000000 : vad (07:07) -- VAD is enabled
0x000000000 : s3m_fw_gp_3_2 (06:06) -- General Purpose Bits FW Defined Reg 3 -
see FAS for details
0x000000000 : s3m_fw_gp_3_1 (05:05) -- General Purpose Bits FW Defined Reg 3 -
see FAS for details
0x000000000 : s3m_fw_gp_3_0 (04:04) -- General Purpose Bits FW Defined Reg 3 -
see FAS for details
0x000000000 : ssku (03:03) -- SSKU is supported
0x000000001 : cross_dielet_tsc1588 (02:02) -- Cross dielet TSC1588 is supported
0x000000001 : multi_skt_support (01:01) -- MSB is supported
0x000000001 : integrated_boot_mode (00:00) -- Integrated boot is supported
```

Method2: OpenIPC* command

To utilize the `irldrscan()` function via OpenIPC* and dump the useful S3M TREGs, follow these steps:

1. Ensure that you have already installed OpenIPC* on your system.
2. Open the OpenIPC* environment or terminal.
3. Use the command "`ipc.irldrscan()`" to initiate the `irldrscan()` function. Try the next commands and see if can get response correctly.

```
In [10]:
ipc.irldrscan(device="IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0",instruction=0x35,
bitCount=5, data=0, writeData=None)
Out[10]: [5b] 0x00
In [11]:
ipc.irldrscan(device="IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0",instruction=0x3a,
bitCount=8, data=0, writeData=None)
Out[11]: [8b] 0x00
```

```

In [12]:
ipc.irldrscan(device="IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0",instruction=0x30,
bitCount=96, data=159878884136224024, writeData=None)
Out[12]: [96b] 0x000000000000000000000000
In [13]:
ipc.irldrscan(device="IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0",instruction=0x31,
bitCount=96, data=None, writeData=None)
Out[13]: [96b] 0x00000000003E01078006851B

```

Debugging by customers:

1. Customers can perform register decoding by referring to *Granite Rapids Processor Registers Specification*, document number 637776, EDS Vol.2, Chapter 11, Secured Startup Services Module (S3M) Registers, specifically section 11.2 S3M Control Registers.
2. If customers retrieve the register value from OpenIPC* commands, the least significant 4 bytes should be ignored. For example:

```

In [13]:
ipc.irldrscan(device="IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0",instruction=0x31,
bitCount=96, data=None, writeData=None)
Out[13]: [96b] 0x00000000003E01078006851B

```

The register valid data (8 bytes) 0x00000000003E0107.

Debugging with Intel:

1. If you would like Intel to assist in decoding these registers, dump all the relevant S3M TREGs mentioned in the next table.
2. Compose the dumped registers in the following format:

Figure 23. S3M TREGs

```

CMD:s3m_integ_boot_status_0
[5b] 0x00
[8b] 0x00
[96b] 0x000000000000000000000000
[96b] 0x000000000000EF0258006851B
CMD:s3m_service_completion_status_
[5b] 0x00
[8b] 0x00
[96b] 0x000000000000000000000000
[96b] 0x00000000C000001F8006850B
CMD:s3m_btg_bc_acbs_reg_for_debug
[5b] 0x00
[8b] 0x00
[96b] 0x000000000000000000000000
[96b] 0x0000000000001018006850B

```

3. Provide the result to your PAE for further decoding and analysis.
4. Additionally, you also can consult your PAE for sample scripts that demonstrate how to dump TREGs automatically. Moreover, you can collect all the required registers' data first and request Intel PAE to do register decoding and further analysis.

Registers and command list:

TREGs commands over Cscripts pre-fix:
sv.socket0.io0.uncore.s3m.main_treg.ibl_treg.cr.

TREGs commands over cscripts	irdrscan() over OpenIPC*
{pre_fix}.s3m_cfg_setup	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=175640452039869704, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_enabled_features	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=159878884136224024, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_err_fatal_mask	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=200411280782558488, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_err_sts	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=150871684881483032, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_fw_err_fatal_status	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=186900481900447000, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_fw_err_cor_status	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=184648682086761752, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_fw_err_fatal_status	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=182396882273076504, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_integ_boot_cpu_hw_timer_cfg	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None)
continued...	

	<pre>ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data= 198159480968873240, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</pre>
{pre_fix}.s3m_in teg_boot_misc_0	<pre>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data= 190277150828823816, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</pre>
{pre_fix}.s3m_k ey_provision_sta tus	<pre>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data= 168886083390965016, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</pre>
{pre_fix}.s3m_m ailbox_data_outp ut_reg_config	<pre>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data= 193655881341502744, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</pre>
{pre_fix}.s3m_m ailbox_flow_statu s	<pre>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data= 144116285440427288, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</pre>
{pre_fix}.s3m_m ailbox_grant_dat a	<pre>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data= 177893282645706008, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</pre>
{pre_fix}.s3m_m sb_status	<pre>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data= 157627084322538776, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</pre>
{pre_fix}.s3m_m ulti_skt_cfg	<pre>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data= 155375284508853528, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</pre>
continued...	

{pre_fix}.s3m_nvram_code_se_svn	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=166634283577279768, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_nvram_content_auth_err_sts	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=153123484695168280, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_nvram_update_limit	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=180145082459391256, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_platform_flash_register_attributes	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=240942646636741896, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_provisioning_auth_failure_count	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=171137883204650264, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_provisioning_availability	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=173389683018335512, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_ufacture_code_cfr_svn	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=164382483763594520, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
{pre_fix}.s3m_ufacture_code_revid	dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0" ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=207166680223614232, writeData=None) ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)
continued...	

<code>{pre_fix}.s3m_s3mfw_cfr_svn</code>	<code>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0"</code> <code>ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=162130683949909272, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</code>
<code>{pre_fix}.s3m_s3mfw_revid</code>	<code>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0"</code> <code>ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=204914880409928984, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</code>
<code>{pre_fix}.s3m_soc_ip_rt_patch_flow_status_0</code>	<code>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0"</code> <code>ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=213922079664669976, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</code>
<code>{pre_fix}.s3m_soc_ip_rt_patch_flow_status_1</code>	<code>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0"</code> <code>ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=216173879478355224, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</code>
<code>{pre_fix}.s3m_soc_ip_rt_patch_flow_status_2</code>	<code>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0"</code> <code>ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=218425679292040472, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</code>
<code>{pre_fix}.s3m_soc_patch_image_manifest_params</code>	<code>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0"</code> <code>ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=211670279850984728, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</code>
<code>{pre_fix}.s3m_soc_platformid_info</code>	<code>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0"</code> <code>ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=245446246264112392, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</code>
<code>{pre_fix}.s3m_tsc_transfer_offset</code>	<code>dev_str = "IO0_V1_IOW_GLBW_IDFTTOP_TAP2IOSFSB0"</code> <code>ipc.irdrscan(device=dev_str,instruction=0x35, bitCount=5, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x3a, bitCount=8, data=0, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x30, bitCount=96, data=189151250921981192, writeData=None)</code> <code>ipc.irdrscan(device=dev_str,instruction=0x31, bitCount=96, data=None, writeData=None)</code>


```
d
|00|00|00|1|09a4|8086|20|80|7270|8086|...|Series Mesh to
IAL
|00|00|00|2|09a3|8086|20|80|7270|8086|...|Series Mesh2IIO
RAS
.....
|00|6a|01|0|0b25|8086|00|00|0000|8086|...|Data Streaming Accelerator
(DSA)
|00|6a|02|0|0cfe|8086|00|00|0000|8086|...|Intel Analytics
Accelerator (IAX)
|00|6a|03|0|09a6|8086|00|80|0000|8086|...|ff|...|PECI Out-Of-Band
Management Services Module (OOBMSM)
|00|6a|03|1|09a7|8086|00|80|0000|8086|...|ff|...|PECI OOB-MSM -
Discovery
.....
+--+--+--+--+-----+--+--+--+--+-----+--+--+--+--+
```

3.28 How to Verify First MCE is from CHA Bank and Error Code is TOR Timeout

Method 1 - Checking if first MCERR comes from a CHA bank and decode the bank status register to check if the error type is TOR Timeout

See the previous section to get how to find out the first MCE when multiple MCA bank reports MCEs.

For CHA bank status register bit [31:16] decoding, see the *Granite Rapids Processor Architecture Specification*, document number 639866, MSR 0x41D:

MSCOD: 0x0C = TOR_TIMEOUT

3.29 How to Get CPU SKU Type

Directly check CAPID registers

Bit Rang	Indicates RAS Levels
4:0	00011b = Advanced
	00001b = Standard

For Example: Check uncore.punit. ptpcioregs.ptpcioregs.capid2_cfg register

```
>>> sv.socket0.io0.uncore.punit.ptpcioregs.ptpcioregs.capid2_cfg.show()
0x00000000 : capid2_31 (31:31)
0x00000000 : capid2_30 (30:30)
0x00000000 : capid2_29 (29:29)
0x00000000 : capid2_28 (28:28) -- RSVD.
0x00000000 : capid2_27 (27:27) -- RSVD.
0x00000009 : upi_link_en (26:23) -- UPI_LINK_DIS bits indicating if an IO stack
is enabled/disabled as UPI port UPI...
0x00000001 : max_upi_speed (22:20) -- Indicating MAX_UPI_SPEED to allow the range
of max UPI speeds that a sku may s...
0x00000000 : module_as_thread_en (19:19) -- CAPID bit only indicates the
capability to represent core module as only...
0x00000020 : dmfc_mcr (18:12) -- The maximum
supported MCR memory frequency. ...
```

```
0x00000001c : dmfc_ddr5 (11:05) -- The maximum
supported DDR5 memory frequency. ...
0x000000001 : ras_levels (04:00)
```

In the example, CAPID2[4:0] is 00001b, so current SKU is server SKU with standard RAS.

3.30 How to Find which Event Triggers Triple Fault

Method 1 - Checking shutdown break address

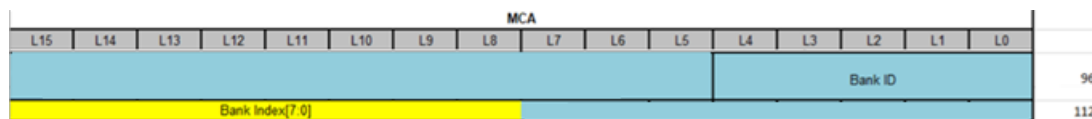
If possible, check shutdown break address assembly code to figure out which exception handler the thread is.

Note that when page fault exception happens, we may not in excetion handler because even page fault expcetion entry address is not able to translate out.

Then we can double check with setting possible exception entry as break point and reproduce issue again.

3.31 How to Check Merged MCA Bank Instance ID

Every merged MCA bank is mapping a group of identical IPs. Every IP with an MCA bank is able to send an merged MCA message on IOSF-SB. There are MCA Bank ID and MCA Index in the message, which indicate to the MCA aggregator, that is, Ubox, about the exact MCA bank identity and which architecture MCA bank to flag to software and firmware.



To access merged MCA MSRs, OOBMSM exposed out-of-band specific MSRID, starting at MSR address 0x8000. In Granite Rapids and Intel® Xeon® 6900/6700-Series Processors with E-Cores, merged machine check banks MC5-MC7, MC9, MC11, and MC12, can be accessed using either the RdIAMSR() or RdIAMSREx() command with processor ID set to zero.

Table 6. Merged MCA Bank Instance IDs

MCE merged Bank	In-Band Access	OOB Access	
Bank 5 UPI	MSR 0x414-0x417 MSR 0x415 for MC_STATUS	MSR 0x416 for UPI_x	MSR 0x8010-0x8014 MSR 0x8012 for MC_STATUS
Bank 6 PCU	MSR 0x418-0x41B MSR 0x419 for MC_STATUS	MSR 0x41A for PCU_x	MSR 0x8045-0x8049 MSR 0x8047 for MC_STATUS
Bank 7 CHA	MSR 0x41C-0x41F MSR 0x41D for MC_STATUS	MSR 0x41E for CHA_x MSR 0x41F for CHA_64 + x	MSR 0x8000-0x8005, 0x8050 MSR 0x8002 for MC_STATUS
continued...			

MCE merged Bank	In-Band Access	OOB Access	
Bank 9 LLC	MSR 0x424-0x427 MSR 0x425 for MC_STATUS	MSR 0x426 for LLC_x MSR 0x427 for LLC_64 + x	MSR 0x803A-0x803E MSR 0x803C for MC_STATUS
Bank 11 MSE	MSR 0x42C-0x42F MSR 0x42D for MC_STATUS	MSR 0x42E for MSE_x	MSR 0x8035-0x8039 MSR 0x8037 for MC_STATUS
Bank 12 B2CMI	MSR 0x430-0x433 MSR 0x431 for MC_STATUS	MSR 0x432 for B2CMI_x	MSR 0x8006-0x800A MSR 0x8008 for MC_STATUS

Take merged Bank 6 PCU as an example, every PUnit will send MCA_MSG to UBox in merged format, the individual instances of the IP can be accessed (read) by OOBMSM over sideband. Instance 0, 4, 9, 10, 11 are accessed using domain ID 0, 4, 9, 10, 11 respectively.

In Band	
PCU	(418h) IA32_MC6_CTL
PCU	(419h) IA32_MC6_STATUS
PCU	(41Ah) IA32_MC6_ADDR
PCU	(41Bh) IA32_MC6_MISC

Out-Of-Band	
PCU	(8045h) PCU_x MC_CTL2
	(8046h) PCU_x MC_CTL
PCU	(8047h) PCU_x MC_STATUS
PCU	(8048h) PCU_x MC_ADDR
PCU	(8049h) PCU_x MC_MISC
PCU	(041Ah) Instance ID

CScripts already show the instance in a summarized output. See the following example, IO0 is the instance.

```
#####
Socket: 0 io0

Uncore MCA from Socket: 0
=====
| Valid | DIE | MC Bank Name | Status | Addr | Misc | Ctl1 | Ctl2 |
=====
| * | io0 | PCU | 0xfa00000000200402 | 0x0 | 0x22a00340400 | 0xf | 0x500000000 |
=====
```

3.32 How to Check First MCERR/IERR Source ID

To consider multiple dies and more IPs that can assert the error on Granite Rapids and Intel® Xeon® 6900/6700-Series Processors with E-Cores, the First MCERR/IERR Src ID expand to 16 bits and also define both Die ID and Port ID.

MCERRLOGGINGREG Example

Check mcerrloggingreg register:

```
>>> sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000001 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x0000570a : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCErr
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x000055b6 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR

0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCErr
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000000 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00000000 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR
```

MCERRLOGGINGREG — Offset A8h

MCERR first/second logging error.

Type	Size	Address	Default
PCI	64 bit	[B:30, D:0, F:0] + A8h	0000000000000000 0h

Register Level Access:

BIOS Access	SMM Access	OS Access	OOB Access
RW	RW	RW	RW

Bit Range	Default and Access	Field Name (ID): Description
63:50	0h RO	Reserved
49	0h RW/V/P	SECONDMCERRSRCFROMCORE: Set to '1 if the SecondMCerrSrcID is from a C2U
48	0h RW/V/P	SECONDMCERRSRCVALID: Set to 1 if the SecondMCerrSrcID is valid.
47:32	0000h RW/V/P	SECONDMCERRSRCID: Physical Port ID of the end point with the second MCErr
31:18	0h RO	Reserved
17	0h RW/V/P	FIRSTMCERRSRCFROMCORE: Set to '1 if the FirstMCerrSrcID is from a C2U
16	0h RW/V/P	FIRSTMCERRSRCVALID: Set to 1 if the FirstSrcID is valid.
15:0	0000h RW/V/P	FIRSTMCERRSRCID: Physical Port ID of the end point with the first MCERR

Register bits definition is same with previous(Eagle Stream)generation. The difference is bits 15:0 are divided to Port ID(bit 10:0) and Die ID(bit 15:11) parts.

Field Name	Field Access	Bit Range	Field Default	Field Description
FIRSTMCERRSRCID (Port ID)	RW/V/P	10 : 0	0x0	Logical Port ID of the end point with the first MCERR
FIRSTMCERRSRCID (Die ID)	RW/V/P	15 : 11	0x0	Logical Die ID of the end point with the first MCERR 00 - I/O (Parent); 04 - I/O (Child) 09 - Compute0; 10 - Compute1; 11- Compute2

See the *UBOX Error Logging Registers Decoding* section in the corresponding *Registers Specification* to decode the source of the MCE logged.

Port ID to IP Mapping:

Port IDs (Hexadecimal)	Source of MCERR error
4800-483F, 5000-503F, 5800-583F	Cores
1D-1F	Ubox
4C8	B2UBOX
3C0, 3C8, 3D0, 3D8, 3E0, 3E8, 3F0, 3F8, 23C0, 23C8, 23D0, 23D8, 23E0, 23E8, 23F0, 23F8	B2CXL
2B2, 2BA, 2C2, 2CA, 2D2, 2DA, 2E2, 2EA, 2F2, 2F8, 22B2, 22BA, 22C2, 22CA, 22D2, 22DA, 22E2, 22EA, 22F2, 22F8	B2HOT
4A0, 4B8, 24A0, 24A8, 24B0, 24B8	B2UPI
116, 616, 2116, 2216, 2516, 2616	UFI2UPI
117, 617, 2117, 2217, 2517, 2617	UPI
27, 80, 81, 6F1, 4027, 2080, 2081, 26F1. 4880, 4881, 4B08, 4EF1, 5080, 5081, 5308, 56F1, 5880, 5881, 5B08, 5EF1	Punit
4D08-4FFA, 5508-57FA, 5D08-5FFA	CHA/SCF
4B61-4B92, 5361-5392, 5B61-5B92	B2CMI
4B9A-94BFC, 539A-53FC, 5B9A-5BFC	MCCHAN

4.0 Error Injection and Verification

This chapter will introduce different types of errors injection methods. With injection methods, audiences can practice error dump recipes, debug flows and familiar system behavior after an error happens.

We had listed some error injection methods, feedback us if any error injection method does not work.

For some type of errors, the injection methods may not be provided as the failures may be from real cases. Register checking results will be still provided for audiences to be familiar with results of these type of errors.

Before error Injection, reset the target, check target BIOS, Host Cscript version as recipes mentioned, especially some BIOS settings, follow the receipt step by step. General BIOS setup setting for error injection. The following BIOS settings paths are from an Archer City CRB. Locations of these settings may be different on your BIOS implementation.

1. EDKII > Socket Configtesturation > Processor Configuration >
 - a. — Lock Chipset = Disable
 - b. — MSR Lock Control = Disable
2. EDKII > Socket Configuration > Uncore Configuration > Uncore General Configuration > Directory Mode Enable = Disable
3. EDKII > Socket Configuration > IIO Configuration > DFX Global Configuration >
 - a. — EV DFX Features = Enable
 - b. — Disable BIOS Done = [X]
4. EDKII > Platform Configuration > System Event Log > Crash Log Enabling >
 - a. — CPU CrashLog Feature = Disable

4.1 Hard Hang Error Injection

4.1.1 Hard Hang.Shutdown

Only two shutdown error injection cases are listed here.

4.1.1.1 Shutdown State With Triple Fault

Tool: *RAS Automation Toolkit*, document number 633201.

Test case: `.\testscripts\BHS\Debug_Handbook
4.1.1.1_Shutdown_State_with_Triple_Fault.py`

Please make sure MSR (60h) SUPPRESS_SHUTDOWN bit 0 is enabled firstly. Note that IERR will be asserted in this case due to the Shutdown Suppression setting. After Injecting page fault error, we will see CPU shutdown break active.

```
>>> halt
[GPC core group] Halt on 128 devices -- 16:45:22.233769 2023-07-07
>>> itp.cv.shutdownbreak = 1
>>> itp.threads[0].state.regs.cr3=0
>>> itp.threads[0].state.regs.cr3
[64b] 0x0000000000000000
>>> mem(0x0,4,0)
>>> mem(0x0,4)
0x0
>>> go()
[GPC core group] Resuming on 128 devices -- 16:48:32.921368 2023-07-07
[GNR0_C0_T0] Shutdown Break at [0x38:0000000071bb5501] -- 16:48:37.452407
2023-07-07
[GPC core group] Break on 127 devices -- 16:48:37.452407 2023-07-07
```

Run Crashdump, SW triple fault shutdown error is logged

```
=====
|socket|
Signature
|
-----
|0      |
HW.MCE.UBOX:Shutdown_Error:_Sw_Triple_Fault_Shutdown.incomplete_dump.mca_same_valu
e|
|1      |
INCOMPLETE_DUMP.MCA_SAME_VALUE
|
=====

##### Status[63]=1 Machine Check summary #####
=====
|s|kt|die|cor|thr|bank|Status      |Address      |Misc      |
CTL      |Error      |
-----
|0  |io0|-  |-  |4  |0xfa0000000005040c|0x0000000000000000|0x0000000000000000|
0x0000000000000001f|Ubox:Shutdown_Error:_SW_triple_fault_shutdown|
=====

...

...
##### Decode each Valid Machine Check #####
*****
*****
=====
|s|kt|die|cor|thr|bank|Status      |Address      |Misc      |
Ctl      |
-----
|0  |io0|0  |0  |4  |0xfa0000000005040c|0x0000000000000000|0x0000000000000000|
0x0000000000000001f|
=====
```

Decoding Status Register:

Range	Field	Description	Value	Value	Decode
Description	Description				
63:63	VALID	Indicates that the information within this register is valid	0x1	Valid	
62:62	OVERFLOW	A '1' indicates that a machine-check error occurred while	0x1	Overflow	reporting
61:61	UC	register bank (the VALID bit was already set)	0x1	UC Detected	
60:60	EN	A '1' indicates that an uncorrectable the error was detected.	0x1	Enabled	
59:59	MISCV	Error reporting enabled. A '1' indicates that reporting is enabled by the associated flag	0x1	IA32_MCi_MISC Valid	
58:58	ADDRV	If this flag is '1', the IA32_MCi_ADDR register content is	0x0	IA32_MCi_ADDR Not Valid	
57:57	PCC	If this flag is '1', the IA32_MCi_ADDR register content is	0x1	Processor Context Corrupted	
56:56	S	Processor Context Corrupt: A '1' indicates that restarting of the processor may not be possible	0x0	is	
55:55	AR	Recovery action required for UCR	0x0	---	
54:53	THRESHOLD_BASED_STATUS	Threshold-based error	0x0	---	
52:38	CORRECTED_ERROR_COUNT	Corrected error count. Reserved for	0x0	---	
37:37	FW	Indicates that the BIOS has updated the contents of the MC	0x0	---	
36:32	OTHER_INFO	Not part of Machine-Check	0x0	---	
31:16	MSCOD	Model Specific Error	0x5	Shutdown Error: SW triple fault	
15:0	MCACOD	Machine Check Architecture Error	0x40c	Shutdown Error	

NOTE

MSR (60h) SUPPRESS_SHUTDOWN bit 0 enhances the log capturing for shutdown cycle. Otherwise, it may be hard to see a triple fault from Crashdump in this case due to system shutdown then reboot without IERR.

4.1.1.2 Shutdown State. MCE when CR4.MCE=0

Tool: *RAS Automation Toolkit*, document number 633201.

Test case: .\testscripts\BHS\Debug_Handbook
4.1.1.2_Shutdown_State.MCE_when_CR4.MCR=0.py

Inject MCERR when CR4.MCE=0

Steps as shown next:

Make sure that MSR (60h) SUPPRESS_SHUTDOWN bit 0 is enabled firstly. Halt CPU and Set CR4.MCE(Machine-Check Enable, bit 6 of CR4) to 0 for all cores.

```
>>> halt
[GPC core group] Halt on 128 devices -- 02:39:24.550756 2023-07-11
>>> itp.threads.state.regs.cr4
NodeGroup([
  ('GNR0_C0_T0.state.regs.cr4', [64b] 0x0000000000000068),
  ('GNR0_C0_T1.state.regs.cr4', [64b] 0x0000000000000060),
  ('GNR0_C1_T0.state.regs.cr4', [64b] 0x0000000000000060),
  ('GNR0_C1_T1.state.regs.cr4', [64b] 0x0000000000000060),
  ('GNR0_C2_T0.state.regs.cr4', [64b] 0x0000000000000060),
  ...
>>> itp.threads.state.regs.cr4=0x620
>>> itp.threads.state.regs.cr4
NodeGroup([
  ('GNR0_C0_T0.state.regs.cr4', [64b] 0x00000000000000620),
  ('GNR0_C0_T1.state.regs.cr4', [64b] 0x00000000000000620),
  ('GNR0_C1_T0.state.regs.cr4', [64b] 0x00000000000000620),
  ('GNR0_C1_T1.state.regs.cr4', [64b] 0x00000000000000620),
  ...
...

```

Inject MCERR after setting CR4.MCE=0

```
>>> ei.mem.injectMemError(0x20000000,errType='uce',immConsume=True)
Halting to do check on injector lock
INFO: BIOS knob 'Disable BIOS Done' is enabled!
Initiating an SMI
[GPC core group] Resuming on 480 devices -- 05:11:26.289775 2023-07-11
Injectors UNLOCKED on GNR0_C0_T0
Injectors UNLOCKED on GNR0_C0_T1
Injectors UNLOCKED on GNR0_C1_T0
Injectors UNLOCKED on GNR0_C1_T1
Injectors UNLOCKED on GNR0_C2_T0
Injectors UNLOCKED on GNR0_C2_T1
[GNR1_C150_T1] SMM entry Break at [0x10:0000000000008000] -- 05:12:24.120367
2023-07-11
[GNR1_C151_T0] SMM entry Break at [0x10:0000000000008000] -- 05:12:24.120367
2023-07-11
[GNR1_C151_T1] SMM entry Break at [0x10:0000000000008000] -- 05:12:24.120367
2023-07-11
...
Injectors UNLOCKED on GNR0_C3_T0
Injectors UNLOCKED on GNR0_C3_T1
Injectors UNLOCKED on GNR0_C4_T0

```

```

Injectors UNLOCKED on GNR0_C4_T1
Injectors UNLOCKED on GNR0_C5_T0
Injectors UNLOCKED on GNR0_C5_T1
...
Exit SMM Mode...

*****
Core Addr_____ =          0x2000_0000 (512.00 M)
Cluster_____ = 0x2
Set Index_____ = 0x0
Tag_____ = 0x1000
Cha_____ = 52
Perspective_____ = FM
*****
FM ddr5 1LM:
Core Addr_____ =          0x2000_0000 (512.00 M)
Ch Addr_____ =          0x2000_0000 (512.00 M)
Sub_Ch Addr_____ =          0x1000_0000 (256.00 M)
pseudo_Ch Addr_____ =          0x800_0000 (128.00 M)
Rank Addr_____ =          0x800_0000 (128.00 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 0x0
Ch_____ = 0x0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 0
Bank in Bank Grp = 1
Bank Addr_____ = 1
Row_____ = 0x800
Column_____ = 0x0
Mirror_____ = 0
Primary_____ = -1
*****
Core Addr_____ =          0x2000_0000 (512.00 M)
Cluster_____ = 0x2
Set Index_____ = 0x0
Tag_____ = 0x1000
Cha_____ = 52
Perspective_____ = FM
*****
FM ddr5 1LM:
Core Addr_____ =          0x2000_0000 (512.00 M)
Ch Addr_____ =          0x2000_0000 (512.00 M)
Sub_Ch Addr_____ =          0x1000_0000 (256.00 M)
pseudo_Ch Addr_____ =          0x800_0000 (128.00 M)
Rank Addr_____ =          0x800_0000 (128.00 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 0x0
Ch_____ = 0x0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 0
Bank in Bank Grp = 1
Bank Addr_____ = 1
Row_____ = 0x800
Column_____ = 0x0
Mirror_____ = 0
Primary_____ = -1

```

```

*****
Discovering Populated Mcs
Discovering Populated Channels
Discovering Channels
System in Non-Mirror mode, and error type: uce
ECC Injection in Non-Mirror Mode, MC=0x0, Channel=0x0, Mirror Scrub Enabled is
True
socket0 Mc0 Ch0 Patrol Scrub Disabled
socket1 Mc0 Ch0 Patrol Scrub Disabled
socket0 Mc0 Ch0 Patrol Scrub Disabled
socket1 Mc0 Ch0 Patrol Scrub Disabled
Current : dev0 = 1, dev0msk = 0x4, dev1 = 5, dev1msk = 0x1.
adddc_flag=None
adddc_errinj_reg=0x0x0
  ret_code:0x0, Dev0:0x1, Dev1:0x5, D0msk:0x4, D1msk:0x1, adddc_mask:0x0,
perspective:FM
Introduce a valid channel
Restoring Original Values!!!

```

Let the processor go, and double check Crashdump log, we can see IERR occurs, and the shutdown error caused by MCE when CR4.MCE=0.

```

##### Signature by Sockets #####
=====
|socket|
Signature
|
-----
|0      |
HW.MCE.MCCHAN:Uncorr_Uncorrectable_Error_For_Demand_Or_Underfill_Reads.incomplete_
dump.mca_zero_value|
|1      |
HW.MCE.UBOX:Shutdown_Error:_Mce_When_Cr4.Mce_Is_Clear.incomplete_dump.mca_zero_val
ue
|
=====

##### Status[63]=1 Machine Check summary #####
=====
==
|skt|die      |cor|thr|bank |Status      |Address      |
Misc      |CTL      |
Error
|
-----
--
|0 |io0      |-|-|4      |0xfa000000001040c|0x0000000000000000|
0x000000000002d0000|0x0000000000000001f|
Ubox:Shutdown_Error:_MCE_when_CR4.MCE_is_clear
|0 |compute0|0 |0 |1      |0xbd80000000100134|0x0000000002000000|
0x00000000000000086|0x0000000000000001f|Core
DCU:UNCORR_LOAD_POISON_0010_0134
|1 |io0      |-|-|4      |0xfa000000001040c|0x0000000000000000|
0x00000000002310000|0x0000000000000001f|
Ubox:Shutdown_Error:_MCE_when_CR4.MCE_is_clear
|0 |soc      |-|-|MCHN0|0xac00000200a00090|0x0000000002000000|
0x2000002040000086|0x000003cc0ffffe5ff|
MCHN:UNCORR_Uncorrectable_Error_for_Demand_or_Underfill_Reads|
=====
==
...
=====
=====

```

skt die cor thr bank Status				Address		Misc			
Ctl									

1 io0 0 0 4 0xfa0000000001040c 0x0000000000000000 0x0000000002310000 0x0000000000000001f									
=====									
=====									
Decoding Status Register:									
Range Field						Value		Value Decode	
Description									
Description									

63:63 VALID register is valid				0x1	Valid	Indicates that the information within this			
62:62 OVERFLOW occurred while				0x1	Overflow	A '1' indicates that a machine-check error			
						the results of a previous error were still in			
the error-						reporting register bank (the VALID bit was			
already set)									
61:61 UC condition				0x1	UC Detected	A '1' indicates that an uncorrectable the error			
detected.						was			
60:60 EN error				0x1	Enabled	Error reporting enabled. A '1' indicates that			
bit(s) in the						reporting is enabled by the associated flag			
register.						IA32_MCi_CTL			
59:59 MISC content is				0x1	IA32_MCi_MISC	Valid			
read.						If this flag is '1', the IA32_MCi_MISC register			
						valid and can be			
58:58 ADDR content is				0x0	IA32_MCi_ADDR	Not Valid			
						If this flag is '1', the IA32_MCi_ADDR register			
57:57 PCC reliable				0x1	Processor	Context Corrupted			
and a reset						restarting of the processor may not be possible			
recommended.						is			
56:56 S error.				0x0	---	Signaling an uncorrectable recoverable			
55:55 AR error.				0x0	---	Recovery action required for UCR			
54:53 THRESHOLD_BASED_STATUS status				0x0	---	Treshold-based error			
---						0x0			
52:38 CORRECTED_ERROR_COUNT IIO				0x0	---	Corrected error count. Reserved for			
37:37 FW of the MC				0x0	---	Indicates that the BIOS has updated the contents			
Bank									
36:32 OTHER_INFO Architecture						Not part of Machine-Check			
---						0x0			
31:16 MSCOD Code						Model Specific Error			
				0x1		Shutdown Error: MCE when CR4.MCE			

```
is clear
15:0 MCACOD Machine Check Architecture Error
Code 0x40c Shutdown Error
```

4.1.2 Hang.Nested_MCE

Tool: *RAS Automation Toolkit*, document number 633201.

Test case: `.\testscripts\BHS\Debug_Handbook\4.1.2_Hang.Nested_MCE.py`

Inject MCERR when MCIP=1

Steps as shown next:

Make sure MSR (60h) SUPPRESS_SHUTDOWN bit 0 is enabled firstly. Set
`itp.cv.machinecheckbreak = 1`

Enable poison under Shell:

```
>>> halt
[GPC core group] Halt on 128 devices -- 12:06:02.654848 2023-07-13
>>> itp.msr(0x178,1)
>>> itp.cv.machinecheckbreak = 1
Check (msr 17Ah) IA32_MCG_STATUS bit 2 MCIP is '0'
>>> itp.msr(0x17a)
NodeGroup([
  ('GNR0_C0_T0.msr', [64b] 0x0000000000000000),
  ('GNR0_C0_T1.msr', [64b] 0x0000000000000000),
  ('GNR0_C1_T0.msr', [64b] 0x0000000000000000),
  ...
  ('GNR1_C31_T1.msr', [64b] 0x0000000000000000)])
```

Inject MCERR (inject memory UCE, then consumed by memory read)

```
>>> ei.utils.resetInjectorLockCheck()
Initiating an SMI
[GPC core group] Resuming on 128 devices -- 12:10:00.185691 2023-07-13
Injectors LOCKED on GNR0_C0_T0
Injectors LOCKED on GNR0_C0_T1
Injectors LOCKED on GNR0_C1_T0
...
>>> itp.threads[0].mem('0x20000000p',4)
[32b] 0x00000000
>>> ei.mem.injectMemError(0x20000000,errType='uce',immConsume=False)
Halting to do check on injector lock
INFO: BIOS knob 'Disable BIOS Done' is enabled!
Initiating an SMI
[GPC core group] Resuming on 128 devices -- 13:22:37.857599 2023-07-13
Injectors UNLOCKED on GNR0_C0_T0
Injectors UNLOCKED on GNR0_C0_T1
Injectors UNLOCKED on GNR0_C1_T0
Injectors UNLOCKED on GNR0_C1_T1
...
Injectors UNLOCKED on GNR0_C10_T1
[GNR0_C0_T0] SMM entry Break at [0x10:00000000000008000] -- 13:22:48.357084
2023-07-13
[GNR0_C0_T1] SMM entry Break at [0x10:00000000000008000] -- 13:22:48.357084
2023-07-13
[GNR0_C1_T0] SMM entry Break at [0x10:00000000000008000] -- 13:22:48.357084
2023-07-13
...
Exit SMM Mode...
[GPC core group] Halt on 128 devices -- 13:22:55.029006 2023-07-13
*****
```

```

Core Addr_____ = 0x2000_0000 (512.00 M)
Cluster_____ = 0x0
Set Index_____ = 0x0
Tag_____ = 0x1000
Cha_____ = 0
Perspective_____ = FM
*****
FM ddr5 1LM:
Core Addr_____ = 0x2000_0000 (512.00 M)
Ch Addr_____ = 0x2000_0000 (512.00 M)
Sub_Ch Addr_____ = 0x1000_0000 (256.00 M)
pseudo_Ch Addr = 0x800_0000 (128.00 M)
Rank Addr_____ = 0x800_0000 (128.00 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 0x0
Ch_____ = 0x0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 0
Bank in Bank Grp = 1
Bank Addr_____ = 1
Row_____ = 0x800
Column_____ = 0x0
Mirror_____ = 0
Primary_____ = -1
*****
Core Addr_____ = 0x2000_0000 (512.00 M)
Cluster_____ = 0x0
Set Index_____ = 0x0
Tag_____ = 0x1000
Cha_____ = 0
Perspective_____ = FM
*****
FM ddr5 1LM:
Core Addr_____ = 0x2000_0000 (512.00 M)
Ch Addr_____ = 0x2000_0000 (512.00 M)
Sub_Ch Addr_____ = 0x1000_0000 (256.00 M)
pseudo_Ch Addr = 0x800_0000 (128.00 M)
Rank Addr_____ = 0x800_0000 (128.00 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 0x0
Ch_____ = 0x0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 0
Bank in Bank Grp = 1
Bank Addr_____ = 1
Row_____ = 0x800
Column_____ = 0x0
Mirror_____ = 0
Primary_____ = -1
*****
Discovering Populated Mcs
Discovering Populated Channels
Discovering Channels
System in Non-Mirror mode, and error_type: uce
ECC Injection in Non-Mirror Mode, MC=0x0, Channel=0x0, Mirror Scrub Enabled is
True

```

```
socket0 Mc0 Ch0 Patrol Scrub Disabled
socket1 Mc0 Ch0 Patrol Scrub Disabled
socket0 Mc0 Ch0 Patrol Scrub Enabled
socket1 Mc0 Ch0 Patrol Scrub Enabled
Current : dev0 = 5, dev0msk = 0x4444444444444444, dev1 = 9, dev1msk =
0x1111111111111111.
    ret_code:0x0, Dev0:0x5, Dev1:0x9, D0msk:0x4444444444444444,
D1msk:0x1111111111111111, adddc_mask:0x0, perspective:FM
MC 0x0 Channel 0
=====
|                               | Device0          | Device1          |
|-----|-----|-----|
| Injectors Status             | Currently unarmed | Currently unarmed |
|-----|-----|-----|
| Mask                         | 00000000 00000000 | 0001ffff ffffffff |
|-----|-----|-----|
| Match                        | 00080000 04000000 | 00080000 00000000 |
|-----|-----|-----|
| crc_err_inj_dev#_5_bits      | 0x5                | 0x9                |
|-----|-----|-----|
| XOR Mask                     | 0x4444444444444444 | 0x1111111111111111 |
|-----|-----|-----|

Restoring Original Values!!!
>>> itp.threads[80].mem('0x20000000p',4)
[32b] 0x73601000
```

Check MCIP value and mca_err_src_log registers

```
>>> itp.msr(0x17a)
NodeGroup([
  ('GNR0_C0_T0.msr', [64b] 0x0000000000000000),
  ('GNR0_C0_T1.msr', [64b] 0x0000000000000000),
  ...
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is 0...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000001 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
```

```

0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is 0...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000001 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

```

Let the processor go, and check the MCIP value

```

>>> go()
[GPC core group] Resuming on 128 devices -- 13:27:40.826120 2023-07-13
[GNR0_C0_T0] Machine check Break at [0x38:00000000719cf501] --
13:27:56.654058 2023-07-13
[GNR0_C0_T1] Machine check Break at [0x38:00000000719cf501] --
13:27:56.654058 2023-07-13
[GNR0_C1_T0] Machine check Break at [0x38:00000000719cf501] --
13:27:56.654058 2023-07-13
[GNR0_C1_T1] Machine check Break at [0x38:00000000719cf501] --
13:27:56.669684 2023-07-13
...
>>> itp.msr(0x17a)
NodeGroup([
  ('GNR0_C0_T0.msr', [64b] 0x0000000000000001),
  ('GNR0_C0_T1.msr', [64b] 0x0000000000000001),
  ('GNR0_C1_T0.msr', [64b] 0x0000000000000001),
  ('GNR0_C1_T1.msr', [64b] 0x0000000000000001),
  ...
  ('GNR1_C31_T1.msr', [64b] 0x0000000000000001)])

```

Use step command and check MCIP value is '1'

```

>>> itp.threads[80].step()
[GNR0_C0_T0] Resuming -- 13:28:35.950976 2023-07-13
[GNR0_C0_T1] Resuming -- 13:28:35.950976 2023-07-13
[GNR0_C1_T0] Resuming -- 13:28:35.950976 2023-07-13
[GNR0_C1_T1] Resuming -- 13:28:35.950976 2023-07-13
...
>>> halt
[GPC core group] Halt on 128 devices -- 13:31:00.296100 2023-07-13
>>> itp.msr(0x17a)
NodeGroup([
  ('GNR0_C0_T0.msr', [64b] 0x0000000000000005),
  ('GNR0_C0_T1.msr', [64b] 0x0000000000000005),

```

```
('GNR0_C1_T0.msr', [64b] 0x0000000000000005),
...
('GNR1_C31_T1.msr', [64b] 0x0000000000000005]))
```

Inject another MCERR when MCIP =1 (inject memory UCE, then consumed by memory read)

```
>>> ei.utils.resetInjectorLockCheck()
Initiating an SMI
[GPC core group] Resuming on 128 devices -- 13:31:44.310164 2023-07-13
Injectors LOCKED on GNR0_C0_T0
Injectors LOCKED on GNR0_C0_T1
...
Exit SMM Mode...

[GPC core group] Halt on 128 devices -- 13:31:58.747607 2023-07-13
>>> ei.mem.injectMemError(0x10000000,errType='uce',immConsume=False)
Halting to do check on injector lock
INFO: BIOS knob 'Disable BIOS Done' is enabled!
Initiating an SMI
[GPC core group] Resuming on 128 devices -- 13:32:45.044562 2023-07-13
Injectors UNLOCKED on GNR0_C0_T0
Injectors UNLOCKED on GNR0_C0_T1
Injectors UNLOCKED on GNR0_C1_T0
...
Exit SMM Mode...

[GPC core group] Halt on 128 devices -- 13:32:59.716480 2023-07-13
*****
Core Addr_____ = 0x1000_0000 (256.00 M)
Cluster_____ = 0x0
Set Index_____ = 0x0
Tag_____ = 0x800
Cha_____ = 6
Perspective_____ = FM
*****
FM ddr5 l1M:
Core Addr_____ = 0x1000_0000 (256.00 M)
Ch Addr_____ = 0x1000_0000 (256.00 M)
Sub_Ch Addr_____ = 0x800_0000 (128.00 M)
pseudo_Ch Addr_____ = 0x400_0000 (64.00 M)
Rank Addr_____ = 0x400_0000 (64.00 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 0x0
Ch_____ = 0x0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 2
Bank in Bank Grp = 0
Bank Addr_____ = 8
Row_____ = 0x400
Column_____ = 0x0
Mirror_____ = 0
Primary_____ = -1
*****
Core Addr_____ = 0x1000_0000 (256.00 M)
Cluster_____ = 0x0
Set Index_____ = 0x0
Tag_____ = 0x800
Cha_____ = 6
Perspective_____ = FM
*****
```

```

FM ddr5 1LM:
Core Addr_____ = 0x1000_0000 (256.00 M)
Ch Addr_____ = 0x1000_0000 (256.00 M)
Sub_Ch Addr_____ = 0x800_0000 (128.00 M)
pseudo_Ch Addr = 0x400_0000 (64.00 M)
Rank Addr_____ = 0x400_0000 (64.00 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 0x0
Ch_____ = 0x0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 2
Bank in Bank Grp = 0
Bank Addr_____ = 8
Row_____ = 0x400
Column_____ = 0x0
Mirror_____ = 0
Primary_____ = -1
*****
System in Non-Mirror mode, and error type: uce
ECC Injection in Non-Mirror Mode, MC=0x0, Channel=0x0, Mirror Scrub Enabled is
True
socket0 Mc0 Ch0 Patrol Scrub Disabled
socket1 Mc0 Ch0 Patrol Scrub Disabled
socket0 Mc0 Ch0 Patrol Scrub Enabled
socket1 Mc0 Ch0 Patrol Scrub Enabled
Current : dev0 = 5, dev0msk = 0x4444444444444444, dev1 = 9, dev1msk =
0x1111111111111111.
ret_code:0x0, Dev0:0x5, Dev1:0x9, D0msk:0x4444444444444444,
D1msk:0x1111111111111111, adddc_mask:0x0, perspective:FM
MC 0x0 Channel 0
=====
| | Device0 | Device1 |
|-----|-----|-----|
| Injectors Status | Currently unarmed | Currently unarmed |
|-----|-----|-----|
| Mask | 00000000 00000000 | 0001ffff ffffffff |
|-----|-----|-----|
| Match | 00080000 02000000 | 00080000 00000000 |
|-----|-----|-----|
| crc_err_inj_dev#_5_bits | 0x5 | 0x9 |
|-----|-----|-----|
| XOR Mask | 0x4444444444444444 | 0x1111111111111111 |
|-----|-----|-----|

Restoring Original Values!!!
socket0 Mc0 Ch0 Patrol Scrub Enabled
socket1 Mc0 Ch0 Patrol Scrub Enabled
>>> itp.threads[80].mem('0x10000000p',4)
[32b] 0x73601000
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg - 0x00000000
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg - 0x00000000
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0x00000000
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0x00000000

```

Use step command, there is no IERR now

```

>>> itp.threads[0].step()
[GPC core group] Resuming on 128 devices -- 15:27:55.591177 2023-07-13
[GNR0_C0_T0] Machine check Break at [0x38:00000000719d966f] --
15:28:02.075720 2023-07-13

```

```

[GNR0_C0_T1] Machine check Break at [0x38:00000000719cf4e2] --
15:28:02.075720 2023-07-13
[GNR0_C1_T0] Machine check Break at [0x38:00000000719cf4e2] --
15:28:02.075720 2023-07-13
...
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMT# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMT_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMT_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMT#. This is 0...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMT_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMT_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000001 : rmsmi (16:16) -- This socket observed recoverable MSMT
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMT
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMT# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMT_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMT_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMT#. This is 0...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMT_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMT_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMT
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000001 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMT
0x00000000 : first_core_ierr (04:00) -- Reserved
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg = 0x00000000

```

```
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg - 0x00000000
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0x00000000
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0x00000000
```

Use step command again, this time we see IERR occurs.

```
>>> itp.threads[0].step()
[GNR0_C0_T0] Resuming -- 15:31:06.841168 2023-07-13
[GNR0_C0_T1] Resuming -- 15:31:06.841168 2023-07-13
[GNR0_C1_T0] Resuming -- 15:31:06.841168 2023-07-13
[GNR0_C1_T1] Resuming -- 15:31:06.841168 2023-07-13
...
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMT# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMT_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMT_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMT#. This is O...
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMT_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMT_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000001 : rmsmi (16:16) -- This socket observed recoverable MSMT
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMT
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMT# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMT_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMT_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMT#. This is O...
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
```

```
to assert MSMT_IERR.
0x00000001 : msmt merr internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMT_MERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmt (16:16) -- This socket observed recoverable MSMT
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000001 : rmsmt_internal (14:14) -- This socket asserted recoverable MSMT
0x00000000 : first_core_ierr (04:00) -- Reserved

>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg - 0x00000026
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_hi_cfg - 0x00000026
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0x4d2elf64
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.pcu_first_ierr_tsc_lo_cfg - 0x4d2elf44
```

Double check the Crashdum log summary, we can see IERR occurs, and the shutdown error caused by MCIP bit is set. However, there is no 3Strike-Time out nor PCU error happens.

```
##### Signature by Sockets #####
=====
|socket|
Signature
|
-----
|0      |
HW.MCE.MCCHAN:Uncorr_Uncorrectable_Error_For_Demand_Or_Underfill_Reads.incomplete_
dump.mca_same_value|
|1      |
HW.MCE.UBOX:Shutdown_Error:_Mce_When_Mcip_Bit_Is_Set.incomplete_dump.mca_same_valu
e
|
=====

##### Status[63]=1 Machine Check summary #####
=====
==
|skt|die      |cor|thr|bank |Status      |Address      |
Misc      |CTL
Error
|
-----
--
|0 |io0      |-|-|4      |0xfa000000002040c|0x000000000000000|
0x0000000000270000|0x0000000000000001f|
Ubox:Shutdown_Error:_MCE_when_MCIP_bit_is_set
|0 |soc      |-|-|MCHN0|0xec00000200a00090|0x0000000020000000|
0x0800002040000086|0x000003cc0ffffe5ff|
MCHN:UNCORR_Uncorrectable_Error_for_Demand_or_Underfill_Reads|
|1 |io0      |-|-|4      |0xfa000000002040c|0x0000000000000000|
0x000000000008d0000|0x0000000000000001f|
Ubox:Shutdown_Error:_MCE_when_MCIP_bit_is_set
|1 |compute0|8 |0 |1      |0xff80000000100134|0x0000000020000000|
0x0000000000000086|0x0000000000000001f|Core
DCU:UNCORR_LOAD_POISON_0010_0134
|
=====
==
...

##### IERR and MCEERR Logs #####
***cpu1***
```

```

=====
|Die|Register          |Value          |
|Description          |
-----
|io0|pcu_cr_mca_err_src_log|0x001c4000    |MSMI_IERR#
Asserted
| | |
| | |
Asserted
| | |
| | |
Asserted
|io0|ierrloggingreg      |0x348340003482c|Bits [15:0] First Ierr Src Valid --
> Core44 compute0 <-- |
| | |
| | |
|io0|mcerrloggingreg     |0x3482c00034807|Bits [15:0] First MCERR Src Valid --
> Core7 compute0 <-- |
| | |
| | |
a CORE
|io0|uboxerrstatus      |0x00000000
|
|io0|rmcerrloggingreg    |0x3480700034807|Bits [9:0] First rMCERR Src Valid --
> Core7 compute0 <--|
| | |
| | |
a CORE
=====
=====

```

4.1.3 Hang.PCU

4.1.3.1 HW.MCE.PCU.0x20

Let Socket 1 VR SVID DIO signal to abnormal during the boot up.

Error log: socket 1 report IERR with Hang. CPU0.PCU.MSCOD.0x20

```

=====
|socket|
|Signature|
|
-----
|0 |
|INCOMPLETE_DUMP.MCA_ZERO_VALUE|
|
|1 |
|HW.MCE.UBOX:Shutdown_Error:_Mce_When_Cr4.Mce_Is_Clear.incomplete_dump.mca_zero_val|
|ue|
=====

##### Status[63]=1 Machine Check summary #####
=====

|skt|die|cor|thr|bank|Status          |Address          |Misc          |
|CTL|Error|
-----

|1 |io0|- |- |4 |0xfa000000001040c|0x0000000000000000|0x0000000000080000|
0x0000000000000000|Ubox:Shutdown_Error: MCE when CR4.MCE is clear|
|1 |io0|- |- |6 |0xba00000000200402|0x0000000000000000|0x0000002240034040|
0x000000000000000f|Punit:Power_Management_Agent_Signaled_Error|

```

```

=====
...
##### Internal and UBOX Errors per socket #####
First Error Assertion SUMMARY by socket
Legend: 1 means yes
=====
|CPU socket |socket0 io0 |socket0 io1|socket0 compute0|socket1 io0 |
|socket1 io1|socket1 compute0|
=====
|Die |io0 |io1 |compute0 |io0 |
|io1 |compute0 |
|First ierr [16:0] |Core31 compute0|--- |--- |0x6F1 PUNIT
|io0|--- |--- |
|First mcerr [16:0] |--- |--- |--- |
|--- |--- |
|Asserted Caterr# |1 |--- |0 |1
|--- |0 |
|Asserted ierr |1 |--- |0 |1
|--- |0 |
|Asserted mcerr |0 |--- |0 |0
|--- |0 |
|Asserted MSMI# |0 |--- |0 |0
|--- |0 |
|Asserted MSMI_ierr |0 |--- |0 |0
|--- |0 |
|Asserted MSMI_mcerr|0 |--- |0 |0
|--- |0 |
|SideBandTrkTO |0 |--- |--- |0
|--- |--- |
|SideBank_Err |0 |--- |--- |0
|--- |--- |
|SMITimeOut |0 |--- |--- |0
|--- |--- |
|PPIN |0x00000000 |0x00000000 |0x00000000 |0x00000000 |
0x00000000 |0x00000000 |
=====
...

***cpu1***

=====
|Die|Register |Value |
|Description |
=====
|io0|pcu_cr_mca_err_src_log|0x18000000 |CATERR#
|Asserted |
| | |
|Asserted |
|io0|ierrloggingreg |0x34800000106f1|Bits [15:0] First Ierr Src Valid --
> 0x6F1 PUNIT io0 <--|
|io0|mcerrloggingreg |0x00000000 |
| |
|io0|uboxerrstatus |0x00000000 |
| |
|io0|rmcerrloggingreg |0x00000000 |
| |
=====
...
##### Decode each Valid Machine Check #####
*****
*****
=====
|skt|die|cor|thr|bank|Status |Address |Misc |

```

Ctl			

1 io0 0 0 6 0xba0000000200402 0x0000000000000000 0x0000022400340400			
0x000000000000000f			
=====			
Decoding Status Register:			
Range	Field		
Description		Value	Value Decode
Description			

63:63	VALID		Indicates that the information within this
register is valid	0x1		
Valid			
62:62	OVERFLOW		A '1' indicates that a machine-check error
occurred while	0x0	Not	
Overflow			the results of a previous error were still in the
error-			
			reporting register bank (the VALID bit was
already			
set)			
61:61	UC		A '1' indicates that an uncorrectable the error
condition	0x1	UC	
Detected			was
detected.			
60:60	EN		Error reporting enabled. A '1' indicates that
error	0x1		
Enabled			reporting is enabled by the associated flag
bit(s) in			
the			
			IA32_MCi_CTL
register.			
59:59	MISCV		If this flag is '1', the IA32_MCi_MISC register
content is	0x1	IA32_MCi_MISC	
Valid			valid and can be
read.			
58:58	ADDRV		If this flag is '1', the IA32_MCi_ADDR register
content is	0x0	IA32_MCi_ADDR	Not
Valid			valid and can be
read.			
57:57	PCC		Processor Context Corrupt: A '1' indicates that
reliable	0x1	Processor	Context
Corrupted			restarting of the processor may not be possible
and a			
reset			
			is
recommended.			
56:56	S		Signaling an uncorrectable recoverable
error.	0x0		

55:55	AR		Recovery action required for UCR
error.	0x0		

```

---
54:53 THRS_ERR_ST      These bits are used to indicate when the number
of corrected 0x0      No hardware status tracking is provided ...
                        errors has exceeded the safe threshold to the
point where
an
                        uncorrected error has become more likely to
happen.

52:38 CORRECTED_ERROR_COUNT Count of filtered correctable MCA errors
including sticky 0x0
---
                        overflow indicator (bit
52)

37:37 FW_UPD          Indicates that the BIOS has updated the contents
of the MC 0x0
---

Bank

36:32 OTHER_INFO      Available when Enhanced MCA is in
use 0x0
---

31:16 MSCOD           Machine Specific Architecture Error
Code 0x20             Power Management Agent Signaled Error
15:0 MCACOD           Machine Check Architecture Error
Code 0x402           PUnit errors

```

NOTE

For the VR SVID error happens during the very early power on stage, and it is also very possible that cannot get the failed CPU error MCA information.

4.1.3.2 HW.MCE.PCU.0x80

Let RTC_CLR_N signal to abnormal during post, system will hang immediately.

Crashdump: Socket 0 report IERR with Hang. CPU0.PCU.MSCOD.0x80

```

=====
|socket|Signature
=====
|0 |HW.MCE.OTHER:Non-Power_Management_Unit_Generic_Signaled_Error.incomplete_dump.mca_same_value|
|1 |INCOMPLETE_DUMP_MCA_SAME_VALUE
=====

##### Status[63]=1 Machine Check summary #####
=====
|skt|die|cor|thr|bank|Status |Address |Misc |CTL |
Error
=====
|0 |io0|- |6 |0xb20000000800402|0x0000000000000000|0x0000000000000000|0x000000000000000f|Punit:Non-
Power_Management_Unit_Generic_Signaled_Error|
...
##### IERR and MCEERR Logs #####
***cpu0***

|Die|Register |Value |Description |
=====
|io0|pcu_cr_mca_err_src_log|0x181c0000|CATERR# Asserted |
| | | |IERR# Asserted |
| | | |MSMT_IERR# Asserted |
| | | |MSMT_MCERR# Asserted |
|io0|ierrloggingreg|0x000106f1|Bits [15:0] First Ierr Src Valid --> 0x6f1 PUNIT io0 <--|
|io0|mceerrloggingreg|0x000106f1|Bits [15:0] First MCERR Src Valid --> 0x6f1 PUNIT io0 <--|
|io0|uboxerrstatus|0x00000000|
|io0|mceerrloggingreg|0x00000000|
=====
...
##### Decode each Valid Machine Check #####
=====
|skt|die|cor|thr|bank|Status |Address |Misc |Ctl |
=====
|0 |io0|0 |0 |6 |0xb20000000800402|0x0000000000000000|0x0000000000000000|0x000000000000000f|

Decoding Status Register:

```

Range	Field	Description	Value	Value Decode
63:63	VALID	Indicates that the information within this register is valid	0x1	
62:62	OVERFLOW	A '1' indicates that a machine-check error occurred while the results of a previous error were still in the reporting register bank (the VALID bit was already set)	0x0	Not
61:61	UC	A '1' indicates that an uncorrectable the error condition was detected.	0x1	UC
60:60	EN	Error reporting enabled. A '1' indicates that error reporting is enabled by the associated flag bit(s) in the IA32_MCI_CTL register.	0x1	
59:59	MISC_V	If this flag is '1', the IA32_MCI_MISC register content is valid and can be read.	0x0	IA32_MCI_MISC Not
58:58	ADDR_V	If this flag is '1', the IA32_MCI_ADDR register content is valid and can be read.	0x0	IA32_MCI_ADDR Not
57:57	PCC	Processor Context Corrupt: A '1' indicates that reliable restarting of the processor may not be possible and a reset is recommended.	0x1	Processor Context
56:56	S	Signaling an uncorrectable recoverable error.	0x0	
55:55	AR	Recovery action required for UCR error.	0x0	
54:53	THRS_ERR_ST	These bits are used to indicate when the number of corrected errors has exceeded the safe threshold to the point where an uncorrected error has become more likely to happen.	0x0	No hardware status tracking is provided...
52:38	CORRECTED_ERROR_COUNT	Count of filtered correctable MCA errors including sticky overflow indicator (bit 52)	0x0	
37:37	FW_UPD	Indicates that the BIOS has updated the contents of the MC Bank	0x0	
36:32	OTHER_INFO	Available when Enhanced MCA is in use	0x0	
31:16	MSCOD	Machine Specific Architecture Error Code	0x80	Non-Power Management Unit Generic Signaled
15:0	MCACOD	Machine Check Architecture Error Code	0x402	PUnit errors

Check S3M error status :

```
>>>
sv.sockets.io0.uncore.s3m.main_treg.ibl_treg.cr.s3m_fw_err_fatal_status.s3m_misc_fw_err_fatal_sts.show()
0x01102017 : s3m_misc_fw_err_fatal_sts (31:00) -- FW misc fatal error status. If FW reports an error they must also update the S3M_FW_ERR_FATA_STS bit.
0x00000000 : s3m_misc_fw_err_fatal_sts (31:00) -- FW misc fatal error status. If FW reports an error they must also update the S3M_FW_ERR_FATA_STS bit.
```

NOTE

If the RTC_CLR_N signal abnormal during power on before phase 4, the S3M error will be signaled as GLB_RST_WARN_N assertion

Figure 24. S3M Error



4.1.4 Hang.3strike.TOR

4.1.4.1 HW.STUCK_TRANSACTION.TOR: TOR_Timeout.PCIe

4.1.4.1.1 Method 1

Show PCIe* topology firstly to find target PCIe* devices and related root port:

```
>>> pcie.topology()
>>> pcie.topology
No PCIe devices present or active on socket 1

*****
*
*   Socket 0 (A1) PCIe Port Mappings Segment #0, Buses
*   0x00,0x05,0x0a,0x1f,0x32,0x45,0x58,0x6b,0x7e,0x7f
*
*****

=====
=====
==
| PCIe port | config as | width | speed | slot/down | ASPM en | secbusno |
subbusno | linkstate |          Vendor          | Vendor ID | Device ID | RevID |
ASPM en |
-----
```

```

--
|      0a      |      x8      |      |      |      |      |
|      |      |      |      |      |      |
-----
--
|      0e      |      x2      |      |      |      |      |
|      |      |      |      |      |      |
-----
--
|      0f      |      x2      |      |      |      |      |
|      |      |      |      |      |      |
-----
--
|      0g      |      x2      |      x1      |      Gen2      |      slot      |      no      |      Bh      |
Ch      |      Up.L0      |      ASPEED Technology, Inc.      |      1a03h      |      1150h      |      06h      |
no      |      |      |      |      |      |
-----
--
|      0h      |      x2      |      x1      |      Gen2      |      slot      |      L1      |      Dh      |
Dh      |      UpL1.1      |      Renesas Technology Corp.      |      1912h      |      0014h      |      03h      |
L1      |      |      |      |      |      |
-----
--
|      1a      |      x16      |      x16      |      Gen5      |      slot      |      L1      |      20h      |
20h      |      UpL1.1      |      Intel Corporation      |      8086h      |      0d52h      |      01h      |
L1      |      |      |      |      |      |
-----
--
|      4a      |      x4      |      |      |      |      |
|      |      |      |      |      |      |
-----
--
|      4c      |      x4      |      |      |      |      |
|      |      |      |      |      |      |
-----
--
|      4e      |      x4      |      |      |      |      |
|      |      |      |      |      |      |
-----
--
=====
=====

(Some blank port entries are not shown)
Address Decoding Per Root Port...
Port 0g MBAS=          91000000h MLIM=          920FFFFFFh
          IOBAS=          2000h IOLIM=          2FFFh
Port 0h MBAS=          92100000h MLIM=          921FFFFFFh
Port 1a MBAS=          99800000h MLIM=          99AFFFFFFh
          PBAS=          1E3FFC000000h PLIM=          1E3FFF0FFFFFFh

```

Now disable completion timeout at root port side:

```
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.devctl2.ctd.show()
0x00000000 : ctd (04:04) -- 1: Disable the completions timeout mechanism for all
NP transactions.\n0: Completion timeout is enabled for all NP transactions.\n(for
VFs, Switch Ports, RC...
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.devctl2.ctd=1
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.devctl2.ctd
0x1
```

Stop the transaction layer on PCIe* to prevent any new TLP happening at the link layer.

Now access PCIe* device's CFG space:

```
>>> halt
>>> pci.config(0x20,0,0,0x100)
>>>
```

Check mca_err_src_log: which shows socket 0 happens MCERR and IERR

```
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions as a Valid bit for the other two
package conditions. It ...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000001 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000001 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); functions as a Valid bit for the other two
package conditions. It has ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (bit 19,18).
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions as a Valid bit for the other two
package conditions. It ...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
```

```
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); functions as a Valid bit for the other two
package conditions. It has ...
0x00000001 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (bit 19,18).
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved
```

Check ubox.ncevents.mcerrloggingreg: which shows first MCERR srcid is 0x00004e60

```
0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000001 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00004e60 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCERR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00004e60 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR

0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000001 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCERR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00000000 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR
```

Run Crashdump: CPU0 happens TOR timeout on CHA26, the address
is0x0000000082000100

```
##### Status[63]=1 Machine Check summary #####
=====
|skt|die      |cor|thr|bank |Status      |Address      |
Misc          |CTL          |             |
Error         |             |
-----
|0 |soc      |-|-|CHA26|0xfe00000000c1136|0x0000000082000100|
0x1a0083f800100086|0x00000003fffffff|
Unmerged_CHA26:TOR_TIMEOUT_OriginalReq[Prd]_TorID[30]|
=====
```

TOR (Table of Requests) Stuck Valid Entries

=====											
SKT	Compute	CHA	TOR	InUse	Valid	InPipe	OpCode	Fsm	System	Address	
Thread	Core	Die	IP	Target	...	B/D/F	Offset	CL MCA	MSCOD		

0	0	26	30	IN USE	1	0	PRd	Wdata_Wcmp	0x0000000082000100		
0	Core0	-	-	PCIE1	...	20/0/0	100	Unmerged_CHA26:	TOR timeout		
=====											

Decode 0x0000000082000100 : which belongs to PCIe* MMCFG, B:0x20; D:0; F:0, offset:0x100

4.1.4.1.2 Method 2

Get PCIe* topology (same as method 1)

Disable completion timeout

```
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.devctl2.ctd=1
>>> sv.socket0.io0.uncore.pi5.pxp1.rp0.cfg.devctl2.ctd
0x1
```

Stop assert to the transaction layer on PCIe*, preventing the link layer from taking any new TLPs.

```
>>> halt
>>> mem(0x1e3ffc000000)
```

Check punit.mca_err_src_log: which shows socket 0 happens MCERR and IERR

```
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions as a Valid bit for the other t...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000001 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000001 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); functions as a Valid bit for the other two p...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (bit 19,18).
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
```

```

0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions as a Valid bit for the other t...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); functions as a Valid bit for the other two p...
0x00000001 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (bit 19,18).
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

```

Check ubox.ncevents.mcerrloggingreg

```

>>> sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000001 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00004e24 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCeRR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00004e24 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR

0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCeRR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000000 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00000000 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR

```

Run Crashdump to know that socket 0 happens TOR time out on CHA14.Address is 0x1e3ffc000000.

```
##### Status[63]=1 Machine Check summary #####
=====
|skt|die      |cor|thr|bank |Status      |Address      |
Misc      |CTL      |
Error      |
=====
|0 |soc      |-|-|CHA14|0xfe000000000c1136|0x00001e3ffc000000|
0x0e0083f800100086|0x0000003fffffff|
Unmerged_CHA14:TOR_TIMEOUT_OriginalReq[Prd]_TorID[30]|
=====

TOR (Table of Requests) Stuck Valid Entries
=====
|SKT|Compute|CHA|TOR|InUse |Valid|InPipe|OpCode|Fsm      |System Address |
Thread|Core |Die|IP|Target|...|B/D/F |Offset |CL|MCA MSCOD |
=====
|0 |0      |14 |30 |IN USE|1      |0      |PRd      |Wdata_Wcmp|0x00001e3ffc000000|
0      |Core0|-|-|PCIE1 |...|c0/0/0|0      |Unmerged_CHA14: TOR timeout|
=====
```

Checking pci.resource we know that this address 0x1e3ffc000000 is belong to PCIe* device (Bus:0x20; Device: 0; Function: 0)

```
|00|20|00|0|#*00001e3ffc000000-00001e3ffcffffff|*.....-.....|
```

4.1.4.1.3 Method 3

Tool: RAS Automation Toolkit, document number 633201.

Test case: .\testscripts\BHS\Debug_Handbook
 \4.1.4.1_HW.STUCK_TRANSACTION.TOR--TOR_Timeout.PCIe--Method3.py

In this method, PCIe* TOR timeout error is injected with PCIe* Error Injector Card (PEI).

Get PCIe* topology to get PEI card BDF.

```
>>> pcie.topology()
No PCIe devices present or active on socket 1

*****
*
* Socket 0 (A1) PCIe Port Mappings Segment #0, Buses 0x00,0x01
* ,0x02,0x03,0x05,0x06,0x07,0x08,0x0a,0x0b,0x0c,0x0d,0x0f,0x10
* ,0x11,0x12,0x14,0x1e,0x29,0x3a,0x4b,0x5c,0x6d,0x7e,0x7f,0xd8
*
*****

=====
=====
=====
| PCIe port | config as | width | speed | slot/down | ASPM en | secbusno |
subbusno | linkstate | Vendor | Vendor ID | Device ID | RevID
```


Access PEI MMCFG space with CScripts meanwhile:

```
>>> itp.threads[0].mem('0x82a00000p',4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    File "cscripts\dependencies\ipccli\node.py", line 99, in newf
    File "cscripts\dependencies\ipccli\ipc_env\ipc_commands.py", line 926, in mem
    File "cscripts\dependencies\py2ipc\IPC2PyUtils.py", line 106, in newfunc
    File "cscripts\dependencies\py2ipc\Service_OperationReceipt.py", line 170, in
Flush
    File "cscripts\dependencies\py2ipc\IPCTypes.py", line 83, in IPC_ErrorCheck
py2ipc.IPCErrorsCodes.IPC_Error: Jtag_Trigger_Timeout == 0x800d0005
.
[XDPA]Bundle_Execute - Error: Jtag Trigger Timeout occurred (Tclk0:RTI,
Tclk1:RTI). Resetting Jtag. A PRDY# pulse from the core was not seen after
submitting a probemode assembly instruction (PIR/PDR). It is possible the pin is
not wired or was not recovered properly (platform bug). If a PRDY# pin issue is
suspected, please try the following workaround:
ipc.config.gpc.RunControl.WaitInstructionCompletionMode = 'DeadWait' and
ipc.config.gpc.RunControl.WaitInstructionTimeout= '2000'(0x00160014)
```

Run Crashdump to know that socket 0 happens TOR time out on CHA2.Address is0x082a00000 .

```
##### Signature by Sockets #####
=====
|socket|Signature|
=====
|0      |HW.STUCK_TRANSACTION.TOR:Tor Timeout.incomplete_dump.mca_same_value|
|1      |INCOMPLETE_DUMP.MCA_SAME_VALUE|
=====

##### Status[63]=1 Machine Check summary #####
=====
|skt|die    |cor|thr|bank|Status      |Address      |
Misc      |CTL      |
Error      |
=====
|0 |soc      |-  |-  |CHA2|0xfe000000000c1136|0x0000000082a00000|
0x020083f800100086|0x00000003fffffff|
Unmerged_CHA2:TOR_TIMEOUT_OriginalReq[PRd]_TorID[30]|
=====
```

NOTE

Before PEI error injection, make sure PEI environment is ready.

4.1.4.2 HW.STUCK_TRANSACTION.TOR:TOR_Timeout.PCIe

4.1.4.2.1 Method 1

Get PCIe* topology byCscrtpts CMD: pcie.topology()

Stop assert to the transaction layer on PCIe*, preventing the link layer from taking any new TLPs.

Access root port CFG

```
>>> halt
>>> pci.config(0x37,1,0,0x100,4)
>>>
```

Check punit.mca_err_src_log: which shows socket 0 happens MCERR and IERR

```
>>> sv.sockets.uncore.punit.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000001 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000001 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is 0...
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved. SKX b302820.

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); ...
0x00000001 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is 0...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
```

```

0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved. SKX b302820.

```

Check ubox.ncevents.mcerrloggingreg: which shows first MCERR happens on socket 0 CHA0.

```

>>>sv.sockets.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000001 : secondmcerrsrcfromcore (27:27) -- Set to '1' if the SecondMCerrSrcID
is from a C2U
0x00000001 : secondmcerrsrcvalid (26:26) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (25:16) -- Core.CCP.0 -- Logical Port ID of the end
point with the second MCERR
0x00000000 : firstmcerrsrcfromcore (11:11) -- Set to '1' if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (10:10) -- Set to 1 if the FirstSrcID is valid.
0x00000080 : firstmcerrsrcid (09:00) -- DecodingBridge.SCF_CHA.0 -- Logical Port
ID of the end point with the first MCERR

0x00000000 : secondmcerrsrcfromcore (27:27) -- Set to '1' if the SecondMCerrSrcID
is from a C2U
0x00000000 : secondmcerrsrcvalid (26:26) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (25:16) -- Core.CCP.0 -- Logical Port ID of the end
point with the second MCERR
0x00000000 : firstmcerrsrcfromcore (11:11) -- Set to '1' if the FirstMCerrSrcID is
from a C2U
0x00000000 : firstmcerrsrcvalid (10:10) -- Set to 1 if the FirstSrcID is valid.
0x00000000 : firstmcerrsrcid (09:00) -- Core.CCP.0 -- Logical Port ID of the end
point with the first MCERR

```

Run Crashdump: CPU0 happens TOR timeout on CHA2, the address is 0x0000000081f10100

```

##### Status[63]=1 Machine Check summary #####
=====
|skt|die      |cor|thr|bank|Status      |Address      |
Misc      |CTL      |
Error      |

-----
|0 |soc      |-  |-  |CHA2|0xfe00000000c1136|0x0000000081f10100|
0x020083f800100086|0x0000003fffffff|
Unmerged_CHA2:TOR_TIMEOUT_OriginalReq[PRd]_TorID[30]|
|1 |io0      |-  |-  |4  |0xfa000000002040c|0x0000000000000000|
0x00000000008b0000|0x0000000000000001f|
Ubox:Shutdown_Error: MCE when MCIP_bit_is_set      |
=====

TOR (Table of Requests) Stuck Valid Entries
=====
|SKT|Compute|CHA|TOR|InUse |Valid|InPipe|OpCode|Fsm      |System Address  |
Thread|Core  |Die|IP|Target|...|B/D/F |Offset CL|MCA MSCOD  |
|0 |0     |2  |30 |IN USE|1   |0     |PRd    |Wdata Wcmp|0x0000000081f10100|
0    |Core0 |-  |-  |PCIE1 |...|1f/2/0|100    |Unmerged_CHA2: TOR timeout|
=====

```

Decode TOR Timeout address '0x0000000081f10100': it is B:0x1f; D:0x2; F:0 which belong to socket 0's PCIe* Root Port 1 according to PCIe* device information.

4.1.4.3 HW.STUCK_TRANSACTION.TOR:TOR_Timeout.MEM

4.1.4.3.1 Method 1

Tool: *RAS Automation Toolkit*, document number 633201.

Test case: `.\testscripts\BHS\Debug_Handbook
4.1.4.3_HW.STUCK_TRANSACTION.TOR--TOR_Timeout.MEM--Method1.py`

Run Intel® Memory Latency Checker (Intel® MLC) tool with reduced TOR Timeout threshold, Intel® MLC tool command example:

```
[root@localhost Linux]# ./mlc --bandwidth_matrix
```

An TOR timeout threshold setting example:

```
>>> pci.config(0x7e,0x0d,0,0x2d0)
0x1fffffff
>>> pci.config(0x7e,0x0d,0,0x2d0,4,0xfffffffffa)
>>> pci.config(0x7e,0x0d,0,0x2d0)
0xfffffffffa
>>> pci.config(0x7e,0x0d,0,0x2d0,4,0x5294a53)
>>> pci.config(0x7e,0x0d,0,0x2d0)
0x5294a53
>>> sv.sockets.compute0.uncore.cha.cha0.sad.qpi_timeout_ctrl_cfg
socket0.compute0.uncore.cha.cha0.sad.qpi_timeout_ctrl_cfg - 0x05294a53
...
socket0.compute1.uncore.cha.cha115.sad.qpi_timeout_ctrl_cfg - 0x05294a53
socket1.compute0.uncore.cha.cha7.sad.qpi_timeout_ctrl_cfg - 0xfffffffffb
...
socket1.compute1.uncore.cha.cha119.sad.qpi_timeout_ctrl_cfg - 0xfffffffffb
```

Check `ncevents.mcerrloggingreg`

```
>>> sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (47:32) -- Core.CCP.0 -- Physical Port ID of the
end point with the second MCErr
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000000 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00000000 : firstmcerrsrcid (15:00) -- Core.CCP.0 -- Physical Port ID of the end
point with the first MCErr

0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000001 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00004d5c : secondmcerrsrcid (47:32) -- Unknown Decode -- Physical Port ID of
the end point with the second MCErr
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00004de8 : firstmcerrsrcid (15:00) -- Unknown Decode -- Physical Port ID of the
end point with the first MCErr
```

Check punit.mca_err_src_log

```
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is O...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is O...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved
```

Run Crashdump: you can see that there is only CHA TOR Timeout but without Intel® MLC 3Strike Timeout. Socket 0's CHA0 TOR Timeout OriginalReq is neither lock nor IO type.

```
##### Signature by Sockets #####
=====
|socket|Signature|
=====
|0|INCOMPLETE_DUMP.MCA_SAME_VALUE|
|1|HW.STUCK_TRANSACTION.TOR:Tor_Timeout.incomplete_dump.mca_same_value|
=====

##### Status[63]=1 Machine Check summary #####
=====
|skt|die|cor|thr|bank|Status|Address|
Misc|CTL|
Error|
-----
|1|soc|-|-|CHA4|0xfe00000000c1136|0x000000016f010940|
0x043c810c00100886|0x00000003ffffffff|
Unmerged_CHA4:TOR_TIMEOUT_OriginalReq[DRd]_TorID[3]|
|1|soc|-|-|CHA5|0xbe00000000c1136|0x000000017feb8e00|
0x053c811400100886|0x00000003ffffffff|
Unmerged_CHA5:TOR_TIMEOUT_OriginalReq[DRd]_TorID[5]|
=====
=====
```

Decode TOR Timeout address " 0x000000016f010940": It belongs to a memory address.

```
*****
Core Addr_____ = 0x1_6f01_0940 (5.73 G)
Cluster_____ = 0x0
Set Index_____ = 0x425
Tag_____ = 0xb780
Cha_____ = 4
Perspective_____ = FM
*****
FM ddr5 1LM:
Core Addr_____ = 0x1_6f01_0940 (5.73 G)
Ch Addr_____ = 0xef01_0940 (3.73 G)
Sub_Ch Addr_____ = 0x7780_84c0 (1.87 G)
pseudo_Ch Addr = 0x3bc0_4240 (956.02 M)
Rank Addr_____ = 0x3bc0_4240 (956.02 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 0x0
Ch_____ = 0x0
Sub-Ch_____ = 0
pseudo-Ch_____ = 1
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 5
Bank in Bank Grp = 3
Bank Addr_____ = 23
Row_____ = 0x3bc1
Column_____ = 0x50
Mirror_____ = 0
Primary_____ = -1
*****
```

NOTE

For the TOR Timeout without IMC or B2CMIError or Intel® MLC 3Strike-Timeout error case, try to disable CHA TOR Timeout and try to see if can make the issue go away.

4.1.4.3.2 Method 2

Disable B2CMI Timeout (Granite Rapids EDS2 Time Out [TIMEOUT] - Offset 3D0h)

```
>>> sv.sockets.computes.uncore.memss.b2cmis.timeout.timeouten
socket0.soc.memss.b2cmi0.timeout.timeouten - 0x00000000
socket1.soc.memss.b2cmi0.timeout.timeouten - 0x00000000
```

Set one specific mesh2mem traffic blocked to its connected memory controllers (Granite Rapids EDS2 B2CMIBIOSQUIESCE - Offset 2D4h)

Access memory

```
>>> itp.threads[0].mem("0x3c44000000p",4)
```

Check mcerrloggingreg and mca_err_src_log register

```
>>> sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000001 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000001 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00004800 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCERR
0x00000001 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00004800 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR

0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000001 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x000026f1 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCERR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x000006f1 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions as a Valid bit for the other two
package conditions. It has no effect when a local core is assoc...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000001 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000001 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000001 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
```

```

error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); functions as a Valid bit for the other two
package conditions. It has no effect when a local core is associate...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (bit 19,18).
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions as a Valid bit for the other two
package conditions. It has no effect when a local core is assoc...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000001 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000001 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); functions as a Valid bit for the other two
package conditions. It has no effect when a local core is associate...
0x00000001 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (bit 19,18).
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

```

Run Crashtump: You can find the corresponding memory address CHA TOR Timeout.

```

=====
|skt|die      |cor|thr|bank |Status          |Address          |
Misc          |CTL          |                  |                  |
Error          |              |                  |                  |
-----
...
|0 |soc      |- |- |CHA0 |0xfe000000000c1136|0x000003c440000080|
0x003c817800100886|0x0000003fffffffff|
Unmerged_CHA0:TOR_TIMEOUT_OriginalReq[DRd]_TorID[30] |
|0 |soc      |- |- |CHA1 |0xfe000000000c1136|0x000003c4400000c0|
0x013c817800100886|0x0000003fffffffff|
Unmerged_CHA1:TOR_TIMEOUT_OriginalReq[DRd]_TorID[30] |

```

```
|0 |soc |- |- |CHA2 |0xfe000000000c1136|0x000003c440000000|
0x023c817800100886|0x0000003fffffffff|
Unmerged_CHA2:TOR_TIMEOUT_OriginalReq[DRd]_TorID[30]
|0 |soc |- |- |CHA3 |0xfe000000000c1136|0x000003c440000040|
0x033c817800100886|0x0000003fffffffff|
Unmerged_CHA3:TOR_TIMEOUT_OriginalReq[DRd]_TorID[30]
|
...

```

4.1.5 Hang.3strike_Only

TAG: HW.STUCK_TRANSACTION.CORE_3STRIKE_ONLY (for Granite Rapids)

TAG: HW.MCE.BUS:Uncorr_Rob_Timeout (for Intel® Xeon® 6900/6700-Series Processors with E-Cores)

One method to inject a 3Strike Only error is to disable TOR Timeout, B2CMI Timeout and set mesh2mem traffic blocked while accessing memory. All the clues could be referred in HW.STUCK_TRANSACTION.TOR:Tor_Timeout.MEM. Next is a brief recipe.

Disable TOR Timeout

```
sv.sockets.computes.uncore.cha.chas.sad.qpi_timeout_ctrl_cfg.enabletortimeout
```

Disable B2CMI Timeout

```
sv.sockets.computes.uncore.memss.b2cmis.timeout.timeouten
```

Set B2CMI traffic blocked

Access memory

```
>>> itp.threads[0].mem("0x3c480000000p",4)
```

Check mca_err_src_log register

```
>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000001 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#.
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000001 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#.
```

```

0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason).
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000001 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#.
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000001 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason).
0x00000001 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#.
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

```

For Granite Rapids:

Run Crashdump, it shows STUCK_TRANSACTION.CORE_3STRIKE_ONLY error, error is logged in core0 MLC bank.

```

##### Signature by Sockets #####
|socket|Signature|
|0|HW.STUCK_TRANSACTION.CORE_3STRIKE_ONLY.incomplete_dump.mca_same_value|
|1|INCOMPLETE_DUMP.MCA_SAME_VALUE|

##### Status[63]=1 Machine Check summary #####
|skt|die|cor|thr|bank|Status|Address|
Misc|CTL|
Error|
|0|compute0|6|0|0|3|0xbe000000e1cd0400|0x0000000000008000|
0x0000000078022000|0x000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xelcd_0x0400|
|0|compute0|8|0|0|3|0xbe000000e1cd0400|0x0000000000008000|
0x000000007803a000|0x000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xelcd_0x0400|
|0|compute0|20|0|0|3|0xbe000000e1cd0400|0x0000000000008000|
0x0000000078026000|0x000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xelcd_0x0400|
|0|compute0|22|0|0|3|0xbe000000e1cd0400|0x0000000000008000|
0x000000007803e000|0x000000000000007f|Core

```

```
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xe1cd_0x0400|
|0 |compute0|28 |0 |3 |0xbe000000e1cd0400|0x0000000000008000|
0x0000000078036000|0x000000000000007f|Core
MLC:WATCHDOG_TIMER_3_STRIKE_ERROR_0xe1cd_0x0400|
```

For Intel® Xeon® 6900/6700-Series Processors with E-Cores: Run Ccrashdump, it shows MCE.BUS:Uncorr_Rob_Timeout error, error is logged in BUS bank.

```
##### Signature by Sockets #####
|socket|Signature|
-----|
|0 |HW.MCE.BUS:Uncorr_Rob_Timeout.incomplete_dump.mca_same_value|
|1 |HW.MCE.BUS:Uncorr_Rob_Timeout.incomplete_dump.mca_same_value|
=====

##### Status[63]=1 Machine Check summary #####
=====
|skt|die |module |core|bank|Status |Address |
Misc |CTL |Error |
-----|
|0 |compute0|module1 |1 |0 |0xf200000000000400|0x0000000000000000|
0x0000000000000000|0x0000000000000000|Core BUS:UNCORR_ROB_Timeout|
|0 |compute0|module1 |2 |0 |0xf200000000000400|0x0000000000000000|
0x0000000000000000|0x0000000000000000|Core BUS:UNCORR_ROB_Timeout|
|0 |compute0|module1 |3 |0 |0xf200000000000400|0x0000000000000000|
0x0000000000000000|0x0000000000000000|Core BUS:UNCORR_ROB_Timeout|
|0 |compute0|module3 |0 |0 |0xf200000000000400|0x0000000000000000|
0x0000000000000000|0x0000000000000000|Core BUS:UNCORR_ROB_Timeout|
|0 |compute0|module3 |1 |0 |0xf200000000000400|0x0000000000000000|
0x0000000000000000|0x0000000000000000|Core BUS:UNCORR_ROB_Timeout|
|0 |compute0|module3 |2 |0 |0xf200000000000400|0x0000000000000000|
0x0000000000000000|0x0000000000000000|Core BUS:UNCORR_ROB_Timeout|
```

4.2 MCE Error Injection

4.2.1 HW.MCE.IFU

4.2.1.1 HW.MCE.IFU.0x150

Tool: *RAS Automation Toolkit*, document number 633201

Test case: BHS/Debug_Handbook/4.2.1.1_HW.MCE.IFU.0x150.py

Set machinecheckentrybreak = 1

Check poison is enabled

```
>>>
halt

>>>

[GPC core group] Halt on 128 devices -- 13:00:21.779749 2023-07-28
>>> itp.msr(0x178,1)
>>> itp.msr(0x178)
NodeGroup([
('GNR0_C0_T0.msr', [64b] 0x0000000000000001),
```

```
( 'GNR0_C0_T1.msr', [64b] 0x0000000000000001),
.....
( 'GNR1_C31_T0.msr', [64b] 0x0000000000000001),
( 'GNR1_C31_T1.msr', [64b] 0x0000000000000001) ])
```

Set first enabled thread RIP and write "jmp \$" to the location

```
>>>itp.threads[0].state.regs.rip=0x10000000
>>>itp.threads[0].asm("$","jmp $","jmp $")
>>>itp.threads[0].asm("$",0x5)
0x38:0000000010000000 ebfe jmp $+0x00 ;a=0x10000000
0x38:0000000010000002 ebfe jmp $+0x00 ;a=0x10000002
0x38:0000000010000004 0000 add byte ptr [rax], al
0x38:0000000010000006 0000 add byte ptr [rax], al
0x38:0000000010000008 0000 add byte ptr [rax], al
```

Inject Memory UCE error at 0x10000000 with immConsume = 0

```
>>> ei.mem.injectMemError(0x10000000,errType = 'uce',immConsume = 0)
Halting to do check on injector lock
INFO: BIOS knob 'Disable BIOS Done' is enabled!
Initiating an SMI

[GPC core group] Resuming on 128 devices -- 13:01:13.342759 2023-07-28
Injectors UNLOCKED on GNR0_C0_T0
Injectors UNLOCKED on GNR0_C0_T1
...
Injectors UNLOCKED on GNR0_C7_T0

[GNR0_C0_T0] SMM entry Break at [0x10:00000000000008000] -- 13:01:24.764189 2023-07-28
[GNR0_C0_T1] SMM entry Break at [0x10:00000000000008000] -- 13:01:24.764189 2023-07-28
...
[GNR1_C31_T0] SMM entry Break at [0x10:00000000000008000] -- 13:01:24.857916 2023-07-28
[GNR1_C31_T1] SMM entry Break at [0x10:00000000000008000] -- 13:01:24.857916 2023-07-28
Injectors UNLOCKED on GNR0_C7_T1
Injectors UNLOCKED on GNR0_C8_T0
...
Injectors UNLOCKED on GNR1_C31_T0
Injectors UNLOCKED on GNR1_C31_T1

[GNR0_C0_T0] Resuming -- 13:01:28.764158 2023-07-28
[GNR0_C0_T1] Resuming -- 13:01:28.779780 2023-07-28
[GNR0_C1_T0] Resuming -- 13:01:28.779780 2023-07-28
...
[GNR1_C31_T0] Resuming -- 13:01:28.842266 2023-07-28
[GNR1_C31_T1] Resuming -- 13:01:28.842266 2023-07-28
Exit SMM Mode...

[GPC core group] Halt on 128 devices -- 13:01:32.404779 2023-07-28
*****
Core Addr = 0x1000_0000 (256.00 M)
Cluster = 0x0
Set Index = 0x0
Tag = 0x800
Cha = 6
Perspective = FM
*****
FM ddr5 1LM:
PSA Addr = 0x1000_0000 (256.00 M)
Ch Addr = 0x1000_0000 (256.00 M)
Sub_Ch Addr = 0x800_0000 (128.00 M)
pseudo_Ch Addr = 0x400_0000 (64.00 M)
Socket = 0
Hbm = -1
Mc = 0x0
Ch = 0x0
Sub-Ch = 0
pseudo-Ch = 0
Mirror = 0
Primary = -1
*****
Discovering Populated Mcs
Discovering Populated Channels
Discovering Channels
Skipping MC1 due to it being inaccessible/unpowered.
Skipping MC6 due to it being inaccessible/unpowered.
Skipping MC7 due to it being inaccessible/unpowered.
Skipping MC1 due to it being inaccessible/unpowered.
Skipping MC6 due to it being inaccessible/unpowered.
Skipping MC7 due to it being inaccessible/unpowered.
System in Non-Mirror mode, and error type: uce
ECC Injection in Non-Mirror Mode, MC=0x0, Channel=0x0, Mirror Scrub Enabled is True
socket0 Mc0 Ch0 Patrol Scrub Disabled
socket1 Mc0 Ch0 Patrol Scrub Disabled
socket0 Mc0 Ch0 Patrol Scrub Enabled
socket1 Mc0 Ch0 Patrol Scrub Enabled
Current : dev0 = 5, dev0msk = 0x4444444444444444, dev1 = 9, dev1msk = 0x1111111111111111.
MC 0x0 Channel 0
=====
| Device0 | Device1 | |
|---|---|---|
| Injectors Status | Currently unarmed | Currently unarmed |
|-----|-----|
| Mask | 00000000 00000007 | 0001ffff ffffffff |
|-----|-----|
| Match | 00080000 02000000 | 00080000 00000000 |
|-----|-----|
| crc_err_inj_dev# 5_bits | 0x5 | 0x9 |
|-----|-----|
| XOR Mask | 0x4444444444444444 | 0x1111111111111111 |
|-----|-----|
Restoring Original Values!!!
```

Check register punit.ptpcfsms.ptpcfsms.mca_err_src_log: no error yet

```
>>> sv.sockets.ios.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00000000
socket0.io1.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00000000
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00000000
socket1.io1.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00000000
>>> sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1' if the SecondrMCErrSrcID is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to '1' if the SecondrMCErrSrcID is valid
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with the second rMCErr
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1' if the FirstrMCErrSrcID is from a C2U
0x00000000 : firstmcerrsrcvalid (16:16) -- Set to '1' if the FirstrMCErrSrcID is valid
0x00000000 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with the first rMCErr
>>> error.check_machine_check_errors()

Checking errors in the following IPs:
machine_check

Please be patient...this operation might take several minutes.

Uncore MCA from Socket: 0
Socket 0 : No errors logged in any Uncore-specific Machine Check Banks

Core MCA from Socket: 0
Socket 0 : No errors logged in any Core-specific Machine Check Banks

Uncore MCA from Socket: 1
Socket 1 : No errors logged in any Uncore-specific Machine Check Banks

Core MCA from Socket: 1
Socket 1 : No errors logged in any Core-specific Machine Check Banks

*****
                        No errors found
*****

The Error registers Dump CSV and JSON files are located in: C:\Users\Administrator\PythonSv
```

Let system go and check those registers again:

```
>>> go
>>>
[GPC core group] Resuming on 128 devices -- 22:09:08.259977 2023-06-25
>>> sv.sockets.ios.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00004000
socket0.io1.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00000000
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00010000
socket1.io1.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00000000
>>>
[GNR0_C0_T0] SMM entry Break at [0x10:0000000000008000] -- 13:04:01.045939 2023-07-28
[GNR0_C0_T1] SMM entry Break at [0x10:0000000000008000] -- 13:04:01.045939 2023-07-28
...
[GNR1_C31_T0] SMM entry Break at [0x10:0000000000008000] -- 13:04:01.139344 2023-07-28
[GNR1_C31_T1] SMM entry Break at [0x10:0000000000008000] -- 13:04:01.139344 2023-07-28
>>> sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000001 : secondmcerrsrcfromcore (49:49) -- Set to '1' if the SecondrMCErrSrcID is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to '1' if the SecondrMCErrSrcID is valid
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with the second rMCErr
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1' if the FirstrMCErrSrcID is from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to '1' if the FirstrMCErrSrcID is valid
0x00000001 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with the first rMCErr
0x00004000 : secondmcerrsrcfromcore (49:49) -- Set to '1' if the SecondrMCErrSrcID is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to '1' if the SecondrMCErrSrcID is valid
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with the second rMCErr
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1' if the FirstrMCErrSrcID is from a C2U
0x00000000 : firstmcerrsrcvalid (16:16) -- Set to '1' if the FirstrMCErrSrcID is valid
0x00000000 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with the first rMCErr
```

Run error.check_machine_check_errors:

For Granite Rapids:

```
>>> error.check_machine_check_errors()
Checking errors in the following IPs:
machine_check

Please be patient...this operation might take several minutes.

Uncore MCA from Socket: 0
=====
| Valid | MC Bank Name | Status | Addr | Misc | Ctl1 | Ctl2 |
|-----|-----|-----|-----|-----|-----|-----|
| * | BANK #13 - MC CH0 merged | 0xa00000200a00090 | 0x0000000010000000 | 0x0800010020000086 | 0x00003cc0ffffe5ff | 0x0000000500000000 |
|-----|-----|-----|-----|-----|-----|-----|

Core MCA from Socket: 0
=====
| Valid | Core | Thread | MC Bank Name | Status | Addr | Misc | Ctl1 | Ctl2 |
|-----|-----|-----|-----|-----|-----|-----|-----|
| * | core0 | thread0 | BANK #0 - IFU | 0xbd80000000c0150 | 0x0000000010000000 | 0x0000000000000086 | 0x00000000000001fff | 0x0000000500000000 |
|-----|-----|-----|-----|-----|-----|-----|-----|

Uncore MCA from Socket: 1
Socket 1 : No errors logged in any Uncore-specific Machine Check Banks
```

```

Core MCA from Socket: 1
Socket 1 : No errors logged in any Core-specific Machine Check Banks

Decoding Machine Checks...

Machine Check Decoder Summary

=====
Uncore MCA from Socket: 0
=====
| MC Bank Name | Status | Addr | Misc | Ctl | Ctl2 |
| BANK #13 - MC CH0 merged | 0xac00000200a00090 | 0x0000000010000000 | 0x0800010020000086 | 0x00003cc0ffffe5ff | 0x0000000050000000 |
=====

MC13 not part of defined merged nor unmerged banks to decode...

=====

Core MCA from Socket: 0
=====
| Core | Thread | MC Bank Name | Status | Addr | Misc | Ctl | Ctl2 |
| core0 | thread0 | BANK #0 - IFU | 0xbd800000000c0150 | 0x0000000010000000 | 0x0000000000000086 | 0x00000000000001ff | 0x0000000050000000 |
=====

Bank name: Core IFU, Error = UNCORR IFU_POISON_ERROR_000C_0150

+ MC0_STATUS 0xbd800000000c0150

Range Field Description Value Value Description
-----
63:63 VALID Indicates that the information within this register is valid 0x1 Valid
62:62 OVERFLOW A '1' indicates that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (the VALID bit was already set) 0x0 Not Overflow
61:61 UC A '1' indicates that an uncorrectable the error condition was detected. 0x1 UC Detected
60:60 EN Error reporting enabled. A '1' indicates that error reporting is enabled by the associated flag bit(s) in the IA32_MCI_CTL register. 0x1 Enabled
59:59 MISCV If this flag is '1', the IA32_MCI_MISC register content is valid and can be read. 0x1 IA32_MCI_MISC Valid
58:58 ADDRv If this flag is '1', the IA32_MCI_ADDR register content is valid and can be read. 0x1 IA32_MCI_ADDR Valid
57:57 PCC Processor Context Corrupt: A '1' indicates that reliable restarting of the processor may not be possible and a reset is recommended. 0x0 Processor Context Not Corrupted
56:56 S Signaling an uncorrectable recoverable error. 0x1 ---
55:55 AR Recovery action required for UCR error. 0x1 ---
54:53 THRESHOLD_BASED_STATUS Reserved 0x0 ---
52:38 CORRECTED_ERR_CNT Corrected error count since last clear of machine check registers 0x0 ---
37:37 FW Firmware update error status indicator 0x0 ---
36:32 OTHER_INFO Not part of Machine-Check Architecture 0x0 ---
31:16 MSCOD Model Specific Error Code 0xc IFU_POISON_ERROR_000C_0150
15:0 MCACOD Machine Check Architecture Error Code 0x150 IFU Icache Data Parity/Data Posion/Tag Parity Error

+ MC0_MISC 0x0000000000000086

Range Field Description Value Value Description
-----
63:9 MODEL_SPECIFIC_INFORMATION Model-Specific Information 0x0 ---
8:6 ADDRESS_MODE Address mode (physical, linear, segment offset, memory, generic) 0x2 ---
5:0 RECOVERABLE_ADDR_LSB Indicates the position of the least significant bit of the recoverable error address (IA32_MCI_ADDR should be ignored) 0x6 ---

+ MC0_ADDRESS 0x0000000010000000

Range Field Description Value Value Description
-----
63:0 ADDR Contains address of code or data memory location that produce the machine check error 0x10000000 ---

The MC decoding log file is located in:
C:\Users\Administrator\PythonSv\Machine_Check_Decoder_Summary_Fri_Jul_28_13_04_37_2023.txt

*****
Fields with non default value
*****

=====
| IP block | # fields with non-default value |
|-----|-----|
| MACHINE_CHECK | 0x2 |
|-----|-----|

*****

The Error registers Dump CSV and JSON files are located in: C:\Users\Administrator\PythonSv

```

For Intel® Xeon® 6900/6700-Series Processors with E-Cores:

```
>>> error.check_machine_check_errors()
Checking errors in the following IPs:
machine_check

Please be patient...this operation might take several minutes.
Collecting socket0 io0 Die
Collecting socket0 io1 Diebank MC6 MC6_CTL2
Collecting socket0 soc0 Diebank MC6 MC6_CTL2
Collecting socket1 io0 Diebank MCHN7 bank mchn_MISC2
Collecting socket1 io1 Diebank UPI1 bank upi_CTL2
Collecting socket1 soc0 Diebank UPI1 bank upi_CTL2
\ Collecting Socket1 Mchbank MCHN7 bank mchn_MISC2

#####
Socket: 0 io0

Uncore MCA from Socket: 0
Socket 0 : No errors logged in any Uncore-specific Machine Check Banks

#####
Socket: 0 io1

Uncore MCA from Socket: 0
Socket 0 : No errors logged in any Uncore-specific Machine Check Banks

#####
Socket: 0 soc

Uncore MCA from Socket: 0
=====
| Valid | DIE | MC Bank Name | Status | Addr | Misc | Ctl | Ctl2 |
|-----|-----|-----|-----|-----|-----|-----|-----|
| * | soc0 | MCHN0 | 0xac00004200a00090 | 0x10000000 | 0x800008018000086 | 0x3cc0ffffe5ff | 0x500000000 |
=====

Core compute0-module1 MCA from Socket: 0
=====
| Valid | Die | Module | Core | MC Bank Name | Status | Addr | Misc | Ctl | Ctl2 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| * | X | compute0-module1 | core0 | BANK #2 - FEC | 0xbd800000010000150 | 0x00000000010000000 | 0x0000000000000040 | 0x0000000000000000 | 0x00000000500000000 |
=====

#####
Socket: 1 io0

Uncore MCA from Socket: 1
Socket 1 : No errors logged in any Uncore-specific Machine Check Banks

#####
Socket: 1 io1

Uncore MCA from Socket: 1
Socket 1 : No errors logged in any Uncore-specific Machine Check Banks

#####
Socket: 1 soc

Uncore MCA from Socket: 1
Socket 1 : No errors logged in any Uncore-specific Machine Check Banks

Socket 1 : No errors logged in any Core-specific Machine Check Banks

*****
Fields with non default value
*****

=====
| IP block | # fields with non-default value |
|-----|-----|
| MACHINE_CHECK | 0x2 |
|-----|-----|

*****

The Error registers Dump CSV and JSON files are located in: C:\Users\Administrator\PythonSv
```

4.2.2 HW.MCE.DCU

4.2.2.1 HW.MCE.DCU.0x134

Tool: *RAS Automation Toolkit*, document number 633201

Test case: BHS/Debug_Handbook/4.2.1.1_HW.MCE.IFU.0x150.py

Check poison is enabled

```
>>>
halt

>>>
[GPC core group] Halt on 128 devices -- 13:35:01.484178 2023-07-28
>>> itp.msr(0x178,1)
>>> itp.msr(0x178)
NodeGroup([
  ('GNR0_C0_T0.msr', [64b] 0x0000000000000001),
  ('GNR0_C0_T1.msr', [64b] 0x0000000000000001),
  ...
  ('GNR1_C31_T0.msr', [64b] 0x0000000000000001),
  ('GNR1_C31_T1.msr', [64b] 0x0000000000000001)])
```

Inject Memory UCE error at 0x10000000 with immConsume = 0

```
>>> ei.mem.injectMemError(0x10000000,errType = 'uce',immConsume = 0)
Halting to do check on injector lock
INFO: BIOS knob 'Disable BIOS Done' is enabled!
Initiating an SMI

[GPC core group] Resuming on 128 devices -- 13:36:56.390466 2023-07-28
Injectors UNLOCKED on GNR0_C0_T0
Injectors UNLOCKED on GNR0_C0_T1
Injectors UNLOCKED on GNR0_C1_T0
...
[GNR0_C0_T0] SMM entry Break at [0x10:0000000000008000] -- 13:37:07.281098
2023-07-28
[GNR0_C0_T1] SMM entry Break at [0x10:0000000000008000] -- 13:37:07.281098
2023-07-28
[GNR0_C1_T0] SMM entry Break at [0x10:0000000000008000] -- 13:37:07.281098
2023-07-28
...
[GNR1_C31_T0] SMM entry Break at [0x10:0000000000008000] -- 13:37:07.343584
2023-07-28
[GNR1_C31_T1] SMM entry Break at [0x10:0000000000008000] -- 13:37:07.343584
2023-07-28
Injectors UNLOCKED on GNR0_C8_T0
Injectors UNLOCKED on GNR0_C8_T1
Injectors UNLOCKED on GNR0_C9_T0
...
Injectors UNLOCKED on GNR1_C31_T0
Injectors UNLOCKED on GNR1_C31_T1

[GNR0_C0_T0] Resuming -- 13:37:11.359225 2023-07-28
[GNR0_C0_T1] Resuming -- 13:37:11.359225 2023-07-28
...
[GNR1_C31_T0] Resuming -- 13:37:11.406090 2023-07-28
[GNR1_C31_T1] Resuming -- 13:37:11.406090 2023-07-28
Exit SMM Mode...

[GPC core group] Halt on 128 devices -- 13:37:14.999845 2023-07-28
*****
Core Addr_____ = 0x1000_0000 (256.00 M)
Cluster_____ = 0x0
Set Index_____ = 0x0
Tag_____ = 0x800
Cha_____ = 6
Perspective_____ = FM
*****
FM ddr5 1LM:
PSA Addr_____ = 0x1000_0000 (256.00 M)
Ch Addr_____ = 0x1000_0000 (256.00 M)
Sub_Ch Addr_____ = 0x800_0000 (128.00 M)
pseudo_Ch Addr = 0x400_0000 (64.00 M)
```

```

Socket_____ = 0
Hbm_____ = -1
Mc_____ = 0x0
Ch_____ = 0x0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Mirror_____ = 0
Primary_____ = -1
*****
Discovering Populated Mcs
Discovering Populated Channels
Discovering Channels
Skipping MC1 due to it being inaccessible/unpowered.
Skipping MC6 due to it being inaccessible/unpowered.
Skipping MC7 due to it being inaccessible/unpowered.
Skipping MC1 due to it being inaccessible/unpowered.
Skipping MC6 due to it being inaccessible/unpowered.
Skipping MC7 due to it being inaccessible/unpowered.
System in Non-Mirror mode, and error_type: uce
ECC Injection in Non-Mirror Mode, MC=0x0, Channel=0x0, Mirror Scrub Enabled is
True
socket0 Mc0 Ch0 Patrol Scrub Disabled
socket1 Mc0 Ch0 Patrol Scrub Disabled
socket0 Mc0 Ch0 Patrol Scrub Enabled
socket1 Mc0 Ch0 Patrol Scrub Enabled
Current : dev0 = 5, dev0msk = 0x4444444444444444, dev1 = 9, dev1msk =
0x1111111111111111.
MC 0x0 Channel 0
=====
|                               | Device0          | Device1          |
|-----|-----|-----|
| Injectors Status | Currently unarmed | Currently unarmed |
|-----|-----|-----|
| Mask            | 00000000 00000007 | 0001ffff ffffffff |
|-----|-----|-----|
| Match          | 00080000 02000000 | 00080000 00000000 |
|-----|-----|-----|
| crc_err_inj_dev#_5_bits | 0x5          | 0x9          |
|-----|-----|-----|
| XOR Mask        | 0x4444444444444444 | 0x1111111111111111 |
|-----|-----|-----|
Restoring Original Values!!!

```

Use a thread to consume the error been injected

```

>>> itp.threads[80].mem('0x10000000p',4)
[32b] 0x73601000
>>> sv.sockets.ios.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log
socket0.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00010000
socket0.io1.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00000000
socket1.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00004000
socket1.io1.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log - 0x00000000
>>> sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000000 : secondrmcerrsrcfromcore (49:49) -- Set to '1 if the
SecondrMCErrSrcID is from a C2U
0x00000000 : secondrmcerrsrcvalid (48:48) -- Set to '1 if the SecondrMCErrSrcID
is valid
0x00000000 : secondrmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second rMCErr
0x00000000 : firstrmcerrsrcfromcore (17:17) -- Set to '1 if the FirstrMCErrSrcID
is from a C2U
0x00000000 : firstrmcerrsrcvalid (16:16) -- Set to '1 if the FirstrMCErrSrcID is
valid
0x00000000 : firstrmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first rMCErr

0x00000000 : secondrmcerrsrcfromcore (49:49) -- Set to '1 if the
SecondrMCErrSrcID is from a C2U
0x00000000 : secondrmcerrsrcvalid (48:48) -- Set to '1 if the SecondrMCErrSrcID

```

```
is valid
0x00000000 : secondrmcerrsrid (47:32) -- Physical Port ID of the end point with
the second rMCErr
0x00000001 : firstrmcerrsridfromcore (17:17) -- Set to '1' if the FirstrMCErrSrcID
is from a C2U
0x00000001 : firstrmcerrsridvalid (16:16) -- Set to '1' if the FirstrMCErrSrcID is
valid
```

For Granite Rapids: run `error.check_machine_check_errors`:

```
>>> error.check_machine_check_errors()
Checking errors in the following IPs:
    machine_check
```

Please be patient...this operation might take several minutes.

Uncore MCA from Socket: 0

```
=====
| Valid | MC Bank Name | Status | Addr
| Misc | Ctl | Ctl2 |
=====
| * | BANK #13 - MC CH0 merged | 0xac00000200a00090 | 0x0000000010000000 |
0x0800010020000086 | 0x00003cc0ffffe5ff | 0x0000000000000000 |
=====
```

Core MCA from Socket: 0

Socket 0 : No errors logged in any Core-specific Machine Check Banks

Uncore MCA from Socket: 1

Socket 1 : No errors logged in any Uncore-specific Machine Check Banks

Core MCA from Socket: 1

```
=====
| Valid | Core | Thread | MC Bank Name | Status | Addr
| Misc | Ctl | Ctl2 |
=====
| * | core8 | thread0 | BANK #1 - DCU | 0xbd8000000100134 |
0x0000000010000000 | 0x0000000000000086 | 0x000000000000001f | 0x0000000000000000 |
|
=====
```

Decoding Machine Checks...

Machine Check Decoder Summary

```
=====
=====
=====
Uncore MCA from Socket: 0
=====
| MC Bank Name | Status | Addr
| Misc | Ctl | Ctl2 |
=====
| BANK #13 - MC CH0 merged | 0xac00000200a00090 | 0x0000000010000000 |
```

```

0x0800010020000086 | 0x00003cc0ffffe5ff | 0x0000000000000000 |
=====
=====

MC13 not part of defined merged nor unmerged banks to decode...

=====
=====
=====

Core MCA from Socket: 1
=====
=====
| Core | Thread | MC Bank Name | Status | Addr
| Misc | Ctl | Ctl2 |
=====
| core8 | thread0 | BANK #1 - DCU | 0xbd80000000100134 | 0x0000000010000000 |
0x0000000000000086 | 0x000000000000001f | 0x0000000000000000 |
=====
=====

Bank name: Core DCU, Error = UNCORR_LOAD_POISON_0010_0134
+ MC1_STATUS 0xBD80000000100134

Range      Field
Description ----- Value Value Description
-----
63:63      VALID
register is valid 0x1 Valid Indicates that the information within this
62:62      OVERFLOW
occurred while 0x0 Not Overflow A '1' indicates that a machine-check error
the error- the results of a previous error were still in
already set) reporting register bank (the VALID bit was
61:61      UC
error condition 0x1 UC Detected A '1' indicates that an uncorrectable the
detected. was
60:60      EN
error 0x1 Enabled Error reporting enabled. A '1' indicates that
bit(s) in the reporting is enabled by the associated flag
IA32_MCi_CTL register.
59:59      MISCV
register content is 0x1 IA32_MCi_MISC Valid
If this flag is '1', the IA32_MCi_MISC
valid and can be read.
58:58      ADDR
register content is 0x1 IA32_MCi_ADDR Valid
If this flag is '1', the IA32_MCi_ADDR
valid and can be read.
57:57      PCC
that reliable 0x0 Processor Context Corrupt: A '1' indicates
possible and a reset Processor Context Not Corrupted
restarting of the processor may not be
is recommended.
56:56      S
error. 0x1 Signaling an uncorrectable recoverable
---
```

55:55	AR	Recovery action required for UCR
error.	0x1	---
54:53	THRESHOLD_BASED_STATUS	Threshold-based error
status	0x0	---
52:38	CORRECTED_ERROR_COUNT	Count of correctable errors in the DCU
including sticky	0x0	---
		overflow indicator (bit 52)
37:37	FW	FW Update Error Status
Indicator	0x0	---
36:32	OTHER_INFO	Not part of Machine-Check
Architecture	0x0	---
31:16	MSCOD	Model Specific Error
Code	0x10	LOAD_POISON_0010_0134
15:0	MCACOD	Machine Check Architecture Error
Code	0x134	DCU data - Load Poison
+ MC1_MISC 0x0000000000000086		
Range	Field	
Description		Value Value Description
-----	-----	-----
63:9	MODEL_SPECIFIC_INFORMATION	Model-Specific
Information		0x0 ---
8:6	ADDRESS_MODE	Address mode (physical, linear, segment
offset, memory,	0x2	Physical Address
		generic)
5:0	RECOVERABLE_ADDR_LSB	Indicates the position of the least
significant bit of the	0x6	---
		recoverable error address (IA32_MCi_ADDR
		should be ignored)
+ MC1_ADDRESS 0x0000000010000000		
Range	Field	
Description		Value Value Description
-----	-----	-----
63:0	ADDR	Address of code or data memory location that produced the
0x10000000	---	machine-check error
The MC decoding log file is located in:		
C:\Users\Administrator\PythonSv		
\Machine_Check_Decoder_Summary_Fri_Jul_28_13_39_21_2023.txt		

Fields with non default value		

=====		
IP block	# fields with non-default value	
-----	-----	-----

```
| MACHINE_CHECK | 0x2 |
=====
```

```
*****
*****
```

For Intel® Xeon® 6900/6700-Series Processors with E-Cores: run `error.crashdump()`

```
Summary failure signature for crashdump_20231109_210719.json
Summarizer version: 0.0.68
```

```
HW.MCE.MEC:Uncorr_0X7.incomplete_dump.mca_same_value
```

```
FaultCPU= SRF
FaultTimeStamp= 2023-11-09_21:07:19
FaultPPIN= 0x33df934af2b09763
FaultCPUID= 0xa06f0
FaultMicrocode= 0x80000320
FaultReason= HW.MCE.MEC:Uncorr_0X7.incomplete_dump.mca_same_value
FaultDevice= N/A
FaultInstance= N/A
FaultCode= CPU0.core0.MS_MCACOD.0x00070134
FaultAction= CPU0:
FaultActionCode= hw.on_repeat_replace.cpu0.core0
```

```
##### Signature by Sockets #####
```

```
=====
|socket|Signature|
-----
|0      |HW.MCE.MEC:Uncorr_0X7.incomplete_dump.mca_same_value|
|1      |INCOMPLETE_DUMP.MCA_SAME_VALUE|
=====
```

```
##### Status[63]=1 Machine Check summary #####
```

```
=====
|skt|die      |module |core|bank |Status          |Address          |
Misc      |CTL
Error
-----
|0 |compute0 |module36 |0      |3      |0xbd8000000070134|0x000000001000000|
0x0000000000000080|0x0000000000000003|Core
MEC:UNCORR_0x7
|0 |soc      |-      |-      |MCHN0|0xac00000200a00090|0x000000001000000|
0x0800008018000086|0x00003cc0ffffe5ff|
MCHN:UNCORR_Uncorrectable_Error_for_Demand_or_Underfill_Reads|
=====
```

```
##### Decode each Valid Machine Check #####
```

```
*****
*****
=====
|skt|die      |module |core|bank |Status          |Address          |
Misc      |Ctl
-----
|0 |compute0 |module36 |0      |3      |0xbd8000000070134|0x000000001000000|
0x0000000000000080|0x0000000000000003|
=====
```

Decoding Status Register:

Range	Field	Description	Value	Value	Decode
Description	Description				
63:63	VALID	Indicates that the information within this register is valid	0x1		
62:62	OVERFLOW	A '1' indicates that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (the VALID bit was already set)	0x0	Not	
61:61	UC	A '1' indicates that an uncorrectable the error was detected.	0x1	UC	
60:60	EN	Error reporting enabled. A '1' indicates that reporting is enabled by the associated flag bit(s) in the IA32_MCi_CTL register.	0x1		
59:59	MISCV	If this flag is '1', the IA32_MCi_MISC register content is valid and can be read.	0x1	IA32_MCi_MISC	
58:58	ADDRV	If this flag is '1', the IA32_MCi_ADDR register content is valid and can be read.	0x1	IA32_MCi_ADDR	
57:57	PCC	Processor Context Corrupt: A '1' indicates that reliable Processor Context Not restarting of the processor may not be possible and a reset is recommended.	0x0	Processor Context Not	
56:56	S	Signaling an uncorrectable recoverable error.	0x1		
55:55	AR	Recovery action required for UCR error.	0x1		
54:53	THRESHOLD_BASED_STATUS	Reserved	0x0		
52:38	CORRECTED_ERROR_COUNT	Count of filtered correctable MCA errors including sticky overflow indicator (bit 52)	0x0		
37:37	FW	Firmware update error status			

```

indicator                                0x0
---
 36:32 OTHER_INFO                        Not part of Machine-Check
Architecture                             0x0
---
 31:16 MSCOD                             Model Specific Error
Code
0x7
 15:0 MCACOD                             Machine Check Architecture Error
Code                                     0x134 CACHE_HIERARCHY_ERROR|DATA_READ|DATA|LEVEL_0
(CORE L1 CACHE)

*****
*****
=====
=====
|skt|die|module|core|bank |Status          |Address          |
Misc          |Ctl          |
-----
|0 |soc|0 |0 |MCHN0|0xac00000200a00090|0x0000000010000000|
0x0800008018000086|0x00003cc0ffffe5ff|
=====
=====

Decoding Status Register:

Range Field
Description
Description
Value Value Decode
-----
63:63 VALID
register is valid 0x1 Valid Indicates that the information within this
62:62 OVERFLOW
occurred while 0x0 Not Overflow A '1' indicates that a machine-check error
the error- reporting register bank (the VALID bit was
already set)
61:61 UC
condition 0x1 UC Detected A '1' indicates that an uncorrectable the error
detected. was
60:60 EN
error 0x0 Not Enabled Error reporting enabled. A '1' indicates that
bit(s) in the reporting is enabled by the associated flag
register. IA32_MCi_CTL
59:59 MISCV
content is 0x1 IA32_MCi_MISC Valid If this flag is '1', the IA32_MCi_MISC register
read. valid and can be
58:58 ADDR
content is 0x1 IA32_MCi_ADDR Valid If this flag is '1', the IA32_MCi_ADDR register
read. valid and can be
57:57 PCC
reliable 0x0 Processor Context Corrupt: A '1' indicates that
and a reset Context Not Corrupted
recommended. restarting of the processor may not be possible
is
56:56 S Signaling an uncorrectable recoverable

```

```

error.          0x0    ---
55:55  AR          Recovery action required for UCR
error.          0x0
---
54:53  THRESHOLD_BASED_STATUS  Threshold-based error
status          0x0
---
52:38  CORRECTED_ERROR_COUNT  Count of correctable errors in the DCU including
sticky        0x0    --- overflow indicator (bit
52)
37:37  FW          FW Update Error Status
Indicator          0x0
---
36:32  OTHER_INFO  Not part of Machine-Check
Architecture      0x2
---
31:16  MSCOD       Model Specific Error
Code          0xa0    Uncorrectable Error for Demand or
Underfill Reads
15:0    MCACOD     Machine Check Architecture Error
Code          0x90    MCCHAN_ERROR(1LM MODE) | MEMORY READ ERROR |
CHANNEL_0

```

4.2.3 HW.MCE.CHA

4.2.3.1 HW.MCE.CHA.0x5

Tool: 633201_RAS_Automation_Toolkit
 Test case: BHS/Debug_Handbook/4.2.3.1_HW.MCE.CHA.0x5.py

HW.MCE.CHA.0x5	SAD_ERR_CORRUPTING_OTHER
----------------	--------------------------

Cacheable read to VGA region (0xa000-0xbffff)

Setup 0xa4000 MTRR to WB type

```

>>> halt()
>>>
[GPC core group] Halt on 48 devices -- 13:38:53.172489 2023-07-05
>>> for i in range(len(itp.devicelist)):
...     try:
...         itp.threads[i].state.regs.cr0=0xc0010013
...     except:
...         pass
...
>>> itp.threads[0].wbinvd()
>>> itp.msr(0x2ff,0)
>>> itp.msr(0x259,0x600)
>>> itp.msr(0x2ff,0xc00)
>>> for i in range(len(itp.devicelist)):
...     try:
...         itp.threads[i].state.regs.cr0=0x80010013
...     except:
...         pass
...

```

Read 0xa4000

```
>>> itp.threads[0].mem('0xa4000p',4)
[32b] 0xFFFFFFFF
```

Check mcerrloggingreg and mca_err_src_log register:

```
>>> sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1' if the SecondMCerrSrcID
is from a C2U
0x00000001 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00004dfc : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCERR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1' if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00004df2 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR

0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1' if the SecondMCerrSrcID
is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCERR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1' if the FirstMCerrSrcID is
from a C2U
0x00000000 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00000000 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR

>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is O...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit2...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
```

```

0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This ...
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is 0...
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000000 : msmi_mcerr internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) - Reserved

```

RunCrashdump:

```

##### Status[63]=1 Machine Check summary #####
=====
====
|skt|die      |cor|thr|bank|Status      |Address      |
Misc      |CTL      |
Error
-----
-----
|0 |soc      |-|-|CHA0|0xbe00000000050408|0x00000000000a4000|
0x0000817881100486|0x00000003ffffffff|
Unmerged_CHA0:SAD_ERR_CORRUPTING_OTHER_OriginalReq[DRd]_TorID[30]|
|0 |soc      |-|-|CHA5|0xbe00000000050408|0x00000000000a4040|
0x0500817889100486|0x00000003ffffffff|
Unmerged_CHA5:SAD_ERR_CORRUPTING_OTHER_OriginalReq[DRd]_TorID[30]|
=====
=====
=====

```

NOTE

For IMC/B2CMI/Ubox/ Intel® UPI bank MCE error injection methods, see the memory/IIO/ Intel®UPI debug handbook respectively.

4.2.4 TOR Timeout without 3Strike-Timeout

4.2.4.1 Method 1

Disable B2CMI transaction, then PCIe* device issue memory read TLP.

In this example, PEI card is used to generate the memory read TLP.

```
>>> halt
[GPC core group] Halt on 48 devices -- 13:49:49.486366 2023-07-06
>>> pcie.topology
No PCIe devices present or active on socket 1

*****
*
* Socket 0 (A1) PCIe Port Mappings Segment #0, Buses 0x00,0x09
* ,0x0a,0x0b,0x0c,0x0d,0x0e,0x0f,0x10,0x11,0x12,0x13,0x1e,0x1f
* ,0xd8
*
*****

=====
| PCIe port | config as | width | speed | slot/down | ASPM en | secbusno |
subbusno | linkstate | Vendor | Vendor ID | Device ID | RevID | ASPM
en |
-----
| dmi | DMI | x8 | Gen3 | down | L1 | | |
| no | Up.L0 | Intel | 8086h | | EBG | | 10h |
-----
| 1a | x16 | | | | | | |
| | | | | | | | |
-----
| 2a | x16 | | | | | | |
| | | | | | | | |
-----
| 3a | x16 | x4 | Gen4 | slot | no | 38h | |
38h | Up.L0 | Intel Corporation | 8086h | 0d4bh | 04h | no |
-----
| 4a | x4 | | | | | | |
| | | | | | | | |
-----
| 4c | x4 | | | | | | |
| | | | | | | | |
-----
| 4e | x4 | | | | | | |
| | | | | | | | |
-----
| 4g | x4 | | | | | | |
| | | | | | | | |
-----
| 5a | x4 | | | | | | |
| | | | | | | | |
-----
| 5c | x4 | | | | | | |
| | | | | | | | |
-----
```

Birch Stream System Hang and MCE
Issue Debug Handbook
159

```

Socket_____ = 0
Hbm_____ = -1
Mc_____ = 6
Ch_____ = 0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 7
Bank in Bank Grp = 1
Bank Addr_____ = 29
Row_____ = 0xff08
Column_____ = 0x0
Mirror_____ = 0
Primary_____ = -1
*****
Hit Enter key to continue

```

Write back invalid on all threads.

```

>>> halt()
>>> itp.threads.wbinvd()

```

Disable B2CMI transaction. And then use PEI card to generate memory read TLP.

```

C:\Program Files\PEI\PCIE>python pei_run.py -t TLP_GEN -f Fmt=1 Type=0
Addr_64=0x2070000000 Length=0x1 BE_1st=0xf -c 1
Starting TLP_GEN()
INFO: Starting PEI.run() on Endpoint image
Count TLP_US: 1
INFO: Trigger disarmed after 1.0sec.

```

Check mca_err_src_log register and mcerrloggingreg register.

```

>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions as a Valid bit for the other two
package conditions. It has no effect when a ...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMT# (for any
reason). It is or(bit 22, bit21);
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMT_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMT_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMT#. This is OR (bit
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMT_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMT_MCERR.

```

```

0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMT
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMT
0x00000000 : first_core_ierr (04:00) - Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29);
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmt_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmt (23:23) -- External error: The package observed MSMT# (for any
reason). It is or(bit 22, bit21);
0x00000000 : msmt_ierr (22:22) -- External error: The package observed MSMT_IERR.
0x00000001 : msmt_mcerr (21:21) -- External error: The package observed
unrecoverable MSMT_MCERR.
0x00000001 : msmt_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMT#. This is OR (bit
0x00000000 : msmt_ierr_internal (19:19) -- Internal error: This socket attempted to
assert MSMT_IERR.
0x00000001 : msmt_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMT_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmt (16:16) -- This socket observed recoverable MSMT
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmt_internal (14:14) -- This socket asserted recoverable MSMT
0x00000000 : first_core_ierr (04:00) - Reserved

>>> sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCERR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00004b92 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR

0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1 if the SecondMCerrSrcID
is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is
valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
the second MCERR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1 if the FirstMCerrSrcID is
from a C2U
0x00000000 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.

```

Check machine check bank. CHA0 machine check bank report TOR timeout.

```

>>> error.check_machine_check_errors()

Checking errors in the following IPs:
    machine_check

Please be patient...this operation might take several minutes.

```

```

Uncore MCA from Socket: 0
=====
| Valid |      MC Bank Name      |      Status      |      Addr      |
|-----|-----|-----|-----|
|      *      | BANK #7 - CHA0 merged | 0xbe00000000c110a | 0x0000000063420000 |
0x0a3cd4800010e886 | 0x0000003fffffff | 0x0000000000000000 |
|      *      | BANK #12 - B2CMI merged | 0xba0000000090400 | 0x0000000000000000 |
0x0000007000000800 | 0x000000000000eff3 | 0x0000000000000000 |
|
=====
=====

```

4.2.5 Uncorrected Recoverable Error Injection under Linux*

4.2.5.1 UCR - UCNA

```

Tool: 633201_RAS_Automation_Toolkit
Test case: BHS/Debug_Handbook/4.2.5.1_UCR_UCNA.py

```

Processor logs Uncorrected No Action (UCNA) for patrol scrubber uncorrected error detection events.

UCNA - Uncorrectable Data is logged in MC Bank with unique signature (PCC=0, UC=1, S=0, AR=0).

BIOS knob setting:

1. "Patrol Scrub Error" = UCNA
2. "EV DFX Features" = Enabled
3. "Disable BIOS Done" = [X]
4. "Lock Chipset" = Disable
5. "MSR Lock Control" = Disable

Boot into Linux*

Check "scrub_en" bit

```

>>> sv.socket0.soc.memss.mc0.ch0.mcchan.scrubctl.show()
0x00000001 : scrub_en (31:31) -- Scrub Enable when set.
0x00000000 : stop_on_cpl (30:30) -- Stop patrol scrub at end of memory range. This mode is meant to be used as...
0x00000000 : ptl_cpl (29:29) -- When stop_on_cpl is enabled, patrol will stop at the end of the address range...
0x00000001 : scrubissued (28:28) -- When Set, the scrub address registers contain the last scrub address issued
0x00000000 : issuance (27:27) -- When Set, the patrol scrub engine will issue the address in the scrub address...
0x00000000 : startscrub (26:26) -- When Set, the Patrol scrub engine will start from the address in the scrub
0x000007e4 : scrubinterval (23:00) -- Defines the interval in HCLKs between patrol scrub requests. The calculation...

```

Inject "Patrol Scrub" UCE Error

```

>>> ei.injectMemError(socket=1, hbm=-1, channel=0, sub_channel=0, dimm=0, rank=0, sub_rank=0, errType='uce',immInject=True, immConsume=True, PatrolConsume=True)

```

Check UCNA status been recorded: Uncorrected No Action required
IA32_MCI_STATUS; (PCC=0, UC=1, S=0, AR=0)

```

>>> sv.socket1.uncore.memss.mc0.ch0.imc0_mc_status_shadow.show
0x00000001 : valid (63:63) -- Error logged in this Bank is valid
0x00000000 : over (62:62) -- (Overflow) Indicates (when set) that a machine check error occurred while the results o
0x00000001 : uc (61:61) -- Error logged in this banks is an uncorrected error.
0x00000000 : en (60:60) -- Indicates (when set) that signaling of the machine check exception for this error was

```

```
0x00000001 : miscv (59:59) -- Indicates (when set) that the MciMISC register contains additional information
0x00000001 : addrv (58:58) -- indicates (when set) that the MciADDR register contains the address where the error
0x00000000 : pcc (57:57) -- Indicates (when set) that the state of the processor might have been corrupted by the
0x00000000 : s (56:56) -- s - Signaling. When set it indicates that the error was signaled with an MCERR. When c...
0x00000000 : ar (55:55) -- AR - Action REquired. When set it indicates that SW needs to take immediate recovery...
0x00000000 : cor_err_stat (54:53) -- These bits are used to indicate when the number of corrected errors has exce...
0x00000000 : cor_err_cnt (52:38) -- A 15 bit counter that is incremented each time a corrected (or uncorrected but
0x00000000 : fw_upd (37:37) -- Indicates that the BIOS has updated the contents of the MC Bank. HW should set this t
0x00000002 : other_info (36:32) -- extra err info vld -- The functions of the bits in this field are implementat...
0x00000010 : mscod (31:16) -- Uncorr PatTol Scrub Error -- Specifies a model specific error code that uniquely...
0x000000c0 : mscod (15:00) -- MC Error (11M), Ch 0, Memory Scrubbing Error -- Specifies the machine check...
```

Linux* is still active. Check OS message, it logs a UCE:

```
Jun 22 07:19:32 localhost systemd[1]: systemd-udevd.service: Watchdog timeout (limit 3min)!
Jun 22 07:19:32 localhost mcelog[2903]: mcelog: Family 6 Model 143 CPU: only decoding architectural errors
Jun 22 07:19:32 localhost kernel: (1)[Hardware Error]: Hardware error from APEI Generic Hardware Error Source: 0
Jun 22 07:19:32 localhost kernel: (1)[Hardware Error]: event severity: recoverable
Jun 22 07:19:32 localhost kernel: (1)[Hardware Error]: Error 0, type: recoverable
Jun 22 07:19:32 localhost kernel: (1)[Hardware Error]: fru text: Card02, ChnA, DIMM0
Jun 22 07:19:32 localhost kernel: (1)[Hardware Error]: section_type: memory error
Jun 22 07:19:32 localhost kernel: (1)[Hardware Error]: error_status: 0x0000000000000400
Jun 22 07:19:32 localhost kernel: (1)[Hardware Error]: physical_address: 0x00000000480000000
Jun 22 07:19:32 localhost kernel: (1)[Hardware Error]: node: 1 card: 0 module: 0 rank: 0 bank: 0 device: 0 row: 0 column: 0
Jun 22 07:19:32 localhost kernel: (1)[Hardware Error]: error_type: 14, scrub uncorrected error
Jun 22 07:19:32 localhost kernel: (1)[Hardware Error]: DIMM Location: NODE 4 CPU1_DIMM_A1
Jun 22 07:19:32 localhost kernel: clocksource: Switched to clocksource hpet
Jun 22 07:19:32 localhost kernel: Memory failure: 0x480000: already hardware poisoned
Jun 22 07:19:32 localhost kernel: core: [Hardware Error]: Machine check events logged
```

RunCrashdump, no MCA error been recorded

```
##### Signature by Sockets #####
=====
|socket|Signature|
=====
| 0 |NO_UNCORR_MCA_ERROR:Good_capture_without_IERR_CATERR_MSMI|
| 1 |NO_UNCORR_MCA_ERROR:Good_capture_without_IERR_CATERR_MSMI|
=====
```

4.2.5.2 UCR - SRAR

Software Recoverable Action Required (SRAR) - Bad data or instructions in the core execution path can cause SRAR errors in MC Bank states with unique signatures (PCC=0, UC=1, S=1, AR=1).

Boot into Linux*, and then Trigger DCU poison UCE via RAK tool (*RAS Automation Toolkit*, document number 633201)

```
Tool: 633201_RAS_Automation_Toolkit
Run test case: "MCA_TC03_SRAR_DCU_EMCA_On_Lmce_On.py" and
"MCA_TC04_SRAR_IFU_EMCA_On_Lmce_On.py"
```

Linux* is still active. Check OS message, it logs a UCE and it has been recovered:

```
[ 70.866440] EINJ: Error INJection is initialized.
[ 91.595133] process '/root/ras-tools-master/einj_mem_uc' started with
executable stack
[ 92.226892] Disabling lock debugging due to kernel taint
[ 92.227211] mce: Uncorrected hardware memory error in user-access at 16bdc4400
[ 92.227686] mce: [Hardware Error]: Machine check events logged
[ 92.236387] mce: [Hardware Error]: Machine check events logged
[ 92.237717] Memory failure: 0x16bdc4: Sending SIGBUS to einj_mem_uc:2897 due
to hardware memory corruption
[ 92.248997] Memory failure: 0x16bdc4: recovery action for dirty LRU page:
Recovered
[ 92.258201] Memory failure: 0x16bdc4: already hardware poisoned
```

Run Crashdump, no MCA error been recorded

```
##### Signature by Sockets #####
=====
|socket|Signature|
=====
|0      |NO_UNCORR_MCA_ERROR:Good_capture_without_IERR_CATERR_MSMI|
|1      |NO_UNCORR_MCA_ERROR:Good_capture_without_IERR_CATERR_MSMI|
=====
```

4.2.6 Ubox Misaligned Error Injection

Verify that the BIOS knob "Disable BIOS Done" is enabled.

First need to find Ubox MMIO space range from serial log as follow:

Figure 25. Ubox MMIO Space Range

CPU Resource Allocation														
	StkId	Seg	Bus	To	ToApic	MmioL				MmioH				StkBmp
CPU0	0	0x00	0x7F	0x0000	0xAFFF	0xFEC00000	0xFECFFFFF	0x90000000	0x377FFFFF	0x00001E00	00000000	0x00001E6F	FFFFFFFF	0x3B5
Ins0	0x00	0	0x00	0x04	0x0000	0x1FFFF	0xFEC00000	0xFECFFFFF	0x90000000	0x907FFFFF	0x00001E00	00000000	0x00001E6F	FFFFFFFF
Ins01	0x03	0	0xFF	0x00	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0x00000000	0xFFFFFFFF	FFFFFFFF	0x00000000	00000000	
Ins02	0x08	0	0x05	0x09	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0x90000000	0x90000000	0x00001E10	00000000	0x00001E1F	FFFFFFFF
Ins03	0x09	0	0xFF	0x00	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	00000000	
Ins04	0x02	0	0x0A	0x21	0x2000	0x3FFF	0xFFFFFFFF	0x00000000	0x91000000	0x9AFFFFF	0x00001E20	00000000	0x00001E2F	FFFFFFFF
Ins05	0x01	0	0x22	0x38	0x4000	0x5FFF	0xFFFFFFFF	0x00000000	0x98000000	0xA4FFFFF	0x00001E30	00000000	0x00001E3F	FFFFFFFF
Ins06	0x04	0	0xFF	0x00	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	00000000	
Ins07	0x05	0	0x39	0x4F	0x6000	0x7FFF	0xFFFFFFFF	0x00000000	0xA5000000	0xAEFFFFF	0x00001E40	00000000	0x00001E4F	FFFFFFFF
Ins08	0x08	0	0x50	0x66	0x8000	0x9FFF	0xFFFFFFFF	0x00000000	0xA6000000	0xB8FFFFF	0x00001E50	00000000	0x00001E5F	FFFFFFFF
Ins09	0x0A	0	0x67	0x7D	0xA000	0xAFFF	0xFFFFFFFF	0x00000000	0xB9000000	0xC2FFFFF	0x00001E60	00000000	0x00001E6F	FFFFFFFF
Ins10	0x06	0	0xFF	0x00	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	00000000	
Ins11	0x07	0	0xFF	0x00	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	00000000	
Ins12	0x00	0	0xFF	0x00	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	00000000	
Ins13	0x0C	0	0xFF	0x00	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	00000000	
Ins14	0xFF	0	0xFF	0x00	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	00000000	
Ins15	0xFF	0	0xFF	0x00	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	00000000	
Rsvd	0x10	0	0xFB	0xFD	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000	00000000	
Ubox	0x11	0	0x7E	0x7F	0xFFFF	0x00000	0xFFFFFFFF	0x00000000	0xC3000000	0xC37FFFFF	0xFFFFFFFF	FFFFFFFF	0x00000000	00000000

Consumed by memory read

```
>>> halt
>>> itp.threads[0].mem('0xC3000000p',8) //Ubox aligned error injection
[64b] 0x0000002200000000
>>> itp.threads[0].mem('0xC3000001p',8) //Ubox misaligned error injection
[64b] 0xFFFFFFFFFFFFFFFF
```

Check register ubox.ncevents.mcerrloggingreg log, this time we will see MCERR occurs.

```
>>>sv.sockets.io0.uncore.ubox.ncevents.mcerrloggingreg.show()
0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1' if the SecondMCerrSrcID is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with the second MCERR
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1' if the FirstMCerrSrcID is from a C2U
0x00000001 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00000001e : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with the first MCERR

0x00000000 : secondmcerrsrcfromcore (49:49) -- Set to '1' if the SecondMCerrSrcID is from a C2U
0x00000000 : secondmcerrsrcvalid (48:48) -- Set to 1 if the SecondMCerrSrcID is valid.
0x00000000 : secondmcerrsrcid (47:32) -- Physical Port ID of the end point with
```

```

the second MCErr
0x00000000 : firstmcerrsrcfromcore (17:17) -- Set to '1' if the FirstMCErrSrcID is
from a C2U
0x00000000 : firstmcerrsrcvalid (16:16) -- Set to 1 if the FirstSrcID is valid.
0x00000000 : firstmcerrsrcid (15:00) -- Physical Port ID of the end point with
the first MCERR

```

Run Crashdump:

```

Summary failure signature for crashdump_20231016_150043.json
Summarizer version: 0.0.67

HW.MCE.UBOX:Ubox.incomplete_dump.mca_same_value

FaultCPU= GNR
FaultTimeStamp= 2023-10-16_15:00:43
FaultPPIN= 0x0aecb5c2bb8bea6a
FaultCPUID= 0xa06d0
FaultMicrocode= 0x80000480
FaultReason= HW.MCE.UBOX:Ubox.incomplete_dump.mca_same_value
FaultDevice= N/A
FaultInstance= N/A
FaultCode= CPU0.Ubox.MSCOD.0x8008
FaultDescription= Misaligned MMIO Rd (Non-SMM)
FaultAction= CPU0: Verify Memory Mapping and Access Method, Contact your OEM
debug team.
FaultActionCode= hw.contact_oem_debug_team

##### Signature by Sockets #####
=====
|socket|Signature|
-----
|0|HW.MCE.UBOX:Ubox.incomplete_dump.mca_same_value|
=====

##### Status[63]=1 Machine Check summary #####
=====
|skt|die|cor|thr|bank|Status|Address|Misc|
CTL|Error|
-----
|0|io0|-|-|4|0xf200000080080407|0x0000000000000000|0x0000000000000000|
0x0000000000000001f|Ubox:General_UBox_error|
.....

##### Decode each Valid Machine Check #####
*****
*****
=====
|skt|die|cor|thr|bank|Status|Address|Misc|
Ctl|
-----
|0|io0|0|0|4|0xf200000080080407|0x0000000000000000|0x0000000000000000|
0x0000000000000001f|
=====

Decoding Status Register:

Range Field
Description Value Value
Decode Description
-----

```

63:63	VALID	0x1	Valid	Indicates that the information within this register is valid
62:62	OVERFLOW	0x1	Overflow Occurred	A '1' indicates that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (the VALID bit was already set)
61:61	UC	0x1	UC Detected	A '1' indicates that an uncorrectable the error was detected.
60:60	EN	0x1	Enabled	Error reporting enabled. A '1' indicates that reporting is enabled by the associated flag bit(s) in the IA32_MCi_CTL register.
59:59	MISCV	0x0	IA32_MCi_MISC Not Valid	If this flag is '1', the IA32_MCi_MISC register content is not valid and can be read.
58:58	ADDRV	0x0	IA32_MCi_ADDR Not Valid	If this flag is '1', the IA32_MCi_ADDR register content is not valid and can be read.
57:57	PCC	0x1	Processor Context Corrupted	Processor Context Corrupt: A '1' indicates that restarting of the processor may not be possible and a reset is recommended.
56:56	S	0x0	---	Signaling an uncorrectable recoverable error.
55:55	AR	0x0	---	Recovery action required for UCR error.
54:53	THRESHOLD_BASED_STATUS	0x0	---	Threshold-based error status
52:38	CORRECTED_ERROR_COUNT	0x0	---	Corrected error count. Reserved for I/O
37:37	FW	0x0	---	Indicates that the BIOS has updated the contents of the MC Bank
36:32	OTHER_INFO	0x0	---	Not part of Machine-Check Architecture
31:16	MSCOD			Model Specific Error
15:0	MCACOD			Machine Check Architecture Error
				0x8008 Misaligned MMIO Rd (Non-SMM)
				0x407 General UBox error

4.3 Error Pin Injection and Verification

4.3.1 RMCA Error Pin

RMCA_N is used to signal recoverable errors.

The following tables summarize RMCA error signaling capabilities:

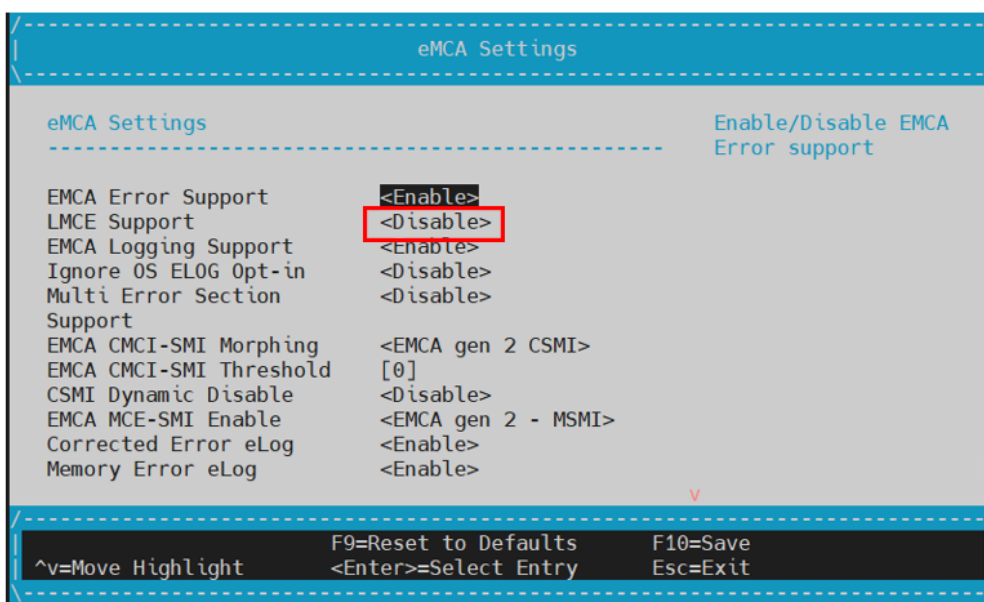
Error Type	MCI_STATUS Fields [Bit Position]				Error Signaling	
	UC [61]	PCC [57]	S [56]	AR [55]	RMCA_N Pin	Interrupt Type1
Uncorrected Recoverable (SRAR)	1	0	1	1	Pulse (RMCA_N in non LMCE mode)	INT18

NOTE

LMCE:Local Machine Check. When LMCE enable, it indicates to the OS that the error logged was not broadcast(localized.); When LMCE disable, it indicates to the OS that the error was broadcast to all threads.

Error Injection Steps:

First need to disable "LMCE Support" mode in BIOS knob as shown next.



Then enter to OS and inject the SRAR errors (Taking DCU errors as an example here). Check if "LMCE Support" mode is truly disabled (0x4d0h).

```
>>>
halt

>>>
[GPC core group] Halt on 192 devices -- 15:58:12.891120 2024-03-13
>>> msr(0x3a)
0x5
>>> msr(0x4d0)
0x0
```

Inject Memory UCE error at 0x10000000 with immConsume = 1

```
>>> ei.mem.injectMemError(0x10000000,errType = 'uce',immConsume = 1)
Halting to do check on injector lock
INFO: BIOS knob 'Disable BIOS Done' is enabled!
Initiating an SMI

[GPC core group] Resuming on 192 devices -- 16:02:22.990686 2024-03-13
Injectors UNLOCKED on GNR0_C0_T0
Injectors UNLOCKED on GNR0_C0_T1
Injectors UNLOCKED on GNR0_C1_T0
...
[GNR0_C0_T0] SMM entry Break at [0x10:0000000000008000] -- 16:02:45.085473
2024-03-13
[GNR0_C0_T1] SMM entry Break at [0x10:0000000000008000] -- 16:02:45.085473
2024-03-13
[GNR0_C1_T0] SMM entry Break at [0x10:0000000000008000] -- 16:02:45.085473
2024-03-13
...
[GNR1_C82_T1] SMM entry Break at [0x10:0000000000008000] -- 16:02:45.132116
2024-03-13
[GNR1_C83_T0] SMM entry Break at [0x10:0000000000008000] -- 16:02:45.132116
2024-03-13
[GNR1_C83_T1] SMM entry Break at [0x10:0000000000008000] -- 16:02:45.132116
2024-03-13
Injectors UNLOCKED on GNR0_C3_T0
Injectors UNLOCKED on GNR0_C3_T1
Injectors UNLOCKED on GNR0_C4_T0
...
Injectors UNLOCKED on GNR1_C83_T0
Injectors UNLOCKED on GNR1_C83_T1

[GNR0_C0_T0] Resuming -- 16:02:53.663472 2024-03-13
[GNR0_C0_T1] Resuming -- 16:02:53.663472 2024-03-13
...
[GNR1_C83_T0] Resuming -- 16:02:53.929047 2024-03-13
[GNR1_C83_T1] Resuming -- 16:02:53.929047 2024-03-13
Exit SMM Mode...

[GPC core group] Halt on 192 devices -- 16:02:55.569694 2024-03-13
*****
Core Addr_____ = 0x1000_0000 (256.00 M)
Cluster_____ = 0x0
Set Index_____ = 0x0
Tag_____ = 0x800
Cha_____ = 14
Perspective_____ = FM
*****
FM ddr5 1LM:
Core Addr_____ = 0x1000_0000 (256.00 M)
Ch Addr_____ = 0x400_0000 (64.00 M)
Sub_Ch Addr_____ = 0x200_0000 (32.00 M)
pseudo_Ch Addr_____ = 0x100_0000 (16.00 M)
Rank Addr_____ = 0x80_0000 (8.00 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 2
Ch_____ = 0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 0
Bank in Bank Grp = 0
Bank Addr_____ = 0
Row_____ = 0x80
```

```

Column_____ = 0x0
Mirror_____ = 0
Primary_____ = -1
*****
*****
Core Addr_____ = 0x1000_0000 (256.00 M)
Cluster_____ = 0x0
Set Index_____ = 0x0
Tag_____ = 0x800
Cha_____ = 14
Perspective_____ = FM
*****
FM ddr5 1LM:
Core Addr_____ = 0x1000_0000 (256.00 M)
Ch Addr_____ = 0x400_0000 (64.00 M)
Sub_Ch Addr_____ = 0x200_0000 (32.00 M)
pseudo_Ch Addr_____ = 0x100_0000 (16.00 M)
Rank Addr_____ = 0x80_0000 (8.00 M)
Socket_____ = 0
Hbm_____ = -1
Mc_____ = 2
Ch_____ = 0
Sub-Ch_____ = 0
pseudo-Ch_____ = 0
Chip Select_____ = 0
Log Rank_____ = 0
Chip Id_____ = 0
Dimm_____ = 0
Rank_____ = 0
Bank Grp_____ = 0
Bank in Bank Grp = 0
Bank Addr_____ = 0
Row_____ = 0x80
Column_____ = 0x0
Mirror_____ = 0
Primary_____ = -1
*****
Discovering Populated Mcs
Discovering Populated Channels
Discovering Channels
System in Non-Mirror mode, and error_type: uce
ECC Injection in Non-Mirror Mode, MC-2, Channel-0, Mirror Scrub Enabled is True
socket0 Mc0 Ch0 Patrol Scrub Disabled
socket0 Mc1 Ch0 Patrol Scrub Disabled
socket0 Mc2 Ch0 Patrol Scrub Disabled
...
socket1 Mc6 Ch0 Patrol Scrub Disabled
socket1 Mc7 Ch0 Patrol Scrub Disabled
Current : dev0 = 1, dev0msk = 0x4, dev1 = 5, dev1msk = 0x1.
adddc_flag=None
adddc_errinj_reg=0x0x0
ret_code:0x0, Dev0:0x1, Dev1:0x5, D0msk:0x4, D1msk:0x1, adddc_mask:0x0,
perspective:FM
Socket0 MC 2 Channel 0
=====
| Device0 | Device1 | |
|---|---|---|
| Injectors Status | Currently unarmed | Currently unarmed |
|-----|-----|
| Mask | 00000000 00000000 | 0001ffff ffffffff |
|-----|-----|
| Match | 00080000 00800000 | 00080000 00000000 |
|-----|-----|
| crc_err_inj_dev#_5_bits | 0x1 | 0x5 |
|-----|-----|
| XOR Mask | 0x4 | 0x1 |
|-----|-----|
=====

Restoring Original Values!!!
socket0 Mc0 Ch0 Patrol Scrub Enabled
socket0 Mc1 Ch0 Patrol Scrub Enabled

```

```
socket0 Mc2 Ch0 Patrol Scrub Enabled
socket0 Mc3 Ch0 Patrol Scrub Enabled
socket0 Mc4 Ch0 Patrol Scrub Enabled
socket0 Mc5 Ch0 Patrol Scrub Enabled
socket0 Mc6 Ch0 Patrol Scrub Enabled
socket0 Mc7 Ch0 Patrol Scrub Enabled
socket1 Mc0 Ch0 Patrol Scrub Enabled
socket1 Mc1 Ch0 Patrol Scrub Enabled
socket1 Mc2 Ch0 Patrol Scrub Enabled
socket1 Mc3 Ch0 Patrol Scrub Enabled
socket1 Mc4 Ch0 Patrol Scrub Enabled
socket1 Mc5 Ch0 Patrol Scrub Enabled
socket1 Mc6 Ch0 Patrol Scrub Enabled
socket1 Mc7 Ch0 Patrol Scrub Enabled
```

When the injection error is completed, you can also see that the waveform of RMCA is triggered at the same time.

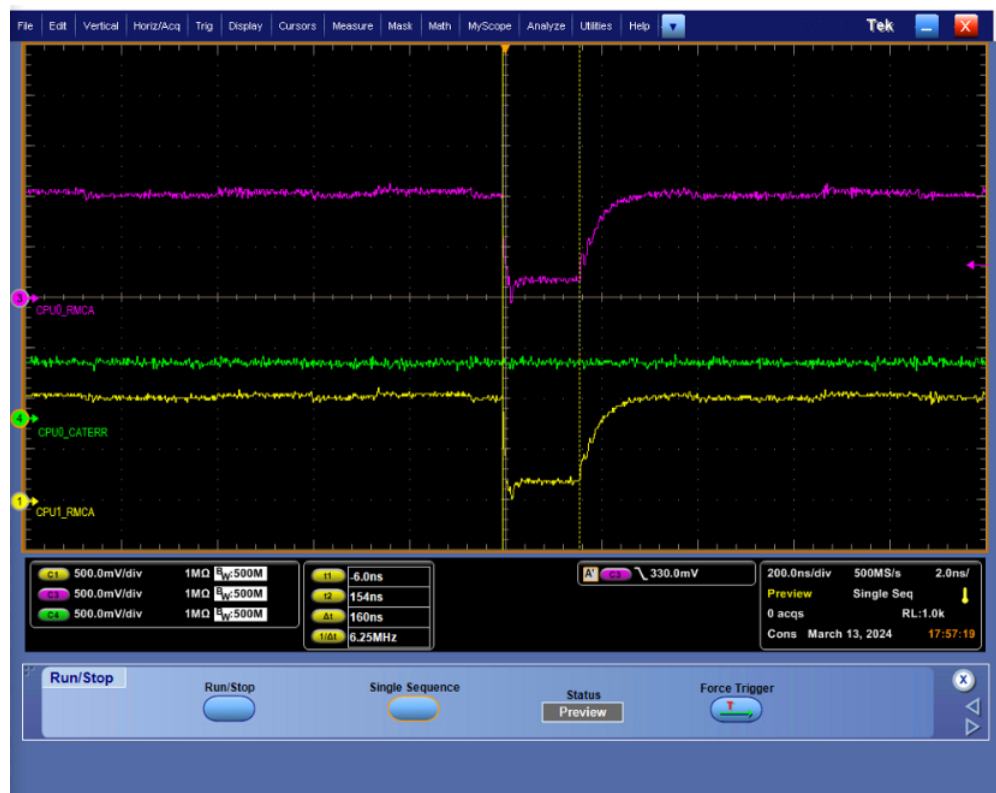


Table 7. Summary

	Enter OS		Before Enter OS	
BIOS Knob	Enable LMCE (BIOS Knob)	Disable LMCE (BIOS Knob)	Enable LMCE (BIOS Knob)	Disable LMCE (BIOS Knob)
MSR 3Ah bit status	0x100005 Enable	0x5 Disable	0x100005 Enable	0x5 Disable
MSR 4d0h bit status	0x1 Enable	0x0 Disable	0x0 Disable	0x0 Disable
Pulse (RMCA)	No	Yes	Yes	Yes

NOTE

Only after entering the OS will the true value of LMCE match the set value of the bios knob; the LMCE has always been disabled under boot progress (for example BIOS setup or uEFI Shell), the true value of LMCE not match the set value of the bios knob. Even if you set enable in the bios knob, but the LMCE read out is still disabled.

4.3.2 CATERR_N Pin

CATERR_N pin is used to signal Uncorrectable errors detected. To signal an MCERR, the processor drives CATERR_N active low for 16 BCLKs. To signal an IERR, CATERR_N is driven active low until a warm or cold reset happen.

For PCIe/IIO errors, with the IOMCA feature enabled, platform firmware/software will program Global IEH so fatal and non-fatal PCIe errors are signaled to BMC via CATERR and not ERR[2:1] pins.

Case 1: CATERR_N pin driver low for 16 BCLKs indicted MCERR happen.

Inject method:

For an example, see [HW.MCE.CHA.0x5](#) on page 155.

After error injection, check register:

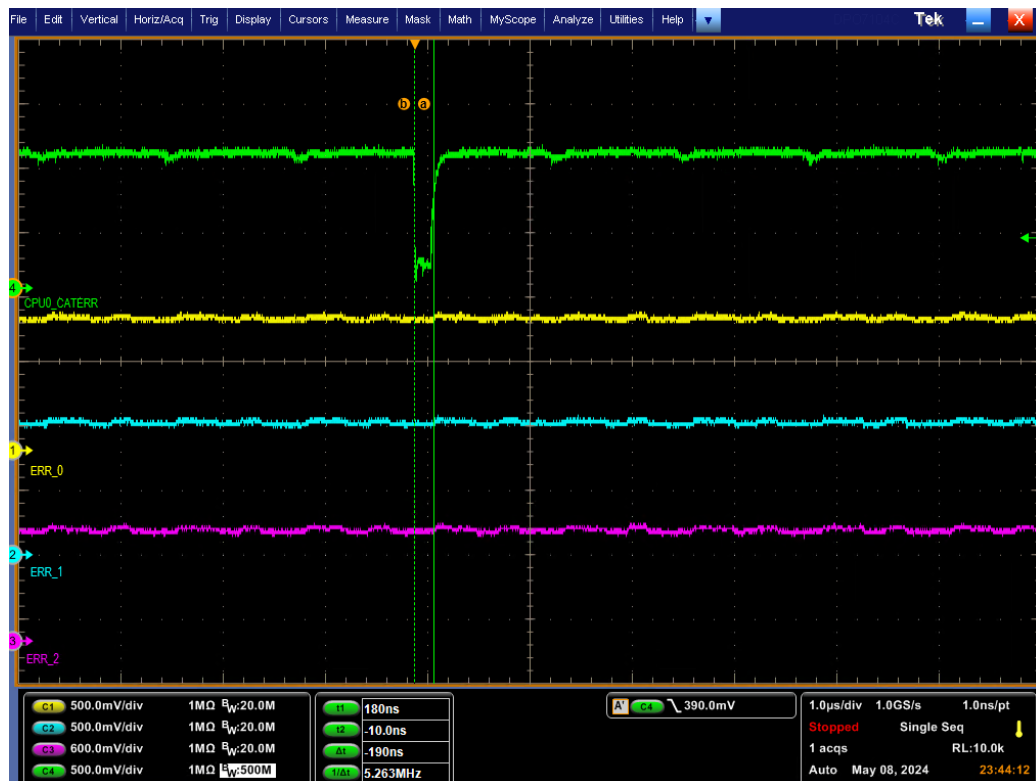
```
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions a...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); functions as a ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (bit 19,18).
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions a...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
```

```

MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMT# (for any
reason). It is or(bit 22, bit21); functions as a ...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMT_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMT_MCERR.
0x00000000 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMT#. This is OR (bit 19,18).
0x00000000 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMT_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMT_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMT
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMT
0x00000000 : first_core_ierr (04:00) -- Reserved
  
```

Waveform:



Case 2: CATERR_N is driven active low until a warm or cold reset happen indicated IERR happen.

Inject method:

For an example, see [Hang.3strike_Only](#) on page 140.

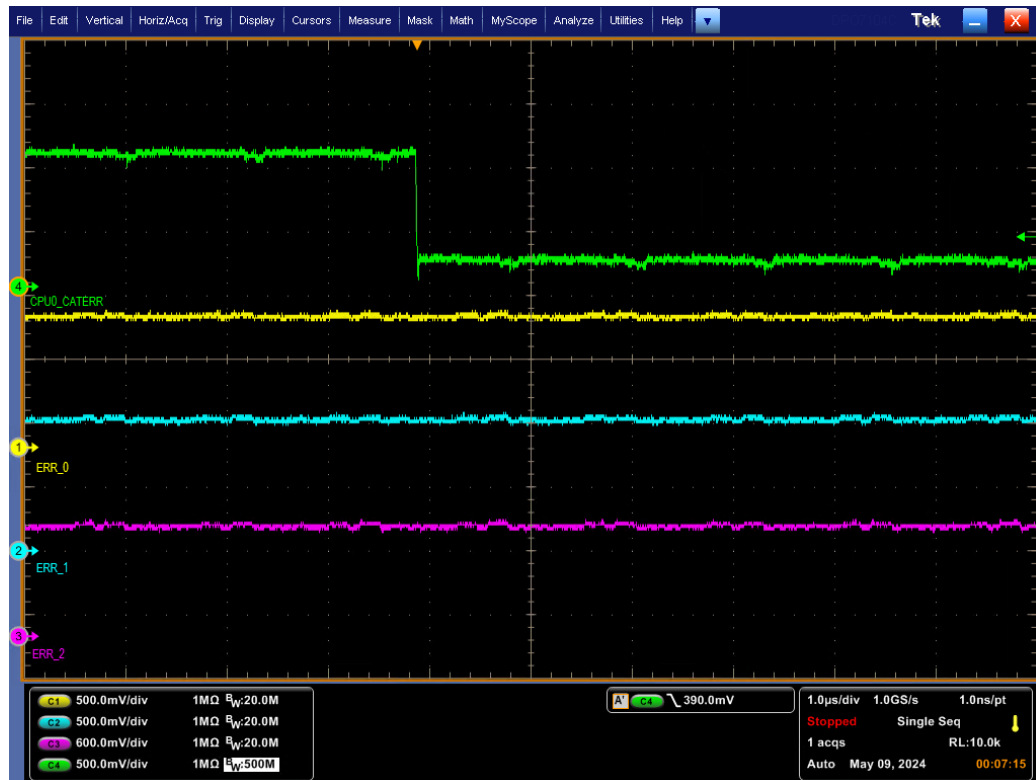
After error injection, check register:

```
>>> sv.sockets.io0.uncore.punit.ptpcfsms.ptpcfsms.mca_err_src_log.show()
0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions as a Valid bit for the other two
pac...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000000 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); functions as a Valid bit for the other two
package...
0x00000000 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000000 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (bit 19,18).
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMI_IERR.
0x00000001 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMI_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMI
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMI
0x00000000 : first_core_ierr (04:00) -- Reserved

0x00000000 : caterr (31:31) -- External error: The package observed CATERR# (for
any reason). It is or(bit 30, bit29); functions as a Valid bit for the other two
pac...
0x00000000 : ierr (30:30) -- External error: The package observed IERR.
0x00000000 : mcerr (29:29) -- External error: The package observed unrecoverable
MCERR.
0x00000000 : caterr_internal (28:28) -- Internal error: This socket asserted or
attempted to assert a CATERR#. This is OR (bit 27,26).
0x00000000 : ierr_internal (27:27) -- Internal error: This socket attempted to
assert IERR.
0x00000000 : mcerr_internal (26:26) -- Internal error: This socket asserted
unrecoverable MCERR.
0x00000000 : cam_parity_err (25:25) -- Internal error: CAM Logical to Physical
lookup has detected parity error.
0x00000000 : msmi_mcp_smbus (24:24) -- Internal error: MCP SMBus has detected
error on the bus.
0x00000001 : msmi (23:23) -- External error: The package observed MSMI# (for any
reason). It is or(bit 22, bit21); functions as a Valid bit for the other two
package...
0x00000001 : msmi_ierr (22:22) -- External error: The package observed MSMI_IERR.
0x00000001 : msmi_mcerr (21:21) -- External error: The package observed
unrecoverable MSMI_MCERR.
0x00000001 : msmi_internal (20:20) -- Internal error: This socket asserted or
attempted to assert a MSMI#. This is OR (bit 19,18).
```

```
0x00000001 : msmi_ierr_internal (19:19) -- Internal error: This socket attempted
to assert MSMT_IERR.
0x00000000 : msmi_mcerr_internal (18:18) -- Internal error: This socket asserted
unrecoverable MSMT_MCERR.
0x00000000 : rmca (17:17) -- This socket observed recoverable MCA
0x00000000 : rmsmi (16:16) -- This socket observed recoverable MSMT
0x00000000 : rmca_internal (15:15) -- This socket asserted recoverable MCA
0x00000000 : rmsmi_internal (14:14) -- This socket asserted recoverable MSMT
0x00000000 : first_core_ierr (04:00) -- Reserved
```

Waveform:



4.3.3 Error_N[0,1,2] Pin

Advanced Error Reporting (AER) uses three pin ERROR_N[2:0] to indicate system/Platform detected IIO/PCIe errors if PCIe device support AER and system IOMCA is disable.

IIO/PCIe Error Type Classifications	Reporting Domain	Signaling ¹
Correctable errors and advisory (nonfatal)	AER (Severity 0)	ERROR_N[0] pin
Uncorrected but recoverable errors (nonfatal)	AER (Severity 1)	ERROR_N[1] pin
Uncorrected Fatal Error	AER (Severity 2)	ERROR_N[2] pin

NOTE

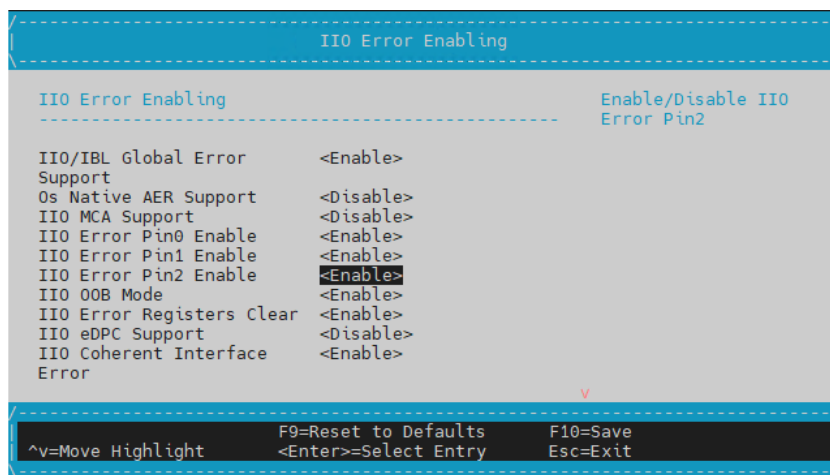
1. When system detected Memory Corrected error also will use Error_N[0] to signal this kind of error.
2. With crash log feature enable, Error_N[2] will de-assertion to indicate crash log feature is in progress.
3. With OS native AER feature enable, OS will handle error first, error Pins will not de-assert.
4. With PCIe eDPC support feature enable, when Uncorrected Fatal and Uncorrected Recoverable or Non-Fatal happening ,Error_N[1 ,2] pin will not assert,only Error_N[0] pull low as a pulse to indicate PCIe error happened.

Error injection:

BIOS setting:

Follow Chapter 4 error Injection BIOS requirement setting, addition BIOS setting as below:

- EDKII Menu → Platform Configuration → System Event Log → IIO Error Enabling → IIO MCA support→ Disable
- EDKII Menu → Platform Configuration → System Event Log → IIO Error Enabling → IIO Error Pin0/1/2 → Enable



error_N[0]:

Method: Inject PCIe CE error with cscripts cmd

```
halt()
pci.error_clear_all()
ei.iio.injectPcieError(0,"pxp2.rp0","ce")
```

Register Check:

```
>>> sv.socket0.io0.uncore.pi5.pxp2.rp0.cfg.devsts.show
0x00000000 : tp (05:05) -- When set, this bit indicates that the function has
issued Non-Posted Requests that have not been completed. For Root or Switch
port,...
0x00000000 : apd (04:04) -- Auxiliary Power is not supported.
```

```
0x00000000 : urd (03:03) -- This bit indicates that this function received an
unsupported request from PCI Express link. Errors are logged in this register
reg...
0x00000000 : fed (02:02) -- This bit indicates that this function has detected a
Fatal error. Errors are logged in this register regardless of whether error re...
0x00000000 : nfed (01:01) -- This bit indicates that this function has detected a
Non-Fatal error. Errors are logged in this register regardless of whether err...
0x00000001 : ced (00:00) -- This bit indicates that this function has detected a
Correctable error. Errors are logged in this register regardless of whether er...
```

Waveform:



Error_N[1]:

Method: Inject PCIe UCE error with cscripts

```
halt()
pci.error_clear_all()
ei.iio.injectPcieError(0,"pxp2.rp0","uce")
```

Register Check:

```
>>> sy.socket0.io0.uncore.pi5.pxp2.rp0.cfg.devsts.show
0x00000000 : tp (05:05) -- When set, this bit indicates that the function has
issued Non-Posted Requests that have not been completed. For Root or Switch port,
it applies ...
0x00000000 : apd (04:04) -- Auxiliary Power is not supported.
0x00000000 : urd (03:03) -- This bit indicates that this function received an
unsupported request from PCI Express link. Errors are logged in this register
regardless of w...
0x00000000 : fed (02:02) -- This bit indicates that this function has detected a
```

```
Fatal error. Errors are logged in this register regardless of whether error
reporting is e...
0x00000001 : nfd (01:01) -- This bit indicates that this function has detected a
Non-Fatal error. Errors are logged in this register regardless of whether error
reporting...
0x00000000 : ced (00:00) -- This bit indicates that this function has detected a
Correctable error. Errors are logged in this register regardless of whether error
reportin...
```

Waveform:



Error_N[2]:

Method: Inject PCIe Fatal error with cscripts

```
halt()
ei.io.injectPcieError(0, 'pxp2.rp0', 'uce', severity='fatal', verbose=True)
go()
```

Register check:

```
>>> sv.socket0.io0.uncore.pi5.pxp2.rp0.cfg.devsts.show
0x00000000 : tp (05:05) -- When set, this bit indicates that the function has
issued Non-Posted Requests that have not been completed. For Root or Switch port,
it applies ...
0x00000000 : apd (04:04) -- Auxiliary Power is not supported.
0x00000000 : urd (03:03) -- This bit indicates that this function received an
unsupported request from PCI Express link. Errors are logged in this register
regardless of w...
0x00000001 : fed (02:02) -- This bit indicates that this function has detected a
Fatal error. Errors are logged in this register regardless of whether error
```

```
reporting is e...
0x00000000 : nfed (01:01) -- This bit indicates that this function has detected a
Non-Fatal error. Errors are logged in this register regardless of whether error
reporting...
0x00000000 : ced (00:00) -- This bit indicates that this function has detected a
Correctable error. Errors are logged in this register regardless of whether error
reportin...
```

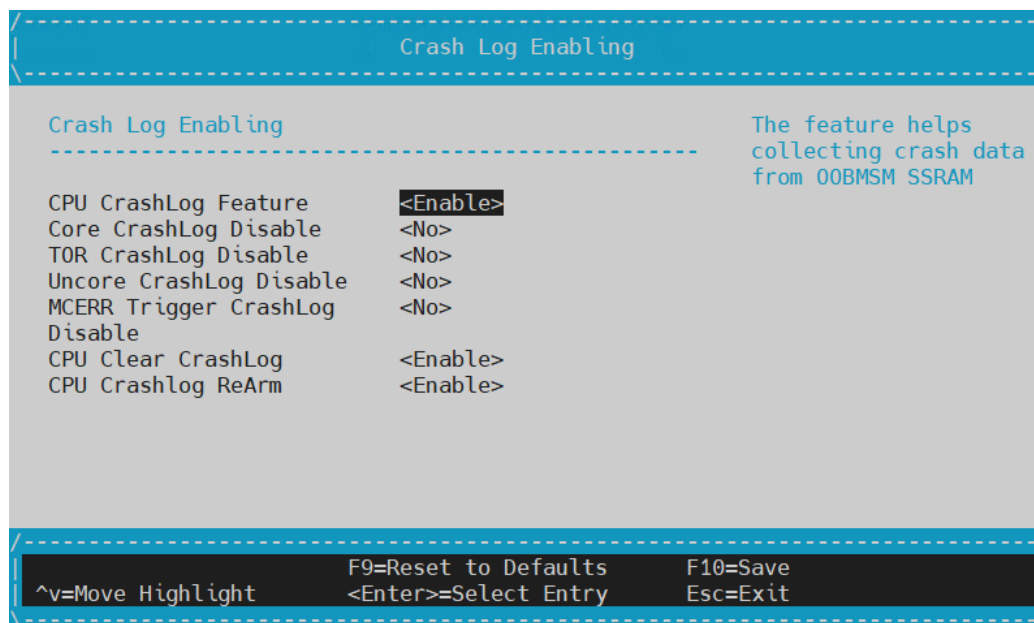
Waveform:



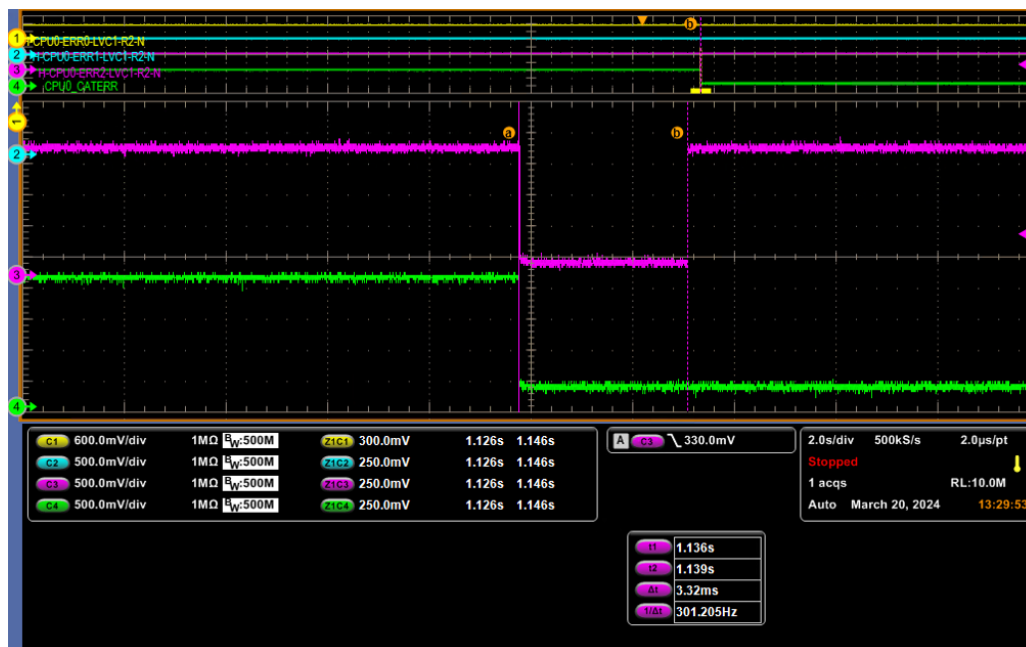
Error_ N[2] used as Crash log indictor.

With crash log feature enable, when caterr happen, Error_ N[2] will drive low, until the crash log done.

BIOS setting:



Waveform:



4.3.4 GLB_RST_Warn_N Pin

In early boot phase before the CPU come out of reset, S3M errors will not log into PCU bank but assert GLB_RST_WARN#.

Error Injection:

Method 1:

Set RTC Profile to eSPI RTC, and program the BIOS to RP .

Filter...
By order
By feature
IBL straps by...
IBL strap 0
IBL strap 1
IBL strap 2
IBL strap 3
IBL strap 4

Straps settings

Name	Value	Description
IBL strap 3	0x19C1E8C1	
eSPI CRC Check Disable for Device 1	CRC checking Disabled	For Device 1 (EC/BMC): CRC Enable Note: This soft-strap is recommended to be left in the default state (disabled) until initial platform bring-up is completed.
eSPI bus low frequency (divider by 8) mode override	Low frequency test mode disable	eSPI bus low frequency (divider by 8) mode is set as follows based on the strap Only applied for non-production system
RTC Profile	eSPI RTC	Select RTC Profile from either dRTC or eSPI RTC.

Power on the RP, the system cannot power on , GLB_RST_Warn_N will be kept low.

Method 2:

Short the RTC_I2C_SDA signal to GND at the begin of AC on, The system cannot power on, GLB_RST_Warn_N will be kept low.

Waveform:



5.0 Power-On Preparation

5.1 Reference Documents

Document	Document ID
<i>Birch Stream Platform Power-On Checklist</i>	751941
<i>Granite Rapids Processor Architecture Specification</i>	639866
<i>Granite Rapids Processor Registers Specification</i>	637776
<i>Birch Stream-SP Platform Electrical Specification</i>	645781
<i>Sierra Forest Processor Architecture Specification</i>	736624
<i>Sierra Forest Processor Registers Specification</i>	737845
<i>Birch Stream: Granite Rapids-AP/SP Processors: Design Discovery Form</i>	740237
<i>Birch Stream AP Platform Design Guide</i>	635354
<i>Birch Stream-SP Platform Design Guide</i>	633502
<i>Birch Stream-AP Pin List</i>	636245
<i>Birch Stream - SP Pin List</i>	642711
<i>Birch Stream Platform - SPI Programming Guide</i>	642713
<i>Birch Stream Platform BIOS Writer's Guide</i>	647898
<i>UBA Porting Guide for Birch Stream</i>	WIP
<i>Intel® Modular Flash Image Tool (Intel® Modular FIT) for Birch Stream Boot Image</i>	685970
<i>2022/2023 Birch Stream Advance Debug Roadshow content</i>	750089
<i>Birch Stream FW Deep Dive Event</i>	752114

NOTE

UBA Porting Guide for Birch Stream is work in progress.

5.2 Preparation Check

Table 8. Preparation Check

Category	Domain	Description
Hardware	VR	See <i>Birch Stream Platform Power-On Checklist</i> , document number 751941, 'Voltage Regulator (VR)' and 'VR Quick Check'.
	Signal and Power Sequence	See <i>Birch Stream Platform Power-On Checklist</i> , document number 751941, 'Apply Power without CPU'.
continued...		

Category	Domain	Description
Firmware		Probe the power sequence signals to verify they were asserted/de-asserted at the correct time according to Birch Stream AP/SP Platform Design Guide.
	CPU Hardware Pin Straps	See <i>Birch Stream Platform Power-On Checklist</i> , document number 751941, "CPU HW PIN Straps".
	Soft Straps	See <i>Birch Stream Platform Power-On Checklist</i> , document number 751941, "Platform Soft Straps" and "eSPI Quick Check". <i>Birch Stream Platform - SPI Programming Guide</i> , document number 642713.
	BIOS	See <i>Birch Stream Platform Power-On Checklist</i> , "BIOS (1-5)" and BIOS Preparation Check on page 183
	BMC	BMC Preparation Check on page 184
General	Design Discovery Form	See <i>Birch Stream: Granite Rapids-AP/SP Processors: Design Discovery Form</i> , document number 740237

5.2.1 BIOS Preparation Check

See *Birch Stream FW Deep Dive Event*, document number 752114, for BIOS porting. And we strongly suggest customer to implement basic porting only which related to platform power on, refer to check items listed in the next table.

Items	Notes
Birch Stream Intel® Simics® environment is functional	Boot to UEFI shell and OS with RP BKC BIOS in Intel® Simics®
Use latest BKC to build power on BIOS	Boot the BIOS into UEFI shell with Intel® Simics®
BIOS knob settings are aligned with power on requirements	Make sure current Knob settings in Intel® Simics® are aligned with requirements in <i>Birch Stream Platform Power-On Checklist</i> , document number 751941.
Make sure all security features are disabled in power on BIOS image	1. Make sure security features are disabled in BIOS setup knobs, including Intel® Trusted Execution Technology (Intel® TXT), Intel® SGX, Intel® Trust Domain Extensions (Intel® TDX), Intel® Total Memory Encryption (Intel® TME), Intel® Platform Firmware Resilience (Intel® PFR). 2. Make sure security features are disabled in Intel® Modular Flash Image Tool (Intel® Modular FIT) (Configure Intel® Boot Guard Profile = 0, Intel® TXT supported = No, TPM on SPI Disabled (IBL strap 3 bit 28), Intel® PFR supported = Disabled.)
Check GPIO table based on board designed	Hard GPIO and vGPIO programming see: <i>Birch Stream FW Deep Dive Event</i> , document number 752114. Make sure vGPIO will not gate system power on
Check PCIe* bifurcation table based on board design	1. See <i>Birch Stream FW Deep Dive Event</i> , document number 752114 - UEFI Platform Porting to configure IIO bifurcation. 2. Check PCIe* bifurcation in Intel® Simics® serial logs is aligned with BIOS UBA configuration.
Check UART configuration based on board design	1. UART selection (IBL/BMC) should meet HW design. 2. If BMC UART in use, make sure Host and BMC communicate with eSPI peripheral channel and BMC is configured to eSPI device 0.
Make sure VGA stack assignment is aligned with HW design.	Real case:
continued...	

Items	Notes
	[Display Issue]No display on monitor when LegacyVgaStack setting do not match Platform design
Make sure Board ID and Board Rev ID are correct.	Make sure board ID is correctly configured to TypeAvenueCityRP or TypeBeechnutCityRP based on your project.
Build up power on BIOS according projects design based on latest BKC BIOS source code	1. Run this power on BIOS with Intel® Simics®, and boot up successfully. 2. Check all ingredients' version is aligned with latest BKC under Intel® Simics®.

5.2.2 BMC Preparation Check

Items	Notes
Make sure that during the power-on phase, BMC sends eSPI Virtual Wire (VW) handshake to CPU.	BMC works as eSPI device 0 by default. Disable eSPI device 1 from softstrap if CPLD has an eSPI device as eSPI device 1 but its FW is not ready to support eSPI VW, or it will gate system power-on. See <i>Birch Stream Platform Power-On Checklist</i> , document number 751941 for details.
Make sure BMC UART configuration/driver is workable if host debug serial port uses BMC UART	BMC provides UART capability to host thru eSPI. If there plans to enable BMC vUART, contact BMC vendor to ensure that the vUART driver is up to date. We have seen BIOS booting slow issue caused by vUART driver.
BMC_INIT_Done checking	Check whether CPLD FW treats this signal as an important power-on indicator. If so, build a special CPLD FW to ignore it to avoid potential BMC early issue blocking platform power-on.
BMC has capability to enter to recovery mode.	To avoid potential BMC features impact power-on, make sure BMC can boot into recovery mode, or stop at uboot.
Make sure that BMC multi-function GPIOs connection (especially power management related signals)	Multi-function GPIOs may be unstable during BMC initial, make sure that those unstable will not gate power-on, especially important power management related signals which connect to those GPIOs.

6.0 Real Cases

This chapter presents actual debug cases for Birch Stream series platforms.

6.1 [System hang] System Hang at BIOS Checkpoint 0xB7 When Memory VDD Voltage over 1.145v

WW03'23 BKC VccD_HVx VRs are exceeding Vmax thresholds, root caused to VCCD_HVx VR Offset not being accounted for in Pcode, and Pcode issues MCA based on this invalid input, BIOS gets stuck.

Fail Condition:

Memory VDD voltage over 1.145v.

Fail Signature:

System stuck at check point at 0xB7

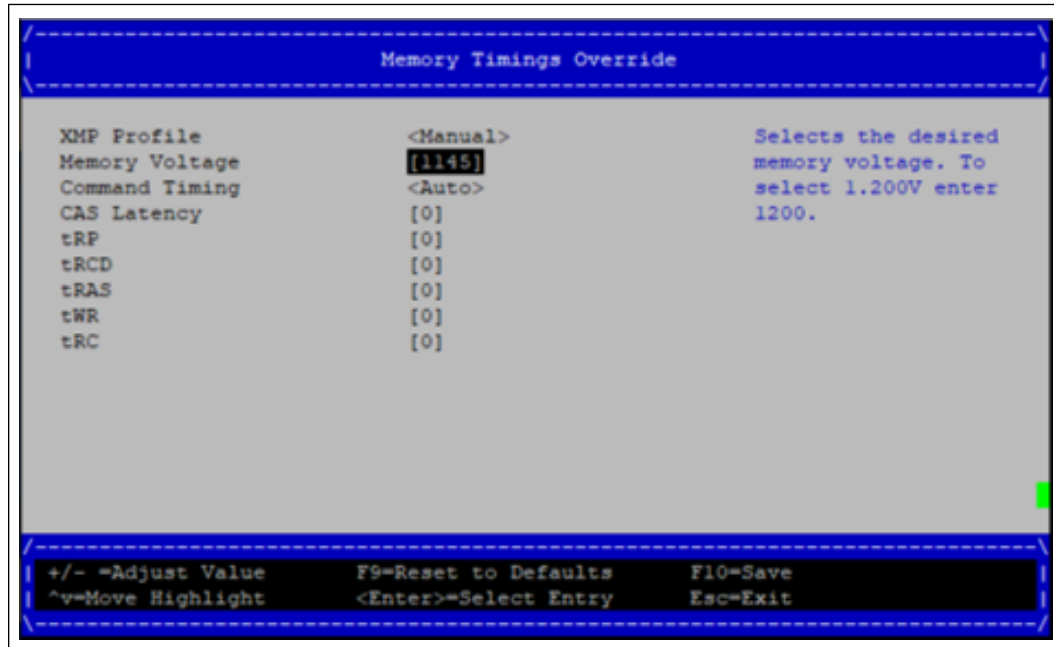
PCU bank report MCCOD 0x402 error.

Debug Process:

BIOS check:

BIOS set the value of Memory voltage to 1145 mV.

Figure 26. BIOS Memory Voltage



VR check:

Check VCCD_CPUx VR config file the voltage is 1.1V at 0A, and config VR offset 30 mV from VR settings.

Root Cause:

According to *Birch Stream Platform Message of the Week [MoW]*, document number 645322, this setting results in a normal output voltage of 1.175V (based VR 1.1V + VR config offset 30mV + BIOS programs 45mV), which exceeds 1.145V.

Solution:

BIOS Side:

Check BIOS source code as following to set the correct VR value based on platform design:

```
#define VDD_DEFAULT 1125 // This is for POR
gStructPcdTokenSpaceGuid.PcdSocketMemoryConfig.Vdd|0x465
```

VR Side:

Customers must update the VccD_HVx VR Offset to 0 mV in the config file for the PWM to achieve the desired nominal output voltage from processor BIOS (AP=1.125V and SP=1.115V). Customers will not have the ability to apply an offset through the PWM config file with this change. If an offset is needed for platform testing (for example, Margin testing), then the voltage adjustment would be made through BIOS. The BIOS range allows for an offset range of 0mV to +45 mV, from 1.1V (Vboot).

After both BIOS side and VR side solution implemented, the normal output voltage now is 1.125V in this sample (based VR 1.1V + VR config offset 0 mV + BIOS programs 25mV), meets the requirement.

NOTE

If an offset is needed for platform testing (for example, Margin testing), then the voltage adjustment would be made through BIOS. The BIOS range allows for an offset range of 0 mV to + 45mV, from 1.1V(Vboot). Range keeps in 1.1V to 1.145V.

6.2 [Display Issue] No Display on Monitor When LegacyVgaStack Setting Do Not Match Platform Design

Fail Condition:

System boot with on board DC-SCM BMC VGA.

Fail Signature:

BMC VGA is not workable during BIOS POST. When system boots into EFI shell, only BIOS console screen works, no display on monitor.

Debug Process:

1. Check VGA device with PCIe* topology and configuration dump.

It can be checked with below Cscripts command (Avenue City RP as example). Make sure configure space registers dump have values return, not all "FF".

```
>>> pci.devices()
.....
|00|15|00|0|f1a8|8086|03|00|390d|8086|...|ff|INTA|SSD 660P Series |
|00|16|00|0|1533|8086|03|00|0000|ffff|...|ff|INTA|I210 Gigabit Network
Connection |
|00|17|00|0|1150|1a03|06|01|1150|1a03|18|18|ff|INTA|AST1150 PCI-to-PCI Bridge
|
|00|18|00|0|2000|1a03|52|00|2000|1a03|...|ff|INTA|ASPEED Graphics Family |
|00|19|00|0|0014|1912|03|00|....|....|...|ff|INTA|uPD720201 USB 3.0 Host
Controller |
.....
>>> pci.config_dump(0x18,00,0)
+-----+
| PCI Dump for S:B:D:F = (0:0x18:0x00:0) |
+-----+
| Offst | f e d c b a 9 8 7 6 5 4 3 2 1 0 |
+-----+
| 00f:000 | 00000000 03000052 02100007 20001a03 |
| 01f:010 | 00000000 00004001 93000000 92000000 |
| 02f:020 | 20001a03 00000000 00000000 00000000 |
| 03f:030 | 000001ff 00000000 00000040 00000000 |
| 04f:040 | 00000000 00000000 00000000 ffc35001 |
| 05f:050 | 00000000 00000000 00000000 00840005 |
/ ..... / ..... / ..... /
+-----+
>>>
```

2. Check P2P bridge and VGA Device ID/Vendor ID present.

In this Avenue City RP example, P2P bridge's vendor ID is 0x1a03 and device ID is 0x1150. VGA's vendor ID is 0x1a03 and device ID is 0x2000.

```
.....
|00|15|00|0|f1a8|8086|03|00|390d|8086|...|ff|INTA|SSD 660P Series |
|00|16|00|0|1533|8086|03|00|0000|ffff|...|ff|INTA|I210 Gigabit Network
Connection |
|00|17|00|0|1150|1a03|06|01|1150|1a03|18|18|ff|INTA|AST1150 PCI-to-PCI Bridge
|
Document Number: TBD, Revision: 0.3 87
Intel Confidential
|00|18|00|0|2000|1a03|52|00|2000|1a03|...|ff|INTA|ASPEED Graphics Family |
|00|19|00|0|0014|1912|03|00|....|....|...|ff|INTA|uPD720201 USB 3.0 Host
Controller |
.....
```

3. Check MMIO resource assigned for P2P bridge and VGA device.

```
>>> pci.resources()
+-----+
|S|B|D|F| | |
|e|u|e|n| Memory Range | I/O Range |
|g|s|v|c| (*=disabled, #=prefetchable) | (*=disabled) |
+-----+
.....
|00|17|00|0| 0000000092000000-00000000930fffff| 00004000-00004fff|
|00|17|00|0| #*ffffffffff00000-00000000000fffff| .....-.....|
|00|18|00|0| 0000000092000000-0000000092fffff| .....-.....|
|00|18|00|0| 0000000093000000-000000009303fffff| .....-.....|
|00|18|00|0| .....-.....| 00004000-0000407f|
.....
```

4. Check BIOS code for PCIe* bifurcation and Board ID based on platform board design. For board ID setting, see real cases.
5. BMC VGA is workable in Intel® Simics®. Double-check Intel® Simics® PCIe* topology. Intel® Simics® PCIe* topology different with real PO system.
6. Based on board design, check IIO stack's BIOS VGA settings. In this case, we discovered IIO stack that BIOS had assigned to VGA device was misaligned with platform design.

```
Reference Code:
gStructPcdTokenSpaceGuid.PcdSocketMplinkConfig.LegacyVgaSoc|0x0 //
Legacy VGA Socket
#ifdef(GNRSRF_AP_HOST)
gStructPcdTokenSpaceGuid.PcdSocketMplinkConfig.LegacyVgaStack|0x2 //
Legacy VGA Stack
#endif
#ifdef(GNRSRF_SP_HOST)
gStructPcdTokenSpaceGuid.PcdSocketMplinkConfig.LegacyVgaStack|0x1 //
Legacy VGA Stack
#endif
```

For more details about CPU PE port and Stack ID mapping, see *Birch Stream Platform BIOS Writer's Guide*, document number 647898, chapter "PE and Stack ID Numbering".

In Intel RP Avenue City for Granite Rapids/Intel® Xeon® 6900-Series Processor with E-Cores design, DC-SCM 2.0 module is connected to socket0 PE0. Its stack ID is 2.

In Intel RP Beechnut City for Granite Rapids/Intel® Xeon® 6900-Series Processor with E-Cores design, DC-SCM 2.0 module is connected to socket0 PE1. Its stack ID is 1.

You can collect BIOS serial logs to check if VGA stack assignment is correct.

```
LegacyVgaSoc: 0  
LegacyVgaStack: 2
```

Root Cause:

Incorrect LegacyVgaStack assignment in BIOS code will result VGA malfunction. VGA works after fixing the incorrect VGA stack assignment.

Solution:

Make sure LegacyVgaStack is assigned correctly based on platform board design.

Appendix A Overview

This appendix will attempt to explain some technical terms. Feel free to give feedback or discuss with us if you find something needs updating.

A.1 What is MCERR and IERR?

MCERR stands for Machine Check Error, though sometimes people use it interchangeably with the term MCE, which stands for Machine Check Exception.

Typically, MCERR means a UCE error with PCC = 1 or a UCE error with PCC = 0 but with a SRAO or SRAR type. If the related error is enabled in its control register of the MCA bank, MCERR will signal this error and jump into the INT18 (Machine Check Exception: #MC) handler when eMCA is disabled. If eMCA is enabled, a SMI will be triggered before jumping into the INT18 handler when MCERR happens, whether to trigger #MC depends on SMI handler.

IERR is asserted by a processor when the processor is experiencing a serious internal error. IERR will be asserted until the processor is reset. Some source of IERR may have log in related MCA bank with both UCE and PCC are set.

Currently 5 popular sources may cause IERR:

- 3Strike-Timeout (see [What is 3Strike-Timeout?](#) on page 192 to get definition)
- PCU errors including pcode errors and PCU hardware errors (see XRefto get more debug information)
- MCERR while the processor is handling another MCERR (Nested MCE) (see [What is Nested MCE?](#) on page 192 to get definition)
- CR4.MCE = 0 while an MCE happens
- Fuse errors

4th Gen Intel® Xeon® Processor Scalable Family, Codename Sapphire Rapids, redefines functionality of the sideband error signaling as follows:

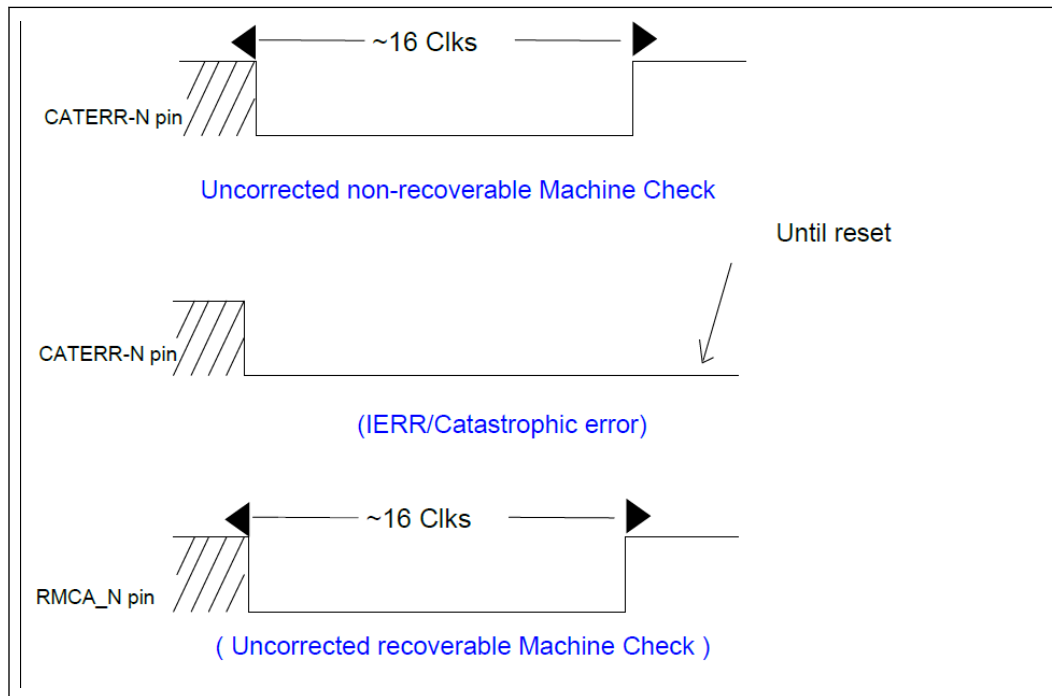
- CATERR_N: Combines signaling of uncorrected non-recoverable and fatal error to the remote processors per node controller in the partition:
 - The pin combines functionality of legacy CATERR_N and MSMI_N pins.
 - The pulse signaling occurrence on an Uncorrected non recoverable error condition
 - Level signal to indicate detection of IERR/catastrophic error in the core/uncore
- RMCA_N: Recoverable MCA event.
 - 16 Bclks pulse signaling an UCR Error
 - It will not toggle on the Local Machine Check Exceptions (LMCE) nor the UCNA event

NOTE

CATERR_N and RMCA-N is not expected to be asserted for the same error.

In case of IERR/MCERR happened, it may not be possible for UEFI-FW/OS/SW to collect all the error logs immediately after the fault is detected and signaled, a platform reset (warm reset) may be required instead.

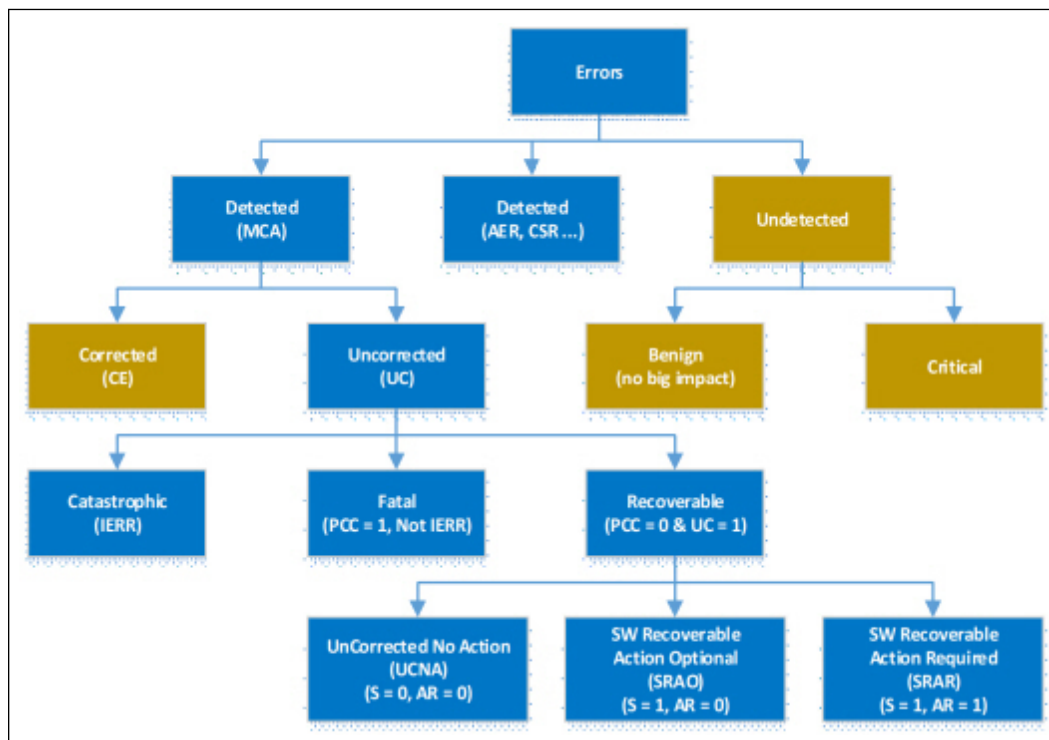
Figure 27. MCERR and IERR



A.2 Errors Classification

Based on MCA, we define errors classification as shown:

Figure 28. Error Classification



Our debug handbook will cover all blue blocks.

A.3 What is Nested MCE?

Nested MCE means a MCERR happens while another MCE handler is running.

There is an MCIP bit in bit 2 of MCG_STATUS register. This bit indicates (when set) that a machine check exception was generated. MCIP will be set by hardware directly when first instruction inside MCE handler is executed. The occurrence of a second MCE while MCIP is set will cause the processor to enter a shutdown state. The RMCA_N or CATERR# signal will be asserted.

A.4 What is 3Strike-Timeout?

3Strike-Timeout: if a transaction issued from a core to uncore did not complete within a programmable amount of time (generally 12 around 20 seconds), then the processor determines that it is not able to make forward progress, keeps the CATERR# or RMCA_N (When eMCA2 is enabled) pin at a low level, and stops execution.

The timer is in the MLC controller (for Granite Rapids)/BUS controller(for Intel® Xeon® 6900/6700-Series Processors with E-Cores).

When a core transaction does not hit in the L1 or L2 cache or need to directly access the uncore (for example, a request that maps to an uncacheable region), a transaction will be sent out of the MLC (for Granite Rapids) BUS (for Intel® Xeon® 6900/6700-

Series Processors with E-Cores) and the timer will start counting. If the request receives no response from the uncore before the timer expires, the 3Strike-Timeout will happen.

How to check 3Strike-Timeout:

For Granite Rapids:

- MLC BANK: IA32_MC3_STATUS
- The register value is end of with MCACOD 0x400

For Intel® Xeon® 6900/6700-Series Processors with E-Cores:

- BUS BANK: IA32_MC0_STATUS
- The register value is end of with MCACOD 0x400

A.5 What is TOR Timeout?

TOR stands for Table of Requests. The TOR is a structure that maintains attributes for multiple transactions in each CHA slice until each transaction is completed. There is a programmable timer mechanism for each transaction in the TOR which tracks outstanding transactions from either the core, IIO, or Intel® UPI. If CHA generates a request on behalf of a transaction, and the request does not get the response within the programmed time window, the CHA will issue a TOR timeout error in the CHA MCA bank. The machine check bank _STATUS register will contain an indication that a TOR timeout events has occurred and the _ADDR and the _MISC registers can be decoded to determine what kind of transaction timed out and what address it was attempting to reach.

TOR timeout is normally accompanied by 3Strike-Timeout for cases where the source of the transaction was a core in the socket. See [What is 3Strike-Timeout?](#) on page 192.

If the requester is from IIO or external PCIe* devices, the transaction will only trigger a TOR timeout but no accompanying 3Strike-Timeout. However, if the external PCIe* device has a completion timeout counter, then the completion timeout status bit in PCIe* device AER register may be set.

A.6 What Is PCIe* Completion Timeout?

See the *PCIe* Specification* for more detailed information. On 4th Gen Intel® Xeon® Processor Scalable Family, Codename Sapphire Rapids, IIO root port, PCIe* completion timeout mechanism can be enabled or disabled by devctrl2.ctd bit (bit 4).

When this mechanism is enabled, a non-posted transaction which need completion back will trigger PCIe* completion timer when the transaction was sent on PCIe* link.

If no completion message comes back after programable time expired, the IIO root port will log completion_time_out_status bit in AER UCE status register. In the meantime, the IIO root port will help return completion to higher layer such as CHA for this transaction.

A.7 What Is Double Fault (#DF)?

See Chapter 6 of *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*, document number 671200, to get detail information about Double Fault.

A Double Fault indicates that the processor detected a second exception while calling an exception handler for a prior exception. Normally, when the processor detects another exception while trying to call an exception handler, the two exceptions can be handled serially. If, however, the processor cannot handle them serially, it signals the Double Fault exception.

A.8 What Is Triple Fault?

See the *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*, document number 671200, to get detail information about Triple Fault.

A Triple Fault occurs when a logical processor encounters an exception while attempting to deliver a Double Fault. Note that Triple Fault is not an exception and will not have a handler for that.

First step of triple fault debug is to understand the instructions that were executing leading up to the triple fault. It is possible for a Triple Fault to be caused by an OS/BIOS bug or feature (yes, Triple Fault can be intentionally performed by the OS as part of requesting a reboot).

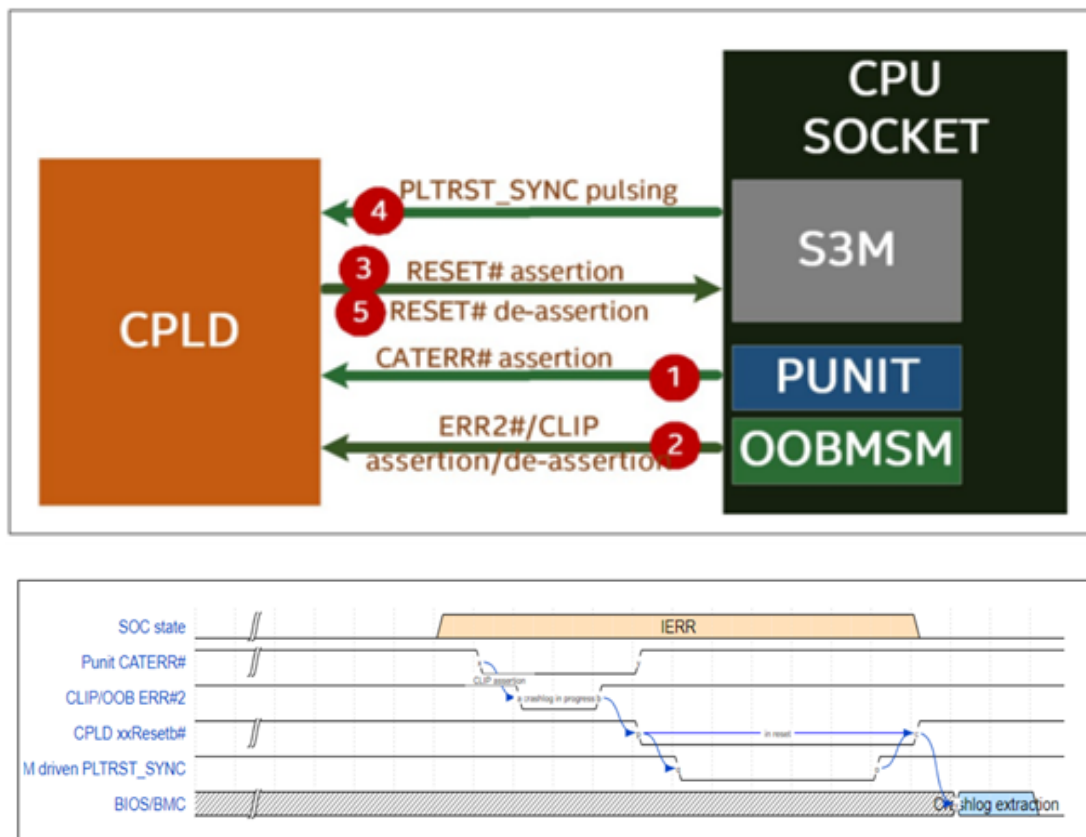
So, it is important to start with SW based approach to debug.

A.9 What Is AWR and AGR?

A.9.1 Asynchronous Warm Reset (AWR)

Asynchronous Warm Reset (AWR) is a specific type of reset flow utilized in Intel platforms to handle certain error conditions and facilitate post-reset error harvesting. It is distinct from traditional warm reset flows in that it can be initiated directly by asserting the CPU RESET_N pin without going through the usual reset preparation flows. This direct assertion can be beneficial in scenarios where the standard reset preparation might lead to timeouts or other issues.

The Birch Stream Platform supports an AWR flow, which is particularly useful following an Internal Error Report (IERR). This flow involves the direct assertion of the CPU RESET_N pin by the Complex Programmable Logic Device (CPLD). The CPLD monitors the PLTRST_SYNC output signal from the CPU to determine when the reset is complete and when to de-assert the RESET_N pin.



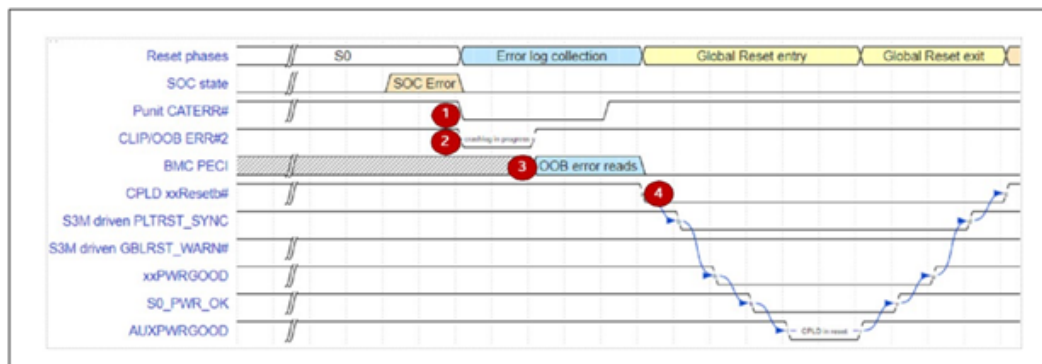
A.9.2 Asynchronous Global Reset (AGR)

The Asynchronous Global Reset (AGR) is a process that involves the direct pulsing of the RESET_N, PWRGOOD, S0_PWR_OK, and AUXPWRGOOD signals without a prior GBLRST_WARN_N assertion. This is in contrast to a graceful global reset, which includes a GBLRST_WARN_N assertion before executing the global reset sequence.

The AGR flow is executed in scenarios where the graceful flow path may not succeed. For example, AGR can be used to recover a platform in an IERR hung state if post-warm reset error harvesting is not a consideration.

AGR is particularly important for platform recovery in situations where a graceful reset is not possible or has failed. It is a critical recovery mechanism for platforms that encounter severe errors, such as an IERR hung state, and need to be restored to a functional state. When implementing AGR, it is essential to consider the potential consequences, as the reset entry flow may not complete based on the severity of the error. Additionally, platforms must be designed to handle AGR flow execution to ensure robust recovery from critical error states.

AGR Following a CATERR_N Assertion

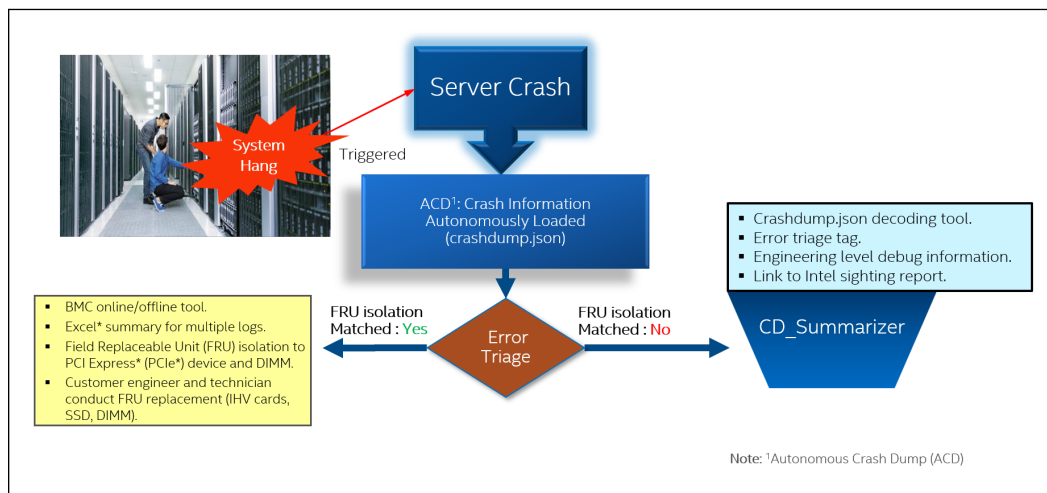


A.10 BMC Crashdump Online Parsing Tool: BMC Assisted FRU Isolation (BAFI)

On Birch Stream platform, Intel recommends using crashhump.json to perform system hang error triage. Crashdump.json could be either collected through Intel® BMC Autonomous Crashdump over PECI or Cscript error.crashdump().

Intel provides two methods to decode and parse crashdump.json. cd_summarizer.py included in Cscript can provide error triage tag with engineering level debug information as well as Intel sighting while BAFI provides real time FRU replacement suggestions in BMC according to crashdump parsing result such as PCIe* device, DIMM and CPU as shown:

Figure 29. Server Crash



BAFI is crashdump.json parsing tool that can run in BMC or standalone in off-BMC environment to quickly isolate FRU in manufacturing line or field. Note that BAFI is not used to replace Cscript's cd_summarizer tool. For those cases where BAFI cannot be used to isolate failure device or requires engineering to perform deep debug, Intel still suggest using cd_summarizer.

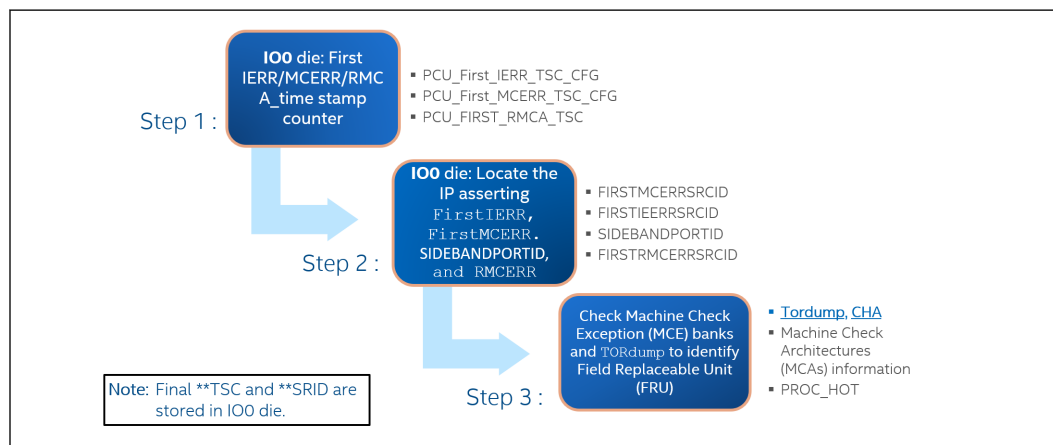
In BAFI output, crashdump.json raw data is decoded and divided into several parts. The most important instance is where the first error occurred that includes the socket and error information. The first error (FirstMCERR, FirstIERR and FirstRMCA) is further

used to isolate FRU in Bus number/Device number/Function number for failure PCIe* device. Additionally, BAFI also shows IMC UCE and CE, PCIe* UCE and CE UPI errors independently as well as CPU package thermal status. See *BMC Assisted FRU Isolation (BAFI) Solution for Autonomous Crash Dump (ACD) for Birch Stream*, document number 789161, for further detailed.

Instructions to read BAFI output:

1. Step 1: In Summary TSC section, locate which TSC register has "Occurred first between all TSCs" tag.
2. Step 2: From the first MCERR or first IERR information in step 1, locate the IP and MCE bank number that asserted FirstIERR, FirstMCERR, SIDEBANDPORTID and RMCA_FirstMCERR (for example, FirstMCERR = CHA27 soc, bank 7, 0x3C4 MCHN11 soc, bank 16).
3. Step 3: Correlate above MCE bank number with error information from MCE section or find device Bus/Device/Function number in "First.MCERR" or "CHA_Error" section if available.

Figure 30. BAFI Output



BAFI full output example with 3 steps highlighted with bold:

```
"TSC": {
  "IERR occurred": "true",
  "MCERR occurred": "true",
  "RMCA occurred": "false",
  "pcu_first_ierr_tsc_cfg": "0xd337a520c",
  ""pcu_first_mcerr_tsc_cfg": "0xc2b0cf40 (Occurred first between all TSCs)",
  "pcu_first_rmca_tsc_cfg": "0x00000000"
}
"Errors Per Socket": "msmi_mcerr_internal|msmi_ierr_internal|msmi_internal|mcerr_internal|ierr_internal|caterr_internal|FirstIERR
= Core0 compute0, bank 0-3, FirstMCERR = CHA3 compute0, bank 7, CA_FirstMCERR = 0x0, SIDEBANDPORTID = CHA3 compute0, bank 7"
}
{
  "First MCERR": {
    "Address": "0x81448000",
    "BDF": "Bus: 0x14, Device: 0x9, Function: 0x0",
    "CBAR": "0x3",
    "Description": "Gen5 PCI Express Root Port 7",
    "ErrorSubType": "Type 1: PCIe* MMCFG access cause TOR Timeout",
    "ErrorType": "TOR Timeout Error",
    "Port": "PCIe2"
  }
}
```

BAFI triage output example:

```
"trriage": {
  "BAFI_version": "2.92",
  "CPU_type": "Granite Rapids SP",
  "Crashdump_timestamp": "2023-04-11T11:52:59Z",
  "First_Error_Occurred_On_Socket": "0",
}
```

```

"socket0": {
  "IMC_Error": [
    "bank MCHN0 | MCACOD: 0x90, MSCOD: 0x80"
  ],
  "HBM_Error": {},
  "UPI_Error": {},
  "Error_Information": [
    "S0.MCERR.CHA27 soc, bank 7 | MCACOD: 0x1136, MSCOD: 0x0c"
  ],
  "FRU": [
    {
      "Address": "0x1f0fee000000",
      "Bus": "0x80",
      "Device": "0x3",
      "Function": "0x1",
      "Description": "PMON MSM Registers"
    }
  ],
  "CHA_Error": {
    "Address": "0x1f0fee000000",
    "BDF": "Bus: 0x80, Device: 0x3, Function: 0x1",
    "CHA": "CHA27"
  },
  "PPIN": "0x0ba92744c4869233",
  "Ucode_Patch_Ver": "0x800004a1",
  "CorrectableMemoryErrors": [
    "J8J1, IMC 0 Channel 0 SLOT: 0",
    "J8J1, IMC 0 Channel 0 SLOT: 0"
  ],
  "UncorrectableMemoryErrors": [
    "J8J1, IMC 0 Channel 0 SLOT: 0",
    "J8J1, IMC 0 Channel 0 SLOT: 0"
  ],
  "PcieAerUncorrectable": {
    "B08_D00_F4_0x104": "uncorrectable_internal_error|"
  },
  "PcieAerCorrectable": {},
  "PackageThermalStatus": {
    "compute0": "Ok",
    "compute1": "Ok",
    "compute2": "Ok",
    "io0": "Ok",
    "io1": "Ok"
  }
},
"socket1": {
  "IMC_Error": {},
  "HBM_Error": {},
  "UPI_Error": {},
  "Error_Information": [
    "S1.N/A"
  ],
  "FRU": {},
  "CHA_Error": {},
  "PPIN": "0x0ba9adc2582b175c",
  "Ucode_Patch_Ver": "0x800004a1",
  "CorrectableMemoryErrors": {},
  "UncorrectableMemoryErrors": {},
  "PcieAerUncorrectable": {
    "B01_D02_F0_0x104": "completion_timeout|"
  },
  "PcieAerCorrectable": {},
  "PackageThermalStatus": {
    "compute0": "Ok",
    "compute1": "Ok",
    "compute2": "Ok",
    "io0": "Ok",
    "io1": "Ok"
  }
},
},

```

FRU replacement Priority (User can see the priority list to replace failed devices):

```
"FRU_Replacement_Priority": {
  "DIMM": {
    "IMC_Error": {
      "socket0": [
        "bank MCHN0 | MCACOD: 0x90, MSCOD: 0x80"
      ]
    },
    "Memory_errors": {
      "socket0": {
        "Uncorrectable": [
          "J8J1, IMC 0 Channel 0 SLOT: 0",
          "J8J1, IMC 0 Channel 0 SLOT: 0"
        ]
      }
    },
    "Suggested_actions": "Replace DIMM"
  },
  "PCIe/IIO": {
    "Failing_FRUs": [
      {
        "Address": "0x1f0fee000000",
        "Bus": "0x80",
        "Device": "0x3",
        "Function": "0x1",
        "Description": "FMON MSM Registers"
      },
      {
        "Address": "0x1f0fee000000",
        "BDF": "Bus: 0x80, Device: 0x3, Function: 0x1",
        "CHA": "CHA27"
      }
    ],
    "Suggested_actions": "Replace PCIe/IIO or the device behind the root port"
  },
  "CPU": {
    "First_Error_Occurred_On_Socket": "0",
    "Suggested_actions": "Replace CPU"
  }
}
```